



现代计算机科学与技术教材系列

1

计算机科学导论

SUAN KEXUE DAOLUN

第三版

JI

赵致琢 著

XIANDAI JISUANJI KEXUE
YU JISHU JIAOCAI XILIE



科学出版社

www.sciencep.com

(TP-2427.0301)

计算科学导论

第三版

JI

SUAN KEXUE DAOLUN

XIANDAI JISUANJI KEXUE
YU JISHU JIAOCAI XILIE

ISBN 7-03-013024-3



9 787030 130242 >

科学出版社 技术分社
<http://www.abook.cn>

ISBN 7-03-013024-3

定价: 24.00 元

现代计算机科学与技术教材系列

计 算 科 学 导 论

(第三版)

赵致琢 著

教育部计算机科学与技术教育与教学改革项目成果

科 学 出 版 社

北 京

内 容 简 介

本书是我社出版的《计算科学导论》一书的第三版。

本书基于计算机科学与技术一级学科(简称计算科学)人才培养科学理论体系,按照学科系列教材一体化设计的纲要,从科学哲学的角度出发,系统地介绍了计算科学的定义、特点、范畴、形态、历史渊源、发展变化、知识组织结构和分类体系,学科专业培养模式和课程体系等内容,并以学科方法论为切入点,系统地介绍了计算科学的基本问题、学科形态、核心概念、典型方法、典型实例、学科基本工作流程方式等科学哲学范畴内学科范型的内容,系统阐述了计算科学发展的特点、规律,以及学科教学和人才成长的内在规律。全书概念清楚,内容丰富,其高级科普的深度定位,通俗流畅的语言文字,深入浅出的描述和严谨的构思设计有助于读者比较全面地了解计算科学,认识计算科学和学习计算科学。

本书可作为大学计算机科学与技术专业计算机科学导论课程的教材或教学参考书。

图书在版编目(CIP)数据

计算科学导论/赵致琢著. —3版. —北京:科学出版社, 2004

(现代计算机科学与技术教材系列)

ISBN 7-03-013024-3

I. 计... II. 赵... III. 计算机科学—高等学校—教材 IV. TP3

中国版本图书馆 CIP 数据核字(2004)第 016036 号

策划编辑: 王淑兰 陈晓萍/责任校对: 耿 耘

责任印制: 吕春珉 /封面制作: 王 浩

科 学 出 版 社 出 版

北京东黄城根北街16号

邮政编码:100717

<http://www.sciencep.com>

新 蕾 印 刷 厂 印 刷

科学出版社发行 各地新华书店经销

*

1998年2月第 一 版 开本: B5(720×1000)

2004年3月第 三 版 印张: 18 1/4

2004年3月第十六次印刷 字数: 333 000

印数: 44 001—49 000

定价: 24.00 元

(如有印装质量问题,我社负责调换〈路通〉)

赵致琢简历

赵致琢，1957 年出生于上海，河南省焦作市人。高中毕业后曾在工厂当工人，后考入大学学习，1982 年毕业于贵州大学数学系计算机软件专业，获理学学士学位，1992 年毕业于中国科学院数学研究所计算机软件专业，先后获工学硕士、博士学位，现为厦门大学计算机科学系教授，在多所大学兼职，并被聘为客座教授。从事计算模型、分布式算法与程序理论、计算机教育研究，发起并连续主持全国暑期计算机科学与技术高级研讨班，担任国防工业出版社《现代计算机科学与技术教材系列》编审委员会执行副主编。

赵致琢先后参加和主持承担了国家“863”计划、国家自然科学基金、教育部教育与教学改革、福建省自然科学基金等科学研究项目，发展了一些新的计算模型和分布式算法，并率先创立了一个一级学科人才培养的科学理论体系，提出了“科学办学，内涵发展”的教育思想和“依法管理，无为而治”的大学办学理念，系统地探索了中国高等教育与研究型大学科学办学的理论体系。先后以独立或第一作者名义在国内外计算机科学与技术权威刊物发表了一批学术论文，独立创作并由科学出版社出版了两本学术专著，“教育与教学改革”成果获 2001 年国家级教学成果奖二等奖。

再 版 前 言

—— 秋 日 的 笔 记

1997 年,《计算科学导论》正式出版前,曾在内部印刷了几百册作为试用讲义。其中,有几十本赠送给了一些学校和好友。那一年,全国有若干所高等学校计算机科学系(专业)采用这本讲义作教材、教学参考书或供教师讨论。1998 年初,《计算科学导论》由科学出版社正式出版。两年以后,《计算科学导论(第二版)》出版发行。这次的修改本将成为《计算科学导论》的第三版。

承上海交通大学侯文永教授的推荐,该校姚天昉副教授采用了本书承担了授课任务。姚老师是一个认真、严谨的学者,他花了很多时间备课,增加了一些本书没有的内容,一共讲了 18 个学时,完成了预定的教学任务。课程结束前,每一个学生就此写出一篇读书报告,内容包括对学习课程的心得和对教材的批评意见。从姚老师的来信和寄来的 8 个同学的文章看,内容反映了来自不同中学、地区、家庭背景的学生对本课程和教材的真实感受。这些来自教学第一线的真实的试验反馈信息,无论是热情赞赏的还是中肯的批评意见,都极大地鼓舞我并使自己坚信正在进行的教学改革方向的正确性。我由衷地感谢侯老师、姚老师和他们的学生们为《计算科学导论》再版提供的重要修改意见。

湖南大学教授张大方博士,中国地质大学(武汉)孟永良教授,汕头大学于津副教授,河北石家庄经济学院刘坤起副教授对我十分信任,他们在没有看到样书的情况下,毅然决定从 1997 年起使用本书作为教材。湖南大学甚至在暑期的计算机短训班和专科学生中也使用了本教材。而且,据我所知,不少学校至今仍在沿用这本教材。同样对我十分信任的还有哈尔滨工业大学(现在北京工业大学任教)的蒋宗礼教授,华侨大学的张全伙教授和陈永红副教授,贵州大学的陈笑蓉副教授,石家庄铁道学院的梁新来教授,他们也都是在没有看到样书而仅仅得知教材概要的情况下,就决定购买《计算科学导论》作为教学参考书或教材。同行的信任使我深受感动,他们中一些带着对教材诚恳批评意见的来信更令我肃然起敬,中国计算机科学教育事业和科学研究事业尤其需要一大批热心的参与者和勇于实践、探索的先锋!

本书出版后,我曾应邀回到母校贵州大学,为计算机科学系 1998 年入学的新生讲解了 8 个学时,不包括布尔代数部分。这次试讲使我有机会深刻反省教材中的不足之处,为今天的修改和再版写下了许多笔记,其中包括不少第一次提出的

观点和第一版因拿不定主意而未写入教材的内容。复旦大学计算机科学系的朱洪教授在看到本书后,利用自己的“学术网络”向复旦大学、郑州大学、兰州大学、武汉大学、华东师范大学、山东大学等学校的计算机科学系推荐《计算科学导论》,同时对本书提出了一些中肯的意见。他的好意使我觉得更要认真听取各方面的意见,把教材修改好。向所在学校计算机科学系推荐《计算科学导论》的还有中国科学技术大学陈国良教授和岳丽华副教授,南京大学教授许满武博士和宋方敏博士,北京大学教授李晓明博士,北京工业大学教授刘椿年博士,中山大学李师贤教授,山东大学马绍汉教授,浙江大学何钦铭教授,武汉大学戴大为教授,天津民航学院王锦彪副教授,北京邮电大学白中英教授,太原理工大学余雪丽教授,广西大学李陶琛教授,桂林电子工业学院董荣胜副教授,贵州民族学院何光优副教授和杨承中副教授,烟台大学范辉教授(现在山东工商学院任教),云南大学李彤教授,四川大学唐常杰教授,华中科技大学李桂兰教授,西华大学宋文副教授,厦门大学叶仰明教授和王一为、黄俊清老师,以及可能有的但我尚不知道的学者。我愿借此机会,向他们表示衷心的感谢。

两次再版改正了一些文字错误和叙述不当之处,使论述更加严谨。此外,主要是根据第一、二版中有些内容太少而增加了这些内容稍稍详细一点的论述,如计算机系统结构、并行与分布式计算、计算机网络、程序理论和形式语义学、非单调逻辑等内容。当然,这些内容在叙述时采用了自然语言且尽可能通俗的形式。但正如以前我们一直有所担心的,读者可能理解起来有困难。不过,即使一些比较深入的内容理解起来有困难,也不会影响他们获得大概的意思和对有关内容的初步了解。作者衷心感谢武汉大学哲学系桂起权教授,他以科学哲学家的身份对我在科学哲学与学科方法论的结合中所做的探索性研究成果给予的充分肯定,以及后来我们的进一步交流促使我下决心在本书第三版的修订和创作中,将科学哲学与学科方法论的内容作为本书的主线之一加以贯彻和体现。配合本书第三版的修订,我们即将推出《计算科学导论教学辅导书》和《计算科学实验教程(第1分册)》,供学生课外阅读和各校实验课程选用。

原武汉大学计算机科学系的陈灏教授通过朱洪教授从国外来信,指出本书第一版介绍的由洪加威教授讲的那个童话不是他最先提出的“三个中国人算法”这一错误,并在后来我们通越洋长途电话的时候建议将这个童话取名为“证比求易算法”。实际上,在此之前,我一直不知道“三个中国人算法”并非指称这个童话,而是指洪加威教授提出的一个在无向图中回路检测问题的算法。这个回路检测算法实际上后来在洪加威教授写的一本《计算:可计算性,相似性与对偶性》(英文)的专著中被简化成“两个中国人算法”。尽管我本人1982年秋天在中国数学会第四次全国代表大会(武汉)上亲耳聆听了洪加威教授的报告,包括本书中引用的那个童话,也曾读过他在国际会议论文集上发表的文章,但因岁月流失,十几年

过去,加上洪加威教授当年(指1980年)的论文发表在一本今天不易找到的由美国出版的计算理论学术年会的论文集上,以至我“张冠李戴”,把“三个中国人算法”的语义弄错了。陈溧教授在信中还建议在适当增加内容后,将本书作为研究生的教材。我认真地思考了几天,觉得有三个因素不宜这样做。一是本书主要供本科生入学时为了对学科有一个全面的初步了解和更好地完成学业作为参考,对研究生许多内容似乎迟了一些;二是对研究生来说,许多学科前沿的研究内容从更高起点的基础而言,主要集中在本书中提到的四个方面的核心基础知识,他们应该努力掌握这些内容,而不是面面俱到地了解所有各个方向的进展情况。而且,有的内容介绍时,由于大多数学校的离散数学和理论计算机科学基础知识的教学在深度上远远不够,因此,在理解上就比较容易产生误解;三是在增加内容时,究竟应该增加哪些方面的内容我难于把握。有些内容很重要,但是必须要引入大量篇幅的预备知识才能讲清楚,这显然不合写作的初衷。感谢陈溧教授的好意,现在尚无必要实现。在此,我向远在美国的陈溧教授表示深深的感谢并向读者致歉。

不时收到一些从书店里购买了《计算科学导论》的在校学生和教师的来信,对自己从第四章里获得了未来努力的方向而感到有所收获。其中,有一些学校的若干名学生在读了本书后,基本上按照书中提出的培养模式和方式方法学习数学基础课程。这些同学非常辛苦但意志坚定,他们不仅要完成所在学校开设的课程的学习任务,而且,还得腾出时间来学习更深入的数学基础课程,弥补正常教学中基础知识的不足。更难能可贵的是,他们基本上是在没有人辅导的情况下艰难地探索前进。于是,这些学生只能牺牲大量的休息时间和寒暑假,以比常人付出更多的努力来获取知识。他们的做法使我深受感动,也令我为他们这样做是否会产生消极影响担忧,因为,这对他们来说意味着“冒险”或某种“风险投资”。令人高兴的是,他们相继来信谈到经过一段时间的努力探索,自己的思维方式正在变得比以前更为严谨,在学习专业(基础)课程时比较易于掌握,学习效率出现“加速”的喜人现象。师生们的来信无疑为《计算科学导论》中的许多观点和思想作了最好的注记。我在心底里感谢这些信任并自愿充当教学改革“先锋”的同时,为国内极少有学校能够迅速地在正常的教学中对至今提出的若干系统的教学改革方案进行全面试验感到遗憾。这使我意识到在目前国内对计算机科学与技术专业教育的办学指导思想、办学模式存在比较大的认识上的差异的情况下,继续保留第四章中计算机科学与技术专业参考办学模式,仍有其积极的意义。

也有一些同学和老师来信中谈到了他们学校目前执行的教学计划与课程体系同《计算科学导论》中给出的参考模式之间存在着较大差别,学校里同学之间的热门话题也主要是 Windows 和 Internet 网络,大多数家庭条件好的学生都在积极购买计算机系统,放在学生宿舍里以便随时上机。他们对大学生究竟应该将主

要精力放在基础课程、专业（基础）课程的学习上，还是应该放在熟练掌握计算机操作和软硬件一般开发技术上感到彷徨和困惑。特别是一些高年级的学生，在接触一些有一定深度的专业知识后，开始觉得自己的基础比较薄弱，学习往往事倍功半，而要根据《计算科学导论》的观点，重新复习或加深基础课程，特别是数理基础课程的知识基础，又感到很难静下心来认真学习。面对数理基础课程的教材，一片茫然，无从下手。这实际上是国内大多数学校计算机科学系学生日常学习、生活、业余活动热点问题的真实写照。在我所了解的不少学校中，目前能够沉下心来认真听课、看书、思考、完成大量作业的同学已属少数，中国计算机科学事业的发展前途令人堪忧。经验告诉我们，作为计算机科学与技术专业的学生，第一、二年基础课程的学习十分重要。如果一个学生能够在教学中认真听课，反复读书，大量完成课外作业，仔细思考和体会，学会分析、归纳、提炼、总结所学的知识，那么，随着时间的推移，就会逐步入门，就像“入道”一样。反之，尽管有可能学习成绩不错，但事实上并没有真正“入道”。可惜，这个问题在大多数学校一直没有引起重视。

今天，由于经济的快速发展和受外来生活方式、文化的影响，大学里浮躁的情绪不免有所抬头，计算机科学与技术专业教育在不少学校出现“文科化”的现象和趋势，这无论对国家和求学的学生来说都不是一个好的现象。现在，国内已经有几家计算机产品制造商制定了向世界经济五百强企业进军的计划，但它们能否实现多年的梦想，真正成为具有世界级竞争实力的企业，关键取决于国内高等学校能否培养出融中西文化于一体，掌握现代高级计算机科学与技术的世界一流人才。因为，若干年之后，我国计算机科学与技术包括产业发展能否赶上国外先进水平，最终要靠今天和未来培养的一代又一代新人去实现。他们能否肩负重任，取决于大学本科学习时基础是否厚实。而教学计划与课程体系是否科学，最终要由实践来检验。

对于许多读者提出的有关对计算机科学与技术的认识问题，我经常对来信者和本系的学生说：“每一个计算机科学系的学生都应经常想到自己五十岁之后干什么？二十几年之后，当你的手脚不再像年轻人那样麻利时，你会不会成为高科技行业下岗职工中的一员？同样都是高科技产品，为什么一个办公自动化软件与一个三维动画软件之间在价格上差别那么大？”我以为，目前用这样一种提问的方式回答同学们提出的类似问题比较合适，因为，从正面直接回答同学的这类提问，本书中的信息已经足够了。

有一些专家对《计算科学导论》和 A 类教改模式是否能行得通表示担忧，这也是我和许多人共同关心的问题。带着这些问题，在贵州大学和国内部分高等学校专家的支持下，1999 年夏天起我们在贵州大学举办了“计算机科学与技术高级研讨班”，向研究生和中青年教师陆续开设“高等计算机体系结构”、“并行与分布

式算法设计基础”、“高等逻辑”和“形式语义学”课程（注：高级研讨班目前课程范围已扩大到“数理逻辑基础”、“可计算性与计算复杂性”、“形式语言与自动机理论”、“算法设计与分析”等内容，2003 年经批准列入了教育部高级研讨班计划，联合主办单位已扩展到十几所高等学校，学员人数达到 260 人，2004 年仍将继续在贵州举办）。期间，一些教师放弃了暑期的度假，放弃了单位组织的游昆明“世界园艺博览会”的活动，一些教师每天往返三十几公里，风雨无阻，坚持听课，其情其景令人感慨。尽管这四门课程，因受制于目前本科教学层次和听课者的基础，受制于各种因素，尚无法达到真正意义上的 72 学时的教学内容和理想的目标，但是，人们从这一尝试得到国内外一大批高水平专家学者的肯定、鼓励而令参加者深受鼓舞。许多听完暑期两门课程的教师和学生，深感受益匪浅，并且从高级研讨班已经开始走出取得重要科研成果的学者。

历史是一面镜子。从贵州大学计算机专业 20 世纪 70 年代末、80 年代初实际执行的本科生教学计划与课程体系同 A 类模式除实验课程外比较接近且效果良好这一情况总结、分析，从暑期高级研讨班课程讲授和听课者的反映，从学科教学计划的科学性，从国内外学科发展前沿正在进行的重要课题的研究内容以及若干学校计算机科学与技术类专业本科生学习《计算科学导论》的反映等几方面初步综合分析，A、B 两类教学模式的科学性和先进性基本上不应再成为有争议的问题。事实上，国内不少大学，包括贵州大学计算机科学系的大部分教师已经在认识上逐渐趋于一致。

尽管认识上日渐明朗，然而，由于国内目前教育界面临众多困难，各校情况差异较大，在计算机科学与技术教学改革方面处于无所适从的状态。实际情况是：B 类模式面向经济建设主战场开展教学，人才培养着眼于学生将来成为专业技术熟练的、优秀的软硬件生产、开发、经营、维护工程师，以及专业技术管理工程师和跨学科复合型人才为具体目标，不把学生未来深造攻读硕士学位作为主要培养目标，所以国内几百所学校努一把力就不难做到，也与目前国内各校正在进行的学分制管理和人才培养的办学风格基本一致，借助于综合性大学多学科的优势，可很好地支持培养跨学科的信息处理复合型人才。可是，由于有 A 类模式这一“更高层次”的培养模式的存在，其以培养学科优秀人才，着眼于他们未来成为主任工程师、总工程师、教授和学者为具体培养目标，人才培养层次按传统的、世俗的价值取向较 B 类模式要高。若单纯执行 B 类模式，许多学校不甘心，而实行 A 类模式，师资队伍压力较大，改革的风险较大，学校必须对学科专业教育与教学中现行的外延发展模式和管理方式进行系统的修正，从而出现了既回避 B 类模式，又不愿意正视 A 类模式的两难境地。确实，要执行 A 类模式，因为涉及到师资队伍的普遍进修提高，涉及到执行单位和教师既得利益的重新调整，涉及到学校管理层面上许多重大改革，不确定性、不稳定性等风险性因素增加，没有极大的勇

气、科学精神、压力或动力，各校普遍采取观望态度。所以，我们常说，如果 A、B 类模式是科学的模式，那么，执行 A 类模式将是对各校学风、校风、科学精神和人文精神面貌的一次严峻考验。要很好地解决这个问题，还必须依赖于国家高等教育分层次分类办学格局的形成，有赖于国家高等教育公开、公平、公正的分级竞争秩序的形成。

正因为如此，在部分高等学校一些专家的鼎力支持下，贵州大学从自身成为研究型大学发展目标的角度出发，率先对计算机科学与技术进行从研究生到本科生教学的系统改革就显得十分难得。作为西部地区贫困省份的一所最高学府，在办学条件远比国内大多数同类学校差得多的情况下，能够排除各种干扰和困难，迅速组织师资队伍进行实际操作，没有良好的校风和传统是不可能的。今年，贵州民族学院数学与计算机科学系在新生中进行了 A、B 两类办学模式的同步改革与试验，除了实验课程因受制于现行校内办学体制尚无法彻底变革外，其他方面已经基本按照 A、B 两类办学模式的要求开展教学。

中华民族是一个富于想象力的民族，高等学校是这个民族发挥想象力的思想乐园，而青年是乐园中最具活力的人群。他们辛勤耕耘去探索，创作春天里科学的故事；他们坚韧不拔而持久，编织秋日下金色的童话。只问耕耘，不计收获，在迈向新知识的海洋中，在攀登自由王国的天梯上，即使没有鲜花与掌声，献身科学事业的志向始终与孤独的行路人时刻相伴。纵使成功与他们永远无缘，但校园的科学精神将是永恒的、宝贵的民族财富。

艺术源于生活。我一直觉得就像某些电影和小说中关于艺术是一种宗教的说法极富哲理性一样，科学技术，对所有为之献身的人来说何尝不是一种宗教。我们一直希望，大学能够培养出更多的献身科学技术这类“宗教”的人才。

感谢贵州大学、理工学院及其计算机科学系有关领导的信任和坚定不移的支持，使该校 1998 年秋入学的学生采用接近于 A 类模式的教学计划进行试验，并在后来进一步加大教学改革力度，大量培训师资，朝全面实施 A 类模式的方向又迈进了一步。我还要感谢贵州民族学院数学与计算机科学系的许多老师，是他们坚定不移地支持高级研讨班的举办，并在 2003 年按照 A、B 类模式对教学计划进行了重大的变革，教学改革正在有条不紊地进行。南京大学教授宋方敏博士、上海交通大学教授傅育熙博士最近告诉我，经过他们系里一些教师的多次讨论，已经在教学计划修订中大量吸收了我们提出的新的思想和办学理念。而令我感到欣慰的是国内若干所第一流大学计算机科学系的研究生培养方案中的学位课程与我们倡导的四门学位课程的距离正在趋近。

啰哩啰嗦地写了这么多。本来是准备用来为再版修改《计算科学导论》书稿的，没想到把“有用的”和“没用的”都记在了脑子里。现在要动手修改了，才想到要把这些东西按照先后顺序预先系统地写下来，重新记在笔记里。这些内容

看似与专业知识无关，但是，它对读者了解国内目前各高等学校的现状提供了有益的补充说明，有助于读者在学习中的决策和规划。因为两次再版都是在秋天整理的，内容也还算同前言之类的短文大抵一致，不妨就叫做“秋日的笔记”，当成再版前言。

通信地址：

361005 福建 厦门 厦门大学计算机科学系

E-mail: zzzhao@xmu.edu.cn

赵致琢

1999年10月20日（第二版）于万石山

2003年10月20日（第三版）于鼓浪屿

前 言

长期以来,计算机科学与技术专业教学计划中是否需要开设“计算科学导论”课程,如果开设这门课程,学时数安排多少,重点讲授什么内容等一系列问题在计算科学教育界存在不同认识。根据“教育部计算机科学类专业面向 21 世纪教学内容与课程体系改革课题(13-22)”研究工作的进展和体会,我们认为,还是有必要在计算机科学与技术专业教学中于第一学期开设“计算科学导论”这一课程。考虑到目前不少学校新生入学时安排几周军训,课程的学时数以安排 15—20 学时为宜。

在过去的二十几年里,已有不少学校的计算机科学系或计算机科学与技术专业开设了“计算科学导论”课程,并相继编写、编译和陆续出版了若干种《计算科学导论》的教材,广泛地应用于高等学校的教学中。然而,随着教育界对计算机科学与技术专业教育认识的不断深化,原有教材的不足之处日渐呈现出来,主要表现在以下两个方面:

(1) 大多数《计算科学导论》教材在内容上写成了计算机科学与技术本科专业(基础)课程教学内容的—个简洁的压缩版,没有从一年级学生学习中普遍关心的问题出发,就学科特点、学科形态、历史渊源、发展变化、典型方法、学科知识组织结构和分类体系、各年级课程的重点,以及如何认识计算科学,学好计算科学等问题从科学哲学和高级科普的角度去回答大家的疑问,因而难以起到后续课程导引的作用;

(2) 导论的内容与后续课程的衔接缺乏科学的论证,也由于一年级学生尚缺乏学习后续课程必要的基础知识,而该课程以往在学时数的安排上又偏大,从而导致在内容上增加了不少本来应该由其他课程(包括实验课程)承担的教学内容,如计算机操作命令与汉字编码等,也增加了一些不应该进入教学计划与课程体系中的内容,如某些高级语言和某些数据库系统语言及其应用等。其中有一部分是学生进入高年级后如果有必要,能够很自然、很容易地自学掌握的知识。

由此可见,编写一本能够真正起到计算科学导论作用的新教材,已是一项紧迫的任务。这项看似简单的工作之所以一直做得不尽圆满,除了认识上的原因之外,很重要的一条是对作者的要求较高。如果没有对计算科学学科历史渊源、学科特点、学科知识组织结构、学科发展规律和趋势等内容的整体把握,没有长期在第一线从事计算科学研究工作的体会、经验和教训,要写好这本书是不容易的。我们创作的这本教材便是在这样一种背景下面向计算机科学与技术专业一年级学生所作的一次探索与尝试,是否合适,还有待实践检验。

本书的写作是建立在科学研究基础之上的。对内容的取舍，我们着重引用“教育部计算机科学类专业面向 21 世纪教学内容与课程体系改革课题（13-22）”的一些研究结果来设计各个章节，具体内容系参考国内外一大批科学技术文献后，综合作者多年来从事计算科学研究工作的心得与体会写成。本书可作为计算机科学与技术专业一年级“计算科学导论”课程的试用教材，参考学时数为 15—20 学时。第一章至第四章的内容是课程的重点，建议各系应选派在第一线从事计算科学研究，学有所成的高水平的科学家担任本课程的教学并使用投影仪进行教学。我们希望授课教师能在一种轻松的气氛中以类似于讲故事或聊天的形式，将前四章的主要内容介绍给学生。第五章的内容是由其在 A、B 类人才培养模式下学科专业整个教学计划与课程体系中的地位以及受教学计划进度表的时空限制等因素确定的，约占 8 个学时。其中，打“*”的小节也可以不学。前四章的内容虽然重要，但却应以学生自学为主，教师辅导为辅，原因是这些内容对学生只是起一个导论的作用，要真正弄明白其内涵，只能在今后的学习中逐步完成。采用课堂讨论的方式初步解决学生在学习前四章的内容时存留的问题也许值得一试。

与本书相呼应的教学课程可以是第一学期的“计算科学实验”。在实验课的教学过程中，学生应该在实验课教师的指导下具体动手操作计算机，掌握一些简单的操作技术，提高感性认识。由于近年来入学的学生在熟悉使用计算机方面差别较大，“计算科学实验”课尤其应注意因材施教。各系应积极针对一年级学生编写《计算科学实验教程》。我们在附录中开列了一些实验单元的参考目录。对一些已经有了程序设计经历的学生，可以考虑安排熟悉各种流行的、重要的计算机软件的使用，特别是网络软件。而对那些从未接触过计算机的学生，则一切应从头开始。要注意的是，教师不必强调学生在“计算科学导论”课程中对主要内容掌握的熟练程度，也不必强调学生在计算科学实验课程中对操作的熟练程度，这些不是一年级学生学习的重点内容。我们建议“计算科学导论”和“计算机科学实验”课程期末考核方式定为考查。

本书的内容重在引导学生怎么从科学哲学的角度去认识和学习计算科学，也包括为学习后续课程准备的布尔代数的基础知识。这些内容对学生学好计算科学，顺利完成学业是有益的。在学习过程中，学生可能会对前四章的一些内容理解不准确、不深刻、不全面，这是正常的，无须大惊小怪。但是，本书中带有结论性的观点、方法和认识学生应牢记在心。这不仅因为它们为计算科学（教育）界多年来经验的积累，而且，随着同学们学习的不断深入，知识的不断积累，会进一步加深对这些观点、方法的认识，有助于大家学好计算科学，顺利完成学业。教学中，有条件的学校可以配发一些计算科学的课外读物，诸如科学人物评传，与计算科学有关的一些经典的综述性论文、报告文献等。

我国计算机科学类专业已有几十年的办学历史，在实践中积累了一些经验。

从最近 10 年内的办学情况来看,许多学校,特别是 1985 年以后建立计算机科学系的学校,由于师资队伍水平、办学经费投入等许多方面的因素,对计算机科学与技术专业的认识存在偏差,主要体现在教师和学生支撑计算机科学与技术发展所需要的基础估计不足,对计算机科学类专业学习的艰巨性估计不足。这两种估计不足导致专业教学计划对基础课程和专业基础课程的重视程度不够,对计算机科学实验课程的重视程度不够,其结果是大多数学校的教师和学生把这一技术科学当成一般工科专业来对待,基础课程和专业基础课程在全部课程中所占的比重下降,基础课程和专业基础课程的教学深度普遍下降。师生过多地将教学精力投入到简单的上机实践中而倾向于知识扩展的外延发展,忽视了课堂教学和实验教学本来应该走内涵发展的道路这一学科的客观要求,淡化了对基础理论知识的学习,忽视了从理论联系实际的角度出发,在教学中加强对计算机基本实验技术的掌握,从而使人才培养偏向职业技术教育。从根本上说,计算机科学这样一种人才培养倾向直接威胁到我国计算机科学学科(科学研究和高等教育)的发展和计算机产业参与国际竞争。

计算机科学与技术专业的学习是一项十分艰巨的劳动,不少近年来成长起来的青年科学家和工程师都有同感。经验告诉我们,学习计算机科学甚至比学习基础数学还要困难,因为其不少理论课程在深度上不比数学课程更简单,同时学生又要面对大量实践内容的学习,知识更新周期很短。理论与实践相结合,理论与实践的统一是计算科学的一大特点,它决定了在学习中学生要经常不断地在严密的逻辑思维与形象的实验操作之间转换学习方式,这对大多数人不是一件轻松的事。何况计算科学学科发展极快,在工作中对知识组织结构的补充与更新任务犹如泰山压顶,让人喘不过气来。难怪一些计算科学大师们感叹:“计算科学是年轻人的科学。”这就是说,一旦你选择了计算科学作为你终生为之奋斗的专业领域,就等于你选择了一条布满荆棘的道路,一条充满艰辛的人生之路。一个有志于从事计算科学研究与开发的学生,必须在大学的几年学习中打下坚实的基础,才有可能在将来学科的高速发展中,或在计算机产品的开发和快速更新换代中有所作为。当然,这样形容不一定能让读者信服,还是应该从学科本身的发展和内在规律方面去找出根据,让学生从根本上认清学好基础课程的重要性。

教材的写法理应贯彻让不同水平的读者都有收获的文学创作原则,此正所谓“深者得其深,浅者得其浅”的作品境界。当然,要做到这一点,不是一件容易的事情,何况书中的内容作这样的安排仅反映了作者的认识,至多也只是一家之说,难免出现谬误或不足之处,这是需要说明的。作者真诚希望使用本书的教师和学生能够将心得体会告诉我们,欢迎读者批评指正。倘若这本教材能够成为有志于计算科学事业的学生和读者在各个不同的学习时期的一本有用的导引,真正起到一些“导引”的作用,作者将感到由衷的欣慰。

作者在写作时，参考了大量国内外的学术著作和文献资料，但未列出参考文献的目录。这并非作者有悖于科学道德，而是对所有一年级的的大学生，我们确实不希望他们将学习的重点放在这门课程之上，更不希望他们在对科学的热情和好奇心的驱使下花费大量的时间去阅读许多专业参考文献，因为本书的内容不是大学一年级的学习重点，仅仅是一个导论而已。这一点，尤其需要向读者作一个简单的说明和交代。

作者

1996年8月于厦门大学海滨寓所

目 录

再版前言	iii
前言	xi
第一章 引论	1
1.1 计算科学一词的来历	1
1.2 科学哲学与学科方法论简介	4
1.3 一般的科学思想方法	8
1.4 计算科学初学者的正确选择	11
1.5 使用本书应该注意的事项	13
第二章 计算科学的基本概念和基本知识	15
2.1 计算模型与二进制	15
2.1.1 计算模型与图灵机	16
2.1.2 二进制	21
2.2 存储程序式计算机的基本结构与工作原理	24
2.3 数字逻辑与集成电路	27
2.4 机器指令与汇编语言	30
2.5 算法、过程与程序	32
2.6 高级语言、程序设计技术与方法	42
2.7 系统软件与应用软件	44
2.8 计算机图形学、图像处理与模式识别	46
2.9 逻辑与人工智能	47
2.10 计算机组织与体系结构	49
2.11 并行计算机、通道与并行计算	51
2.12 计算机网络与通信	53
2.13 高性能计算	58
第三章 计算科学：它的意义、内容和方法	60
3.1 什么是计算科学	60
3.2 学科的基本问题	65
3.3 计算科学发展主线	70
3.3.1 计算科学的知识组织结构	70
3.3.2 计算模型与计算机	72
3.3.3 计算模型、语言与软件开发方法学	92

3.3.4 应用数学与计算机应用	111
3.3.5 学科发展带给我们的启示	133
3.4 计算科学的分类与分支学科简介	136
3.4.1 构造性数学基础 (数理逻辑、代数系统、图论、集合论等)	136
3.4.2 计算的数学理论 (计算理论、高等逻辑、形式语言与自动机、 形式语义学等)	144
3.4.3 计算机组成原理、器件与体系结构 (计算机原理与设计、体系结构等)	147
3.4.4 计算机应用基础 (算法基础、程序设计、数据结构、数据库基础、 微机原理与接口技术等)	148
3.4.5 计算机基本应用技术 (数值计算、图形学与图像处理、网络、多媒体、 计算可视化与虚拟现实、人工智能等)	149
3.4.6 软件基础 (高级语言、数据结构、程序设计、编译原理、数据库原理、 操作系统原理、软件工程等)	149
3.4.7 软件开发方法学 (并行与分布式计算机系统、智能计算机系统、 软件开发方法学等)	151
3.5 计算科学与数学和其他相关学科的关系	152
3.5.1 为什么说数理逻辑和代数是计算科学的主要基础	152
3.5.2 计算科学与其他相关学科的关系	154
3.6 范型及其科学意义	156
3.7 计算科学的学科形态与核心概念	161
3.8 计算科学的典型方法与典型实例	163
3.9 学科基本工作流程方式及其科学意义	173
3.10 计算科学学科特点、发展规律、趋势及其社会影响	175
3.11 计算科学知识组织结构及其演变	180
3.12 计算机产业发展前景	184
第四章 如何学习计算科学和健康成长	189
4.1 计算科学 (专业) 的培养规格和目标	189
4.2 一个计算科学 (专业) 参考教学计划与课程体系	193
4.2.1 计算科学专业本科生 A 类教学计划与课程设置进度表	193
4.2.2 计算科学专业本科生 B 类教学计划与课程设置进度表	198
4.3 如何学习计算科学和顺利完成学业	203
4.3.1 思维方式的数学化及其实现的途径与方法	204
4.3.2 计算科学 (专业) 各学期重点课程	219
4.3.3 实验课程在计算科学教学计划中的作用和地位	221
4.3.4 提高专业技术能力的途径与方法	225

4.3.5 科学思想方法的训练与形成.....	227
4.4 理解科学与科学素养	232
4.4.1 理解科学与科学素养	233
4.4.2 对科学研究活动和一些热点问题的重新认识.....	234
第五章 布尔代数基础	243
5.1 集合的基本概念与基本运算	243
5.1.1 从属与包含关系	243
5.1.2 集合的基本运算和基本关系	244
5.2 自对偶的公理系统	251
5.2.1 布尔代数公理系统	251
5.2.2* 标准形式和公理系统的完备性	261
习题	266
附录 计算机科学系一年级计算科学实验参考目录	268
一年级(上)计算科学实验单元参考目录 (54—72 学时)	268
一年级(下)计算科学实验单元参考目录 (54—72 学时)	269
后记	270

第一章 引 论

本书主要用于大学计算机科学系一年级学生“计算机科学导论”或“计算机科学与技术导论”课程。

面对计算机科学与技术专业的学生，面对有可能对计算机科学与技术知识知之甚少的初学者，作为导引性课程的教材，我们希望能从如何成长为一个优秀的专业技术人才这样一个问题谈起，从更一般的认识层面上掌握如何学习一门新的学科专业知识的方式方法，解决如何认识计算机科学与技术，如何学习计算机科学与技术的问题，以便帮助同学们在大学期间顺利完成学业。

1.1 计算科学一词的来历

从 20 世纪 30 年代到 60 年代初，计算机科学与技术的研究与学科发展基本上处在萌芽状态，当时从事计算机科学与技术研究的科学家主要来自数学和电子科学领域。数学家最初的工作主要是围绕什么是计算开展理论探索，寻求计算的数学理论模型，弄清计算的极限。由于图灵和冯·诺依曼（Von Neumann）等人的贡献，使得存储程序式通用电子数字计算机在 40 年代诞生，人类使用自动计算装置代替人工计算和手工劳动的梦想成为现实。在此基础上，吸引了大批数学家在现代计算机系统的支持下开展计算应用研究，解决了许多过去难以解决的科学计算问题，有力地推动了计算数学的快速发展。

20 世纪 50 年代后期高级程序设计语言的发展促进了硬件、软件与理论的融合，计算的数学理论、通用电子数字计算机系统、科学计算、高级语言程序设计等多个方向的研究进展催生了计算机科学作为一个学科的出现。尽管在上个世纪的整个 60 年代，计算机科学能否作为一个学科在学术界还存在争议，但这种争议随着计算机科学本身的快速发展和取得的令人惊叹的大量成果而逐渐平息。社会对计算机专业人才的广泛需求使许多大学的计算机科学系应运而生，迅速成燎原之势。由于最初各高等学校产生计算机科学系的学科生长点所在的院系不同，到了 70 和 80 年代，众多大学的计算机科学系逐步分化为计算机科学系和计算机工程系等两大阵营，以后又出现了一些变形，如计算机科学与工程系、数学与计算机科学系、电子与计算机工程系、管理与信息科学系等。由于对计算机科学领域开展工作的侧重点不同，对学科的认识不同，产生了对学科发展道路与人才培

养方面的不同认识和诸多争议。为了解决学术界和教育界对计算机科学的认识和分歧,为学科人才培养探索一个科学的发展模式,从 1985 年起,美国 ACM 和 IEEE/CS 两个学会联手组成攻关小组,试图解决这个问题。

联合攻关小组在走访了全美几乎所有知名计算机科学家的基础上,经过近四年的工作,于 1989 年提交并发表了一份《计算作为一门学科》的研究报告。在这个报告中,他们认为计算机科学与计算机工程没有区别,本质上是相同的,并建议使用计算科学一词来涵盖计算这一领域内的所有工作。次年,联合攻关小组在这份报告的基础上,又提交了关于计算机科学教学计划的 *Computing Curricula 1991*。但《计算作为一门学科》报告对计算作为一门学科的论证在内容上不够充分。

在中国大陆,计算机科学与技术的研究始于 20 世纪 50 年代,中国科学院在数学研究所的基础上迅速组建了计算技术研究所,同时开始培养学科专业技术人才。高等学校中最早开展计算机科学与技术人才培养工作的要数清华大学和哈尔滨工业大学等学校,之后一些学校也相继成立了计算机专业。但早期大多数学科专业人才的培养是附设或结合在其他专业内开展教学的,如不少学校设置了自动控制与计算机专业、导弹与计算机专业、电机制造与计算机专业等,学科专业教学内容还只相当于一个专门化方向。从 70 年代初起,一些学校的数学系开始设置计算机专业,人才培养着眼于软件与理论方向发展的需要。从那时至 80 年代中期,大多数学校经过调整,逐步将专业更名为计算机科学理论、计算机软件、计算机系统与结构、计算机应用专业、计算机器件与外部设备专业,以后又从五个专业经过合并改为计算机科学理论、计算机软件、计算机及应用三个专业,研究生继续保留四个专业。最后,在 90 年代后期又进行了一次调整,本科专业统一为计算机科学与技术,研究生教育继续保留软件与理论、体系结构、计算机应用技术三个专业。目前,在中国大陆的学科分类中,涵盖计算这一领域研究工作的一级学科专业名称为计算机科学与技术。

在计算机科学与技术的发展历程中,随着对计算理论研究和应用的深入,大量先进的思想、方法和技术开始被应用于各行各业,并在工作中与具体的行业或学科的内容交织在一起,很难分清哪些属于计算机应用的范畴,哪些属于其他学科应用计算机技术的范畴。尽管界限模糊,但绝大多数计算机科学家都认为,计算机应用是指计算机科学与技术 in 各行各业、各个学科开展具体应用的过程中对其共性理论、概念、方法、技术研究的总称,研究工作具有共性、创新的特点和未来应用普适性方面的考虑,而将已经成熟的计算机科学与技术的理论、概念、方法和技术应用于具体的行业和学科的研究,不应该属于计算机科学与技术的范畴,它们属于计算机具体应用研究,应该分属于具体学科的研究范畴。例如,将

微型计算机应用于电子测量装置进行计算机辅助测量,如果这里涉及的计算机科学理论、概念、方法和技术都是成熟的,那么,这项研究就不应该属于计算机应用而应该属于计算机具体应用,应该列入电子测量专业领域。我国国家自然科学基金委员会和一些政府科技主管部门已经意识到了这个问题,并在与计算机科学有关的科技项目的申请中进行了一些分类,将属于计算机科学与技术在各行业、各个学科开展具体应用的过程中对其共性理论、概念、方法、技术的所有研究与开发用计算机应用技术一词来概括,将已经成熟的计算机科学与技术的理论、概念、方法和技术应用于具体的行业和学科的研究与开发统称为计算机应用。尽管学术界已经对计算机应用和计算机具体应用的内涵作出了比较明确的划分,但目前社会公众范围内,认识上还比较模糊。

现在的问题是:计算科学能否作为一门学科呢?

实际上,计算科学作为一门学科是没有问题的,有关计算科学定义,研究对象,研究范畴以及读者将要看到的学科来龙去脉和学科的内涵都将有力地正面回答这个问题。但是计算科学一词却包含着两方面的内涵,有狭义和广义之分。

狭义的计算科学指称的就是我们前面所提到的计算机科学与技术,其研究内容覆盖了对计算问题的一般研究。而广义的计算科学包含的内容要广得多,它不仅覆盖了计算机科学与技术的研究范畴,而且还包含更多的内涵。

下面,我们来叙述广义的计算科学的含义。

回顾科学技术的发展史,我们知道,历史上一共有两大学科类形态,即理论与实验。不同的学科类形态支持按照自己特有的方式方法和学科发展轨道开展学科的研究与发展,由此产生了理论科学与实验科学两个学科类。所谓学科形态是指从事该学科领域工作的文化方式,所谓学科类形态是指从事一类学科研究与发展工作且具有共性的文化方式。随着计算机科学与技术学科研究和应用的深化,一些学者开始认识到计算已开始作为理论与实验之外的第三种学科类形态出现,成为第三种从事一类学科研究与发展工作的文化方式。随着信息科学与技术的发展,在图形学与图像处理、数据库系统、人机界面、人工智能、虚拟现实等技术的支撑下,人们已经开发了许多计算可视化的系统,使计算机可以模拟真实处理问题的实验场景或人的思维过程来解决许多问题。例如,像学习机床的操作过程,学习汽车、轮船、飞机的驾驶技术这类问题,并不需要学员通过传统的实验和实际操作进行练习,而只需要在一个虚拟现实的模拟环境中就可进行练习。又如,经过长期的实验数据积累和理论模型的深入研究,美国的核武器研制现在对一般当量的核试验并不需要每次都通过实际的核爆炸来收集数据,而可以根据过去积累的数据和核爆炸模型通过计算机系统进行试验,从计算机模拟系统的运行中获得真实、可靠的实验数据,并对获得的数据进行分析处理,从中发现问题和新的规

律。由此可以看到，继理论科学、实验科学之后，计算科学已成为从事科学研究的第三种学科类形态。当然，作为一种学科类形态，计算科学并不能独立存在和发展，必须借助理论科学、实验科学两个学科类形态的支持获得自身的发展。

学科形态和学科类形态属于科学哲学的范畴，它们都属于库恩（T. S. Kuhn）在他著名的学说《科学革命的结构》一书中提出的科学范型（Paradigm）¹⁾的内容。有关这方面的内容，我们将在后面的章节中系统介绍。

本书除非特别说明，否则所述计算科学均指称狭义计算科学。而广义计算科学除了狭义计算科学的内容，还包括计算作为一个学科类形态所包含的学术范畴和内涵。

1.2 科学哲学与学科方法论简介

亚里士多德（Aristotle），古希腊伟大的思想家、哲学家，他创立了科学研究的三段论方法，最早给出了形式逻辑的一些基本定律，如矛盾律、排中律等，其一生的主要科学贡献是为形式逻辑奠定了基础，成为演绎逻辑主义的代表人物。在19世纪中叶以前，自然科学的发展一直是依附在哲学的母体之中的，这不仅是因为当时的社会发展水平较低，宗教势力强大，还由于自然科学本身的发展比较缓慢所致。在亚里士多德时代，自然科学的发展主要从古典哲学中获得科学思想方法，演绎逻辑成为科学发展最重要的逻辑基础，三段论成为判断科学知识真理性的主要方法论工具。

17世纪初，近代自然科学开始形成，科学的发现和科学的发展亟需要拓展科学的思维方式。英国科学家、哲学家培根（F. Bacon）研究了科学方法论，系统地提出了归纳法，发表了其代表作《新工具》。培根主张科学是一个归纳系统：普遍的原理性和定律性知识是由大量来自经验观察的具体而特殊的知识经归纳得到的，普遍的知识是否可靠依赖于采用的归纳方法是否正确。以后，由于一些科学家的进一步工作，形成了历史上著名的逻辑经验主义。事实上，归纳法的思想最先并非出自培根，在亚里士多德时代就已经由亚里士多德的学生苏格拉底（Socrates）提出，只是没有培根提的系统。然而，演绎逻辑主义和逻辑经验主义这两个科学方法论学派各自都有其自身无法克服的缺陷。演绎逻辑主义的方法论没有逻辑上的漏洞，演绎逻辑保证了从真的前提通过演绎方法得到的知识的科学真理性，整个科学的真理性得到了保证。但问题是作为科学之基础的这些前提究

1) 在通常的科学哲学的中文文献中，Paradigm一词常译为范式。为区别数学和计算机科学中的范式，根据范式概念多个层次的含义，我们译为范型。

竟是什么？它们的真理性又由什么来保证呢？逻辑经验主义的立场有实践上的依据，有自然科学的实证知识作为它强大的后盾。可是逻辑经验主义在方法论的科学性上存在致命的不足，这就是其科学性后来受到怀疑的问题：从具体的、有限的经验知识永远不可能得出普遍有效的、必然的结论。换言之，普遍的真理性知识不能通过有限的经验性知识归纳出来，因为这样做常常是靠不住的。

作为自然科学理论知识的基础且真理性的东西可以称为基础之基础，这样下去，总要达到一个其本身的真理性不需要其他东西来作为基础保证的最原始的基础。作为这一最原始的基础的东西可以称之为知识的本体，对一个学科最原始的基础的可靠性或真理性进行研究的学问称为该学科的本体论。随着自然科学发展的不断分化，自然科学本体论的研究又逐渐形成了“自然科学哲学”和“专门科学哲学”。

在近代自然科学于 17 世纪刚刚建立的初期，当自然科学尚未从哲学的母体中分离出来时，哲学曾经在方法论上给了自然科学很大的推动，但随着自然科学向纵深发展，自然科学的日新月异却使哲学大大地落后了。自然科学家中学术权威的建立并不需要哲学家的帮助，因为对于大量日渐深化的自然科学新知识的获得和新知识的真理性判断问题，哲学家并不拥有比自然科学家更多的发言权。普通人从自然科学本身发展带来的好处中对科学家产生了信任，他们相信科学并不是因为哲学家对知识真理性的判断，而是亲眼所见科学解决了许多问题，帮助人们改善了生活的质量，许多人甚至从不怀疑科学知识的真理性。随着知识的不断积累和深化，对知识真理性问题和使用价值的探讨常常只能在自然科学家中进行，而哲学家所能做的，或者是接受，或者是回避。终于有一天，科学的发展遇到了危机，科学大厦的根基发生了动摇。

19 世纪至 20 世纪初，由于自然科学的迅猛发展，数学、物理学和其他自然科学在发展中因危机而发生了剧烈的变革，一些原来被认为在任何情况下都不容怀疑的科学理论（如欧氏几何、牛顿力学等）受到了严重的挑战，如非欧几何学的提出对欧氏几何学理论提出了严峻的挑战，数学的发展中也产生了不少悖论。长期以来，欧氏几何学一直是数学思维的典范，连牛顿的《自然科学的哲学原理》都仿照欧几里德《几何原本》的格式。数学中悖论的出现使得采用公理化方法改造数学成为一种潮流。欧洲文艺复兴以后，从几何学本体论研究的角度出发，很多人尝试对欧氏几何中的平行公理做出证明，以探索平行公理的基础，但他们的努力都失败了。这使得人们从另一个角度来考虑问题，把平行公理换成另一条公理，导致了 19 世纪 20 年代非欧几何学的诞生。大约同期射影几何学也正式形成。后来，牛顿经典力学中的定律因爱因斯坦相对论的提出而开始受到挑战。诸如此类的情况迫使科学家反思，由此提出了一系列自然科学的哲学问题，如科学的认

识论基础和逻辑基础问题，吸引了包括爱因斯坦、罗素在内的许多科学家和哲学家的注意。对科学知识的真理性讨论和自然科学、技术科学的哲学基础的讨论先后产生了诸多不同的哲学思潮和哲学流派，如新黑格尔主义、科学主义思潮、反理性主义思潮、逻辑实证主义、逻辑经验主义、反归纳主义，概率逻辑方法，直至历史主义的回归和否定主义的形成，现代科学哲学就是在这样一个背景下从研究这些自然科学的哲学问题中产生和发展起来的。数理逻辑的发展为这种研究提供了形式化的数学工具，同时自然科学社会地位的不断提高又对科学的哲学基础研究提出了迫切的需要，于是，西方哲学思想的发展逐渐分裂为两个不同的、独立的研究领域：科学哲学和人文哲学。从 20 世纪初至 20 世纪 40 年代，逐步发展并形成了一门独立的学科——科学哲学。

科学哲学的诞生使自然科学自脱离哲学的母体以来第一次与哲学成为相互参照、互动的学科，哲学从自然科学的发展中不仅获得了大量的科学知识，而且获得了许多新的思想方法与科学方法论工具，而自然科学也从科学哲学的发展中得到更多的启示和反省，形成了相互参照，相得益彰，相互促进，共同发展的态势。

科学哲学的产生和发展有着深刻的历史必然性。19 世纪末的欧洲，曾经发生了一场反对“形而上学”的浪潮，对发现和建立一切非实证的“知识”，亦即神秘的、独断的、纯思辨的“知识”的哲学方法进行了猛烈的抨击。问题的实质在于：究竟是科学高于哲学还是哲学高于科学？换言之，人们是否只有通过科学方法才能获得真的知识？在科学方法之外是否还存在另外一种更高的方法即某种哲学方法，通过这种非科学的哲学方法人们可以获得更高的知识？在这场浪潮中，一大批主张科学高于哲学的科学家认为不存在一种比科学方法更高的哲学方法，人们所能掌握的最高方法（最有效的和最可靠的方法）就是科学方法。因此，对科学方法合理性的研究应该是哲学研究的中心内容之一，哲学必须以科学为依据，为科学发展服务。由此，形成了科学主义。

科学哲学为科学发现提供了理论支持。所谓科学发现，可以分为两类：一类是从自然界中发现新的事实，即通过新的观察、实验手段发现新的自然现象；一类是在科学研究中提出新的概念、假说、定律，通过逻辑演绎、实验验证或计算模拟的方法建立新的理论体系，揭示科学现象或事物发展的规律。新的理论如果在自然界中找不到概念实体的直接对应，那么这种理论是一种非自然科学理论。前述两种发现是相互联系、相互促进、彼此不可分割的。新事实是新理论发现的前提，新理论可以促进新事实的发现，而且可以更深刻地理解和解释以前得到的科学发现。显然，没有科学哲学理论的支撑，科学发现的合理性和可接受性是有疑问的。

从这样一种科学主义的立场出发，西方哲学围绕自然科学中的哲学问题开展

研究,以历史上和现存的科学知识、科学实践和科学方法为研究对象,研究科学之所以成为科学,科学之所以异于非科学,科学知识之所以为真理,科学方法之所以合理和可靠等问题,逐步形成了科学哲学。

哲学的目的之一是为了寻求真理。为了保证所获得的知识是真理,哲学家认识到必须先研究判别真理的方法。正是因为这个原因,在17世纪,西方哲学的研究实现了从本体论向经典认识论的转变。

所谓经典认识论是从人们获取知识的过程来研究所获得的知识的真伪,它将人的认识活动分为三个方面:认识主体、认知对象和认知过程。认识主体是人的精神,认知过程是人的精神活动或心理活动,因此,经典认识论是以认知心理学为基础的。但是,到了20世纪,科学的发展使许多哲学家承认:科学已经走在哲学的前面。这些持科学主义态度的哲学家相信,如果世界上有真理,那么真理首先得到科学中去寻找,因为科学提供了真理的样板。哲学的任务主要是向科学学习,从哲学上对科学知识和科学方法进行系统的总结,而不是将科学的发展纳入一个事先规定的框架。于是,科学主义的哲学家从研究哲学意义上的真理转而去研究科学,他们坚信科学知识是最具有真理性质的知识。要想弄清楚科学究竟是什么(科学与非科学的区别是什么)?科学所揭示的普遍有效性和必然的发展规律究竟有什么根据?是否真的可靠(科学的基础是什么)?还得去研究知识,研究科学。于是,西方哲学在20世纪初实现了从经典认识论向知识论的转变。

知识论是要对具体科学知识和获取知识的方法进行研究,通过系统总结知识的构成成分和相互之间的关系,考察和总结科学发展中具有普遍意义的思想、概念、方法、原理、技术的方法论结论和模式,为科学家从事科学研究提供科学指导,促进科学研究认识真理过程的有效性,同时,通过研究科学的特点,探讨科学之所以为科学,非科学之所以不科学的原因。

通过上面的叙述,我们不难认识到,科学哲学既要研究科学理论,也要研究科学活动或科学实践。一般地说,科学哲学包括下列三方面的研究:

(1) 对各部门科学的逻辑结构和经验内容的分析,形成专门的科学哲学,如数学哲学等;

(2) 对科学理论和客观世界的关系进行分析,研究理论对象的“存在”问题和理论陈述的“真伪”问题,这就形成了科学哲学研究对象的本体论的内容;

(3) 对科学理论和科学家的关系进行分析,研究科学家认知科学理论和真理的活动的合理性问题,形成了科学方法论的内容。

科学的认识论、方法论和科学的逻辑基础是科学哲学最主要的研究内容,科学方法论的研究与一个具体学科相结合,就产生了学科方法论。学科方法论固然需要研究一个学科具体的各种方法,但它的主要任务不是研究如何建立各学科具

体的科学方法，而是要研究学科中不同学科领域和方向内一类科学概念、原理、方法、技术的特性，相互之间更为抽象的共同特性、特征，相互之间的关系，以及使用这些概念、原理、方法、技术的更一般化的原则、原理、方法、技术及其合理性问题。

学科方法论与学科方法学是完全不同的两个概念。学科方法论属于科学哲学层面和该学科科学哲学的研究范畴。学科方法学则是对该学科范围内一类具体学科方法进行研究的学问。例如，程序设计方法学，主要对软件开发方法学领域内各种程序设计方法进行研究，发现和建立新方法，探讨如何运用这些方法获得高质量的程序。而程序设计方法论则不同，它主要以现存的程序设计方法作为自己的研究对象，研究各种方法的特点、特性，相互之间的关系，方法的科学性和合理性，研究运用这些方法的原则、原理、方法、技术，等等。

1.3 一般的科学思想方法

每一个初入大学有理想、有抱负的理工科专业学生，都希望在大学学习的几年里，通过自己的努力，为未来成长为高素质的专业技术人才打下坚实的基础。然而，对于刚刚踏进科学园地，初入校园的一年级学生来说，面对一个陌生的环境和全新的科学技术领域，常常感到一片茫然，不知道如何来开展学习，实现自己心中的理想。即使对许多已经工作多年的成年人，在如何学习和掌握一门新的学科知识方面，无论是教育界还是学员，未必能够轻而易举地开展学习。学习就是工作，这实际上涉及到一个人如何来开展工作的科学的思想方法。

通常，我们面对一个待解决的问题，有没有什么更一般的方式方法或工作原则来指导和帮助我们较好地解决这个问题？

在高等学校长期的办学实践中，以工科为主的院校和以理科为主的院校在培养人才的过程中对这一问题的看法往往是不同的。以工科为主的院校由于其校内的学科（专业）常多为经验科学主导的技术科学和工程科学学科，专业办学比较注重实用知识和经验的传授，科学技术研究比较注重应用研究和解决实际问题，看重技术上的创新，对基础理论的重视程度不如以理科为主的院校，对各类事务的处理常希望获得立竿见影的效果。而对于理科为主的院校来说，由于其校内的学科（专业）多为非经验科学主导的理论科学、实验科学、技术科学学科，专业办学比较注重基础理论教学和知识体系的系统性，科学研究比较注重（应用）基础研究和对事物变化规律的全面认识，看重理论上的创新和学术深度，因此，对各类事务的处理不求马上获得立竿见影的效果，而是希望从机理上获得对于事物的全面的、系统的、深入的认识，在认识的基础上来指导实践，解决实际问题。

由于计算机科学与技术学科的属性和特点方面是一门更为偏理的技术科学，因此，以理科见长的院校在学科专业办学实践中整体上具有一定的优势，但工科为主的院校在开展工程研究和应用方面所拥有的协调和管理优势也是不容忽视的。

问题的实质并不在于是工科院校还是理科院校，而是在于（应用）基础研究同应用与技术开发工作特点之间的差异产生了这样的现象。因为在中国的高等学校，全面展开科学技术研究是最近 20 多年内的事情，由于国家经济上还不富裕，除了文理科等基础学科之外，对于同生产实践紧密相关的应用科学领域和工程科学技术领域的学科，大量的科学技术研究项目主要是以应用研究和技术开发研究为主，这就使得在承担这类项目有一定优势的工科院校在工作中自觉或不自觉地产生了上述对工作方式的认知和风格。正是由于文理院校大量的科学研究项目主要是以（应用）基础研究为主，研究目标不以是否产生经济效益为主，因此，在长期的工作中形成了重视基础理论的传统和风格。

无论是以理科为主还是以工科为主的院校的毕业生，凡是训练有素的科学技术人员，他们面对一个具体的待解决的困难问题，一般不是依赖于经验匆忙地开展工作，而是首先对这个问题及其相关的理论课题开展（应用）基础研究，其目的在于从多个视角、多个层面立体地对问题及其对象进行深入的剖析，系统地了解和掌握问题及其对象的特点、性质、（内在）变化规律，从根本上对被研究的对象建立认知的科学理论体系。在此基础上，解决与此对象有关的问题被认为是相对比较容易的。有了科学的认知作为基础，有了坚实的（专业）基础，就比较容易发展和建立解决相应问题的科学的方法。有了解决问题的科学的方法，基于科学的认知基础，部分地借助以往的实际工作经验，只要按照工程实践的一般原理和方式方法，建立解决相应问题的一套严密的程序，具体解决问题应该是不困难的。正如人们在日常生活和工作中总结出来的一条经验一样：只有认识世界，才能改造世界。

因此，针对理论科学和偏理的技术科学，甚至所有的学科和我们日常生活中遇到的难题，面对所要处理和解决的问题，我们所倡导的一般的科学思想方法概括起来就是：

一个科学的认知：建立在对于事物性质、特点和发展变化规律的深入的认知基础之上。

一套科学的方法：基于科学的认知，通过寻找、建立，改进或引用，发展解决这个问题的一套科学的方法。

一个科学的程序：着眼于具体解决这个问题，在科学认知的基础之上，依据确定的一套科学的方法，制定实际解决问题的一个严密的、科学的程序，确定第一步做什么，怎么做，第二步做什么，怎么做……确定每一步怎么检验，出了问

题怎么处理，等等。

一个人只要善于使用这样一种思想（方式）方法开展工作，那么，他在处理科学问题乃至日常事务的过程中往往会做得比较好。有了这样一种认识与实践的方法学概念，我们做什么事情就比较容易做好。

对于一个具有良好的科学素养的专业技术人员，正确的思想方法不仅能够帮助他解决专业领域的实际问题，而且能够帮助他迅速了解和进入一个可能完全不熟悉的领域，并对该领域内的许多事物提出自己的见解。而对于一个缺乏良好的科学素养的专业人员来说，离开了自己熟悉的专业领域，就很难使自己对不熟悉的领域内的许多事物迅速形成正确的认识和基本的判断，有时难免人云亦云，随波逐流。

关于如何提高个人的科学素养，我们将在第四章进行详细讨论。下面，我们首先给出一批看似希奇古怪的问题。读者不妨思考一下，看这些问题如何回答，我们将在后面的章节或本书的教学辅导书中给出参考答案。注意，有些问题并不一定存在标准答案。但是回答这些问题可以训练和培养我们科学的思想方法。

问题 1 考虑一般情况下，字应该怎样书写才算写得比较好？请总结一下判别字写得好的一般标准与写字应该遵循的准则。

问题 2 “凡是美的，都是简单的。”这句话对吗？如果认为不正确，请给出反例。如果认为正确，请就这句话结合你的生活、学习经验给出自己的解释，并用自己的一段话对它进行概括。

问题 3 数学、逻辑与新闻之间有什么联系？

问题 4 没有深入，就不可能有真正高水平的创新。但是，一个人读的书越多，思想受到的束缚也可能越多，创新也就变得越困难，这似乎是一个矛盾。怎么认识这一点？

问题 5 什么是艺术？请你根据自己的理解和体会给艺术下一个比较准确的定义。

问题 6 如何培养创新人才？教师、学生和社会应该怎么做，才能说我们在培养创新人才方面做得比较好？

问题 7 “不识庐山真面目，只缘身在此山中。”请根据这句话的涵义，在你学过的科学知识中寻找一个对应的实例。

问题 8 有人说，20 世纪计算机科学是领头学科，21 世纪生物学或生命科学是领头学科，此话对吗？

问题 9 一个新的概念、思想、方法、技术或理论，普通人在一时无法通过实验或实践验证的情况下，如何对它做一个初步的判断？

问题 10 在计算机科学中,一般认为求解一个问题往往比较困难,但验证一个问题就比较简单。所以,很多人猜测:证比求易。这个讲法是否正确?请用教材以外的实例举证。从这个问题可以联想得出什么新的结论吗?

上述这些问题看似可能同计算机科学与技术无关,但是,解决这些问题所需要的科学思想方法应该具有广泛的适用性。因为,一个训练有素的科学技术工作者,不仅能够在本专业做出好的成果,而且也应该在日常生活中表现出其科学智慧。

1.4 计算科学初学者的正确选择

对于一个初入大学有理想、有抱负的计算机科学与技术专业的学生,希望通过自己的努力,逐步成长为高素质的专业技术人才,这是一种非常正常的想法。问题是对于计算机科学与技术专业的初学者,如何来选择自己的发展道路,如何实现自己心中的理想呢?

按照科学的思想方法,显然他(她)首先应该弄清楚什么是高素质的计算机科学与技术专业人才?或更一般地应首先弄清楚什么是高素质的(理工科专业)人才?其次,按照高素质专业人才应该具备的条件,努力去做和实现作为高素质专业人才应该具备的这些要求和条件。

我们说在通才教育观下,第一流的专业人才应该具备三个条件:

- (1) 具有高尚的品德和良好的人文素养;
- (2) 具有坚实的专业基础和深厚的专业功底;
- (3) 富有创新意识,具有科学的思想方法。

所谓通才教育观在大学教育阶段是指教学内容的重点立足于一级学科的范围开展内工作,在此基础上,根据学生的兴趣适当拓展其他学科的知识,而且,这种拓展应该是在学习科学哲学的同时,以学科方法论和科学的思想方法作为重要的工具逐步扩大自己的学科知识结构和领域,形成良好的科学素养。因为,一个人在学习其他学科专业知识的时候,完全按照自己学习第一学科专业的方式方法去学习,并不是一个好方法,既费时,又不易掌握学科专业知识要领。而运用科学哲学的思想方法去学习一个新的学科专业知识,效果比较好,往往事半功倍。当然,在学习的方式方法上应该以自主提高、自我完善为主。

作为高素质的理工科专业人才,除了应该具备高素质专业人才应有的内涵和共性特征外,还应该具备良好的理工科专业的实践能力,较好地掌握解决本专业实际工作中提出的理论问题、技术问题和工程问题的方式方法和技能。而作为高素质的计算机科学与技术专业人才,与其他理工科学科高素质专业人才的区别仅

仅在于人才所处的学科背景和所具备的专业知识的不同，更多地了解和掌握计算机科学与技术发展的一般规律。

按照高素质、第一流专业人才应该具备的三个条件，如何来学习计算机科学与技术知识，使自己具有坚实的专业基础和深厚的专业功底，富有创新意识，具有科学的思想方法呢？显然，按照科学的思想方法，读者应该首先建立对整个计算机科学与技术学科科学的认识基础，然后基于科学的认识来指导自己的学习和研究。也许读者会问：作为初学者，他既没有经验也不具备专业基础，自然谈不上如何按照对计算机科学与技术学科的科学认识来学习计算机科学与技术，可是他又需要学习计算机科学与技术，否则他就不是一个初学者了。这似乎是一个先有鸡蛋还是先有鸡的难题。我们说，当一个人在一个系统中无法解决这个问题时，正确的做法应该考虑跳出这个系统，从系统之外去观察和解决这个问题。既然我们面对的主要是刚刚进入大学的高中毕业生，那么，我们要解决大学一年级学生在很短的时间内对于一个待学习的学科专业的认知难题，就应该考虑从专业层面认知这个学科这样一个系统跳出来，到另一个能够适合中学生认知这个学科的高级科普系统中去考虑解决这个问题。于是，这个难题便迎刃而解。如同一个人去一个陌生的地方旅游，尽管他对那个地方的情况可能一无所知，但仍然可以通过有关当地的旅游指南进行一次愉快的旅游。不过，即使有了旅游指南，如果他缺乏对旅游基本常识的了解，仍然可能使这样一次旅游成为一个悲剧或不愉快的旅行。这说明，存在某种方式可以帮助初学者顺利地学习计算机科学与技术。这个方式应该是借助科学哲学的观点和高级科普深度定位的计算机科学与技术导论。

具体地说，当一个初学者对整个学科还缺乏深入、全面了解的情况下，应该从科学哲学的角度，通过对本学科导引性课程的学习，设法对本学科的定义、范畴、特点、基本问题、发展主线、学科分类、知识组织结构、学科发展的特点和内在规律、学科教学的特点和内在规律，以及从事学科工作的基本工作流程方式等作一个初步的了解，建立起一种基本的、科学的概貌性的认识，对如何使自己富有创新意识，逐步成长为创新人才建立起一种基本的、正确的科学认识。如果做到了这一点，就能够较好地帮助自己按照科学的方式方法来选择未来的发展道路，学习本学科（计算机科学与技术）专业知识，顺利完成学业。

本书基于这样一种认识，从科学哲学的角度对计算机科学与技术的定义、范畴、特点、基本问题、发展主线、学科分类、知识组织结构、学科发展的特点和内在规律、学科教学的特点和内在规律，以及从事学科工作的基本工作流程方式等方面进行了系统的、全面的阐述。书中丰富的内容使初学者可以在很短的时间内了解到整个学科的概貌，初步了解这个学科的特点和一些重要的内在变化规律，初步了解学科的来龙去脉和未来发展的走向，初步了解本学科专业教育与教学活

动的内在规律，并对自己未来学习计算机科学与技术学科的发展道路做出正确选择，坚定自己学好计算机科学与技术专业的信念。

当今社会高新技术的快速发展和人类生活节奏的加快，五彩斑斓的世界难免使许多人心气浮躁，对于在学习本学科专业知识的过程中需要学习的大量被一些人认为是枯燥乏味的甚至是没有多大用处的数理基础知识，常感到困惑并采取实用主义的态度。然而，读者应该坚信，本书所阐述的大量内容，是对整个学科过去半个多世纪以来学科发展、学科教育与教学活动内在规律的系统的、科学的总结，特别是关于如何学习本学科的论述实际是在对计算机科学与技术进行重新科学认识的基础之上提出的，具有很强的规律性。作为初学者，学习和研究科学技术，理应按照科学的规律行事。当然，在万花筒般的计算机科学与技术世界，初学者很难不受各种社会思潮的影响，我们所希望的是读者在学习计算机科学与技术时首先应该对本书的概要和注意事项作一点起码的了解，希望在大浪淘沙中产生更多中流砥柱式的科学技术人才。

1.5 使用本书应该注意的事项

本书作为一个导引性教材，目的在于从科学哲学的角度用高级科普的形式为初学者提供一个了解和学习计算机科学与技术的导引，读者不能指望从导论中学到大量计算机科学与技术领域的专门的、具体的、系统的专业技术知识，特别要注意力戒焦躁情绪的发生。本书并不专门系统地阐述科学哲学和学科方法论的内容，而是将科学哲学的观点和学科方法论中大量成熟的内容融入到各章节之中，自始至终贯穿在各个章节的字里行间。作为高中毕业刚刚进入计算机科学与技术专业学习的学生，第一次读本书的内容，有可能对一些内容理解不深甚至不理解，这些都是正常的现象，无须大惊小怪，也不必对本书很少涉及具体的、系统的专业知识特别是操作使用计算机的技术知识感到困惑，因为本书仅仅是为读者学习基础课程和后续计算机科学与技术课程提供一个导引。书中没有解释的一些名词和术语，读者今后会有机会在一些具体的分支学科的课程中学到，书中的许多观点和思想方法，或许在陪伴读者度过四年的大学生涯中是有裨益的。为了帮助学生虽然是初步但比较准确地了解计算科学，帮助任课教师比较顺利地开展教学和组织讨论，我们根据读者的要求和出版社的安排，专门创作了一本《计算科学导论教学辅导书》，对许多重点内容、背景知识、疑难问题和实例分析进行了进一步的阐述和说明，同时还根据教育与教学改革的 A、B 类模式创作了一本《计算科学实验教程（第 1 分册）》（一年级用），两本书将稍后出版发行，有兴趣的读者可用作参考。

作为一个计算机科学与技术学科专业的学生，不能把自己混同于一个非计算机专业的人员，采取实用主义的态度学习计算机科学与技术知识，满足于能够操作使用计算机系统来处理一些各行各业的简单的问题，满足于掌握一点操作使用计算机或程序设计的小技巧，更不能因其他专业的同学在操作使用计算机方面处于暂时领先的状态而焦虑，而应该以一种科班的态度，对自己提出更高的要求，通过若干年的艰苦学习，系统地掌握本学科最基本的、比较完整的专业知识，使自己拥有比较合理的专业知识结构和科学的思想方法，对许多计算机科学与技术的问题和与之相关的领域知识，不仅要知其然，而且更要知其所以然。

第三、四章的内容是全书的重点。读者应该尽快将全书第一至第四章通读一遍，然后认真跟随老师的讲授学习。对于书本中提到的大量的思想、观点和方式方法，读者可以结合自己的生活经历，理论联系实际，通过自己的反复思考和体会，在学习中加深理解，在理解的基础上指导自己的学习、工作乃至生活实践。

在完成“计算科学导论”课程的学习之后，我们建议每一位读者独立地完成一个读书报告，对自己所学习的本门课程的成效和体会作一个恰当的小结，也便于梳理一下自己对计算机科学与技术学科的初步认识，并设计一个自己未来几年学习的规划，或者对自己未来几年的学习规划进行必要的、合理的调整。对于初学者来说，虽然读书报告的内容暂时还很难有创新，对他人也很难有较高的参考价值，但是，这样一个学习、分析、思考、总结、归纳、形成报告的过程，对于任何一个严肃的、从事科学技术工作的专业技术人员来说，都是一次十分重要的、基本的初始训练，是对个人能力的一种锻炼和考验。因此，每一个读者在起草这样的报告的过程中，应该贯彻独立思考，精益求精的原则，在文章的字里行间中体现出开门见山，言之有物，立论明确，论证严密，结构合理，主题鲜明，表述准确，行文规范的特点，能够写出自己的观点、体会和意见。

如果一个计算机科学与技术专业的学生能够在大学的学习阶段，对重要课程和每一个单元的实验都能在完成后写出有一定质量的读书报告和实验报告，那么，随着时间的推移，当他（她）顺利完成学业时，其在科学论文和科学报告写作方面的能力必将达到相对较高的学术水准，具备较好的科学素养和综合素养，这一点是毋庸置疑的。

第二章 计算科学的基本概念和基本知识

本章主要介绍一些计算科学的基本概念和基本知识，以便帮助读者较为顺利而又自然地学习后面章节的内容。请注意：有些概念我们采用先说明后使用的办法，有些概念或结论我们则采用了先使用或默认，后说明的办法。

2.1 计算模型与二进制

数学不等于计算，但数学确实起源于对计算的研究。在数学发展的初期，针对实际问题研究计算方法和技术是当时的一大特点。之后，从古希腊数学家开始，出现了对计算的根本问题，即可计算性的研究。不过，那时的可计算性研究与本世纪不同，其成果只是关于某些问题类的不可计算性。例如，直角三角形的直角边与斜边之长是不可通约的；一元五次以上方程式不存在通解公式等。这是中学时读者就已经知道的两个基本事实。随着许多计算问题得到解决和不可计算问题类的不断积累，由于问题的种类繁多，互不相同，不仅从认识上难于统一已有的成果，而且也使几百年来几代数学家渴望制造出计算机器的梦想因缺乏科学理论而未能成功。无数次的失败使科学家认识到发展计算模型，研究最一般的问题类的可计算性的重要性。

说到计算模型和数学，总免不了要涉及到形式化与形式化方法的概念。我们说，形式是事物的内容存在的外在方式、形状和结构的总和。所谓形式化是将事物的内容与形式相分离，用事物的某种形式来表示事物。形式化方法是在对事物描述形式化的基础上，通过研究事物的形式变化规律来研究事物变化规律的全体方法的总称。下面的例子可以帮助读者理解。

有两个幼儿园的小朋友，他们天天在一起。开学的第一天，下午放学前，老师给他们每人发了两个梨。两人没有舍得吃，将梨放在一起，数了数一共是四个。第二天，老师给每人发了两个桔子。他们还是不舍得吃，又将桔子放在一起，数了数，也是四个。两人感到奇怪，梨和桔子明明白白是不一样的，怎么放在一起都是四。于是，他们去问老师，得到回答：你们的问题涉及到数学的加法运算。

在数学家看来，两个梨与两个梨合在一起和两个桔子与两个桔子合在一起形式上是一样的，可以抽象地看成是 $2+2=4$ 。一旦不考虑梨、桔子或其他东西的内在区别，而只考虑其外在的某种形式，譬如数量，那么，每个小朋友分到的梨

就是 2，分到的桔子也是 2，合起来都是 4。由此，可以抽象出加法中的一条形式化的运算规则： $2+2=4$ 。

另外，本书的第五章介绍布尔代数的基础知识，读者可以在那里了解和体会到前四章所阐述的许多抽象的概念，如形式化、形式化方法、形式系统、公理、公理化方法、数学系统、模型，等等。

2.1.1 计算模型与图灵机

20 世纪 30 年代是计算模型研究取得突破性进展的时期。由于受到数理逻辑发展中判定问题引起的计算模型研究和构造性数学学派的影响，哥德尔（K. Gödel）、丘奇（A. Church）、图灵（A. M. Turing）、波斯特（E. L. Post）等人在研究中陆续提出了一批计算模型，如递归函数、 λ 演算、图灵机、波斯特系统等，并称这些模型是用算法方法解决问题的极限。即凡是能用算法方法解决的问题，也一定能用这些计算模型解决；反之，这些计算模型解决不了的问题，任何算法也解决不了。进一步的研究发现，这些计算模型之间在计算能力上是等价的。其中，以图灵机的特点和性质更接近普通人计算的思想方法，又因其好用而被现代计算机系统的研究、开发者所采纳。

所谓判定问题是对这样一类问题的总称：这类问题要求解题人对问题所主张的某种断言的真假给出准确的值的判断。例如，“爱尔兰的首都是都柏林”是一个断言，而且我们知道这个断言是真的。又如，“英国的尼斯湖和中国的长白山天池中生活着某种大型水生动物（俗称‘水怪’）”，这这也是一个断言。尽管人们现在还没有弄清楚是否真的存在“水怪”，但这个断言不是真的就是假的，因为是否存在“水怪”在现实世界中是一种客观存在，它不以人们的意志为转移。后面这个例子使我们看到，一个问题是否为判定问题，与这个问题能否判定其真假值没有关系，它们属于不同的概念。对一个判定问题，如果存在一个算法使得无论在什么情况下算法运行终止时都能准确地给出这个判定问题的真假值，那么，这个问题是一个可判定问题。有了这个概念，联想到可计算性的概念，读者可能会问，是否有某种问题是不可判定其真假值的？答案是肯定的。实际上，在这方面的研究中，人们将判定问题分为三类，即可判定问题、半可判定问题和不可判定问题。当然，读者暂时没有必要作深入探究，因为对判定问题的深入研究，涉及到计算机科学的许多方面，没有坚实的基础是很难弄清楚和有所作为的。

所谓计算模型是刻画计算这一概念的一种抽象的形式系统或数学系统，而算法是对计算过程步骤（或状态）的一种刻画，是计算方法的一种能行实现方式。在计算科学中，我们通常所说的计算模型，并不是指在其静态或动态数学描述基础上建立求解某一（类）问题计算方法的数学模型，如梯形面积的计算公式和一

元二次方程根的计算公式等，而是指具有状态转换特征，能够对所处理的对象的数据或信息进行表示、加工、变换、接收、输出的数学机器。

从更一般的角度认识计算模型，我们可以将计算模型分成几个不同的层面，分别确定其指称的对象和内涵：

第一个层面：计算某个（类）具体问题的计算方法，如计算面积的辛普森公式。

第二个层面：能够支持某种语言执行，按照计算方法对应之程序完成一类特定问题计算所需要的平台，如 Pascal 语言和 C 语言程序设计环境，其抽象的形式表示可以是某种数学机器，如图灵机。

第三个层面：一类数学机器或与数学机器这类计算模型在计算能力上具有某种等价性的形式系统或数学系统，能够体现算法计算过程的本质特点和思想，如递归函数、 λ 演算、图灵机、波斯特系统、一阶逻辑系统等。

第四个层面：一切计算模型都应该内含的一种机理。这种内在机理保证了计算模型对一类问题的处理所使用的计算规则、运算操作与该模型的内涵本质属性之间保持一致，反映了计算模型结构与运行机制设计的合理性，逻辑上的可靠性。这种内在机理就是计算模型的模型，如布尔代数系统的模型、一阶逻辑系统的模型，欧氏几何公理系统的模型等。平面几何理论可以看成是欧氏几何公理系统的一个模型。

第二和第三层面之间的分界线似乎不是很清楚，但其实不然，因为第三层面上的计算模型的种类是很多的，这些模型按照计算的能力可以分成许多个类，不同类的计算模型相互之间在计算能力上并不等价。由此可以看出第二层面计算模型和第三层面计算模型之间的分界。对于第四个层面的模型，可能理解起来有困难，但请读者不必急躁，等进入下一节时，结合程序的概念，读者应该能够比较容易地明白这里所讲的大致含义。

历史上，由于观察计算的角度不同，产生了各种不同的计算模型。例如，递归函数是将可以计算的问题用函数来表示，从考察可以计算的函数应具有怎样的构造入手研究计算，而图灵机则是从计算的一般化过程来研究计算。

在递归函数的研究中，从少数几个初始函数出发，通过有穷次地使用函数的代入运算、复合运算以及原始递归等运算，就可以构造出大量复杂的函数。初始函数可以是：

- (1) $s(x)=x+1$ （后继函数）；
- (2) $o(x)=0$ （零函数）；
- (3) $U_j^{(n)}(x_1, x_2, \dots, x_n)=x_j$ （射影函数）。

由于初始函数十分简单, 它们显然都是可以计算的, 因此, 如果能够保证今后使用的函数的代入运算、复合运算以及原始递归等运算都是能行可以计算 (如图灵机可计算) 的, 那么, 我们就可以构造出大量可以计算的函数。对此处可计算的概念, 读者可以理解为用一张纸和一支笔做有限次运算就可以完成的计算。注意: 有限次可以是 0 次或 1 次, 也可以是 100 次、1000 次、10000000 次或 $1000^{3000000}$ 次……运算的次数可以是一个非常大的数字, 反正是有限次就行。换言之, 我们希望一个函数是否可以计算的判定问题, 能够化归为对函数的构造的研究。早期, 关于递归函数理论的研究是先于图灵机建立的, 但后来, 在图灵机出现之后, 关于可计算性理论的研究更多地转向对以图灵机为基础的研究。20 世纪 50 年代, 后人比较完整地建立了图灵机与递归函数之间的关系。

下面, 我们介绍图灵机的基本概念。

图灵机由一条两端可无限延长的带子, 一个读写头和一组控制读写头工作的命令 (控制器) 所组成, 如图 2.1 所示。这条带子上划分了无穷多个可写、可擦的小格 (可以想象成计算机的存储器)。读写头可以沿带子方向左右移动并在带上读写 (可以将读写头与带子的关系理解成录音机中的磁头和磁带的关系)。每个图灵机有一个状态集 Q , 其中包括一个开始状态和一个结束状态; 它还有一个符号集 S , 其中包括一个空白符号, 一般只使用 0 和 1 两个符号, 0 代表空白符。也可以增加一个空白符 b , 与只用两个符号的机器的功能一样。图灵机工作时, 首先从开始状态启动, 每次动作都由控制器根据图灵机所处的当前状态和读写头所对准的符号决定下一步的动作 (或称操作)。其中, 每一步操作包含三件事: 第一, 把某个符号写到读写头当前对准的那个小格内, 取代原来的那个符号 (注意, 这里隐含了可擦、不写的功能); 第二, 读写头向左或向右移动一格, 或者不移动; 第三, 根据控制器的命令用某个状态 (可以是原状态) 取代当前的状态, 使图灵机进入一个新状态。

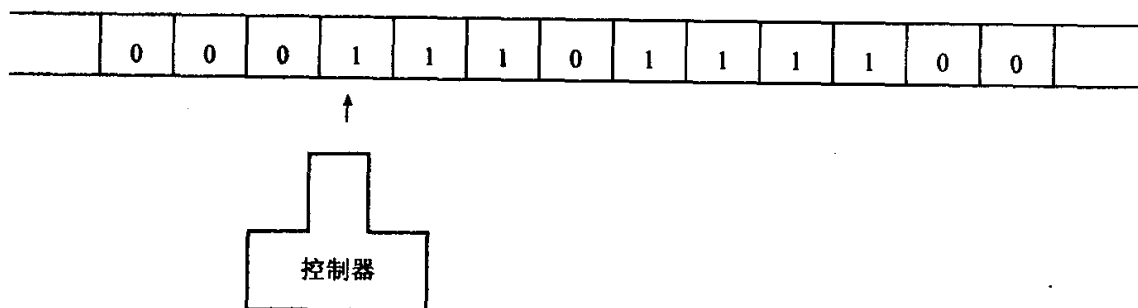


图 2.1

控制器的命令可表示为

(状态, 符号) $\rightarrow$ (写符号, 移动, 状态)

一旦图灵机在运行中进入了一个状态,而且这个状态是一个结束状态,那么,图灵机就停机,计算任务宣告完成,此时带上的内容就是计算的输出结果。显然,当带上的信息是用图灵机的两个符号 0 和 1 表示的数据时,一台图灵机的运行是可以表示机器对带上信息或数据进行处理的过程的。

例 1 图灵机 M 。

M 的字母表 $A=\{0, 1, b\}$, b 表示空格。状态集 $Q=\{q_1, q_2, q_3\}$, 其中, 可以指定 q_1 是开始状态, q_3 是终止状态。 M 的程序(控制器的命令)如下:

$q_1 0 1 R q_1$
 $q_1 1 0 R q_1$
 $q_1 b b R q_2$
 $q_2 b b L q_3$
 $q_2 0 0 H q_1$
 $q_2 1 1 H q_1$

其中, L, R, H 分别代表读写头向左移动一格、向右移动一格、保持不动三个基本操作。

如果此时 M 的输入是 1100b0011, 读写头对准第一个 1, 状态为 q_1 , 那么, 按照我们上面给出的图灵机的非形式的定义, 注意到图灵机的读写带子两头无穷长(隐含了输入的两端可添加任意多个 b), 读者应能自行完成计算, 输出正确的计算结果。

由上面的非形式定义我们已经知道, 图灵机是一种数学自动机器, 控制器的命令相当于程序指令。对图灵机的工作过程从不同的角度考察, 可以给予不同的解释。

第一, 把图灵机看作识别器, 即判断带子上最初的内容能否被图灵机所接受。假定图灵机从左向右扫描完带子上的内容后停机则为接受, 否则为不接受。

例 2 一台图灵机可以设计成识别下面的序列:

1000110, 10011101, 010101011

第二, 把图灵机看作生成器, 对给定的输入集合, 考察输出集合, 并研究输入输出集合性质之间的关系, 这就研究了图灵机的生成能力。

例 3 设一台图灵机的输入集合为 $In=\{1^n 0^n \mid n \in N\}$, 可设计一台图灵机, 对给定的输入集合 In , 得到输出集合 $Out=\{0^n 1^n \mid n \in N\}$ 。其中, N 是全体自然数的集合。

第三, 把图灵机看作计算器, 相当于一个函数。图灵机的输入是函数的自变量的值, 图灵机的输出是函数的值。

例 4 图灵机可以计算下列函数:

$$(1) s(x)=x+1;$$

$$(2) o(x)=0;$$

$$(3) \begin{cases} A(0, y)=y+1, \\ A(x+1, 0)=A(x, 1), \\ A(x+1, y+1)=A(x, A(x+1, y)). \end{cases}$$

第一和第二个函数读者不难从图灵机的定义出发感悟到它们是图灵机可以计算的函数, 而第三个函数就比较复杂, 一时难于判断。顺便提一下, 第三个函数叫做阿克曼函数, 它是阿克曼 (W. Ackermann) 在研究原始递归函数和递归函数的关系时给出的。原始递归函数和递归函数是对一大类可计算函数从函数构造方式的角度对它们进行的分类, 在某种程度上反映了函数结构的复杂程度。阿克曼给出的这个著名的函数是一个递归函数, 但不是原始递归函数。这个函数在计算理论中具有重要价值, 因为它清楚地说明原始递归函数集是递归函数集的一个真子集, 而且, 阿克曼函数在构造性证明递归函数论的一些结论时是一个重要的工具。事实上, 图灵机还可以计算形式上比第三个函数更复杂的函数。当然, 这超出了本书的内容, 它们属于可计算性理论或递归函数论课程所讨论的范畴。

沿着这样一种思路, 图灵机被证明具有很强的计算能力, 它与 20 世纪 30 年代发展的递归函数论 (一种能行可计算性理论) 中一类最一般的可计算函数 (部分递归函数或部分可计算函数) 在计算表达能力上是等价的。然而, 图灵机简洁的构造和运行原理隐含了存储程序的原始思想, 深刻地揭示了现代通用电子数字计算机最核心的内容。尽管在递归函数论的研究中, 通用机定理在揭示存储程序式思想方面也与图灵机有着异曲同工之妙, 但没有图灵机来得明显和易于为一般人所理解。原因是图灵机可以动态地描述计算的整个过程, 与人们日常生活、学习、工作的程序行为方式保持一致, 而递归函数只可能从结构上给出一种静态表示。

20 世纪 40 年代, 在图灵机提出后不到十年, 世界上第一台存储程序式通用电子数字计算机就诞生了。随着 70 年代英国政府对第二次世界大战时期一些档案的解密, 从科学史的研究和另一位对现代电子数字计算机研制作出突出贡献的科学家冯·诺依曼本人的谈话记录都证实了图灵对存储程序式计算机设计思想的杰出贡献。

由于图灵对计算科学的杰出贡献, 美国计算机协会 (ACM) 于 1966 年设立了以图灵名字命名的计算机科学大奖——图灵奖, 以纪念这位杰出的科学巨匠。同时, 在每一年的 ACM 年会上表彰对计算科学作出卓越贡献的科学家并颁发此奖。到目前为止, 已有几十位学者荣获了图灵奖。这些获奖人所做的工作, 极大

地影响了计算科学的发展方向，基本上反映了计算科学在各个时期主流方向上最重要的科研成果。

在图灵研究可计算模型之前，波斯特也曾提出了一种反映计算过程的数学机器，但没有图灵机来得自然，用途也不一样。在图灵工作的影响下，后人针对各种应用背景，提出了许多数学自动机器模型和理论，包括许多图灵机的变形机器，但都没能超出图灵机的计算能力。尽管如此，它们丰富了计算科学的内容，构成了计算科学理论部分最重要的基础。

2.1.2 二进制

也许是图灵机读写带上只出现两个符号启发了研究者，在当时的技术条件下，从便于元器件的设计和制造考虑，计算机的研制很自然地选择了二进制。后来的实践也证明了这种选择具有极大的优点。

我们先来看一下十进制数的表示。例如，1997.630 这个数可以写成：

$$1997.630 = 1 \times 10^3 + 9 \times 10^2 + 9 \times 10^1 + 7 \times 10^0 + 6 \times 10^{-1} + 3 \times 10^{-2} + 0 \times 10^{-3}$$

一般地，任何一个十进制数 S 都可以表示为

$$\begin{aligned} S &= k_n k_{n-1} \cdots k_0 \cdots k_{-m} \\ &= k_n \times 10^n + k_{n-1} \times 10^{n-1} + \cdots + k_0 \times 10^0 + \cdots + k_{-m} \times 10^{-m} \\ &= \sum_{i=n}^{-m} k_i \times 10^i \end{aligned}$$

其中，10 称为十进制的基数； $k_i \in \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ ； m, n 为正整数。小数点的位置不言自明。

二进制和十进制一样，是一种数制，它用于表示数的符号（数字）由于在书写中的位置不同而具有不同的值。二进制表示数的符号只有两个，即 0 和 1，其数值与十进制中的 0 和 1 相同。此外，与十进制不同之处在于二进制数是逢二进一。

例如，十进制数与二进制数中的一些数可做如下对应：

十进制	二进制
(0)	(0)
(1)	(1)
(2)	(10)
(3)	(11)
(4)	(100)
(5)	(101)
⋮	

(9) (1001)

(10) (1010)

⋮

一般地, 任何一个二进制数 S 都可以表示为

$$\begin{aligned} S &= k_n k_{n-1} \cdots k_0 \cdots k_{-m} \\ &= k_n \times 2^n + k_{n-1} \times 2^{n-1} + \cdots + k_0 \times 2^0 + \cdots + k_{-m} \times 2^{-m} \\ &= \sum_{i=n}^{-m} k_i \times 2^i \end{aligned}$$

其中, 2 称为二进制的基数; $k_i \in \{0, 1\}$; m, n 为正整数。

进一步, 读者可从十进制数和二进制数的一般表示公式得到启发, 将这种表示推广到更一般的任意进制, 不同之处只是基数不一样。

有了二进制数的表示, 就可以发展二进制数的算术运算。不过对于二进制的运算和二进制数与十进制数之间的转换, 这里不作详细介绍, 读者将来在“数字逻辑”或“计算机组成原理”的课程中会得到系统的训练, 我们在此仅就二进制的特点进行讨论。

很多人都会提出这样一个问题: 计算机的设计为什么要采用二进制呢? 要回答这个问题, 还得从二进制的特点出发来说明。

首先, 二进制只取两个数码 0 和 1, 因此, 二进制数的每一位都可以用任何具有两个不同的稳定状态的元器件来表示, 而十进制数用元器件表示时就需要更多不同的稳定状态。显然, 制造具有两个稳定状态的元器件比制造具有多个稳定状态的元器件要容易得多, 这是二进制数应用于计算机或其他数字系统的一个优点。

其次, 二进制数的运算规则简单。我们知道, 在进行算术运算时, 必须记住体现运算规则的整数加法和乘法的运算表。对十进制数的运算, 人们必须记住 $10(10+1)/2$ 个和运算规则与 $10(10+1)/2$ 个积运算规则, 才能进行计算。如果是二进制, 则只须记住 $2(2+1)/2$ 个和运算规则与 $2(2+1)/2$ 个积运算规则。如二进制加法表为

$$0+0=0$$

$$1+0=0+1=1$$

$$1+1=10$$

乘法表为

$$0 \times 0 = 0$$

$$1 \times 0 = 0 \times 1 = 0$$

$$1 \times 1 = 1$$

而十进制的加法表为

$$0+0=0$$

$$0+1=1+0=1$$

$$0+2=2+0=2$$

⋮

$$0+9=9+0=9$$

⋮

$$1+1=2$$

$$1+2=2+1=3$$

⋮

不难想象，二进制数的运算要比十进制来得简单。

第三，计算机的理论基础是逻辑和代数。当二进制与同样只使用“真”和“假”两个值的逻辑代数建立了联系后，这就为计算机的逻辑设计提供了便利的工具。

当然，二进制也有它的缺点，一方面书写冗长，不易读懂，另一方面，人们习惯上仍然使用十进制，在使用计算机进行计算时，输入的十进制数还需要进行转换。不过，一旦建立了转换规则和相应的算法，一切可以让计算机机械地去做，无非是多花费一些时间。

现在，人们早已对二进制与十进制数之间的互换给出了通式，很容易通过反映这种数与数之间互换的程序或电路来自动进行。

用二进制方式制造的存储程序式计算机是不认识英文字母、数字和中文汉字的。要对这些英文字母、数字和中文汉字等信息进行处理，首先必须对它们进行编码。由于美国是早期发展计算机系统的国家之一，美国国家标准局最先公布了美国信息交换标准码（ASCII），简称 ASCII 码。ASCII 码规定了计算机上键盘所能打出的 127 个不同字符的内部二进制信息表示，现已被大多数国家所采纳。除了 ASCII 码以外，还有其他的编码，如 EBCDIC，EBCDIK 码等。

与西文不同的是，中文汉字的编码要困难得多。虽然中国在 1980 年公布了常用汉字内部信息交换码的国家标准，确定了汉字无论以何种编码方式输入计算机，都应转换成标准的内部信息交换码表示。但是，如何在计算机键盘上利用几十个键以平均较短的编码区分几千个常用汉字，曾进行了多年的汉字编码研究。现在，已经有了若干种比较成熟的汉字编码方案并在计算机界广泛使用，如汉字拼音码，五笔字型码，等等。

2.2 存储程序式计算机的基本结构与工作原理

图灵机的出现为现代计算机的发明提供了重要的思想。图灵机的带子可以看成是计算机的存储设备，数据可以存储在上面，也可以根据需要擦去。图灵机的命令相当于一组事先设计、存储好了的程序，它们在控制器安排下，决定读写头的每一步操作。这样一种数学机器，如果不考虑它的实用性，就思想和原理而言，确实包含了存储程序的重要思想。

这里，我们引用了程序的概念。但什么是程序呢？也许有的读者从未接触过计算机程序的概念。其实，对程序这一概念大家并不陌生。一个智力正常的人，或者是一个“有头脑的人”，每天都在编制和使用程序。我们每个人经常在规划着自己今后一段时间内的活动，事先想好应该先做什么事，后做什么事，遇到什么情况作怎样的处理，等等。其实这就是程序。我们每天早上醒来，一般都按照预先想好储存在脑子里的某个程序行事，按部就班地去做每一件事，这样实际上就等于由人执行了程序。类似地，通俗地讲，计算机程序是指能够在计算机系统中运行的程序。由于计算机程序的出现，随着科研工作的开展和计算机在各行各业应用的推广，为了提高效率和可靠性，围绕程序的设计、描述、构造、分析、测试和验证等方面，发展了许多技术，它们被统称为程序技术。

联系上面一节介绍的计算模型的概念，我们不难理解，人实际上是一个能支持自己程序行为的计算模型。现在我们再来设法解释前面关于第四层面模型的概念。在一个特定的社会里，我们说某个人是一个 100%的健康人¹⁾或理性的正常人，实际上是说这个人的思想和行为与这个社会所倡导的一个健康人或理性的人所应有的思想方法和行为准则完全一致，即保持公设条件下和论域范围内人的思想与行为的逻辑一致性。一个人为了做到这一点，必须要在自己的头脑中建立一系列的思想方法与行为准则，以及运用这套思想方法与行为准则的一个程序，帮助他在任何情况下处理面对的各种各样的情况和问题。如果这个人的思想方法与行为准则以及那个程序真能够保证他今后所有的思想和行为都符合这个社会对健康人或理性的人的标准，即与社会关于健康人与理性的人的思想方法与行为准则保持一致，那么，他的思想方法与行为准则以及那个程序就是健康人与理性的人思想与行为的一个模型。显然，只要任何一个人胸中装着这个模型并按照这个模型行事，那么，他必然能够成为这个社会所希望的健康人或理性的人。但如果这个人的思想方法与行为准则以及那个程序不能使他的思想与行为同社会关于

1) 此处健康人不是指医学意义上的健康人。

健康人或理性的人的思想方法与行为准则保持一致，那么，他的思想方法或者行为准则或者那个程序是有缺陷的，因为其行为缺乏逻辑上的合理性。比如，这个人的思想方法中包含机会主义的色彩，看到小利不为所动，能够做到拾金不昧，但面对巨额财富的吸引，一旦有可能非法占为己有而自知不会被人发现，就会铤而走险，说明这个人的思想有问题，因为他与我们社会倡导的社会公德（一种行为准则）不一致。还有，如果那个程序是一个随机的程序，面对各种情况，他的思想和行为让人完全摸不着头脑，毫无规律可循，自然人们会认为这个人的神经有问题，他是一个精神病人。上述两种情况都反映了这个模型存在缺陷。注意，如果没有这个程序，这个模型只具有认知健康人或好人的能力，但不具有成为一个健康人或好人的行为能力，即该模型只是一个认知模型或说明性模型，但不是一个操作模型或过程性模型。掌握模型的概念对于未来了解理论计算机科学和研究计算的正确性问题具有特别重要的意义。

图灵机诞生后不到十年，在以冯·诺依曼为代表的一批科学家的努力下，现代存储程序式电子数字计算机的基本结构与工作原理被确定下来。其工作原理可以概括为：“存储程序，顺序控制。”它主要由五部分组成，即存储器、运算器、控制器、输入设备和输出设备，如图 2.2 所示。

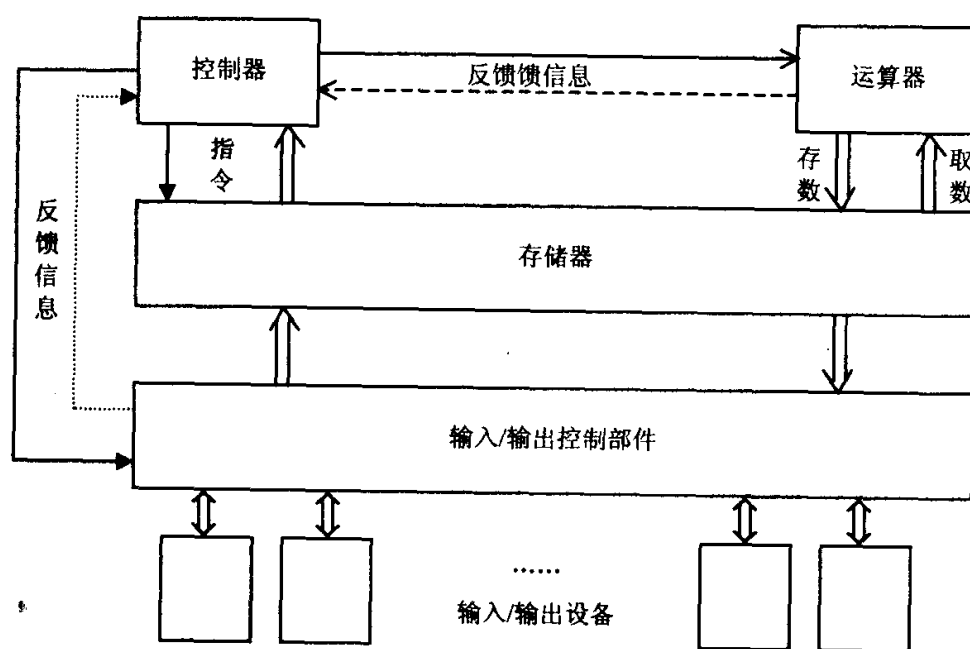


图 2.2 电子数字计算机结构的简单框图

(1) 输入设备：输入设备是人或外部与计算机进行交互的一种部件，用于数据的输入。常见的有键盘、鼠标、光电输入机、磁记录设备、录像机、扫描仪、数字化仪、数码照相机、数字摄像机等。目前，无论哪一种输入设备都以二进制数或代码的形式向计算机输入信息。

(2) 存储器：存储器是一种数据或信息的存储部件，相当于计算机的仓库。它分成很多存储单元，按照一定的方式排列。每个单元都编了号，称为存储地址。原始数据和中间计算结果都可以按地址存放在存储器中。现今，存储器多使用半导体集成电路器件。存储器一般分成内存储器和外存储器两大类。内存储器存取数据很快，外存储器存取数据相对慢得多。存储在内存中的数据只有在机器通电的情况下才能保存，一旦机器被关闭，数据立即消失。但是，外存储器则不然，即使机器被关闭，数据也照样保存在其中。原因是两者所用的器件的材料和工作原理不同。

也有一种描述是将主要部件分成六部分，即把内存储器和外存储器看作两部分。内存储器可对数据进行随机的存储或访问，也叫作主存，外存储器不能对数据进行随机访问，只能将后备的数据和信息调入主存后才能进行随机访问。由于调入数据和信息的方式是分组进行的，常以文件为单位调入调出主存，所以，外存储器有时也称为文件存储器。

(3) 运算器：运算器是计算机对各种数据或信息进行算术运算和逻辑运算的主要部件，由很多逻辑电路组成。它们包括寄存器、加法器、乘法器、移位器和一些控制电路等。

(4) 输出设备：输出设备是人与计算机交互的一种部件，用于数据的输出。它把各种计算结果数据或信息以数字、字符、图形、图像、声音等形式表示出来。常见的有打印机、显示器、绘图仪、磁记录设备、平面绘图机、光盘刻录机等。

(5) 控制器：控制器是计算机的中枢，它控制整个计算机自动地工作。控制器由时序电路和逻辑电路组成，它通过输出电压和脉冲信号来实现对计算机的控制。

习惯上，人们把运算器和控制器看成一个整体，称为中央处理机或中央处理器（CPU），而把输入设备和输出设备以及外存储器称为外部设备。在超大规模集成电路技术日益发展和成熟的今天，将运算器和控制器做成一个集成块电路已是平常事。

当计算机工作时，CPU 每次从内存储器中“依次”读取程序的一条指令（相当于一个动作的命令），存入指令寄存器（一个临时存放等待执行的指令的器件或部件），然后，根据需要从内存储器或寄存器中读取数据，并通过与该指令对应的逻辑与控制电路或加法器等部件执行这条指令。之后，将结果存入寄存器或内存储器，或者根据指令的要求作跳转和其他工作，如通过发送和接受信号进而控制输入和输出设备的操作。当 CPU 遇到停机指令或执行某条指令出错时，计算机将出现停机。

计算机（裸机）系统制造出来以后，人们在应用中不断地发现和感到计算机

系统的使用与运行存在着机器与人之间的沟坎,在思维方式、语言、操作习惯等方面很不自然。要充分地发挥计算机的各项功能,使用户能够更自然地、更方便地使用计算机系统,需要程序员和管理者对机器的资源、功能和原理有更多、更深入的了解,无疑这对用户是一个负担。为了从人机工程学原理的角度改善用户与机器之间的关系,充分发挥计算机系统的各项性能,人们从汇编程序开始,围绕合理分配、调度和管理系统与用户的各种资源、信息与数据,平滑用户与机器之间的沟坎,提高计算机系统资源的使用效率,发展了操作系统、高级语言编译系统、数据库系统、各种面向不同计算机用户的管理工具软件,包括系统管理软件和应用程序库中的各种软件。并在此基础上形成了不同层次、不同类别的计算机系统环境。

在学科的发展历程中,习惯上,人们将不带有软件系统的存储程序式电子数字计算机系统称为硬件裸机,将硬件裸机,参与构成硬件裸机的各类部件及其研究范畴统称为硬件(研究方向)。将包含计算机裸机系统、系统软件、应用软件,形成计算机系统环境的软硬件系统称为计算机系统。这个系统指称的是硬件和软件的综合体。

2.3 数字逻辑与集成电路

数字逻辑是数字电路逻辑设计的简称,其内容是应用数字电路进行数字系统逻辑设计。电子数字计算机是由具有各种逻辑功能的逻辑部件组成的,这些逻辑部件按其结构可分为组合逻辑电路和时序逻辑电路。组合逻辑电路是由与门、或门和非门等门电路组合形成的逻辑电路;时序逻辑电路是由触发器和门电路组成的具有记忆能力的逻辑电路。有了组合逻辑电路和时序逻辑电路,再进行合理的设计和安排,就可以表示和实现布尔代数的基本运算。而布尔代数只使用0(假)和1(真)两个数,这样,当二进制的加法、乘法等运算与布尔代数的运算建立了对应关系后,就可以用逻辑部件来实现二进制数据的加法、乘法等各种运算。

布尔代数本身是一种代数系统,但更是一种逻辑系统。布尔当初采用代数运算的方法发展的这个形式系统所要表达的首先不是代数系统而是逻辑系统。有了布尔代数,当人们用电路表示二进制的许多运算时,其实很多运算的组合反映的是代数运算或逻辑运算,指称的是数字电路的某种组合,事实真是如此!可见,布尔代数在逻辑、代数和电路三者的相互之间起到了桥梁作用。于是,当人们将电路与布尔代数建立联系之后,从计算模型到技术支持,存储程序式通用电子数字计算机的设计与制造可谓真正建立在了数学基础之上,建立在了逻辑与代数的基础之上,特别是数理逻辑的基础之上。关于这方面的内容,以后随着对后续课

程学习的深入,读者会有更多的了解和感悟。

下面是基本的门电路的例子。

例5 基本的“与”、“或”、“非”门电路。

“与”门电路一般有两个以上的输入和一个输出。图 2.3 (a) 表示了一个“与”门电路,其输出 P 和输入 A, B, C 之间的逻辑关系可用下面的式子表示:

$$P = A \cdot B \cdot C$$

在具体的电路设计中,如果我们用高电平信号表示 1,低电平信号表示 0,那么,“与”门电路只有当输入 A, B, C 同时为 1 时,输出 P 才为 1,否则, P 为 0。

“或”门电路可以用图 2.3 (b) 表示。一般地说,“或”门电路是一种具有逻辑加法功能的电路,它有两个以上的输入和一个输出,其输出 P 和输入 A, B, C 之间的逻辑关系可用下面的式子表示:

$$P = A + B + C$$

在具体的电路设计中,如果我们用高电平信号表示 1,低电平信号表示 0,那么,“或”门电路仅当输入 A, B, C 中有一个为 1 时,输出 P 就为 1,否则, P 为 0。

“非”门电路可以用图 2.3 (c) 表示。一般地说,“非”门电路是一种具有逻辑取反功能的电路,它只有一个输入和一个输出,其输出 P 和输入 A 之间的逻辑关系可用下面的式子表示:

$$P = \sim A$$

在具体的电路设计中,如果我们用高电平信号表示 1,低电平信号表示 0,那么,“非”门电路当输入 A 为 0 时,输出 P 就为 1,否则,当输入 A 为 1 时,输出 P 为 0。

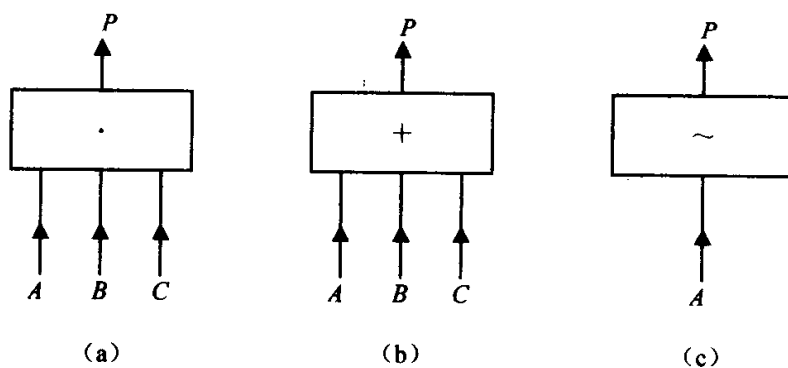


图 2.3

将布尔代数(见第五章)和前面谈到的二进制联系起来,我们可以看出,“与”、“或”、“非”门电路的作用与集合运算“交”、“并”、“补”是一致的。一旦门电路中仅使用两个电平信号 0 和 1,引入二进制及其运算规则,那么,用门电路及其

组合就可以实现二进制的各种运算，如二进制的加法、乘法等，而对复杂电路的计算，如电路的化简就有可能通过布尔代数的方法进行。事实上也正是如此。

利用“与”、“或”、“非”门电路进行有机的组合，可以设计出很复杂的数字逻辑电路。这些电路可以是执行算术运算的加法器，也可以是执行控制任务的触发器，还可以是用于信号加密和解密的通信密码电路。各种单一功能的数字逻辑电路再进行合理的、有机的结合，就有可能形成具有多种复杂配套功能的数字系统。目前，电子数字计算机的设计主要采用的是这种电子技术。

一台计算机是复杂的，因为它涉及到很多功能部件。众所周知，一台计算机有很多指令，这些指令的执行是通过电路来实现的，这就涉及到用数字电路来进行数字系统的逻辑设计。早期，数字系统的设计是用分离元件的某种组合来实现的，其结果是机器的体积很大，速度不快。采用硅片作主要材料将一组数字逻辑电路做在硅片上的半导体集成电路技术是一大发明。随着微电子技术的迅速发展和微处理器的广泛应用，数字系统的设计从应用小规模集成电路进行组合发展到直接用大规模集成电路和超大规模集成电路来实现。其结果是计算机体积愈来愈小，速度越来越快。

在这里，我们谈到了用电路来实现简单的数学运算。从图灵机与计算理论可知，实际上，一种抽象的计算机只需要很少几种基本运算指令就可以有强大的计算能力，其他一些指令只是从使用方便的角度提出来的，它们并不能增强机器的计算能力。至于这种计算能力用什么技术来表现，则完全取决于当时社会的工业技术发展水平，取决于成本和效率这两个基本因素，对用户来说也就是大家购买商品时常常考虑的性能价格比。例如，存储程序式计算机可以用机械技术制造，也可以用电子技术制造，将来甚至可能使用其他新技术制造。由于电子技术在成本、效率（速度和精度）上远远超过了机械装置，因此，在发展中，机械式计算机必然会被电子计算机所取代。又如，一个计算过程，既可以用高级语言描述的程序来实现，也可以用电路来实现。这就是说，电子技术和程序技术只是计算科学的两种基本的技术形式。

由此可见，真正构成计算科学基本的、核心的内容是围绕计算而展开的大量带有规律性的知识，而不是具体的实现技术。因为，在将来，我们完全可能发展一种能表示二进制数及其运算的新技术，它比今天的微电子技术精度更高，生产价格更便宜。如果真是那样，这种技术就可能取代微电子技术 in 计算科学中的地位。近年来科学研究的发展已显现出一些很有希望但尚不成熟的技术，如光电子技术，金属表面处理技术等。当然，程序技术在可预见的将来尚不太可能被别的技术所代替，原因是它与各种应用相联系，而且在形式上易于反映能行性这一本质的计算特征，在表达形式上与通常算法的描述较为接近，设计和生产的成本也比

较低, 又不存在工业污染的问题, 所以不易被取代。因此, 我们常说, 从这个意义上讲, 软件技术比微电子技术对计算科学更重要一些。

现在我们已经知道了数字电路逻辑系统可以用于电子数字计算机的部件设计和制造。事实上, 数字逻辑的用途不只是这一方面, 许多其他的数字电路也可以使用数字逻辑技术。例如, 录音机、录像机、摄像机、照相机、洗衣机、对讲机、手机、电视机、CD、VCD、DVD 等产品在设计中都采用了数字逻辑技术, 在元器件的制造中采用了集成电路技术。

2.4 机器指令与汇编语言

我们已经知道, 为了用计算机求解一个问题, 必须事先编制好程序。程序是由一系列的指令组成的。每一台电子数字计算机中都设计规定了一组指令集合, 称为机器指令系统。机器指令的格式一般分为两部分, 如下所示:

指令格式:

操作码	地址部分
-----	------

其中, 操作码指出运算的种类, 如+, -, $\times$, $\div$, 跳转等, 地址部分用来指示参与运算的数据保存在什么地方, 如存储器的某个地址或某个寄存器等。操作码和地址部分都用二进制代码表示。

机器指令一般可根据其功能划分为以下几类:

- (1) 控制指令;
- (2) 算术运算指令;
- (3) 逻辑运算指令;
- (4) 移位操作指令;
- (5) 传送操作指令;
- (6) 输入/输出指令。

应当注意的是, 不同的机器, 其指令系统是不同的。指令系统的设计, 不仅与具体应用领域有关, 而且还与用户对机器的性能要求有关。一般地说, 就通用计算机系统而言, 基本的各种运算指令每一台计算机系统都有, 由这些基本指令还可以通过微程序技术实现其他指令的功能。计算机系统的指令, 一般是采用数字逻辑电路或微程序设计技术实现的。所谓微程序是一种用微指令编制的程序。将一个指令的执行分成若干步, 每一步对应一条微指令, 于是, 可以采用微指令编制微程序的方式来实现指令的功能。微指令的实现是用与微指令对应的数字逻辑电路的方式实现的, 该指令具有自己的地址和寻址功能, 可用来设计并对应一个程序, 存储在只读存储器中, 其执行实现了指令的功能或一个部件的功能, 信

号在该微程序电路上的经过反映了该指令或部件对数据或控制信息的处理过程。随着大规模集成电路的发展和基于用户对运算速度与使用方便方面的考虑,大多数现代计算机都设计了比较庞大的指令系统。

在基本指令系统的基础上扩展指令的种类和数量并不能从根本上提高计算机系统的计算能力,因为一台计算机系统的计算能力是由基本指令系统决定的,理论上,它与图灵机的计算能力是等价的。这说明,只需要少数几条指令就可以决定计算机系统的计算能力,其他指令只是从方便用户使用计算机系统进行程序设计的角度考虑设计的。当然,涉及到从计算机存储器单元和寄存器中存取数据的附加操作指令对实用化的计算机系统还是需要的,也应该归入基本指令集。1964年,中国科学院软件研究所¹⁾的唐稚松教授在《数学学报》发表了“论指令系统的递归性”的论文,从数理逻辑的角度指出了计算机基本指令系统集决定了计算机系统的计算能力,并且,任何一个程序原则上只需要用反映程序基本结构的三条基本指令(不考虑存取数据的附加操作指令)就可以写出来,转向语句不是必需的。这一结果比国外雅可比(G. Jacobi)等人关于程序三种基本控制结构的类似的工作大约早了两年。

指令系统的日渐增大虽然给用户的程序设计带来了方便,但也带来了硬件设计复杂性的增加以及因译码、存储、寻址等开销的增大而使运算速度下降。事实上,研究人员经过对大量运行的程序进行认真的分析,发现一个指令系统中大约五分之一的指令是在程序中经常反复使用的,其代码量大约占整个程序的五分之四;而该指令系统中剩下的大约五分之四的指令是很少使用的,其代码量只占到整个程序的五分之一。这一发现使人们认识到计算机体系结构发展的潜力。

真正使研究人员对指令系统进行较大改进的原因是超大规模集成电路(VLSI)技术的发展和采用微程序技术实现体系结构设计思想的过程中硬件复杂性的不断增长带来的技术上的困难,使人们开始认识到在进行计算机系统设计时,应公平对待硬件与软件的地位,使两者平衡负担整个系统的复杂性。这一认识的转变直接导致了精简指令系统计算机(RISC)的诞生。所谓RISC是根据指令系统中各种指令应用的规律,将最常用的指令,以及一些控制较为简单的寄存器—寄存器的操作与寄存器等一起直接做在VLSI芯片上,靠减少译码、存储、寻址方式和次数等开销而大大增加运算速度。实际上,RISC主要是一种体系结构设计思想,而不只是一种计算机产品。RISC技术由于极大地提高了计算机的运算速度,因而它的问世被认为是计算机体系结构发展史上的一个里程碑。

计算机系统的机器指令构成一个集合,它不是一种高级语言,而是一种使用

1) 当时该所尚未从中国科学院计算技术研究所分出来。

微程序设计技术实现的指令级语言。所以称它是指令级语言而且不是一种高级语言，原因是一方面该语言只有计算机（裸机）系统能够理解和执行，更重要的是另一方面，该语言的每一个句子都只相当于一个单词和短句，缺乏表达复杂的长句子的语法结构和能力，可读性很差，在表达和理解语言的思维结构方面与人类普遍使用的自然语言之间存在较大的沟坎。在计算机科学发展的早期，人们最初使用机器指令编制程序。然而，由于用以二进制形式表示的机器指令书写的程序很难阅读和理解，而且程序员还必须详细了解机器的许多细节。如早期计算机的使用中，程序员要记住计算机的几个寄存器中哪一个具有累加功能，哪一个是指令寄存器，哪一个地址寄存器，等等。因此，研究工作中很快就发明和产生了对比易于阅读理解的汇编语言。汇编语言实际上是由一组汇编指令构成的语言，每一条汇编指令用该指令西文名称的某个缩写的字符串来表示其所代表的操作，用符号来代表数据的二进制、八进制和十进制数字序列。大多数情况下，一条汇编指令对应一条机器指令，少数对应几条机器指令。例如，下面是几条汇编指令的操作符，右边中文是名称。

add	加法	idiv	有符号除法
mul	无符号乘法	neg	求补
xchg	交换	test	逻辑比较
jmp	无条件转移		

有了汇编语言，就得编写和设计汇编语言翻译程序（简称汇编程序），专门负责把使用汇编语言书写的程序翻译成可直接执行的机器指令程序。因为计算机（裸机）只能识别机器指令而不认识汇编指令，汇编指令写的程序不能在计算机（裸机）系统上直接执行，需要汇编程序做上述翻译。一般说来，汇编程序被看成是系统软件的一部分。

当然，汇编语言在可读性和编写程序时仍然是不能令人满意的，这导致进一步发展了高级程序设计语言。不过，由于高级语言在使用时通常还是要通过编译程序逐步将高级语言写的程序翻译成机器指令的程序，而这种翻译的结果往往不如机器指令或汇编语言写的程序效率高，所以，直到今天，不少工程师在系统软件的开发中还在使用机器指令和汇编语言。

2.5 算法、过程与程序

求解一个给定的可计算或可解的问题，不同的人常常会编写出不同的然而都是正确的程序。这是为什么呢？

经过分析, 科学家发现这里存在两个层面的问题, 一个是与计算方法密切相关的算法问题, 另一个是程序设计的技术问题。我们先举一个例子。

例 6 给定两个整数, 求它们的最大公因数。形式上, 可以描述如下:

不失一般性, 设有不全为 0 的整数 x, y , 记 $\gcd(x, y)$ 为它们的最大公因数, 即同时能整除 x, y 的公因数中的最大者。显然, $\gcd(x, y)$ 可表示为

$$\gcd(x, y) = \max\{z \mid z \mid x, z \mid y\}$$

这个问题许多中学生都会解, 最常见的有著名的欧几里德辗转相除计算方法。当然, 还有许多种解法。我们考虑从函数 $\gcd(x, y)$ 的性质出发来求解。函数 $\gcd(x, y)$ 具有如下性质:

(1) $\gcd(a, b) = \gcd(b, a)$;

(2) $\gcd(a, b) = \gcd(-a, b)$;

(3) $\gcd(a, 0) = |a|$;

(4) $\gcd(a, b) = \gcd(b, a \bmod b)$, $0 \leq a \bmod b < b$ (欧几里德辗转相除计算方法)。

根据以上性质, 我们可以设计如下算法:

算法 A (计算函数 $\gcd(x, y)$)

A₁. 输入 x, y ; z 是辅助变量;

A₂. 重复执行如下操作步骤:

(1) 若 $y=0$, 则输出 $|x|$, 算法停止;

(2) 若 $y \neq 0$, 则 $z \leftarrow x \bmod y$, $x \leftarrow y$, $y \leftarrow z$ 。

有了计算函数 $\gcd(x, y)$ 的算法, 用程序设计语言很容易写出可在计算机系统上执行的程序。算法 A 的核心是利用了函数 $\gcd(x, y)$ 的上述 (4) 的性质。倘若我们使用的不是 (4) 的性质, 那么, 算法就会发生改变。

不难想象, 不同的求解方法将产生出不同的算法, 不同的算法将使我们设计出不同的程序, 而决定这个程序功能的本质是计算方法及其算法。一般地说, 对不同计算方法过程的抽象描述就产生了相应的不同算法, 但同一算法由不同的人来写程序则完全可能设计出差别很大的程序。

对欧几里德计算方法, 高中毕业生都能很快理解, 因为上面这种计算方法的算法与程序之间在形式上几乎一致。但是, 如果采用其他程序设计技术, 如程序推导技术或程序变换技术, 算法与程序之间的差别就可以变得很大, 甚至简单的几行程序一时都会很难看明白它的功能。

例 7 用程序变换技术设计的计算函数 $\gcd(x, y)$ 的程序。

利用一种叫做程序变换技术的程序设计方法, 可以产生计算函数 $\gcd(x, y)$ 的

如下递归过程性程序:

```
Procedure gcd(x, y:int, var z:int);  
  begin if x=0 then z:=y;  
        x≤y & x≠0 then gcd(x, y-x, z);  
        x>y & x≠0 then gcd(y, x, z);  
      endif  
  end;
```

很明显, 例 7 的这个程序要一眼看出其功能是不容易的。实际上, 例 7 的程序是反映辗转相除计算方法的。但是, 经过程序变换, 形式上已发生了很大变化。

这两个例子进一步引发我们的一些思考: 如何确保算法和程序的正确性? 既然求解同一个问题可以有不同的方法, 不同的算法, 不同的程序, 那么, 怎么来判断算法和程序的好坏呢? 如果我们依据一个算法设计了一个计算某一问题的程序, 然后对其进行程序变换。假如经过程序变换得到的程序依然是计算这个问题, 那么, 这个新程序所对应的算法和原来的算法一样吗? 这就是说, 程序变换是否改变算法呢? 等等。

这里我们不要求大家对求最大公因数问题作深入的探讨, 也不要求大家马上都能回答这些问题, 这些内容将来在“算法设计与分析”和“程序设计方法学”等课程中会涉及, 而只是想提醒读者注意, 上述这两个例子或许已使大家隐隐约约感觉到算法、程序与数学之间存在着密切的联系。要解决上面的问题, 没有数学和计算科学理论的支持恐怕不行。多年来大学计算科学本科教学的实践已反复证明, 实际情况正是如此。

在计算科学研究中, 事实上存在一条定律: 一个问题, 当它的描述及其求解方法或求解过程可以用构造性数学描述, 而且该问题所涉及的论域为有穷, 或虽为无穷但存在有穷表示时, 那么, 这个问题一定能用计算机来求解; 反过来, 凡是能用计算机求解的问题, 也一定能对该问题的求解过程数学化, 而且这种数学化是构造性的。这是由于构造性数学的构造性特征保证了计算方法的能行性, 两者之间是相容的。在计算科学的发展中, 方法同算法始终是紧密联系在一起, 有时候计算方法就是算法, 有时候计算方法离算法还有一定的距离, 如某一方法仅仅只能表示为一个过程而不能表示成算法, 或该方法只说明了解应该满足的性质或条件而没有说明如何一步一步计算得到这个解。

当一个问题的求解获得了计算方法和算法时, 并不等于万事大吉了。在许多情况下, 找到求解一个问题的算法只是走完了第一步。至于现实是否可以计算, 则取决于算法的存在性和计算的复杂性, 即取决于该问题是否存在求解算法, 算法所需要的时间和空间在数量级上能否被接受。要注意的是, 有的问题虽然存在

求解问题的计算方法,但是,不存在算法。原因有两种可能:一是计算方法可能不是构造性的;二是虽为构造性的,但计算方法不能保证计算过程在任何初值的情况下都能结束。

例 8 设一元函数 $f(x)$ 在闭区间 $[a, b]$ 上连续, $f(a) < 0$ 且 $f(b) > 0$, 求 $f(x)$ 在闭区间 $[a, b]$ 上的精确零点。

根据闭区间上连续函数的性质,我们知道, $f(x)$ 在闭区间 $[a, b]$ 上一定存在零点 c , 满足 $f(c) = 0$ 。现在的问题是,假设只有一个零点而且不考虑数的分数表示,那么,无论我们采用什么方法设计算法,如果这个零点是一个无理数的话,算法是不可能终止的,因为算法永远无法获得一个精确的零点值 c 。要使算法终止,只能降低要求,允许计算结果是函数 $f(x)$ 的一个近似的零点值。

类似于上面问题的计算,任何算法都面临着某种付出。即使我们有足够的金钱,我们也必须考虑计算所需要耗费的机器时间和占用的机器存储空间。这就涉及到算法性能的分析,具体地说涉及到算法复杂性的分析。

什么是算法的复杂性呢?我们知道,使用计算机计算各种问题,需要一定的存储空间存放我们的程序和数据。即使我们使用的计算机具有巨大的存储空间,也必须付出为等待计算机系统完成计算任务所需要的时间。依据不同的算法写成的程序拿到计算机系统上去计算时,计算所需要的时间和空间是不同的。所谓算法的复杂性就是对算法计算所需要的时间和空间的一种度量。注意,我们不能将这种度量简单地理解成一个算法实际计算所需要的时间和空间。由于不同的计算机系统千差万别,运算速度可以相差很大,因此,不可能用算法在某一台计算机系统上计算所需要的实际的时间和存储单元(空间)去衡量这个算法的复杂性。

那么,怎样才能准确刻画算法的计算复杂性呢?实际工作中,科学家采用了与问题规模大小相联系的一个近似函数,一个叫做渐近增长率的概念来刻画算法的计算复杂性。该函数表示了算法在求解一类问题的过程中随问题规模大小变化引起的算法中抽象的基本运算(即可在某一抽象的计算模型上执行的基本运算操作)执行的次数或存储空间量的变化规律。如某个计算问题,当问题的输入规模分别为 $1, 2, 3, \dots, n$ 时,经分析算法中抽象的基本运算次数分别为 $1, 4, 9, \dots, n^2$, 那么,我们就可以用函数 n^2 来刻画这个算法的计算时间复杂性,并称这个算法的复杂性度量为 $O(n^2)$ 级。空间复杂性的表达也是类似的。有了复杂性的度量函数,一旦选定计算所用的具体计算机,人们就可以根据这台计算机对某个 n 值的实际运算速度比较准确地估算出对其他的 n 值完成计算所需要的时间。于是,对各种算法的复杂性进行分析的关键是要设法去找出这样的相近函数。然而,在大多数情况下,寻找这样的函数常常不是像刚才例子的情形那么简单,它涉及到与数学密切相关的算法的设计原理、思想和方法。因此,算法的复杂性分析就常

常成为具有智力挑战性的研究课题。

下面举一个计算第 n 个素数的问题。这是一个有趣而又带挑战性的问题。这个问题的求解有许多种方法, 最简单的算法设计思想是从素数的定义出发, 对自然数从小到大一个一个地去试。显然, 采用这个算法, 当 n 很大时, 算法所需要的计算时间是我们难以承受的。因此, 必须采用在对素数规律研究的基础上, 借助一些素数的定理或性质进行计算, 以便为计算寻找更为有效的算法。例如, 我们都知道, 素数的一个基本性质是除 2 以外, 所有的素数都是奇数。依靠这个性质, 显然在算法中可以不考虑偶数情况的判别。

例 9 用筛法求第 n 个素数。

在数论的研究中, 对素数的计算有一个著名的方法, 叫做筛法。现在, 如果我们对任意给定的正整数 N , 能够用筛法计算得出 $2, 3, 4, \dots, N$ 中的所有素数, 那么, 求出第 n 个素数也就不困难了。

筛法计算素数的基本思想是: 对给定的一个自然数 N , 我们可以将 $2, 3, 4, \dots, N$ 这 $N-1$ 个数组成一个集合, 然后依次从这个集合中筛掉 2 的倍数, 3 的倍数, 5 的倍数, $\dots$, 直到某个素数的平方大于 N , 此时, 集合中剩下的全是素数, 而且, 剩下的素数可按从小到大排定序号。由于已知 2 是素数, 于是, 可以在计算中设定两个变量 M 和 N , M 存放当前已经计算得到的最大的素数, 同时使 N 从 3 开始递增变化, 依次向后计算第 2 个素数, 第 3 个素数, $\dots$, 第 n 个素数。

根据俄国数学家的一个定理的结论: 任意给定一个素数, 在这个素数和它的两倍之间还存在至少一个素数。据此, 我们实际上在计算时可以使 N 的变化加快。

下面是筛法求素数的一个简单算法:

算法 B (筛法求素数)

B₁. 输入 n , 输出第一个素数 2;

B₂. $M \leftarrow 2, N \leftarrow M*2$;

B₃. 从 2 开始, 依次筛除 $M+1$ 到 N 之间所有 2 的倍数, 3 的倍数, $\dots$, 直到某个已求得的素数的平方大于 N 为止;

B₄. 依小到大输出 $N+1$ 到 M 之间剩下的数 (均为素数);

B₅. 如果已求得第 n 个素数, 则输出第 n 个素数, 否则 $M \leftarrow$ 当前得到的最大素数, $N \leftarrow M*2$;

B₆. 转 B₃;

这个算法并不是最好的筛法计算素数的算法, 但是, 它比从素数的定义出发一个一个去判定素数的计算方法 (算法) 要好得多。现在的问题是, 读者可能会担心上面的算法能否计算得出我们所要的结果。从算法中我们可以看出, 每次循

环前, 我们都是先做 $N \leftarrow M * 2$, 接下来找素数也是在上次找到的最大素数与该素数的两倍 N 之间进行, 那么, 这样的区间内一定存在素数吗? 还有, 怎么就能保证通过筛选剩下的都是素数呢? 要准确弄清这些问题, 需要数论知识, 要靠数学定理及其证明来保证这一算法的正确性, 一年级的大学牲暂时没有必要深究。

上面两个算法的例子告诉我们, 根据我们对问题求解的直觉理解设计的算法往往不是比较好的算法。要想设计出高效率的快速算法, 首先必须通过对问题本身的深入分析, 发现或找出问题的内在规律。也只有这样, 才有可能从中获得巧妙的算法设计原理和思想, 最终设计出高效率的算法。显然, 要弄清问题的内在规律并设计高效算法, 借助数学方法揭示算法设计的思想是比较可行的构想和方式方法, 可见数学特别是离散数学在解题过程中的重要作用。由上面的算法我们有理由提出一个疑问, 既然算法的计算复杂性涉及到时间和空间这样两个因素, 那么, 算法所需的时间和空间之间是否存在某种关系或联系呢?

关于算法及其复杂性的有关问题, 中国科学家洪加威教授曾经讲了一个童话, 在国外被称为又一个“三个中国人算法”¹⁾。这个童话, 能够很好地帮助读者理解与计算和计算复杂性有关的一些概念。

很久以前, 有一个年轻的国王, 名叫艾述。他非常喜欢数学, 聘了当时最大的数学家孔唤石当宰相。

邻国有一位聪明美丽的公主, 名字叫秋碧贞楠。艾述国王爱上了这位公主, 便亲自登门去求婚。

公主说: “你如果向我求婚, 请你先求出 48770428644836899 的一个真因子, 一天之内交卷。”艾述听罢, 心中暗喜。他想, 我从 2 开始, 一个一个地试, 看看能不能除尽这个数, 还怕找不到这个真因子吗?

艾述国王十分精于计算, 他一秒钟就算完一个数。可是, 从早晨开始一直到晚上, 他总共算了三万多个数, 还是没有找到那个真因子。于是, 艾述只好去向公主求情。公主说: “既然你求不出来, 那么请你验证一下, 223092871²⁾ 是它的一个真因子。”国王如获大赦, 在一秒钟之内就做了一个除法, 验证了这个数恰好能除尽 48770428644836899。公主于是说: “你求婚不成, 将来做我的证婚人吧。但我可以给你一个机会再求一次。”

国王急急回国, 召见宰相孔唤石。这位大数学家说: “这个问题我也没有好的

1) 这个算法虽然也涉及三个中国人, 但与洪加威最初提出的关于无向图中回路存在的判定问题的“三个中国人算法”不同, 实际上称其为“证比求易算法”比较合适。

2) 洪加威教授给出的这两个数字, 实际上仅仅是为了说明问题, 其本身可能并不符合要求, 作者注。

办法,不过可以试一下。公主给的数不超过 17 位,如果这个数可以分成两个真因子的乘积,那么,较小的因子一定不超过 9 位。你把全国的老百姓按自然数的顺序编上号,让大家记住自己的编号。等公主给了一个数目之后,你把这个数字通报全国,让每个老百姓用自己的编号去除公主给的这个数。谁除尽了就赶快报上来,赏黄金 300 两。”

于是,国王发动全国上下的民众,再度求婚,终于取得成功。

在这个童话中,艾述国王最先使用的是一种顺序算法,后面由孔唤石提出的是一种并行算法。如果我们用黑板来计算除法,按照第一个算法,只要一块黑板就行了。因为前面的计算对后面并没有用处,可以擦去。按照第二个算法,就得每人有一块黑板,以便大家同时进行计算。这里使用的黑板可以看成是计算所需的存储空间,相当于计算机的存储器。显然,第一个算法复杂在时间方面,第二个算法复杂在空间方面。

一个问题的规模是指这个问题的大小。像上面童话这个问题的规模可以取秋碧贞楠公主给的 48770428644836899 这个数的长度,我们通常用 $n=17$ 表示。一个算法的计算复杂性如何,直接决定了这个算法可以用到多大规模的问题之上。假设现在有求解同一个问题的两个算法,第一个算法的计算复杂性是 n^3 ,第二个算法的计算复杂性是 3^n 。用每秒百万次的计算机来计算,当 $n=60$ 时,第一个算法要用时 0.2 秒,而第二个算法就要用时 4×10^{28} 秒,也就是 10^{15} 年,相当于 10 亿台每秒百万次的计算机算一百万年。

考察两个算法复杂性的表示函数,前者 (n^3) 是一个多项式函数,后者 (3^n) 是一个指数函数。当 n 很大时,这两个算法的效率差别是很大的。因此,一个问题如果没有多项式时间计算复杂性的算法,这个问题就被称为是难解型问题。不过,要断定一个问题是不是难解型问题是很困难的。一个问题即使长期没有找到多项式时间计算复杂性算法,也不能保证明天找不到多项式时间计算复杂性算法,更不能据此证明这个问题不存在多项式时间计算复杂性算法。为了研究这样的问题,必须从计算模型出发从理论上去研究和探讨,这样就有了计算复杂性理论。

国王有臣民的帮助,求亲成功是自然的。但是,如果一个平民百姓的小伙子去求婚,他就很难了。不过,办法还是有的。从国王求亲取得成功所采用的并行算法可以得到启发,他可以随便猜一个数,然后验证这个数。这样做当然成功的可能性很小,不过,万一猜着了呢?反正猜对猜不对都没有关系。由于一个数和它的因子之间存在一些有规律的联系,因此,数论知识水平较高的人猜中的可能性就大。我们说,这个小伙子使用的算法叫做非确定性算法。这样的算法需要有一种假想但实际并不存在的非确定性计算机才能运行,其理论上的计算模型是非确定性图灵机。

在算法计算复杂性的研究中,一个算法如果存在图灵机可计算的多项式时间计算复杂性算法,就将这个算法归入 P 类,如果存在非确定性图灵机可计算的多项式时间计算复杂性算法,就将其归入 NP 类。对大多数实际问题来说,找到一个解可能很难,检验一个解常常比较容易,所以都属于 NP 类。现在,计算科学研究中一个悬而未决的重要问题是 $P=NP?$ 。

$P=NP?$ 这个问题不仅具有理论上的价值,而且具有重大实用价值。因为到目前为止,已经发现了一批可计算但有相当难度的问题是属于 NP 类的,并且常通过证明一个问题与已知属于 NP 类中的某个问题(如可满足性问题,简称 SAT 问题)等价将其归入 NP 类。不过,该问题是否属于 P 类,即是否能找到多项式时间计算复杂性算法求解该问题,或证明该问题不存在多项式时间计算复杂性算法求解,至今尚未解决。现在,很多人都猜测秋碧贞楠的看法是对的:求解一个问题总是比验证一个问题的解要难,用公式表示也就是 $P \neq NP$ 。20 世纪 70 年代初,库克(S. A. Cook)和卡尔普(R. M. Karp)指出, NP 类中有一小类问题具有以下性质:迄今为止,这些问题多数经过深思熟虑还没有人找到多项式时间计算复杂性算法。但是,一旦其中的一个问题找到了多项式时间计算复杂性算法,这个类中的其他问题也能找到多项式时间计算复杂性算法,那么就可以断定 $P=NP$ 。换句话说,如果确属这个类中的某个问题被证明不存在多项式时间计算复杂性算法,那么,就等于证明了 $P \neq NP$ 。通常,将这类问题称为 NP 完全性问题。

正是由于这些原因,算法被认为是计算科学的核心问题之一。

然而,究竟什么是算法呢?我们先来看一看算法的定义。

定义 1 (算法) 给定问题和设备,一个算法是用该设备可理解的语言表示的,解决这个问题的一种方法的精确刻画。特别,算法具有下列特征属性:

- (1) 算法应用于一个具体的输入集合或问题描述将在有穷步动作序列之后得到结果;
- (2) 该序列有一个惟一的初始动作;
- (3) 该序列中的每一个动作有一个惟一的后继动作;
- (4) 该序列终止时或者得到这个问题的解,或者因该问题不可解而获得不可解说明。

定义 1 是一种百科全书¹⁾式的定义,在专业上似乎不够严谨,而且也不适应于不确定性算法、分布式算法和并行算法。因为,这里所讲的设备和动作都难以让初学者把握,更不用说后继动作是不确定的情况了。

因对计算机算法和程序设计语言研究作出突出贡献而获图灵奖的著名计算科

1) 引自美国出版的《计算机科学与工程百科全书》。

学家克努特 (D. Knuth) 曾对算法这一概念给出了进一步的描述。这个描述性定义后来为学术界广泛接受, 而且出现在许多专著和教科书中。

定义 2 (算法) 一个算法, 就是一个有穷规则的集合, 其中之规则确定了一个解决某一特定类型问题的运算序列。此外, 算法的规则序列须满足如下五个重要条件 (特性):

- (1) 有穷性。算法必须总是在执行有穷步之后结束;
- (2) 确定性。算法的每一个步骤必须是确切地定义的;
- (3) 输入。算法有零个或多个输入;
- (4) 输出。算法有一个或多个输出, 即与输入有某个特定关系的量;
- (5) 能行性。算法中有待执行的运算和操作必须是相当基本的, 即是说, 它们原则上都是能够精确地进行的, 而且用笔和纸做有穷次就可以完成。

有穷性和能行性是算法最重要的两个特征。对有些问题来说, 如果我们给出了一个类似于算法的有穷运算序列, 而它对某些输入可能不停机, 那么, 我们就称这样的运算序列为一个过程。算法和过程都满足能行性和确定性, 惟一的本质区别是算法的执行必须终止, 而过程则不然, 它可以永不停止。需要指出的是, 高级程序设计语言中也有过程的概念, 但与这里所讲的过程不同, 那里实际上指的是一种子程序。

关于算法的定义, 近年来又有新的研究。有人从状态变换的角度给出了一种新的定义。这种定义直接与某一抽象的屏蔽了运算操作定义的计算模型联系在一起, 同时通过引入具体的、功能足够强的若干运算操作定义可证明这种分布式或并行计算模型与图灵机等价, 而且允许某些通信运算操作, 从而使这种模型上定义的算法可以涵盖分布式算法或并行算法, 涵盖确定性算法和非确定性算法, 同时, 只需要作一些简单的限制就可以得出一般的算法定义实际上只是分布式算法定义或并行算法定义的一种特殊情况。然而, 对刚刚开始学习计算科学的读者来说, 克努特的定义因与程序的概念比较接近, 或许比较容易被大家所接受。将来, 随着专业知识的不断积累和增加, 有兴趣的同学可以对算法理论作进一步的了解。

20 世纪 60 年代, 克努特把计算科学定义为是研究算法的学问, 并在美国科学院会刊和一份计算科学杂志上两度发表文章, 介绍计算科学并阐述他对计算科学的一些认识观点。他的文章曾经在很长一段时间内影响了 ACM 对计算科学教程的制定, 这一点从美国 ACM 公布的早期的几个计算科学教程的课程设置中不难看出。不过, 由于 80 年代计算科学中并行与分布式算法的发展同计算机体系结构密切相关, 学科发展不断深化并渗透到更为广泛的领域, 以及学科研究中发展多种不同层面计算模型的需要, 包括发展非图灵-冯·诺依曼型计算模型, 使许

多人认识到研究计算模型与体系结构具有与研究算法同等的重要性,从而使今天学术界对计算科学下的定义变得比过去更为准确。

一般地说,对任何一个问题,如果有了解决该问题的算法,就可以编制相应的程序。所谓程序,是一种事先编制好了具有特殊功能的指令序列。其中,指令既可以是机器指令,汇编语言指令,也可以是高级语言的语句命令,甚至还可以是用自然语言描述的运算、操作命令。当然,用自然语言编写程序是计算科学进入非常高级阶段的标志之一,有赖于自然语言理解取得重大突破,目前看来还是一个十分遥远的设想。

程序这一概念的出现,得益于人类长期的生活实践。我们每一个人从很小的时候起,就在日常生活中潜移默化地被灌输了程序的概念。我们每做一件相对比较复杂的事情,不知不觉地就会学习、模仿前人的经验和方法,编制相应的处理程序,或调用现存头脑中而且是过去使用过的有效的程序。这些程序确定了处理事情时,第一步、第二步、……、第 n 步等每一步所要做的操作。虽然,我们在头脑中编制的各种程序有时没有使用具体的某一种语言进行描述,但是,如果需要,大多数程序是可以用自然语言描述的。那些难于描述的程序,不是人的智力不够,就是该程序“只能意会,不可言传”。至于这些不经过语言描述的程序在人体的大脑中是怎么存储和控制的,目前还是一个谜。这说明,人体中的各种细胞组织结构,具有思维、存储、学习、创造的功能。不难联想,人体脑细胞的存储功能类似于计算机的存储功能。从这个意义上说,每一个具有正常行为能力的人都是程序员。

这样一看,似乎程序设计并不神秘,事实也确实如此。但是,程序设计是一种高智力的活动,不同的人对同一事物的处理可以设计出完全不同的程序。我们每一个人的生活经历已经告诉自己,知识和阅历(经验)是处事(设计程序)的基础,符合时代发展要求的理念、科学的思想方法、先进的科学技术能够帮助我们更好地处理遇到的各种社会事务。正因为如此,在计算机发展的早期,程序设计被认为是一个与个人经历、思想和技艺相联系的一种技艺和技巧。只是到了后来,软件开发规模的扩大和开发方式的变化,科学思想、方法的引入,一些程序与程序设计规律被揭示,程序设计才开始被当成一门科学来对待。

既然程序如此重要,又为人类经常使用,就有必要对程序及其运行的规律进行深入的研究。于是,对程序各种性质如程序的结构、程序的正确性、程序的运行效率等的研究产生了计算科学十分重要的一个方向——程序理论。

有一种程序的定义,用公式给出:

程序=数据结构+算法

这种定义初看起来有新意,它从程序的特征入手,高度抽象和概括。然而,仅有

学术研究上的辅助参考价值，不能作为教学中的定义。因为，按照这一定义，一旦数据结构和算法的定义被确定下来，程序的概念应该被随之确定，而实际上，程序的概念比数据结构加算法的涵义更广。考虑到我们前面给出的算法定义都明确要求满足终止性的属性，而程序可以不停机，甚至采用非常高级的程序设计语言设计的程序可以没有任何数据结构，有的程序中看不到算法（如 Prolog 程序中一些推理计算的过程）。所以，我们还是只取前一种程序的定义更合适一些。

前面提到的用汇编语言和高级语言编写的程序都是不能在计算机（裸机）系统上直接执行的程序，必须首先将它们翻译成机器指令表示的程序才能在计算机上运行。高级语言的翻译主要有两种方式，一是解释，二是编译。解释方式是指使用解释程序对高级语言书写的程序进行逐条解释执行，而编译方式是借助编译程序对高级语言书写的程序进行整体上的识别与理解，将其翻译成一个功能等价的程序。这些将是读者今后要学习的内容。

2.6 高级语言、程序设计技术与方法

所谓高级程序设计语言（简称高级语言）是指用于描述计算机程序的类自然语言。这种语言只是自然语言的一个很小的子集，在语法结构上比较简单而且规范，在语义上较少二义性，能够以比较准确、易读的形式描述各种计算机程序。例如，人们常见的 Fortran 语言、Pascal 语言、C 语言、LISP 语言、Ada 语言、Prolog 语言，等等。

高级程序设计语言是程序设计发展的产物。在计算机诞生后不久，Fortran 语言就于 20 世纪 50 年代中期出现并投入使用。随着计算机应用领域的不断拓展，针对各个应用领域的不同特点和程序设计的经验，科研人员设计和发展了一批高级程序设计语言。在高级语言发展的历程中，人们不断地将程序设计的新理念和一些成熟的程序设计方法和技术融入程序设计语言的设计之中，不断地推出新型程序设计语言。进入 60 年代中期以后，计算科学理论的快速发展，对程序设计技术与程序设计方法的研究，以及计算机在各个领域的应用极大地促进了高级程序设计语言的发展，先后产生了一大批高级程序设计语言。

今天，许多人在没有上大学之前，已在中小学有了程序设计和上机调试程序的经历。有程序设计经历的、细心的读者可能早已注意到，对于一个已经有了计算该问题算法的待解问题，不同的人根据同一算法可能编写出完全不同然而都是正确的程序。这种不同除了程序的书写形式有区别外，更重要的是这些程序的结构反映在程序设计的构思和易读性方面有差别，程序运行的效率（主要指速度）也不一样，有时相去甚远。这是为什么呢？

实际上,这里存在一个程序设计方法和技术的问题。在程序设计发展的早期,由于对一个可解的问题(那时往往是比较简单的问题),事实上采用程序设计的途径可以设计出很多种程序来计算求解,也由于早期的程序开发常常由一两个人包揽,因此,程序设计技术不免被看成是一种与个人思想、经验和技能相联系的技巧或技艺,程序设计是一种个人行为。然而,随着乔姆斯基(N. Chomsky)和巴科斯(J. W. Backus)等人对语言理论研究取得进展,当 Algol 60 语言出现之后,人们开始认识到程序设计语言是一门科学,高级语言本身有着很强的规律,它与程序设计的思想、方法、技能之间存在密切的联系。20 世纪五六十年代,克林(S. C. Kleene)从计算模型的角度对程序中的递归关系、递归过程、递归条件表达式等进行了研究,雅可比则对程序流程的基本结构与图灵机的关系进行了研究,使人们对程序结构的本质有了更深入的了解。在克林和雅可比等人研究了构成程序的基本结构并进而在学术界统一了关于程序的好坏的概念之后,也由于大型程序和软件的开发需要由集体来完成,需要参加者之间的密切合作,于是,程序理论支持下的软件开发方法学研究使大多数人开始承认程序设计是一门技术科学。从那时起,程序设计方法与程序设计技术取得了很大的进展。

经过多年的研究,在计算科学中已经发展了许多程序设计方法和技术。例如,自顶向下逐步求精的程序设计方法与技术、自底向上的程序设计方法与技术、基于程序推导的程序设计方法与技术、基于程序变换的程序设计方法与技术、面向对象的程序设计方法与技术、函数式程序设计技术、逻辑程序设计技术、程序验证技术、约束程序设计技术、并发程序设计技术,等等。又如,对于许多问题的计算,可以用类似于计算函数的方法来进行,也可以用表(一种数据结构)处理的方法进行,甚至还可以用逻辑公式演绎推导的方法进行。在实现技术上,既可以用递归技术计算,也可以用迭代技术或其他技术进行计算。

在上一节,我们已经知道,任何算法中的基本运算操作,都必须能在某一(抽象)计算(模型)机器上运行,基本的运算操作实际上在划分成充分小的单位后,每个单位可以对应于一条机器的指令或计算模型的基本动作。机器的指令与模型的基本动作事实上可以看成是机器语言或模型语言,其形式是机器或模型的外在表现形式。实际上,20 世纪 50 年代和 60 年代在计算科学理论的研究工作中已经指出:不同复杂程度的语言文法分别与某种计算模型在处理语言的计算能力上相等价,计算机的指令系统具有递归性。有了这些认知之后,就应该能够认识到语言本身也是一种计算模型,它在计算机裸机系统与人的思维、描述方式之间架起了一座沟通的桥梁,只是在描述、刻画、表达计算时所处层面和方便程度不同而已。这就不难帮助读者理解语言是计算模型的外在表现形式。

需要指出的是,不同的程序设计方法和技术,都是从不同的角度对程序及其

设计和产生的过程的特性和规律进行观察，经抽象、分析和总结之后得出的。其中，凡是有生命力的方法和技术，都在以后的发展中建立了比较坚实的数学理论基础，并在实践中被反复检验证明是有效的。这些内容，读者将来在后续课程中会陆续接触到，眼下还没有必要作太多的了解。

作为一门科学，高级语言和程序设计确实对计算科学的发展产生了巨大的影响。程序设计方法和技术在各个时期的发展不仅直接导致了一大批风格各异的高级程序设计语言的诞生，而且许多新思想、新概念、新方法和新技术不仅在语言中得到体现，同时还渗透到了计算科学的各个方向，从语言理论、硬件支撑条件、语言实现技术到计算机应用技术等多方面深刻影响了计算科学的发展。难怪人们把对高级语言和程序设计的学习与训练看成是计算科学专业的基本功之一。

2.7 系统软件与应用软件

计算机（硬件裸机）制造出来以后，人们在机器上开发了许多各种各样的软件。所谓软件是对事先编制好了具有特殊功能和用途的程序系统及其说明文件的统称。曾长期从事软件工程研究的美籍华裔计算科学家叶祖尧教授认为，软件是一个发展的概念，早期软件和程序几乎是同义词。后来，随着软件开发中各种方法和技术的出现，以及软件开发中程序及其开发工作量所占比重的降低，软件的概念在程序的基础上得到了延伸。1983年，IEEE对软件给出了一个较为新颖的定义，指出：软件是计算机程序、方法、规范及其相应的文档以及在计算机上运行时所必须的数据。

这个定义虽然在学术上有重要参考价值，它将程序与软件开发方法、程序设计规范及其相应的文档联系在一起，将程序与其在计算机上运行时必需的数据联系在一起，实际上是在考虑了软件生存周期中的各项主要因素之后提出的。但是，当一些程序运行所必需的数据只能动态地获得时，特别是针对人工智能程序和实时程序时，这一定义在实际工作中会引出问题。因为，完全认可IEEE的定义，意味着软件的销售允许出售不完整的软件；对软件及其质量的评价可以通过对不完整的软件的评价进行。

不管怎么说，我们已经明白，软件是相对于机器硬件而言的，它与计算机硬件一起构成了一个完整的计算机系统。至于计算机系统的性能怎么样，功能是否强大，则取决于硬件和软件的性能和功能的总和。

在软件的发展过程中，随着各种各样软件数量的增加，人们将各种软件分成两大类，一类称为系统软件，一类叫做应用软件。所谓系统软件是指那些参与构成计算机系统，提供给计算机用户使用，用于扩展和充分发挥计算机硬件功能、

维护整个计算机硬件和软件系统,平滑用户思维方式、操作习惯与计算机硬件设备之间沟坎的软件。应用软件是相对于系统软件而言的,它是对用户在计算机系统上针对各种具体的应用问题开发的一类专用程序或软件的总称。

一般地说,操作系统、汇编程序、解释程序、编译系统、数据库管理系统、软硬件故障诊断程序、子程序库、各种软件开发工具和软件包及其说明文件等属于系统软件,其他由用户开发的各种形形色色的应用程序及其说明文件或应用软

件系统被称为应用软件。

在系统软件中,初学者最先接触到的常常是操作系统。什么是操作系统呢?通俗地讲,操作系统是一种系统软件,它负责管理计算机系统中的各种资源并控制各类程序的运行。它通过接受来自用户的操作命令,按照用户的要求对计算机的工作进行控制。计算机配上操作系统之后,能够提高计算机系统的工作效率,便于用户使用。

汇编语言是将汇编指令翻译成对应的机器指令的一种程序,属于系统软件的范畴,能够支持用汇编指令编写的程序在机器上运行。解释程序和编译程序都是将高级语言写的程序(简称源程序)翻译成用另一种语言(可以是机器指令、汇编指令或高级语言)编写的等效程序的程序(简称目标程序),它们也都属于系统软件的范畴。解释程序和编译程序的一个本质的区别是在翻译的过程中,解释程序对源程序采取的是根据程序语句的执行顺序进行逐条语句翻译的方法,而编译程序是在整体理解源程序的基础上进行翻译的。

数据库系统包括数据库和数据库管理系统两部分内容。随着计算机处理数据种类和数量的增长,亟需要有一整套数据表示、存储、管理的方法来对大量的各种数据进行管理,为不同的数据用户提供不同的服务,支持各种数据处理软硬件系统的开发。于是,数据库管理系统应运而生。早期,由于数据表示、组织和存储的方式不同,曾经产生了层次数据库系统和网状数据库系统,很快,当基于关系代数演算理论的关系数据库理论出现后,关系数据库系统的开发取代了其他数据库系统。值得指出的是,对数据库管理系统的类属划分存在不同认识,有人认为它应属于系统软件,也有人认为它属于应用软件。其实,由于系统软件和应用软件的定义本身就不是十分严格,因此,存在这种情况不足为奇,读者完全没有必要在这种枝节问题上耗费过多精力。将来一旦大家更深入地掌握了计算科学的知识,就完全有能力自己作出准确的判断。

随着计算机系统中软硬件资源的不断增加,系统的维护就成为一个保持系统正常工作十分重要的任务。系统管理员维护系统的主要方式是通过借助于系统的维护工具来进行正常的维护。系统维护的工具种类很多,最常见和最常用的工具有软硬件故障诊断程序,通过它可以及时发现系统存在的问题。其他工具还有磁

盘碎片整理程序、进程监视程序、垃圾回收程序,等等。

这里所述的子程序库通常是指操作系统向用户开放的一部分系统调用的集合,每一个系统调用以类似于函数形式的子程序存在于系统之中,用户可以根据自己的需要调用这些子程序,作为自己开发的程序的一部分。这种子程序库中各种子程序的功能往往与操作系统内核(核心程序)经常使用的各种程序功能有关,它们是在操作系统开发的过程中形成的,在功能上比较接近于机器内部的宏操作功能,其函数表示形式从高级语言的角度看一般与开发操作系统使用的语言保持一致,既可以看成是操作系统功能的开放性延伸,也可以视为高级语言成分的扩展。当其他的语言使用这种子程序时有可能存在语言不兼容的问题,但可以通过开发操作系统使用的语言与其他语言之间的接口转换程序通过间接调用的方式使用子程序库。

2.8 计算机图形学、图像处理与模式识别

通用电子数字计算机的出现,使人们很容易联想到能否用计算机来处理图形和图像,因为图形和图像具有直观性强的特点,许多学科在阐述自身的大量科学内容时,常常借助于图形和图像来进行说明,取得了很好的效果。然而,由于早期的计算机系统存储容量很小,一幅很小的画计算机都存不下,更不用说如何处理了。后来,随着计算机性能的不断提高,特别是海量存储器的发明,使得图形学和图像处理得到快速的发展。

计算机图形学是研究怎样用数字计算机生成、处理与显示图形的一门分支学科。计算机图形学主要处理两类图形,一类是由线条形成的图形,如地图、工程图、曲线图表等,用点和线这样一些基本的图元按照构建图形的数学模型形成各种图形;另一类是明暗图,与照片相似,用点阵数据出现的色彩程度的不同构成图形,也就是我们通常认为的图像。为了让计算机生成图形,首先需要有图形的原始数据和构建图形的数学模型。比如,矩形由哪几个点的数据和相互之间的关系构成等。图形的原始数据与最终计算机生成的图形之间可能存在一定的误差。例如,工程技术人员在设计图形时的构思和草图,不可能从一开始就很规范、精确、美观。但是,通过将原始数据输入计算机进行处理,可以得到精度很高的图形。计算机图形学应用研究的一个最著名的成果是 CAD 技术。今天,许多工业生产行业和部门已经广泛使用 CAD 技术用于产品的设计与研发。

与计算机图形学密切相关但相对独立的一门技术是图像处理,也叫数字图像处理。图像处理主要研究对客观世界中已经存在的物体映象处理成新的数字化图像,如卫星遥感中的资源勘测,医学中对人体的 CT 扫描,彩色感光摄影等。由

于处理图像的各种设备存在精度上的局限性和技术上的误差,图像处理关心的问题主要是如何滤掉噪声,对图像数据进行压缩便于存储、传输和还原图像,如何对图像进行处理,使图像的质量得到提高等。

与图像处理有密切关系的另一门技术叫做模式识别。模式识别研究怎样分析和识别各种图像,找出其中图像数据蕴涵的内在联系或构建图像的抽象模型。例如,高速公路上自动收费站对通过的车辆牌照进行扫描,将得到的图像进行模式识别,弄清楚刚才是哪一种牌照、多少编号的车辆通过了收费站,并作出相应处理。

显然,与计算机图形学的研究不同,图像处理重点讨论怎样将数据和几何对象的模型变成图像,而模式识别是研究怎样从图像提取识别对象的数据和模型,两者之间正好是一个相反的过程。由于图形的种类繁多,千变万化,其几何模型必然很多。于是,在图形学、图像处理与模式识别的研究过程中又产生了一门新的学科,这就是计算几何。计算几何研究几何模型及其数据处理,重点讨论几何形体的计算机表示、分析与综合,研究怎样建立几何形体的数学模型及其数据存储和管理,提高算法的效率等。例如,三维立体造型,曲线曲面的构造与拼接等。

2.9 逻辑与人工智能

计算机的发明与应用,其根本目的在于代替人的各种劳动。科学研究特别是军工科技领域内的尖端科技研发中大量复杂、烦琐的计算任务极大地促进了电子数字计算机的研制。早期的计算机应用主要集中在科学计算方面,针对大量的科学问题,依据计算方法编制程序,让计算机来代替人从事那些机械、刻板、令人生厌的计算任务。在第一台电子数字计算机诞生之后,一些人开始考虑让计算机具有某种思维的能力,以便让它像一个训练有素的人一样能够做一些需要一定的思维、推理的工作。对于这样一个美好的愿望,人们自然很容易想到首先让计算机模拟人来下棋,证明和发现数学定理或做语言翻译之类的事情,并且也确实有一些在当时看来属于惊人的发现和成功,这些应该可以看成是人工智能研究的发端。

1956年夏天,一批平时经常聚在一起进行学术讨论的科学家发起并在美国的达特茅斯大学召开了世界上第一次人工智能学术大会,经麦卡锡(J. McCarthy)教授提议,会上正式决定使用人工智能这个词来概括这个研究方向。那次会议期间被认为是人工智能正式诞生的日子。

自从第一次人工智能大会之后,人工智能的研究先后在许多学术机构开展起来,发表了大量的研究成果。然而,很快人们就发现自己的设想是过于乐观了。

因为,要使机器具有某种思维能力,代替人来做一些需要思维和推理的工作,需要基础理论研究的支持。问题是什么理论能够成为人工智能的基础?或者说人工智能需要什么样的理论基础?人工智能也许是世界上争议最多的学科之一,因为即使是今天也很难有人就智能和人工智能给出准确的定义和研究范畴。长期以来,围绕机器是否具有智能问题的争论从来就没有停息过。随着研究的不断深入开展,人工智能的研究也曾几起几落。每当研究工作取得一些重要的发现和进展,人工智能研究就会形成一次热潮,而一旦研究遇到挫折,研究工作很快就会陷入低谷。以至于学术界有一些人始终认为,人工智能研究工作的一个致命的弱点是我们对于什么是智能的本质没有真正弄清楚。

围绕智能的本质和机器能否达到人的智能水平,学术界存在不同的看法。冯·诺依曼教授认为机器决不会有智能,可是图灵却认为计算机能够达到人的智力水平。但图灵十分谨慎,一直不愿意正面回答机器能否思维的问题,而是通过提出著名的“图灵试验”从哲学上设置一个“陷阱”,巧妙地绕开了这个问题的正面回答。尽管如此,人们还是要问:什么是人工智能?

所谓人工智能是研究如何让机器来模拟人的行为的科学。根据这一定义,我们不难猜测,人工智能是一个涉及内容很广的多学科交叉的领域。人的行为不仅受到人的思维和思想的支配,而且,要让机器来表现这样一种行为能力,必然涉及到生理学、心理学、行为科学、语言学、信息论、控制论、思维科学等许多方面。也许早期从事人工智能研究的学者主要来自具有数学专业背景的人,也可能从其他学科研究人工智能并与现代计算机技术结合比较困难,在相当长的一段时间内,人们将更多的注意力集中在人的思维结构和逻辑推理的研究。

事情涉及到了思维和推理的研究,人们自然首先想到了逻辑,原因是逻辑学是研究思维结构和推理的科学。许多科学家认为,如果不把人的思维结构和推理过程弄清楚,恐怕要让机器来代替人进行思维和推理将是行不通的。此时,追溯往事,学术界发现:在历史上,亚里士多德、培根、莱布尼兹(G. W. Leibniz)、布尔(G. Boole)、哥德尔、图灵、香农(C. Shannon)、莫克利(J. W. Mauchly)、麦卡锡等一批学者已经从不同的角度和不同的层面对思维结构和推理机制进行了研究,取得了一些重要的进展。但是,人的智能又是一个十分复杂的问题,涉及到许多方面,过去的大量研究仅仅只是从某个角度对智能及其应用的某个方面开展了有一定成效的研究工作,但是,对于人的智能的内在机理我们还远远没有弄清楚。逻辑是反映人的思维过程的一个重要的方面,但不是全部。研究各种逻辑系统对于发展人工智能具有十分重要的科学意义,因为一个成熟的逻辑系统常常不仅能够帮助我们理解人的行为依据,而且能够帮助我们模拟人的某些智能行为。

逻辑并不是研究人工智能的惟一的发展道路。历史上,在人工智能的研究中还产生了认知心理学派,认为人的心理活动是可以通过信息的形式加以研究和认识的。以耐威尔(A. Newell)和西蒙(H. Simon)教授为首的认知心理学派主张人工智能的研究应该与心理学相融合,借用信息论建立心理活动的信息模型,借用计算机技术进行信息的处理,由此揭示人的心理活动过程,解释人的智能行为。近年来正处于热点研究的演化计算也开创了智能计算的另一条战线。

人工智能的逻辑学派和认知心理学学派虽然在研究中发表了大量的研究论文,但并没有取得实质性的根本突破,在发展中几度沉浮,步履艰难。倒是演化计算在多目标整体优化方面解决了一些著名的世界难题,表现出生机和希望。尽管人工智能的发展多次遇到挫折,但我们应该看到,在计算机下棋、数学定理证明、专家系统、机器人、人工智能的逻辑基础、自然语言处理等许多方面,人工智能的研究依然在探索中继续向前发展。

2.10 计算机组织与体系结构

最初,计算机的研制都是逐个进行设计、制造的,每一台计算机有自己的指令系统,用机器指令或汇编语言编写的程序在不同的计算机系统上相互之间是不能兼容的,也就是说用一台计算机的指令编制的程序并不能在另一台计算机系统上运行。高级语言的出现使计算机系统的兼容问题得到某种程度的缓解。但是,由于编译程序在实现时有一些内容与计算机硬件密切相关,仍然不能完全解决兼容问题。于是,计算机产品的系列化和软件的兼容性在20世纪60年代受到重视。

IBM公司在1964年推出的IBM 360系统标志着计算机体系结构(Architecture,也称系统结构)研究的开端。通俗地说,计算机体系结构是使用机器语言编写程序的用户可看到的一个机器的抽象结构,而对这一结构实现的硬件组成属于计算机组成原理研究的范畴。

随着大规模和超大规模集成电路(LSI/VLSI)技术的成熟,一方面器件速度和可靠性在不断提高,目前已逼近极限;另一方面器件的体积和价格在持续下降,这极大地增强了计算机系统的性能。然而,高质量的器件并不是产生高性能计算机的惟一因素。虽然,今日大多数计算机都是图灵-冯·诺依曼型存储程序式电子数字计算机,但人们早已认识到计算机不仅仅是一个硬件组织的问题。一个现代的计算机系统一般被认为是由存储器、处理器、功能部件、互联网络、汇编程序、编译程序、操作系统、外部设备、通信通道等内容组合而成的。

早期,人们对计算机系统的研究与开发有两个基本的特点:一是研究与开发

工作中,对硬件和软件的研究与开发基本上是独立进行的;二是对硬件的研究与开发更多的是从计算机组成原理上去关心各部件中分离元器件及其相互之间的连接关系,关心各部件的内部构造和外部特性。但是,集成电路的发展使得计算机 CPU 的速度和存储器的容量成倍增长,各种多功能部件不断引入,如何设计和配置计算机系统使其具有更高的性能价格比,适应不同用户的要求,成为亟待解决的问题。同时,集成电路的发展也使许多专业人员可以不再关心各部件的内部细节,而只需考虑如何以一种统一的观点从硬件器件和一些软件系统出发,通过组合配置,设计功能更强,性能价格比更高、更可靠的计算机系统。于是,计算机体系结构应运而生。

一个典型的、有助于读者理解计算机系统结构的方向是所谓的网络计算机系统。20 世纪 60 年代末至 70 年代初,计算机通信技术的发展和成熟使网络计算机系统成为现实,同时,也对多处理机并行计算机系统的发展产生了很大影响。与传统的放置在一个机房里的顺序执行指令的计算机系统相比,地域上分布的广域网络计算机系统和地域上虽不分布但有多多个终端计算机的局域网络计算机系统在使用时与顺序计算机有很大的不同。如果用户面对的是一台未开发各种通信软件的系统,那么,用户要在这样的网络计算机上设计分布式程序是十分困难的,因为,此时他不仅要熟悉网上每一种通信资源(设备)的配置情况,而且还必须了解每一种通信资源的基本操作方法。当分布式程序进行通信时,设计者还必须在程序中自己考虑通信时可能出错的情况,以及如何纠错。显然,对于用户来说,无论是从工作的效率还是工作的质量考虑,都需要有一种能够屏蔽计算机硬件系统的技术细节,仅对用户的功能透明的系统层,使用户看到的和实际使用的是一个与使用者思想方式、使用习惯比较接近,无需具体关心网络计算机通信时一些十分繁琐的技术细节的分布式计算机系统。尽管这似乎与计算机环境有许多相似之处,但体系结构与环境之间还存在不少区别,因为一个成熟的、良好的程序设计或软件开发环境对用户来说,不仅要能够提供一系列的工具支撑程序设计或软件开发的全过程,而且,在用户人机界面的友好性方面较之计算机体系结构有更高的要求,必须符合人机工程学的原理。对用户来说,计算机体系结构仅仅是一种比较初步的环境。

一般地说,硬件的互连结构和软件结构及相互关系形成的计算机系统的总体结构,支持这种结构的基本算法,还有以总体结构为基础的面向用户的程序设计语言等内容构成了计算机体系结构的技术范畴。显然,体系结构既不属于硬件范畴,也不只是属于软件范畴,而是一个软硬件兼而有之的综合研究方向。

2.11 并行计算机、通道与并行计算

超大规模集成电路技术的发展极大地提高了计算机器件的速度，也提高了器件的可靠性，这使得计算机硬件的价格和物理尺寸不断地降低和减少，大大地增强了计算机的性能指标。

早期的计算机 CPU 里只有一个算术逻辑部件 (ALU)，而且，在每一时刻 ALU 只能执行一种功能，因此，要执行一长串算术逻辑指令需要很长的时间。实际上，一串算术逻辑指令可以划分成若干组相互独立且可以由多个能并行工作的专用功能部件来完成。当 CPU 中添加多个功能部件和多个寄存器后，只需要增加一个记录被请求的功能部件和寄存器可用性的记录板就可以通过控制器实现算术逻辑指令的并行执行，从而使计算机的运算速度大大提高。运用这些技术，我们就可以实现具有单个处理器的并行计算机系统。例如，流水线向量计算机，阵列式计算机。

在我们日常使用的非网络型计算机系统中，大多数的计算机系统只有一个中央处理器，亦即单处理机系统。使用单个处理器虽然能够做成并行计算机，但是，这种并行计算机的性能在通用性方面有很大的局限性。随着计算机应用的不断深入与拓展，由于大量的计算问题本身是由一组相对独立但各不相同的子问题构成的，在计算的过程中隐含着大量并行性的特点，也由于单处理机系统不仅在速度上而且在功能部件方面不能满足并行处理的要求，人们开始设想用多个处理机按照某种结构连接在一起，构成并行计算机系统，亦即多处理机系统，试图采用并行的思想方法和技术方法来并行处理各种复杂问题。到目前为止，世界上已经开发了许多不同种类的并行计算机系统。

并行处理要求在计算机上并行地运行许多程序。根据使用的计算机系统的不同，我们可以在四个程序的级别上提出并行处理的问题：

- (1) 作业或程序级并行；
- (2) 任务或过程级并行；
- (3) 指令级并行；
- (4) 指令内部级并行。

用户请求计算机计算的一个计算任务叫做一个作业。它可能包含几个程序的顺序执行，也可能包含几个程序的并行执行。最高的作业或程序级常通过作业调度与分配算法实现并行处理，最低的指令内部级并行常直接用硬件方法实现。

采用不同的并行处理思想和技术，产生了许多不同的并行计算机系统。从使用方便的角度考虑，用户更希望看到的是系统提供作业或程序级并行，这样用户

可以不必考虑实现并行的细节。而从实际计算提高效率的角度考虑,用户更希望系统已经实现了指令级并行或指令内部级并行。那么,面对众多的并行计算机系统,我们应该怎么来认识和区别它们的系统结构呢?注意到了程序在计算机上的执行实际上是指令在数据集上的一个操作序列这样一个基本的事实,菲林(M. J. Flynn)在1966年提出了一种著名的、也是现今比较常用的系统结构分类方法。他将计算机系统的系统结构分为四类,分别是:单指令流单数据流计算机(SISD)、单指令流多数据流计算机(SIMD)、多指令流单数据流计算机(MISD)、多指令流多数据流计算机(MIMD)。

传统的顺序执行指令的计算机就是 SISD 类型的典型代表,但在指令的执行阶段可以是流水线重叠处理。这类计算机可以有一个以上的功能部件,所有功能部件都由一个控制器管理。阵列式计算机对应于 SIMD 类型。同一个控制器管理了多个处理单元,控制器每发出一条指令,每一个处理单元都能收到,但各处理单元执行指令时,操作对象或数据却是来自不同的数据流的数据组。与 MISD 类型对应的机器目前还没有实际的机器,不过,从原理上说,这种类型的机器共有 n 个处理单元,每个处理单元接受不同的指令,操作对象或数据来自于同一条数据流。大多数多处理机系统和多计算机系统可以划归为 MIMD 类型,特别,网络计算机系统对应于 MIMD 类型。一台内在的 MIMD 计算机意味着 n 个处理器之间存在相互作用,因为所有存储数据流都来源于这 n 台处理机共享的一个数据空间。如果一台内在的 MIMD 计算机其处理机之间的相互作用程度很高,则称为紧密耦合系统,否则,称为松散耦合系统。由于紧密耦合的计算机系统在硬件的设计上较为复杂,用户使用计算机系统时的通用性受到影响,除非机器应用具有很强的背景,大多数情况下,商品化的 MIMD 计算机系统多为松散耦合系统。

由多机系统技术的扩展及网络互联与通信技术的引入,发展了分布式网络计算机系统,并在此基础上,提出了分布式计算机体系结构、分布式算法与分布式并行处理的概念,进而又拓展了相应的许多研究方向,如分布式操作系统、移动计算、分布式人工智能等。

并行计算机系统的出现,带来了信息并行化处理的概念。并行处理是信息处理的一种有效形式,它着重于发掘计算过程中的并发事件。并发性包含宏观上的并行性,同时性以及微观上的流水线处理。并行事件可在同一时间间隔内在多个资源(如多个处理器)里发生;同时事件可在同一时刻上发生;流水线事件可在部分重叠的时间内于一个(或多个)资源里出现。并发通常是指宏观上一个资源里并行发生了两个或两个以上的事件,但在微观上是顺序发生的,而并行通常是指多个资源里微观上同时发生了两个或两个以上事件。显然,一组事件是并行的也是并发的,但一组事件是并发的却不一定是并行的。

并行计算机运算速度的提高并非大功告成了。在我们的各种计算机系统上，连接着大量的外部设备。众所周知，外部设备的运行速度是比较慢的，它与高速的 CPU 之间形成了鲜明的对照。如果我们有一个程序在运行中经常调用或频繁地使用输入/输出 (I/O) 设备的程序片段，那么，I/O 设备的低速将直接影响这个程序的运算速度。显然，解决如何在 I/O 处理中获得与处理机最大的并发性将是提高并行计算机性能的关键问题之一。为了解决这个问题，引入了通道的概念。通俗地讲，通道是数据或信息传送的通路。传统的计算机系统中，I/O 通道包含一个小处理器，能够为一组外部设备进行主存的随机访问。为了解决低速的 I/O 设备与高速的 CPU 之间的不匹配，可以用一个专门进行 I/O 处理的处理器来加强 I/O 子系统。I/O 处理器一般不包括算术逻辑运算部件，不具备算术逻辑运算能力，而只负责与 CPU、主存和外部设备控制器打交道，负责大量的数据和信息的传送工作。当我们将这个处理器通过总线连接到系统上，负责从主存中选择、检索出各个 I/O 命令并执行这些命令时，由这个处理器和若干传统的 I/O 通道以及外部设备控制器一起，提供了外部设备之间，CPU、主存和外部设备之间的通信通道。通信通道的引入，使得计算机的 CPU 能够与通道并行工作，这极大地提高了计算机的吞吐能力和运算速度。

利用并行计算机系统进行数据与信息的并行处理称为并行计算。从处理对象的角度划分，并行计算分为数值并行计算和非数值并行计算。与顺序计算类似，并行计算也分成几个步骤：研究并行计算方法，研究并行算法，设计并程序，调试与运行，分析结果。由于许多问题的计算已经有了计算方法，而这些计算方法中隐含了大量的并行性运算操作，稍作变化就可支持并行算法和并行程序设计，而且，由于各种并行计算机的系统结构不同，各处理器和各多功能部件之间在体现算法时的相互作用不同，算法不能通用。因此，除了并行计算机体系结构的研究外，当前和今后相当长的一个时期内并行处理的一个工作重点将是研究各种并行与分布式计算机系统上的并行算法或分布式算法。

2.12 计算机网络与通信

随着计算科学及其应用的高速发展，在现实世界中，一方面，用户迫切需要实现不同计算机上软硬件和信息资源的共享，另一方面，有一大类问题本身具有地域上分布，其处理需要分布计算技术的特点，这些客观要求导致了计算机网络的发展和网络计算机系统的诞生。

所谓计算机网络是使用通信设备和通信线路将一组地理上分布的相同（称为同质）或不同（称为异质）的计算机、终端及其附属设备按照某种方式互联起来

得到的一个计算机硬件系统,也叫网络计算机。在这种计算机硬件系统的基础上,通过开发能协调各台计算机系统工作的通信系统或更进一步的网络操作系统,就能使一组计算机实现软硬件资源共享、协同计算,合作求解一个问题。由这种通信系统或网络操作系统连同网络计算机一起,就形成了网络计算机系统。

有关网络发展的背景和过程是这样的。20 世纪 60 年代中后期,美国国防部远景研究规划署 ARPA(后改称国防远景研究规划署(DARPA))设立了 ARPANET 项目,它是美国政府最初进行信息分组交换实验研究的一个项目。该项目的研究目的是为了开发一组网络协议,使在不同地区的计算机可通过各种分组交换网络透明地通信,使分布在各地的计算中心的研究人员能够共享硬件资源和软件资源。1969 年,使用射频与卫星技术进行通信的其他试验网通过 DARPA 资助的网络互联技术与 ARPANET 网联通了。80 年代初,原来的 ARPANET 网分解成两个网,即 ARPANET 网和 MILNET(非机密军事情报网),但两网互联,允许相互之间继续通信。起初,这种军用网和实验网的互联网,叫作 DARPA Internet,后来干脆简称为 Internet,中文叫做因特网。

在最初的几年里,ARPANET 网的使用仅限于军事工作的需要。70 年代后期,美国的大学和科研机构出现了一些分散的协作网,如世界范围的 UNIX 通信网和用户网 USENET,使用基本上是免费的。20 世纪 80 年代,更多的协作网,如计算机科学网 CSNET 和 BITNET 开始为学术机构提供全美范围的联网。这些网络不是 Internet 的组成部分,但后来经过专门的互联,相互之间可以交换信息。

在 Internet 的发展史上,一个重要的时刻是 1986 年美国国家科学基金网 NSFNET 的诞生。该网通过五个大型计算中心把全国的研究人员联系起来,并迅速扩展,联通了一些中等规模的全国性的学术网络,为科学研究和学术交流提供网络服务。由于这个学术网络上大量的用户来自网络开发与应用排头兵的计算机公司实验室、大学、科研机构的科学家和工程师,丰富的、多学科的网络资源使用户可以在网络上获得许多先进的思想、技术、软件和信息,因而促使更多的人加入到这个网络中来,NSFNET 网开始逐步取代 ARPANET 网的位置。1990 年 3 月,ARPANET 网光荣退役。之后,CSNET 网的管理人员也很快发现,其许多早期成员都已联接到 NSFNET 网上,于是,在 1991 年 CSNET 也退出了舞台。

按照数据传输范围和实现技术的不同,计算机网络存在局域计算机网络和广域计算机网络之分。局域计算机网络是一个数据通信系统,其传输范围在中等地理区域,使用中等或高速数据传输速率,使用专用数据通信线或总线进行通信,可联接大量独立设备,在物理通信通道上互相通信。

广域计算机网络把不同城市、不同国家中的计算机或计算机网络通过分级互联技术联接起来,其传输范围可达到相当远的距离,目前最常见的是使用公用或

专用电话线通信,主干网和一些局域网使用可进行数字通信的光纤光缆数据通信专用线。今后,卫星、微波和光纤通信会进一步普及。例如,Internet是成千上万个分布于世界各地的计算机网络的松散联合体,这些网络使全世界的几千万人保持着联系。在Internet中,一些国家已经建立了国家科研教育网,国际商业性服务网等网中之网。联接到网上的网络用户,可以通过自己的计算机与远在天边的其他网络用户进行通信,通过网络软件查询网上的各种信息资源。有了计算机网络,加上近年来宽带通信介质和设备的开发,以及通信数据传输速率的不断提高,今后,利用互联网教学、收看实时新闻节目、网上订票和购物、网络电视会议、网络医院门诊,等等,就不再是科学幻想。今天,由于通信技术的发展,网络通信的数据传输速率已经实现了质的飞跃,成千上万的数据在微秒级即可实现异地传送,地球突然间变得很小,就像一个村庄一样。

网络互联的拓扑结构是计算机网络的重要特性。网络的拓扑结构是一种抽象的由点和线组成的图。网络上的每台计算机用一个结点表示,机器与机器之间的链路用线和路径表示,它们构成了网络计算机体系结构中一些基本算法研究中数学描述的对象表示基础。

支持计算机网络的重要技术是通信,即实现计算机之间信息传输的一种技术方式。网络通信的核心内容是通信协议。所谓通信协议是网络通信中一组约定的集合,由它确定了经由通信网络传输的信息或存储在报文和数据库中的信息的格式和控制方式。研究通信协议主要是为了在网络计算机系统中实现可靠的、高效的数据交换,差错控制,信息编码,线路利用,同步,使通信数据具有透明性。

一般情况下,网络上连接着大量的计算机系统,任何时刻,每一台计算机系统上可能有多个用户在同时使用计算机与其他网上用户进行通信,而网络的通信线路通常设计成公用资源,这样,网络通信为了实现可靠的数据交换,因需要做许多具体的操作运算而变得十分复杂。由于从用户发送或接收可以识别的符号信息到实际在正确的通信线路上传递物理信息之间存在转换、线路利用、分组交换、差错纠正等一系列的操作,为了便于协议的有效实现和对不同的用户开放,最大限度地实现线路的有效利用,有必要对网络计算机系统进行通信结构分层。网络计算机系统在通信的系统结构(注意,不是指互联拓扑结构)上可分成一个一个的层次,从而可以得到一种分层结构或分层模型。每一层包含一组通信功能和相应的层间通信协议。协议不仅支持通信双方在不同的层间进行通信,而且提供了实现通信的具体思想和方法。

按照国际标准化组织(ISO)的建议,目前实际广泛使用的网络结构模型是开放系统互连模型(OSI)。这是一个七层协议,包括物理层、数据链路层、网络层、传输层、会话层、表示层和应用层。相应地,产生了七层协议,如图2.4所

示。

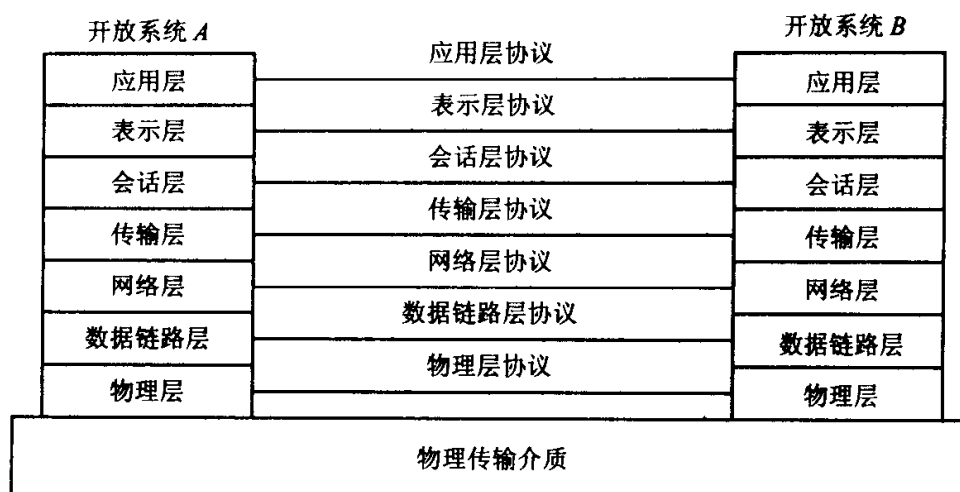


图 2.4 OSI 参考模型及同等层协议

物理层协议实现物理上互连系统间位流信息的透明传输，即实现一位（组）数据在两个通信实体之间的可靠传送通信，它描述了经通信介质在数据链路实体之间建立、维护和拆除物理连接。

数据链路层协议主要是对高层屏蔽传输介质的特性，为网络通信实体之间提供建立、维护和释放数据链路连接的功能和手段，实现无差错的数据以帧为单位的可靠传送。

网络层协议主要是为通信子网与高层结构之间提供界面连接，其主要任务是对通信子网实现路径选择，实现通信实体之间端-端的透明的数据传送，对高层屏蔽了数据传送经过的路径。

传输层协议也称为主机-主机层协议，它为会话层的通信实体之间提供透明的数据传送，其主要任务是接收会话实体送来的数据，根据需要把它们分成若干比较小的单元，保证所有数据单元经下面三层正确地到达另一个会话实体。

会话层协议也称进程-进程层协议，它通过协议提供的一组命令为网上两个进程之间的通信建立会话连接和释放会话连接，并管理它们在该连接上的对话。

表示层协议以对应用实体有意义的形式提供有关信息表示的服务。这些服务有文本压缩、代码转换、数据加密与解密、文件格式变换、信息格式变换、终端属性的转换等。

应用层协议是用户访问网络的接口层，直接为正在通信的端点用户的应用进程服务，如电子邮件、远程操作访问、虚拟终端等。

随着计算机网络与通信技术的出现，以及网络上各种用户和各种软件的开发，网络上的各种数据、软件等网络信息的安全问题日益突出。在国外，已经发

现了一些计算机爱好者通过破译密码进入网络上一些严禁局外人进入的软件系统。如两个中学生进入了美国国防部的计算机网，在如入无人之境地访问了一些结点之后安全地退了出来；各种计算机病毒通过计算机网络感染网上的一些结点计算机系统并使之瘫痪的事情也时有发生。通过上述方式窃取对手的各种情报，在必要时破坏对手的计算机网络系统已成为未来战争中电子对抗的一种重要方式和手段。针对这些问题，对信息安全技术，尤其是网络信息安全技术的研究被提到了一个重要的高度来认识。对计算机病毒，目前主要是研究各种防病毒技术和清病毒技术，据此开发出各种软硬件产品，并从道德和法律方面加强管理。就防止计算机系统及其通信中各种数据的失窃和篡改来说，主要是研究各种数据加密技术和解密技术。

近年来，伴随着计算机网络的普及，一个经常为人们谈虎色变的话题是所谓的计算机病毒。尽管计算机病毒早在很多年前就已为大众所熟悉，但是，在目前的病毒防范技术水平下，所有联网的计算机事实上处于很难设防的状态下。那么，什么是计算机病毒呢？计算机病毒是计算机上人为设计的一段程序，这种程序的运行可以干扰计算机上各种正常程序的运行，有的可通过改变正常程序的数据和指令使其出现错误，严重的甚至可以破坏程序的运行，或导致整个计算机（网络）系统瘫痪。从技术上说，计算机病毒是一种人为编制的对计算机用户无意义且独立或附着在用户程序、数据文件中的一个可运行的计算机程序，它通过各种方式进入计算机系统并通过某种方式被激活进入内存运行。一个计算机病毒是“良性的”是指它在被激活运行时并不破坏用户的数据文件、程序文本等软硬件设备，干扰用户的程序正常运行。大多数的计算机病毒都是“恶性的”，它们被激活运行后将导致用户的数据文件、程序文本和软硬件设备遭到干扰和破坏，严重的将导致整个计算机系统瘫痪。经过其他计算机系统拷贝来的文件和数据很容易被“感染”上病毒，从市场上购置的各种计算机软件产品也并非百分之百的可靠。

现在，计算机网络的普及又使得大量的计算机终端用户处在一种防不胜防的状态之下，病毒可以通过拷贝文件方式、用户通信方式、异地下载文件方式、受“黑客”攻击协议方式等不时地进入分布在世界各地的计算机系统中。我们经常可以在电视和报刊上看到发现了一种新的病毒报道。为了对付计算机病毒的泛滥，目前，各国政府主要是通过三方面的工作多管齐下。一是通过立法规范公民的行为，通过严肃执法惩治罪犯；二是从技术上开发防范病毒侵入的软硬件设备，开发检测并删除病毒的程序，如防病毒卡、网络防火墙、杀毒软件等；三是加强媒体的宣传报道，提高公民的道德水准。

2.13 高性能计算

在结束本章之前,我们还要提到近年来在学术刊物和大众传媒中使用频率很高的三个概念名词:高性能计算与通信(HPC&C, High Performance Computing and Communication),信息高速公路(Information Super-Highway)和数字地球。

通俗地说,高性能计算泛指向量计算、并行计算、分布式计算、网络计算和元计算等一类数据信息量大,运算速度快,算法效率高的计算。按照高性能计算机系统在运行过程中的运行特点,可以将高性能计算分为如下三种类型:

- (1) 计算密集性。主要面向大规模科学计算和多学科交叉大型工程计算;
- (2) 数据密集性。主要面向海量数据处理任务,如数字图书馆、数据挖掘,计算可视化等;
- (3) 通信密集性。主要面向元计算、远程医疗诊断、网络实时监控与数据采集处理等。

1991年,美国提出了为期五年的高性能计算与通信计划。该计划认为:现在计算已发展成为与实验(设计)、理论相并列的第三种科学研究方法(学科形态),在不少领域已取代了实验,如计算流体力学中的飞行器风洞实验,各种量级的核武器实验,模具设计实验,生态环境破坏与恢复实验,深部地质勘探实验等,而且随着计算机硬件、软件和算法的进步,这种趋势还会加强。

当今许多重大应用领域对科学计算、数据处理提出了更高的要求。如中长期天气预报,全球气候变化研究,地球物理勘探,空气动力学模型,受控核聚变与核试验,分子、原子、粒子结构探索,生态环境系统模拟,新材料与新药物的研制,新飞行器和武器系统研究,大气与海洋环境污染研究等都需要比现在的计算机更强的计算能力。这些方面的研究工作对国家及社会发展至关重要,它们形成了对计算科学的“重大挑战”。显然,发展高性能计算与通信工作以支持解决上述挑战问题势在必行。

高性能计算与通信包括:高性能计算机系统,先进的软件技术和算法,国家研究与教育网络,基础研究与人才资源。高性能计算与通信支持研究人员更多地参与科学、艺术、教育、文化、工程技术等多方面的创造性基础研究与交流活动,它将从根本上改变传统的科学技术研究方式。

通用高性能计算机系统必须在计算能力、存储能力、带宽、延迟及 I/O(输入/输出)等五个方面均衡发展,才能保证实际应用系统性能的提升。从 1976 年美国的 Cray-1 计算机以后的系统设计开始失去平衡,特别表现在 CPU(中央处理器)与 Memory(内存)之间数据存取的时间延迟,Cache(高速缓存)带宽、内

存带宽、I/O 带宽等方面发展滞后。同时为增强系统的可靠性、可用性、可维护性及性能价格比等性能，高性能计算机的设计师需要对电耗性、冷却系统、机器体积与运行环境、造价等因素进行综合考虑。

高性能计算与带宽高速通信计算机网络的出现，使得人们意识到地域之间的距离正在缩短，地球正在变得越来越小，人与人之间的交流就像大家共同住在一个村庄里，十分方便，许多人开始把我们生活的地球称之为地球村。于是，有人提出利用计算机、通信和互联网技术，建设数字地球。所谓数字地球，就是将地球按照坐标的方式划分成一系列由坐标位置确定的区域，利用图形、图像、文字等多媒体信息构成的有关该区域地理、政治、人文、经济、社会等各种数据信息，使人们无论处在什么地方，只要用计算机终端的鼠标点击某个区域的坐标，通过互联网络，就可以了解该区域的所有基本情况。数字地球，也就是用数字技术对真实地球的一种现实模拟得到的一个信息化系统。

随着高性能计算与通信计划的提出，以计算机软硬件技术、光纤通信技术和网络互联技术为基础的信息高速公路的概念和建设计划也被提了出来，并由此启动了全球信息社会的建设步伐。有了计算机网络和相应的各种软件，人们可以通过网络开展订票、购物、学习、互通信息、看电影、看新闻、账单结算等多种活动，许多在今天看来十分困难的事情将不再是天方夜谭。在中国，由于各方面的重视，正在建设“金桥”工程、“金关”工程、“金卡”工程、“金税”工程、“金企”工程，以后还将建设“金智”工程、“金农”工程、“金卫”工程，从政府到各行各业的一大批企事业单位都在积极上网，建立网络站点，提供网络站点信息，不少城市已经提出了信息化社会建设的规划，国民经济和社会的信息化进程正在加快。

第三章 计算科学：它的意义、内容和方法

学习和研究任何一门科学技术，人们首先关心的是这门科学技术的定义，它的意义、内容和方法。计算科学也不例外。

在开始本章的内容之前，我们首先介绍三个基本的术语，它们是科学、技术和工程。

科学是关于自然、社会和思维的发展与变化规律的知识体系。技术是泛指根据生产实践经验和科学原理而发展形成的各种工艺操作方法、技能和技巧。工程是指将科学原理应用到工农业生产部门中去而形成的各门学科的总称。本书中所讲的科学、技术和工程，若非特别说明，均指计算机科学、计算机技术和计算机工程。

3.1 什么是计算科学

国际著名计算科学家马纳（Z. Manna）曾在其著作《计算的数学理论（Mathematical Theory of Computation）》中一开头就写道：“什么是计算？我相信，世界上没有两个计算机科学家会就这一概念给出相同的定义。”尽管如此，我们还是要首先讨论计算和计算科学的概念，因为它们是阅读本书后续章节的基础。

计算科学是对描述和变换信息的算法过程，包括其理论、分析、设计、效率分析、实现和应用的系统的研究。全部计算科学的基本问题是，什么能（有效地）自动进行，什么不能（有效地）自动进行。本学科来源于对数理逻辑、计算模型、算法理论、自动计算机器的研究，形成于 20 世纪 30 年代后期。

计算作为数学的研究对象已有几千年了。计算不等于数学，但数学确实起源于对计算的研究。计算的渊源可以深入扩展到数学和工程。数学为计算提供了理论、方法和技术，而工程为实际计算和应用提供了可以能行自动计算的设备，并为更有效地完成计算和应用任务提供了工程方法和技术。

早期，工程与数学期长期合作的一个研究目标一直是试图制造一种能代替人完成计算任务的自动计算机器，如巴斯卡尔（B. Pascal）和莱布尼兹（G. W. Leibniz）的算术计算器，巴贝治（C. Babbage）的分析机，布什（V. Bush）的电子模拟计算机和德国人朱斯（K. Zuse）的机电式数字计算机。而近年来，工程与数学合作的主要目标除了试图制造性能更加优越的计算机器，制造一种能模拟人的思维和

智能行为的自动计算机器外，还包括为更好地开发和应用计算机提供工程和技术方法，如维也纳开发方法（VDM），程序证明技术，演化计算，各种新型计算机体系结构研究，计算可视化与虚拟现实，并行与分布式计算等。

数理逻辑是数学的一个分支，是研究推理的科学。自从欧几里德时代以来，逻辑一直是严格的数学和科学论证的工具。由于计算科学的发展，现在一般认为数理逻辑包含一个基础和四个研究领域，即命题逻辑与谓词逻辑，集合论，模型论，递归论，构造性数学和证明论，其中，命题逻辑与谓词逻辑成为集合论，模型论，递归论，构造性数学和证明论共同的基础，它们分别从逻辑演算，数学基础，构造性数学方法，算法计算，语言系统语法和语义的角度支持了数学和科学论证的科学性。现今对数理逻辑基础的介绍更多的是从直觉主义逻辑的观点出发，从语法和语义的角度刻画逻辑系统，而将逻辑系统的公理化作为一种形式系统从技术上加以处理。

从 19 世纪后期开始，不少人试图建立通用的逻辑系统，验证所有反映客观世界规律的定理，或从少数几条公理和推导规则出发，证明或发现和导出所有的未知定理。这方面的工作以大数学家希尔伯特（D. Hilbert）为代表，数学史上称之为“希尔伯特计划”。可惜，哥德尔 1931 年关于形式系统不完备性的论文宣告了这一幻想的破灭。哥德尔不完备性定理指出，任何一个有一定复杂程度的形式数学系统中，都存在这样一些合法的公式（定理），该形式系统既不能证明它成立，也不能证明它不成立，即不能在该系统内证明或否定所有合法公式（定理）。在哥德尔成果的影响下，几乎是在这同一时期，哥德尔、丘奇、波斯特和图灵等人从不同的角度正在独立地进行着各种计算模型的研究，以探索计算这一概念所包容的内涵。

当时的主要研究问题是怎样判断一类数学问题是否是机械可解的？这其中就涉及到计算模型，需要找到一种最基本的计算模型，以使判断机械可解的判定问题及其结果有一个共同的、合理的基础，以便确定可计算与不可计算的边界。哥德尔从数学演算的思想出发，考虑从简单的直观上可计算的函数出发构造复杂的可计算函数，提出了一般递归函数（类）的概念：凡由一组等式经过代入与替换所能计算的处处有定义的函数叫做一般递归函数。以后，丘奇和克林等人又在这方面做了许多工作，形成了今天的递归函数理论。丘奇也给出了一种计算模型，一种基于演算思想的函数演算，叫做 λ 演算。用函数演算来表达可计算性，必须要有一种能够非常严格地表达函数的抽象形式，因为演算中当函数是一种具有复合、递归、多元特征的函数时，演算不仅要涉及到函数的自变元，还会涉及到函数的值，这就必然要面对处理多个函数变量论域和多个值域的情况，需要有一种统一的函数表示形式。 λ 演算中采用的表示函数的方法称为 λ 表示法，它用一种

叫做 λ 表达式的形式来表示所有的函数。在 λ 演算中,可以从几个最简单的 λ 表达式(函数)出发,通过函数的换位演算(或称置换演算)来定义函数和判定函数的可计算性,包括函数值的计算。波斯特研究了符号的变换问题(这实际上也是一种逻辑推导),提出了著名的波斯特符号变换系统,简称波斯特系统,为程序设计语言奠定了重要的理论基础。不过,这些产自不同背景的计算模型后来都被证明与图灵机在计算能力上是等价的。在这些计算模型中,以图灵的研究与实际计算机更为接近。他提出的形式化的理想计算模型深刻地揭示了计算这一本质概念,将可计算性这一概念同机械程序和形式系统的概念统一起来,为能行理论奠定了基础。在图灵机这一计算模型出现之后,许多人又从不同的角度对可计算进行了深入的研究,相继提出了一批计算模型。其中,有些计算模型具有很强的计算能力。然而,直到今天,绝大多数关于计算科学理论和技术的研究仍然没有跳出图灵机所确定的范围。数理逻辑之所以重要,不仅在于它支撑了计算科学理论基础——递归函数论的发展,揭示了计算的极限,而且在支撑整个学科的发展中发挥了重要的作用,成为计算科学大多数重要分支方向共同的基础。

演算与算法是处理计算过程包括推理过程时一般最常用的两大形式,当然还有归纳、类比等形式。但是,长期以来,在数学研究中,由于分析技术比较发达,人们常常倾向于和习惯于采用演算的思想而忽视了算法的思想。图灵所以取得成功,很重要的一条是他在研究中采用了算法的思想来研究计算的过程,并由此揭示可计算的概念。由于算法的思想与今日计算机系统上运行的程序之间有着更密切的关系,从而使他的理论受到重视并被广泛使用。

围绕着计算科学的教学与科研,长期以来国内外一直对计算机科学与技术究竟属于科学还是属于工程的范畴这一问题存在着争议。这是一个困扰国内外计算科学界很久且争论不休的问题。由美国计算机协会(ACM)和国际电气、电子、工程师学会计算机学会(IEEE/CS)联合小组的二十几名专家从1985年起用了五年的时间进行了深入的调查和分析,他们的结论是:计算机科学和计算机工程之间在本质上没有区别,两者是一回事。联合小组的专家之所以得出这一结论,其深刻的学科内在原因在于与许多别的学科不同,计算科学所研究的全部问题的核心恰是能行问题,能行问题贯穿在整个学科包括硬件和软件在内的理论、方法、技术的研究,以及应用各个方向的研究与开发之中。而且,学科的科学、技术、工程的方法论的主要基础——以离散数学为代表的构造性数学与能行性形成了天然的一致。由于这一原因,代表计算机科学的各分支学科的理论、技术理论(含方法)和代表计算机工程的各分支学科的工程(含开发方法)和工程技术(含技艺和技巧)常常既有理论特征,又有技术特征,甚至还具有工程特征,三者相互之间的界限往往不很清楚。但本质上,它们大都是从不同的角度和层面对各种问

题的能行性及其求解方法和过程的描述，是通过对各种问题反映其能行性的内在规律的描述折射出求解方法和求解过程的描述。在统一了认识之后，近年来，国外使用计算科学涵盖并称谓计算机科学、计算机技术和计算机工程。

在计算科学发展的历程中，随着学科的兴起，陆续产生了一些学术团体。国际知名的计算科学学术团体主要有美国计算机协会（ACM），国际电气、电子、工程师学会计算机学会（IEEE/CS），国际信息处理联合会（IFIP），美国人工智能协会（AAAI），以及由一些国家人工智能学会和协会组织的国际人工智能联合会议（IJCAI）等。在中国计算机科学界，中国计算机学会是目前最有影响的全国性一级学会。影响较大而又比较重要的国际学术会议有世界计算机大会（四年召开一次），IJCAI 年会（两年一次），ACM 年会（一年一次），AAAI 年会（一年一次），欧洲理论计算机科学年会（一年一次，每年会议主题常不同）等。除学术会议的会议论文集外，由学会和一些著名的出版社定期出版有若干计算科学第一流的学术刊物，其中包括与数学有关的刊物。这些刊物中处于领头和重要地位的主要有：

Information and Computation（本学科领头学术刊物，原为 **Information and Control**, Academic Press: 学术出版社（美国）出版，简称 **AP**（美国）出版，下同）；

J. of ACM（本学科领头学术刊物，ACM（美国）出版）；

J. of Computer and System Sciences（本学科次重要学术刊物，AP（美国）出版）；

J. of Symbolic Logic（本学科次重要学术刊物，Association for Symbolic Logic 出版）；

Mathematical Systems Theory（本学科次重要学术刊物，Springer-Verlag（德国）出版）；

IEEE Transaction on Computer（本学科次重要学术刊物，IEEE 协会出版）；

Artificial Intelligence（本学科次重要学术刊物，Elsevier Science（荷兰）出版）；

Cognitive Sciences（本学科次重要学术刊物，AP（美国）出版）；

J. of Algorithms（本学科次重要学术刊物，AP（美国）出版）；

New Generation Computing（本学科次重要学术刊物，Springer-Verlag（德国）出版）；

SIAM J. on Computing（本学科次重要学术刊物，Society for Industry and Applied Mathematics 出版，简称 **SIAM** 出版）；

Theoretical Computer Science（本学科次重要学术刊物，Elsevier Science（荷兰）出版）；

Science of Computer Programming（本学科次重要学术刊物，Elsevier Science（荷兰）出版）；

J. of Parallel Programming（Plenum（荷兰）出版）；

J. of Human-Computer Studies (原 J. of Man-Machine Studies, AP (美国) 出版);
Computing Surveys (ACM (美国) 出版);

Information Sciences (North-Holland (荷兰) 出版);

J. of Computer and System Sciences International (英译俄刊, Scripta Technica 出版。前苏联时为 J. of Computer and System Sciences)。

中国计算机学会最重要的学术期刊有三种, 它们是:

计算机学报 (科学出版社出版);

J. of Computer Science and Technology (计算机学报(英文版), 科学出版社出版);

软件学报 (中国科学院软件研究所出版);

Advanced Software Research (软件学报(英文版), Allerton Press 出版, 已停刊);

计算机研究与发展 (科学出版社出版)。

进入 20 世纪 90 年代, 国外陆续创刊了一批新的计算科学学术刊物, 主要涉及并行计算、人工智能、神经元计算、网络与通信、软件工程、计算几何、计算可视化等较新的学科方向。

虽然从根本上说, 学术论文的水平 and 影响是由学术刊物的内容决定的, 而不是由学术刊物的级别决定的, 但是, 学术刊物在水平上存在着明显的等级是一种客观存在, 因为高水平的学术刊物的论文水平是由一大批在本学科与该刊物办刊领域一致的方向上工作的科学家的学术水平保证的。作为一个严肃的学者, 应该努力争取将自己可以公开发表的学术论文在权威刊物发表, 而不是以各种理由强调或误以为高水平杂志仅发表理论性的文章。事实上, 国内外许多杂志都发表工程技术研究论文, 世界上非军事院校中任何一所高水平研究型大学的大量创新科学研究成果主要以学术论文的形式在各类高水平学术刊物上发表。如果排除了各种非学术性因素后, 高水平的杂志之所以学术水平高, 是因为这些杂志发表的大多数学术论文在研究成果的科学意义、创造性、完整性 (或正确性)、学术深度四个方面, 可能还有行文质量等方面综合来看达到了较高的水准。

20 世纪 90 年代中期, 中国国务院学位委员会以文件形式发布通告, 今后, 学位委员会 (计算机科学与技术) 学科评审组在审议、评估一个学校博士、硕士学位点学位授予权和办学质量时, 对于中文重要刊物认定了 7 个杂志, 它们是: 《中国科学》、《科学通报》、《计算机学报》、《软件学报》、《计算机研究与发展》、《电子学报》、《自动化学报》。第一、二份杂志是综合性刊物, 第六、七份杂志是相近学科杂志, 它们都可能刊登一些计算科学的研究论文和技术报告, 支持较高水平的跨学科研究。

出版计算科学学术著作的重要出版社主要有美国 AP 出版社, 德国

Springer-Verlag 出版社, 荷兰 Addison-Wesley 出版社, 荷兰 Elsevier Science 出版社, 美国 Pitman 出版公司, 美国 ACM 出版公司, 荷兰 North-Holland 出版社, 荷兰 Morgan Kaufmann 出版社, 美国 MIT 出版社; 国内主要有科学出版社、国防工业出版社等几家出版单位。近年来, 由于市场化竞争和其他一些因素, 国内学术著作出版有分散化的趋势。但对于一个严肃的学者来说, 依然比较看重科学出版社和国防工业出版社。在有关部门的支持下, 科学出版社和国防工业出版社先后建立了面向全国科学技术学术界的学术著作出版基金, 专门用于支持高水平学术著作的出版。

国内计算科学教材的出版目前是一种群雄割据的时代, 除了中央一级的出版社之外, 许多地方的出版社和大学的出版社也在争相出版计算科学教材。其中, 出版计算科学教材较多, 质量较好, 影响较大的出版社主要是: 高等教育出版社、清华大学出版社、电子工业出版社、科学出版社、国防工业出版社、机械工业出版社、人民邮电出版社。其中, 以机械、电子、清华、高教、人民邮电为代表的一些出版社近年来还系统地引进了许多国外原版英文教材, 有的已组织力量翻译出版了中文版。

值得一提的是, 在这些出版社中, Springer-Verlag 出版社长期连续出版了一套计算科学系列图书, 系列名称为 *Lecture Notes in Computer Science*, 内容包括各分支学科的专著, 高水平的重要国际学术会议论文选集, 某一方向近年来重要学术论文或成果(学术报告)的汇编等。其中, 关于人工智能专题的图书后来使用 *Lecture Notes in Artificial Intelligence* 名称作为一个子系列。至今, 该系列已出版 2000 多本, 在学术界有重要影响, 不过, 最近五年多来, 该系列将许多学术水平一般的学术会议的论文集收录在其中, 系列图书的学术水平出现良莠不齐的现象。新加坡也有出版社系统地出版计算科学的学术专著和学术会议的会议录, 特别是注重出版一些华裔计算科学家的著作, 其中有一些著作水平较高。

3.2 学科的基本问题

任何一个学科的发展都是由该学科需要解决的科学问题驱动的。如果一个学科没有了科学问题, 该学科发展的生命也就终止了。对于一个正在发展中的学科, 需要解决的科学问题是很多的, 形式上也可能是五花八门的。一个从事学科发展研究工作的科学工作者在具体研究工作中, 首先需要在前人工作的基础上发现科学问题, 然后才谈得上用科学的方法来解决问题。作为一个科学工作者, 要学习一个学科的科学理论知识, 可以通过书本知识和实践, 了解科学问题和解决问题的科学方法、科学原理和技术, 要发现一个学科的科学问题, 可以也应该到这个

学科具体的分支学科各个领域、方向上去寻找,到生产实践中去寻找。依靠自己的基础知识和科学的思想方法去发现问题,解决问题。至于如何去发现某个领域、某个方向的科学问题并具体解决问题,掌握这方面的基本知识,这些问题都应该在相关的各门课程的教学与实践中去解决。

现在的问题是,对于本书的大多数读者,一大批没有理工科基础知识和学科专业基础知识,刚刚进入大学的一年级学生,如何来了解这个学科,认识这个学科,学习这个学科呢?如果我们认同“工作就是学习,学习就是工作,”那么,对于一个初学者,应该如何从研究工作者的角度来了解这个学科,认识这个学科,学习这个学科呢?

我们已经知道,任何一个学科都是伴随着不断发现问题、解决问题而发展起来的,因此,我们有理由相信,认识一个学科应该从认识这个学科的问题开始。可是,对于一个知识体系已经十分庞大,发展速度相对比较平稳的学科,面对大量的科学问题和围绕解决这些问题而发展起来的科学理论、方法和技术,一个几乎没有什么基础的初学者要了解和很好地掌握这些内容,抓住重点是一个十分困难的问题。学生因为不了解计算科学而需要学习计算科学,但要按照科学的方式来学习计算科学,又需要对整个学科有较好的了解。这似乎是一个先有鸡还是先有鸡蛋的问题。怎么办?科学告诉我们,当我们在一个认识体系内解决不了这个体系内的某个问题的时候,应该跳出这个体系,借助于其他的体系来解决这个问题。所幸的是科学哲学的发展为读者铺平了一条认知一个学科的道路。科学哲学使我们可以从科学哲学的观点出发,以学科方法论作为认知一个学科的切入点,从学科的基本问题和重大问题入手,通过系统了解整个学科的来龙去脉,总结学科发展的内在规律,从整体上把握一个学科,指导我们和学生认识一个学科的实践活动。

在一个学科的发展历程中,围绕学科发展的目标会产生一系列的科学问题,其中,问题的提出和解决对这个学科的发展能够产生重大影响的问题称之为重大问题。在学科发展的不同时期,重大问题是相对的,也是比较多的。围绕重大问题的解决,整个学科会形成若干条发展的主线和若干主流方向。

基本问题是科学哲学层面的概念,它不是针对一个学科的具体问题提出的,而是在一个学科一大类具体问题的基础上,通过分析、研究这类问题共同的特点和本质属性,经过总结、提炼得到的更一般化的抽象的问题,学科中的许多具体问题是某一抽象的基本问题在该具体问题所属领域内的一个具体的实例。例如,在数学中,存在性问题、惟一性问题和稳定性问题就是数学的三个基本问题。要解一个微分方程或线性方程,先要考虑这个方程是否有解;要求出一个函数的极值,先要看这个函数是否有极值;要找到函数的一个不动点,先要分析这个函数

是否存在不动点；要求出一个矩阵的逆矩阵，要找出线性空间上一个线性变换的逆变换，要找到一个曲面上的奇点，要寻求一个有向图中的一个回路，要求出一个集合中的最大元，等等，首先需要回答数学对象是否存在，即矩阵是否可逆，是否存在逆变换，曲面上是否有奇点，等等。这类问题的内容都涉及到数学中的存在性问题：一个方程（组）在有解的情况下解是否是惟一的？一个函数的极值和零点是否是惟一的？一个图中给定的两个顶点之间的最短路径是否是惟一的？一个可逆矩阵的逆矩阵是否是惟一的？这类问题的内容都涉及到数学中的惟一性问题：一个微分方程是否是稳定的？一个曲面的曲率怎么样？这类问题的内容都涉及到稳定性问题。问题可能来自不同分支学科，不同领域，不同方向，问题的含义也不相同，但它们的共同特性或特征之一是存在性或惟一性或稳定性。显然，存在性问题、惟一性问题、稳定性问题是高度概括了这样一大类具体的数学问题，并准确反映这类问题共同特性或特征的更为抽象的问题，即数学的基本问题。因为按照数学的这些基本问题，我们可以在数学的各个不同的分支学科、领域和方向上，针对数学研究的各种对象，找到大量数学的具体问题。例如，在集合论中，针对一个集合，集合上定义的一个关系和极大元的概念，我们可以提出这个集合是否存在极大元的问题。又如，在构造性数学的研究中，针对一个函数和它的算法计算问题，我们可以提出该函数是否存在构造性计算模型（指计算方法）的问题。解决了存在性问题，数学上还需要对被研究的数学对象考察其惟一性、稳定性问题，等等。读者可以结合自己所学过的中学数学知识和大学数学知识进行思考和体会，此处不再展开讨论。

从上面的例子和分析我们可以看出，学科的基本问题是一个非常重要的概念，它能够帮助我们认识一个学科，帮助我们发现这个学科的具体问题。至于如何从一个学科的基本问题出发，考察和发现这个学科的具体问题，发展和运用科学的方法解决具体问题，丰富学科领域的科学知识，推动学科的向前发展，同时建立、丰富和完善自己的科学思想方法，读者可以在第四章了解一般的思想方法，从本专业的后续课程中逐步、分别地学习和了解大量具体的内容，在今后的实践中锻炼提高。

那么，哪些是计算科学学科发展的重大问题和基本问题呢？

重大问题是比较容易理解的，而且，相对于不同时期，重大问题既是相对的，也是比较多的。例如，学科发展早期提出的什么是可计算与不可计算的概念，20世纪50年代末60年代初提出的高级程序设计语言的形式化描述问题，60年代末70年代初提出的操作系统中的并发控制问题，等等。然而，在学科经历了几十年的发展后，当我们今天以科学哲学的观点回顾历史的进程，系统总结学科的内容时，可以发现：如同数学等一些基础学科一样，在学科各个分支学科方向的发展

进程中,不断地出现了一些在表现形式上虽然不同,但在科学哲学的解释下本质上是相同或相近的问题,即学科研究与发展普遍关心的基本问题。这些基本问题主要有如下三个:

- (1) 计算的平台与环境问题;
- (2) 计算过程的能行操作与效率问题;
- (3) 计算的正确性问题。

计算的平台与环境问题是不难理解的。历史上,为了实现自动计算,人们首先发明和制造了自动计算机器。在实际的探索过程中,前人逐渐认识到,研究自动计算不仅要从理论上提供计算的平台——观察和描述计算的起点,用于解题或者证明问题本身不可解,而且要实际制造出针对各种待处理问题特点和要求的自动计算机器,这就产生了计算模型、计算机原理与计算机体系结构的研究。那么,是否所有的计算模型都是能行的呢?答案是否定的。众所周知,只有构造性计算模型才有可能是能行的,这类计算模型所采用的构造性数学描述方法保证了计算的能行性。但是,并不是凡具有能行性特点的计算模型都能够造出具体的自动计算装置,如非确定性计算模型。这说明,计算模型的研究从应用的角度考虑,必须注重区分计算模型在理论上的能行性和现实中的能行性。进一步,从广义计算的概念出发,计算的平台在使用上还必须比较方便,于是派生出计算环境的概念。据此,不难看出,研究中提出的各种计算模型、实际的计算机系统、高级程序设计语言、计算机体系结构、软件开发工具与环境、编译程序与操作系统等都是围绕解决这一基本问题发展而来的,都属于计算的平台与环境问题的研究,其内容实质可归结为各种计算模型的研究。换言之,这个基本问题实际上关心的是计算问题在理论上和/或现实中是否能行和使用方便的问题。

计算过程的能行操作与效率问题也是学科的基本问题之一。一个问题在判明为可计算的性质后,从具体解决这个问题着眼,必须按照能行可构造的特点与要求,给出实际解决该问题的一步一步的具体操作,同时还必须确保这样一种过程的开销成本是我们能够承受的。围绕这一问题,学科发展了大量与之相关的研究内容与分支学科方向。例如,数值与非数值计算方法,算法设计与分析,结构化程序设计技术与效率分析,以计算机部件为背景的集成电路技术,密码学与快速算法,演化计算,数字系统逻辑设计,程序设计方法学(程序推导与程序变换技术),自动布线,RISC技术,人工智能的逻辑基础等分支学科的内容都是围绕这一基本问题展开、发展而形成的。显然,计算操作的能行与效率问题也涉及到了计算模型的问题,但是,这里所说的计算模型既包括类似于具有状态转换特征的那一类计算模型,也包括数学建模这类计算模型。不难看出,这一基本问题的核心是算法问题。这也辅助说明了在计算科学发展的早期,当计算的平台与环境、

计算的正确性问题尚不突出时，为什么学术界将本学科看成是算法的学问。

计算的正确性是一切计算工作都不能回避的问题，特别是使用自动计算机器进行的各种计算。一个计算问题在给出能行操作序列的同时，必须确保计算的正确性，否则，计算是无意义的。围绕这一基本问题，长期以来，学科发展了一些相关的研究内容与分支学科，例如，算法理论（数值与非数值算法设计的理论基础）、程序设计语言的语义学、程序理论（程序描述与验证的理论基础）、程序测试技术、电路测试技术、软件工程技术（形式化的软件开发方法学）、计算语言学、容错理论与技术、Petri 网理论、CSP 理论、CCS 理论、进程代数与分布式事件代数、分布式网络协议等都是针对为解决这一基本问题而发展形成的。由于数学为各种计算方法提供了计算正确性的理论基础，而各种计算在计算机系统上的自动进行均采用语言（包括电路）描述，以程序或电路系统的载体形式出现，因此，计算的正确性问题常常归结为各种语言的语法和语义问题，研究的方式方法一般为先发展某种合适的计算模型（如开关电路），用计算模型来描述各种语言的语法和语义。特别，由于描述和实现计算离不开语言（包括电路），而语言的词法、文法已经比较成熟，当前的难点集中在语义，因此，这也从一个侧面揭示了计算的正确性问题常可以归结为语言的语义学问题，揭示了语义学在整个学科中的重要地位。

应该指出，上述基本问题并非仅出现在某些分支学科方向，而是普遍出现在各个分支学科方向之中，并且常常交织在一起，是学科研究与发展中经常面对而又必须解决的问题。学科的三个基本问题，从本质上分析，又都具有能行特点，可以归结为能行性问题。首先，计算的平台与环境问题可以归结为计算模型问题。根据计算概念的定义，计算模型必须能行。这样，第一个基本问题就常归结为计算模型的能行问题。其次，根据上述分析，计算过程的能行操作与效率问题可以归结为基于某一恰当计算模型的算法问题。按照算法的定义，算法问题的研究实际上分成两个方面，一个是算法的设计描述问题，另一个是算法的效率问题。由于所讨论的算法都是基于某种计算模型之上的算法，故计算模型的能行性决定了这个问题本质上可归结为一个理论上的能行过程描述问题和现实条件下的能行问题。第三，计算的正确性问题也可以归结为能行问题。与上述问题不同，前两个问题是从实际解决问题的角度出发，客观上要求计算模型与算法研究应该注重能行性，而计算的正确性研究客观上并不要求一定要满足能行性，只要能够判定计算是正确的即可。但是，由于计算的正确性问题是在上述两个基本问题研究的基础上针对一类诸如程序、逻辑公式、电路系统等对象进行语义判定，而前述基本问题的能行性特点决定了从过程的角度采用外延和内涵的方法进行语义分析研究时，必然使计算的正确性研究具有能行性特点。而且，目前大多数计算语义学研

究从将来语义判定自动化的角度考虑,先后引入了一批成熟的能行计算模型和/或发展了一批构造性数学方法,将语义描述与判定融入在描述计算的过程中。因此,迄今为止,大量关于语义学研究的成果都具有能行性的特点。所以,限于目前学术界对计算的认识,计算的正确性问题可以归结为能行问题。至于计算的正确性问题能否采用非能行的方法或非构造性数学方法进行研究,目前尚未见报道。

显然,通过上面的分析,我们可以看出,计算科学的定义是恰当的、准确的,因为学科的所有基本问题全部可以归结为更一般的能行性问题,它与学科的定义是完全一致的。循着这一线索,我们很快将看到,整个学科正是在着眼于学科发展目标,面对解决各个时期的重大问题和热点问题,围绕学科的这些基本问题而展开的大量具体研究与发展中形成了学科发展的主流方向与学科发展主线。

3.3 计算科学发展主线

着眼于学科发展目标,面对学科各个时期的重大问题和热点问题,围绕学科的基本问题而展开的大量具体研究与发展中形成了学科发展的主流方向与学科发展主线,形成了学科自身的知识组织结构。下面我们就来讨论这些内容。

3.3.1 计算科学的知识组织结构

每一个学科都有其自身的学科知识组织结构,计算科学也不例外。经过半个多世纪的高速发展,计算科学已经形成一个分支学科众多,知识组织结构庞大,内容丰富,学问很深的一个学科。在20世纪的科学史上,还从来没有一个学科能够像计算科学那样高速发展,能够渗透到社会生产、生活乃至人类活动的几乎每一个领域和大多数微观层面,对科学技术的发展,生产水平的提高,社会文明的进步,人类生活和行为方式的改变产生如此深远的影响,并成为20世纪下半叶和21世纪的领头科学。

在上一章初步介绍了计算科学一级学科最重要也是最基本的概念和知识之后,本节我们首先来考察一下计算科学的知识组织结构。通过对学科知识组织结构内容和特点的一些了解,有助于读者在后面的章节中了解学科发展的历程,学科知识组织结构的形成和变化规律。需要指出的是对学科知识组织结构的内容、特点、结构的阐述,不同工作背景的学者可能存在不同的认识和看法,但大同小异,因为学科知识组织结构是一种客观存在的现象,其形成和变化包含着许多学科发展的客观规律,不以人的意志为转移。从一个学科的定义、学科发展的目标、学科发展的主线和学科知识组织结构的形成及其变化规律,可以感触到整个学科发展的脉络,并为进一步探索和认识学科发展的内在规律、特点建立必要的基础。

我们将计算科学的学科内容按照基础理论、基本开发技术、应用以及它们与硬件设备联系的紧密程度分成三个层面，见图 3.1。

计算科学应用层	移动计算与全球定位	计算机自动控制	计算机辅助制造	计算机集成制造系统
	机器人	计算可视化与虚拟现实	数据与信息检索	计算机创作
计算科学专业基础层	科学计算	多媒体信息系统	计算机辅助设计	信息、管理与决策系统
	模式识别与图像处理技术	计算机图形学	计算几何	人工智能与知识工程
计算科学基础层	数据表示与存储	网络与开放系统互联标准	软件测试技术	人机工程学（人机界面）
	软件开发方法学：软件工程技术、程序设计方法学、软件开发工具和环境、软件开发规范	编码理论	密码学	计算机体系结构
数学与物理学层	程序理论	数据表示理论与数据库系统	电子计算机系统基础	计算机接口与通信
	故障诊断与器件测试技术	容错技术	汇编技术	操作系统
其他学科	高级语言	程序设计	数字系统设计	符号计算与计算机代数
	数据系统技术	算法设计与分析	编译与解释技术	控制论基础
自动化技术、电子科学等	数字系统设计基础	形式语义学	网论（Petri 网理论等）	信息论基础
	框图理论	算法理论	可计算性（递归论）	计算复杂性
	程序设计语言理论	计算模型（各种抽象机）	模型论与非经典逻辑	公理集合论
	形式语言与自动机	光电子技术基础	电路基础	电子线路基础
	数字与模拟电路基础	数值分析与计算方法	大学物理学	函数论基础（复变函数、 λ 演算、泛函分析等）
	泛代数	概率与数理统计	常微分方程	偏微分方程
	集合论与图论	组合数学	抽象代数	数理逻辑基础
	空间解析几何	数学分析	布尔代数	高等代数
	数论	计算机硬件与应用	计算科学理论	计算机软件与应用
	数据与信息表示理论、信息处理等	其他学科		

图 3.1

第一层面是计算科学的应用层，它包括人工智能应用与系统，信息、管理与决策系统，移动计算，计算可视化，科学计算等计算机应用的各个方向。其中，人工智能应用与系统涵盖人工智能，机器人，神经元计算，知识工程，自然语言处理与机器翻译，自动推理等方向；信息、管理与决策系统涵盖数据库设计与数据管理技术，数据表示与存储（包括多媒体技术），数据与信息检索，管理信息系统，计算机辅助系统，决策系统等方向；计算可视化涵盖计算机图形学，计算几何，模式识别与图像处理等方向。

第二层面是计算科学的专业基础层，它是为应用层提供技术和环境的一个层面，包括软件开发方法学，计算机网络与通信技术，程序设计科学，计算机体系结构，电子计算机系统基础。其中，软件开发方法学涵盖顺序、并行与分布式软件开发方法学，如软件工程技术，软件开发工具和环境等方向；计算机网络与通信技术涵盖计算机网络互联技术，数据通信技术，网络计算，以及信息安全技术等方向；程序设计科学涵盖数据结构技术，数值与符号计算，算法设计与分析（包括并行与分布式算法设计与分析），程序设计语言，程序设计语言的文法与语义（指

程序设计语言的文法与语义描述), 程序设计方法学, 程序理论等方向; 电子计算机系统基础涵盖数字逻辑技术, 计算机组成原理, 故障诊断与器件测试技术, 操作系统, 编译技术, 数据库系统实现技术, 容错技术等方向。

第三层面是计算科学的基础层, 它包括计算的数学理论, 高等逻辑等内容。其中, 计算的数学理论涵盖可计算性(递归论)与计算复杂性理论, 形式语言与自动机理论, 形式语义学(主要指代数语义, 公理语义等), Petri 网理论等方向; 高等逻辑涵盖模型论, 各种非经典逻辑与公理集合论等方向。

给出这样一种结构并不等于说图中所列都属于计算科学的范畴。有些内容实际上在整个学科中仅仅只用到少量的概念和一些简单的浅层知识, 或是属于跨学科的研究范畴, 如控制论基础、机器人、编码理论、信息论等, 它们主要属于其他学科而不是计算科学, 但为了比较完整地反映计算科学整个学科的知识组织结构, 我们将这些内容罗列出来以帮助读者更全面地理解。我们相信, 随着读者对计算科学知识体系的逐步、系统地学习, 必将能够更准确地了解整个学科。

支撑这三个层面的是计算科学这一学科的理工科基础科目, 包括物理学(主要是电子技术科学)、基础数学(含离散数学)等。近年来, 学科基础有扩大的趋向。

任何一门学科都存在许多分支学科和研究方向, 然而, 在学科的发展过程中, 不同时期, 面对学科的一些重大问题和热点问题, 围绕学科的基本问题而展开的各种具体研究与发展中, 若干方向便构成了所谓的主流方向, 由主流方向又形成了学科发展的主线。

在上述三个层面构成的计算科学发展的历程中, 不断地追求制造出各种新型计算机系统, 拓展和提高计算机的应用领域和应用水平这样两个目标使学科的发展形成了三条相对独立的主线, 它们是计算模型与计算机, 计算模型、语言与软件开发方法学, 应用数学与计算机应用。下面, 我们将围绕学科的这三条主线, 沿着学科发展史的脉络与轨迹, 介绍学科的发展及其主要分支学科的形成, 从中读者也可以了解到学科在各个时期发展的主流方向以及学科发展主线的形成过程。

需要指出的是, 不同的学者, 由于知识背景不同, 学术观点不同, 对计算科学学科发展主线的认识也不一样。这里, 仅仅是根据作者对计算科学的认识和体会提供一种参考意见, 同时, 也便于本书的写作与阅读。

3.3.2 计算模型与计算机

人类社会最早使用手指、结绳、算筹等方式进行计算, 古埃及尼罗河流域由于经常发生洪水泛滥而产生了丈量土地的需要。随着生产的发展和交流的增加,

又发明了算盘——一种最早的计算工具。其中，又以中国人发明的珠算盘成为当时最先进的计算工具。欧洲文艺复兴运动的发展带来了人文主义思想家和科学家对宗教统治的挑战，哥白尼的新学说使天文学的发展进入了一个新的时期。当时，科学研究中的各种计算问题特别是天文学中遇到的大量繁重的计算工作，使得改革旧的计算工具成为一个亟待解决的问题。从那时开始，制造一种能代替人进行计算的机器一直是好几代科学家的梦想。

钟表业的产生和发展，特别是齿轮传动装置技术的发展为机械传动装置计算机的产生提供了重要的技术基础。1623年，威尔海姆·什卡尔特（Wilhelm Schikard）在写给天文学家开普勒（J. Keplen）的信中，最先提出了一台计算机的构思设想，它主要由加法器、乘法器和记录中间结果的机构等三部分组成。可惜，样机尚未完成就被大火焚毁，其思想也很少为后人所知。

法国人巴斯卡尔对计算工具的研究迈出了重要的一步，他于1642年基于齿轮技术制造了一台能够进行加法和减法运算的计算器。这种计算器在今天看来虽然很简单，但它的设计思想比较符合人类的思维习惯，其原理成为后来广泛使用的手摇计算机的基本原理。为了纪念巴斯卡尔，20世纪60年代由语言大师沃斯（N. Wirth）设计的一种高级程序设计语言取名为Pascal。

在巴斯卡尔的思想和工作的影响下，德国人莱布尼兹也致力于计算机器的研究。他于1672年提出了不用连续相加进行机械乘法的思想，并于次年在巴黎科学院表演样机成功。莱布尼兹最重要的成就是提出了改进计算机的设计思想，为手摇计算机的发展奠定了理论基础。莱布尼兹对计算科学的另一个重要贡献是系统地建立了二进制的算术运算法则，指出了二进制在某些理论研究中的优点，为现代计算机系统的发展作了部分理论上的准备。

巴斯卡尔和莱布尼兹的成功使不少人开始从事计算机的改进工作。但由于当时技术的发展落后于计算机设计思想和理论的发展，研制的样机性能不稳定，限制了计算机的发展，从而使比较实用的计算机直到一百多年后才由法国人托马斯（C. Thomas）研制成功并建厂批量生产。巴斯卡尔和莱布尼兹的这类计算机根本的缺陷是只能做简单的四则运算，没有程序控制的机构。

提出用程序控制计算的思想和第一人是英国数学家巴贝治。在现代电子计算机诞生一百多年前，巴贝治在其设计、制造成功的差分机——一种计算机的基础上，提出了几乎是完整的通用数字计算机的设计方案。方案中的计算机主要由齿轮式寄存器、运算装置和专门控制操作顺序的机构三部分组成，非常接近于今天的计算机。可惜，因为得不到政府的经费支持，研制工作没能成功。为了纪念巴贝治作为计算科学的先驱者对学科发展的贡献，总结前人的经验和教训，支持信息处理史的研究，1977年在美国的明尼苏达大学建立了巴贝治信息处理史研究所

(CBI)。

值得一提的是，在计算机研制的发展史上，物理学的发展曾吸引了一批探索者从另一个角度研制计算工具，即研制模拟机。虽然，经过几十年的努力，到20世纪的二三十年代已取得了很大的进展，研制成功的模拟机中最具代表性的有布什的微分分析仪，然而，由于模拟机在通用性、精确度以及速度方面存在的局限性，当数字计算机的研究取得成功后，人们的注意力很快便转到了电子数字计算机的研究之中。不过，模拟机在许多测量和控制问题中仍然有着广泛的应用。今天，人们常通过设计模/数、数/模(A/D, D/A)¹⁾转换接口来处理模拟问题。当然，这是后话。

19世纪的英国，产生了一位杰出的数学家布尔(G. Boole)。□他系统研究了逻辑思维的一般规律，成功地将形式逻辑归结为一种代数演算，即布尔代数。虽然他的一生没有考虑怎样把布尔代数同计算机相联系，但他的工作却为现代计算机的产生与发展提供了重要的理论准备。

布尔代数实现了从一组逻辑公理出发，依靠代数演算来推导逻辑定律或定理，用数学的方法来研究人的思维结构。对数学来说，布尔代数是一个代数系统，它可以建立在集合论的基础之上，通过引入的代数运算规则来研究集合的运算和性质，并由此揭示人的一些重要的逻辑思维的方式方法。对逻辑来说，布尔代数可对应于命题演算，反映逻辑推理的思维结构，可将逻辑研究转化为用严格的数学方法来进行。以后，由皮尔斯(C. S. Peirce)和米切尔(O. H. Mitchell)在19世纪后期扩展到谓词演算的形式。施罗德(E. Schröder)、洛文海姆(L. Löwenheim)和斯科伦(T. Skolem)在这一传统领域也作出了杰出的贡献，最后，由哥德尔在1930年完成了谓词演算系统的完备性，数理逻辑基础得以完整地建立起来。在此基础上，经过许多人的努力，进一步发展形成了一门新的数学分支——数理逻辑。

在布尔工作的基础上，随着数理逻辑的发展，不少人试图建立通用的逻辑系统，验证所有反映客观世界规律的定理，或从少数几条公理和推导规则出发，证明或发现和导出所有的未知定理。大数学家希尔伯特是所谓数学机械化中这种思想的积极倡导者。可惜，哥德尔1931年关于形式数学系统不完备性的论文宣告了这一幻想的破灭。哥德尔的定理指出了这样一个基本事实，具有一定复杂程度的完备的形式数学系统是不存在的。哥德尔的成果使许多数学家感到困惑，可计算与不可计算是那样地令人不可捉摸。于是，关于可计算的概念研究成为当时一些

1) 即将模拟信号转换为数字信号，或将数字信号转换为模拟信号的转换器件或设备。其中，模拟信号可以是电波的连续波形信号，也可以是声音的连续波形信号等。数字信号一般是仅用几个数字，如0, 1两个数字的某种组合或编码表示的信号。

数学家重点关心的问题之一。

在这同一时期，丘奇、波斯特和图灵等人围绕着怎样判断一类数学问题是否是机械可解的这样一个问题，从不同的角度考察了计算这一概念，并探索了计算这一概念所包容的内涵。其中，以图灵的研究与实际计算机更为接近，他提出的形式化的理想计算模型深刻地揭示了计算这一本质概念，为能行计算理论奠定了基础。

有关图灵当时工作的情况是这样的。1936年，正当图灵准备把关于可计算数的论文提交出去的时候，丘奇关于递归函数的论文于3月发表了。从本质上来看，两者的研究论文的结论是相同的，都指出了不可计算函数的存在，但所用概念相去甚远。丘奇是从可计算函数的结构和构造出发讨论计算的概念，而图灵则是从一种简单的数学机器（理想计算机）出发来研究计算的概念。前者在研究可计算函数的结构和构造时使用了算子的概念，而后者通过引入机器状态使用了本质上具有指令特点的程序运算操作。在当时看来，图灵的方法不过更新奇、更简洁一些罢了。可是，10年过去，当存储程序式计算机诞生后，人们才逐渐发现他的理论存在巨大的优越性，因为那正是计算科学理论的基础。当然，我们还应该看到，30年以后，由于计算的正确性问题研究的需要，当计算机程序设计语言的语义学和形式语义学研究受到学术界广泛重视时，人们惊奇地发现，丘奇使用 λ 演算建立递归函数论的研究为形式语义学的发展提供了重要的理论基础和方法论基础。

图灵机计算模型的巨大优越性不仅表现在它为存储程序式电子数字计算机提供了重要的设计思想，又因其结构十分简单，操作运算相当基本，功能出乎意料地强大，以过程形式比较准确地刻画了计算这一基本的概念而为学术界广泛关注。而且，从图灵机出发，可采用相近的构造、定义方法设计，派生出一批计算能力强、使用方便、形式各异的自动计算机器，从不同的观察事物和处理事物的角度为学科的发展奠定了重要的理论基础。

沿着图灵开辟的方向，计算科学理论在后来的几十年里得到迅速发展，有力地推动了计算科学的高速发展。直到今天，绝大多数关于计算科学理论和技术的研究仍然没有跳出图灵机所确定的范围。不仅如此，图灵机的构造和后来进一步研究得到的可计算函数中通用机的定理都隐含了现代通用电子计算机中存储程序式的基本思想。图灵机的构造简单而又基本，然而功能却十分强大，以致学术界在丘奇和图灵等人研究工作的基础上，总结可计算性概念研究中关于计算的本质时提出了丘奇-图灵论题：可计算性等价于图灵机可计算性。丘奇-图灵论题作为可计算性理论研究中的一个重要的观点，已经为学术界所广泛接受，这等于使学术界普遍接受了图灵给出的可计算性这一概念。

在计算科学的发展中，随着一批计算模型包括并行计算模型（如多头多带图

灵机)的出现,带来了这样的问题:相对于各种计算模型设计的算法之间如何比较优劣的问题,其实质是不同机器之间是否可以相互模拟,计算能力是否等价的问题。于是,出现了针对某一个可计算问题,不同计算模型上的算法之间谁优谁劣的问题,出现了针对某一个计算模型,算法计算的时间复杂性和空间复杂性之间存在什么关系的问题。这方面的研究产生了相似性原理与对偶性原理。我国在这一方向上作出贡献的学者是洪加威,是洪加威教授在 20 世纪 80 年代初最先提出了相似性与对偶性原理,引起学术界的广泛重视。

在这里,我们先对算法计算复杂性作一简要的通俗解释。所谓算法计算的时间复杂性是指该算法运行所需要的时间度量,其数学表示在形式上采用与算法输入的某个量相联系,且反映算法运行时运算所需执行次数的变化的近似函数来渐近地表达算法所需要的计算时间。以一元函数在二维空间上的图示为例,这就好比算法的计算时间复杂性度量如果用一个函数来表示的话,时间复杂性的分析是去设法找出另一个近似函数,它的值始终延着靠近实际的计算时间复杂性函数的曲线发展,并与其最多只相差一个正常数因子。如果这个近似函数的值乘上常数因子后大于等于实际计算时间复杂性函数的值,那么,所找到的近似函数就被视为这个算法计算时间复杂性的一个上界函数;如果,这个近似函数值乘上常数因子后小于等于实际计算时间复杂性函数的值,那么,所找到的近似函数就被视为这个算法计算时间复杂性的一个下界函数。算法计算的空间复杂性定义也是类似的,不同之处是考虑算法计算随问题大小的变化所需要的存储空间。在对各种算法进行复杂性分析的时候,人们总是希望找到算法最小的上界函数和最大的下界函数。通过这种方式找到的上(下)界函数就作为该算法的复杂性度量函数,一旦完成算法计算的计算机系统确定,可以通过简单的计算,估算实际完成算法所需要的时间。

例如,一个规模大小为 n 的问题的算法计算所需要的实际时间的增长变化规律可以用一个解析函数 $f(n)=2n^2+3n+1$ 来表示,可是,这个解析表达式只有当所有各种输入情况计算机都算了一遍以后才可能找出来,事先谁也不知道。但是,类似的许多问题在尚未计算开始前是有可能根据算法计算的数学方法,通过分析、演绎、推理等方法找到算法计算时间复杂性的近似函数的。譬如 $g(n)=n^2$ 就可以作为 $f(n)$ 这个函数的上界函数,因为我们很容易从 $2n^2+3n+1 \leq cn^2$ 中确定 $c=5$ 这样一个值。如果我们确实能够通过分析、演绎、推理等方法保证对于任意 n 值规模大小的这个问题及其输入数据,算法计算所需要的时间不会超出 cn^2 的值,其中 $c>0$,那么, $g(n)=n^2$ 就可以作为这个问题算法计算时间复杂性 $f(n)$ 的一个上界函数。有了这个估计函数,人们可以在一台选定的计算机上通过 n 值固定的少数几组实测参数计算算法实际完成计算所需要的时间,然后运用这个估计的上界

函数估算出这台机器在完成更大的 n 值的情况下完成计算实际至多需要的时间，并运用下界函数估算出这台机器在完成更大的 n 值的情况下完成计算实际至少需要多少时间。读者自己分析一下就不难得出， $g(n)=n^2$ 也是 $f(n)$ 的下界函数。

读者从这个例子不难想象，对许多问题来说，要找到其算法计算时间复杂性的近似函数恐怕不是一个轻而易举的事情，实际情况也确实如此。由于这个原因，算法分析就成为一个高智力的、有趣的数学问题。

有一个问题在 20 世纪 70 年代曾引起科学家的思考。一类问题的计算复杂性是否与计算模型有关系呢？会不会有些类的问题在图灵机上计算，所需资源很多，而换一种计算模型去计算，所需资源比较少（或者相反）呢？这就是计算模型的相似性问题，是计算复杂性理论的基本问题，相当于可计算性理论中的丘奇-图灵论题。

最初，库克等许多人在 20 世纪 70 年代针对某些问题在不同计算模型上的计算特性研究了不同计算模型之间的关系，指出某些计算模型之间具有同等的计算能力关系。1980 年，洪加威在总结前人工作的基础上，证明了 12 种有代表性的、基本的计算模型之间可以相互模拟计算，计算能力相互等价，并提出了相似性原理。相似性原理告诉我们：所有合理的、功能足够强的计算模型不仅可以相互模拟计算，而且它们相互模拟时，同时使用本质上是相同的并行时间，本质上是相同的空间，本质上是相同的顺序时间。所谓本质上相同指的是计算复杂性多项式相关，也就是说，在计算复杂性的度量上至多只相差一个增长不快的多项式函数因子。

在上一章国王与公主的童话里，我们曾提到不确定性算法的概念。针对这个童话，不确定性算法的含义是对任意给定的一个自然数 n （输入），随机地任取一个大于 1 且小于这个自然数的整数 x ，判定 x 是否为 n 的一个真因子。当我们作这样一个判定且实际上就是验证时，这个不确定性算法在某个不确定性计算模型的支持下具有这样一条性质：

当且仅当 n 不是一个素数时，算法总能找到 n 的一个真因子 x 并使计算成功地终止；当且仅当 n 是一个素数时，算法将以失败终止。

由于 n 是任意的，我们不可能造出一台具有 n 个处理器（注意： n 是任意可变的）的并行计算机系统，但确实又可以假想这样的机器存在。一旦假设对任意给定的 n ，总存在具有 n 个处理器的并行计算机系统，那么，一个本来在传统的单处理器计算机上用确定性算法需要非多项式计算时间复杂性计算的问题，马上可以用非确定性算法在这样一台假想中的并行计算机系统上以多项式计算时间复杂性计算。因为小伙子仅仅是做一项验证计算，只不过要想提高成功计算的概率，并行计算机系统需要有更多的处理器和存储空间。

“证比求易算法”给了我们一个重要的启示：对任何可计算问题，我们是否可以用增加处理器的数目和存储空间来降低计算所需要的时间呢？或者，反过来，

我们是否可以用增加计算时间来减少处理器的数目和存储空间呢？

洪加威教授提出的对偶性原理告诉我们：在并行计算模型上，计算的时间与空间可以互换。对一个算法，如果要降低计算时间复杂性，常可以通过牺牲空间复杂性换得，反之亦然。也就是说，如果一个计算能在一个小的空间里完成，就一定能用短的时间并行完成；反过来，如果一个计算能用短的时间并行完成，就一定能在一个小的空间里完成。此外，并行计算时间和空间不仅可以相互转换，而且是相互制约的。时间、空间和并行计算时间对任何一种计算模型来说都是对偶要素。

洪加威教授提出的相似性与对偶性原理所以未称为定律或定理，是因为无法对所有各种（包括已经存在的和将来可能存在但现在还没有发明的）计算模型证明其结论成立，但这个结论对现有的有代表性的计算模型都是成立的，而且，这个原理在科学哲学上具有普遍意义，在数学、物理学等学科的发展中得到广泛印证，所以，用原理来称谓这两个重要的结论是完全站得住脚的。相似性与对偶性原理是中国科学家对计算科学的重要贡献之一。

需要指出的是，计算复杂性研究有算法计算复杂性和抽象计算复杂性之分。前者是通过对算法所需要的运算（或操作）步骤和所需存储空间进行度量分析研究计算复杂性，而后者是基于递归函数论在布卢姆（M. Blum）给出的复杂性测度公理（Blum 测度）的基础上研究各种可计算函数的复杂性。但两者之间存在联系，算法的复杂性测度（时间与空间）满足布卢姆的公理。这说明，算法复杂性的定义是合理的，而且也是比较实用的。

19 世纪和 20 世纪前半叶，是计算机发展史上不同寻常的历史时期，一大批杰出的数学家和物理学家在各自熟悉的领域中感到了制造计算机的重要性，并孜孜不倦地努力探索。在此期间，虽然因社会需求不够强劲，得不到理解和支持，制造业技术水平不够发达等各种因素，没有取得成功，但他们的工作为 20 世纪计算科学时代的到来作了必要的准备。

在图灵 1936 年写出关于可计算数的论文之后不到十年，世界上第一台通用程序控制计算机就诞生了。德国工程师朱斯是第一台通用程序控制计算机的设计者。他从 1934 年起就致力于计算机器的研制，在经历了机械计算机研制经验的基础上，他第一个采用电器元件来制造一种二进制的程序控制计算机，并于 1941 年取得成功。差不多是在这同一时期，艾肯（H. Aiken）在 IBM 公司的支持下，于 1944 年研制成功了一台通用程序控制计算机，并在哈佛大学投入运行。可是，由于这两台计算机都采用了继电器作为部件，运算速度受到极大限制。就当时物理学电子技术的发展水平来看，机电式的计算机必将要被使用电子线路的计算机所淘汰。尽管如此，这些机电式计算机的研制为后来电子计算机的诞生积累了重要的经验。

第一台采用电子线路技术研制成功的通用电子数字计算机是由美国宾夕法尼亚大学莫尔电工学院的莫克利(J. W. Mauchly)、艾克特(W. J. Eckert)等人在 1945 年年底设计制造的 ENIAC。这台机器后来被运往阿伯丁弹道实验室,为美国军方所使用。虽然 ENIAC 机的制造采用了当时先进的电子技术,但是,在基本结构上它与机电式计算机没有本质的差别。存储容量太小,程序不能存储等缺陷都极大地限制了机器的运算速度,它的先天不足预示了计算机的研制急需有更合理的结构设计。事实上,从技术的角度看,20 世纪二三十年代就已具备了制造现代通用电子数字计算机的技术条件,但在计算机的结构设计方面还缺乏原理的指导。

1944 年夏天,一个偶然的机会使当时已经成名正在阿伯丁弹道实验室任顾问的大数学家冯·诺依曼参加到新型计算机的研制工作中来,并参与了 ENIAC 机完成前的改进工作。ENIAC 机的研制使莫尔电工学院成为当时美国的计算机研究中心之一。在冯·诺依曼到来后不久,经过对 ENIAC 机不足之处的认真分析和讨论,研究小组很快又考虑并承担了新型计算机 EDVAC 的研究任务。1945 年 3 月,冯·诺依曼参加新型计算机的研制工作之后,在他的主持下,根据图灵提出的存储程序式计算机的思想,研究小组完成了 EDVAC 设计方案报告的初稿。1945 年 6 月,一个全新的存储程序式通用电子计算机的设计方案——EDVAC 诞生了,此时,ENIAC 机尚未完成。

EDVAC 方案的主要内容是确定了计算机由(1)运算器,(2)控制器,(3)存储器,(4)输入,(5)输出等五部分组成,明确地反映出现代电子数字计算机的存储程序控制工作原理和基本结构,对以后计算机的发展产生了深远的影响。今天,人们把具有这样一种工作原理和基本结构的计算机统称为图灵-冯·诺依曼型计算机。

这里存在一个值得注意的问题。冯·诺依曼本是一个数学家,他在此之前对计算机的研制,特别是有关的技术可说是一无所知,然而,他何以能一下子敏锐地觉察到通用电子计算机研制的重要性,马上看出计算机设计的关键在于其逻辑装置呢?

这里有一个历史故事。冯·诺伊曼在第一次离开普林斯顿大学之前,不到 28 岁就担任了普林斯顿大学数学系的教授,他一直从事数学和物理学的研究工作(包括他在欧洲从事的研究工作),先后在集合论的公理化、量子力学的数学基础、遍历理论与算子代数、博弈论等领域做出了一大批高水平的创新研究成果,成为当时十分重要的一位数学家,并在 1940 年和 1943 年分别被美国阿伯丁弹道实验室和洛斯·阿拉莫斯原子武器实验室聘为顾问。1944 年夏天,戈德斯坦(H. Goldstine)在阿伯丁火车站巧遇冯·诺依曼,他早就听说过这位闻名的大数学家。怀着年轻人会见大人物那种揣揣不安的心情,他走近冯·诺依曼作自我介绍,开始与冯·诺

依曼攀谈起来。冯·诺依曼热情友好，毫无大学者的架子，很快使戈德斯坦不觉得拘束，大胆地谈起了自己的工作。当冯·诺依曼知道他们正在莫尔电工学院制造每秒能算 333 次乘法的电子计算机时，谈话的气氛一下子变了。冯·诺依曼严肃而认真的询问，使戈德斯坦好像又经历了一次博士论文答辩，而冯·诺依曼本人也从戈德斯坦的介绍中感到了这项工作的极端重要性。他马上向项目的负责人提出访问实验室的请求，并很快得到了同意的答复。

历史上，当冯·诺依曼准备前往莫尔电工学院计算机研究小组时，莫克利和艾克特并不十分热情，他们想要考一考这位来自普林斯顿高等研究院的数学天才，并称他们只要从冯·诺依曼提的第一个问题就可知道他是否是真正的天才！世上无巧不成书。冯·诺依曼到达莫尔电工学院计算机研究实验室看了研制中的计算机之后，提的第一个问题就是这台计算机的逻辑装置和结构，而这正是莫克利等人所谓判别真正的天才的标志。于是，他们把大量与机器研制有关材料毫无保留地统统拿出来给冯·诺依曼看，并与他进行讨论。冯·诺依曼详细地了解了他们的研究工作进展情况，提出了许多改进意见。在此基础上，他们很快提出了研制新的机器的设想方案。

这段往事常常令后人深思，冯·诺依曼提的第一个问题就是计算机的逻辑装置难道仅仅用“天才”两字就能解释得了的吗？类似地，图灵也是一个长期从事基础数学研究的学者。但是，第二次世界大战使他转入了从事军事上破译密码的工作。据已解密的英国二次大战绝密档案记载，图灵与同事们一起研制了多台专门用来破译密码的计算机，并成功地破译了德军的密码。战争期间，图灵在破译敌军密码的工作中作出了重要的贡献，战后曾被授予大英帝国荣誉勋章，受到嘉奖。难道这也是巧合？事实上，图灵的科学研究生涯中还有更令人惊讶的活动。他曾经系统地研究过生物结构的发生和发展过程，并以“形态形成的化学基础”一文由皇家学会发表在 1952 年的《哲学学报》上，引起生物化学家的极大兴趣。图灵甚至还是机器思维研究的开拓者。显然，坚实的基础和良好的科学素养在他们的科学生涯中起着重要的作用。

ENIAC 机的诞生曾使莫尔电工学院一下子成为全世界关注的焦点。可惜，1945 年底，□由于莫尔电工学院计算机研究小组在 ENIAC 诞生之后，设计组的专家们因发明权而争得不可开交，小组陷于分裂，最终自行解体，致使研究工作一度中断。在这种情况下，冯·诺依曼与戈德斯坦、勃克斯（A. W. Burks）等人离开了宾夕法尼亚大学莫尔电工学院，来到普林斯顿高等研究院继续计算机的研制工作，并在军方的支持下使普林斯顿高等研究院代替莫尔电工学院成为全美计算机研究中心之一。1952 年，冯·诺依曼等人完成了 EDVAC 机的建造工作。

自从第一台通用电子数字计算机 ENIAC 诞生和 EDVAC 方案发表后，美国、

英国、前苏联、法国等国家迅速地加快了计算机的研制步伐，一批计算机相继出现，并于 20 世纪 50 年代形成了生产规模。50 年代的计算机，因其主要元器件采用电子管而为后人称之为电子管时代，即第一代电子计算机时期。在这一时期，美国计算机的发展实现了三个转变：从军用扩展至民用；从实验室研制进入工业化生产；从科学计算扩展到数据处理。

随着计算机的应用和工业化生产的发展，为了提高机器的运算速度和可靠性，对计算机组织结构、电子器件的研究逐渐形成了计算机设计与电子器件、故障诊断与检测、外部设备等方向。

第一代计算机推向社会后不久，由于晶体管技术的发展和成熟，开始采用晶体管研制第二代电子计算机的工作就已在美国的一些著名实验室进行了。至 20 世纪 50 年代后期，第一批晶体管电子计算机产品已经问世，计算机的性能也有了很大提高。到 1960 年，各个公司已能大量生产各种型号的晶体管电子计算机，第二代电子计算机时期即计算机的晶体管时期来临了。几乎与此同时，英国、联邦德国、法国、日本对计算机的研究与开发也进入了第二代。

第二代计算机研制与开发的一个重要特点是：一方面生产满足不同用户数据处理需要的各种类型的计算机，另一方面在继承存储程序式原理的基础上，采用先进的电子元器件和各种技术新思想研制主要用于科学计算的速度更快的计算机。各类计算机的研制与开发，使多种计算机的总体结构设计方案得到了反复的试验和比较，为第三代计算机的发展作了有益的尝试和经验积累。随着第二代计算机的出现，高级程序设计语言及其编译系统、操作系统、各种科学计算方法和标准子程序库等方向的研究也得到了很快的发展。

1958 年，当第二代计算机还处于刚刚准备批量生产的时候，美国德克萨斯州仪器公司制成了第一块半导体集成电路。三年后，德克萨斯州仪器公司在军方的支持下，研制成功了第一台试验性的集成电路计算机。然而，初期的集成电路由于集成度不高且价格昂贵，没有能够马上应用于计算机的工业生产。随着集成电路工艺技术的改进和生产成本的降低，到 1964 年，IBM 公司成功地研制出大型集成电路通用计算机 IBM 360 系统，才拉开了第三代集成电路计算机的序幕。IBM 360 计算机系统的诞生，因其系列化、通用性和标准化极大地影响了世界各国通用系列计算机的发展，成为计算机发展史上一个重要的里程碑。

体积小、速度快的第三代计算机一进入用户市场，很快淘汰了晶体管计算机。随着计算机更多地应用于数据处理，特别是联机系统的应用，极大地促进了软件和计算机应用的发展。20 世纪 60 年代中后期，“软件危机”的出现，软件和计算机应用的发展中对计算机硬件提出的各种要求使科研人员不断地改进计算机的设计，也使计算机通信技术得到了很快的发展。在第三代计算机发展的后期，研究

人员已经意识到计算机的设计和制造, 需要软硬件双方的共同参与, 软件开发中对计算机性能和各种功能的要求必须在计算机的设计中得到体现, 一台计算机系统的成败更多地是取决于软件而不是硬件。

20 世纪 60 年代后期, 随着计算机存储容量和运算速度的成倍增长, 高级语言的发展, 远程终端和多道程序批处理技术的出现, 操作系统的设计急需引入新的技术以支持多个用户同时共享计算机系统资源。于是, 出现了进程的概念和分时操作系统。以多进程技术为基础的分时操作系统的出现, 大大提高了计算机系统的功能, 极大地推动了软件理论和技术的发展。在多进程和分时技术的支持下, 并发程序设计语言和并发程序设计技术得到快速发展, 在刺激和影响数据库系统、程序证明技术、语义学理论、通信技术等诸多方向发展的同时, 也为分布式网络计算机系统的发展在理论和技术上做了一些必要的准备。

当时的情况是这样的。随着计算机硬件技术的发展, 特别是存储器访问和中央处理器速度的极大提高, 外部设备的操作速度与计算机存储器访问和中央处理器速度之间的差距就显现出来了。此外, 过去的计算机系统每次只能由一个用户使用的弊端也已显现出来。于是, 人们考虑首先采用作业描述语言进行批程序的处理。其基本思想是, 将程序视为作业, 多个作业排成一排, 执行的各种操作由作业描述语言所写的作业控制命令来控制。这样做, 可以充分利用夜晚用户较少, 而机时较空闲的时机。在这种思想的指导下, 很快开发了支持多道批程序工作的操作系统。

但是, 多道批程序处理功能的出现并不能完全解决问题, 高速的 CPU 与速度较低的外部设备之间的不匹配使计算机系统在运行中白白浪费了大量宝贵的有效机器时间。此外, 许多用户, 特别是当时的软件开发工程师、商务人员和公务人员迫切希望能有一种支持联机操作的计算机系统, 使他们能够在不同的终端上联机使用计算机开发软件或操作使用计算机。此时, 计算机器件速度的成倍提高和中断指令系统的完善, 以及各种寄存器数目的增加使实现这一设想的条件已经成熟。于是, 在开发支持联机多用户、多任务操作系统的过程中产生了进程 (Process) 和分时的概念。

进程是一个程序在该程序所能识别的某个数据集合上的一次活动。这里的数据集合是指支持该程序运行的所有数据, 包括用户提供的数据, 系统提供的数据和程序运行中自动产生的数据。程序是静态的概念, 而进程是动态的概念。不同的程序在同一个数据集合上的运行生成不同的进程, 同一个程序在不同的数据集合上的运行也生成不同的进程。一个进程可以有就绪、运行和挂起三种基本的系统状态, 自进程创建后运行结束前可在这三种基本状态之间进行转换。进程具备运行条件时, 进入就绪状态; 当它获得操作系统调度分配的处理机时, 即转入运

行状态；进程运行用完分配给它的时间片，就回到就绪状态；如果进程的运行中需要等待某个事件的到来（如外部设备的运行完成反馈信息），就转入挂起状态。分时，即是给系统中运行的各个进程动态分配一段(可能各不相同的)运行时间，使它们能够轮流占有处理机运行，执行程序操作。当然，要实现计算机系统对多个用户的任务分时处理，必然需要以进程为单位提供一种能够为进入就绪状态的进程分配时间片，并为进程实际占用处理机进行运行的技术，这涉及到操作系统中一系列概念和技术，如中断处理、现场数据保护与恢复、进程控制等，以后又进一步提出和解决了进程之间如何进行通信的问题，并将基于进程的程序设计技术用于应用程序的设计之中，推动了并发程序设计的发展。

进程状态的转换图可用图 3.2 表示。

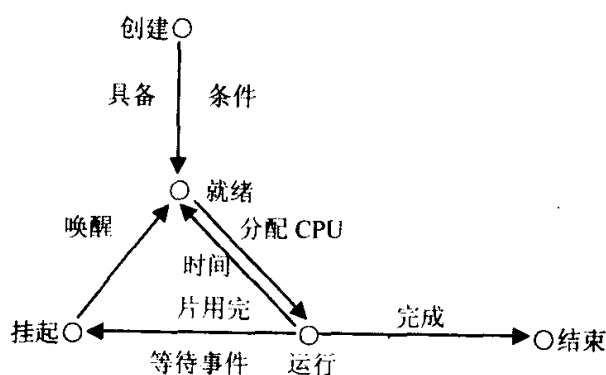


图 3.2 进程状态转换图

在支持多进程运行的分时操作系统中，进程是操作系统调度程序执行和分配系统资源的基本单位。用户的程序以作业的形式进入计算机后，以一个或多个进程的方式接受操作系统的资源分配和调度执行。组成该作业的一个或多个进程运行结束，用户的作业就随即完成。显然，这样设计的操作系统极大地拓展了计算机系统的功能。

以进程为核心概念的操作系统出现，使用户具备了实现并发程序设计的基础。但是，操作系统毕竟是一种重要的系统软件，其安全性是设计者必须考虑的主要问题之一。而要让用户实现并发程序，系统必须向用户开放，这就涉及到系统的安全问题，带来了如何开放的问题。从操作系统的角度考虑问题，不应该也没有必要将整个系统向用户开放。而从一般用户的角度考虑，对进程的管理也不应该全部交给用户自己来管理。于是，人们希望操作系统对用户具有某种功能上的透明性，它能够屏蔽系统的许多内部管理操作，用户只需要通过调用一些功能命令就可以实现各种并发程序。按照这样一种设想，从方便实现和方便使用的角度考虑，需要有一种语言，既能方便在硬件裸机上开发操作系统，又便于用户使

用功能命令，而且还要使这些功能命令易于实现。基于这样的认识，人们首先设计了 C 语言（以后因受面向对象程序设计思想的影响，引入面向对象程序设计的语言成分而发展成为 C++ 语言），进而产生了 UNIX 操作系统。

进入 20 世纪 70 年代之后，半导体集成电路取得了飞速发展和进步。集成度的不断提高和半导体存储器的问世，大规模集成电路的迅速发展使计算机的速度在不断提高的同时，体积不断地缩小，价格逐年下降，这种情况导致产生了采用大规模集成电路的计算机系统，电子计算机的发展进入了第四代。除了使用大规模集成电路技术外，第四代计算机出现了外部设备高性能、多品种、小型化、磁盘存储器向大容量方向发展的特点，还出现了巨型机、小型机、微型机、数据流计算机、向量计算机、直接执行高级语言计算机等各种类型的计算机系统。小型与微型计算机的问世，以及计算机的联机化应用使得过去许多必须要在大型计算机上完成的计算工作得以在小型与微型计算机上进行，大大降低了使用计算机的成本。这不仅促进了计算机在社会各行各业的普及与应用，甚至使计算机进入寻常百姓的家庭，而且促进了对计算机网络的研究与开发。与此同时，并行计算机系统的研究也取得了很大的进展。所有这些，都促进了计算机体系结构技术、并行与分布式算法和计算机网络方向的形成和发展。

所谓向量计算机是高速计算机类型中的一种。这种计算机以向量为单位进行数据的处理。向量是由一串数据组成的，同一个向量中的每个数据称为向量的分量，它们之间是相互独立的。由于事先研制了数据预先读取、高速缓冲存储技术，处理数据时，一条向量指令的多个动作可通过流水线时间重叠顺序处理同一向量中的每个分量，实现向量的流水线时间重叠意义下的并行处理。显然，采用流水向量处理技术，只要引入执行流水线向量运算的功能部件，指令译码次数和可能的辅助计算工作量将减少，可大大提高机器运算的速度。在向量计算技术的基础上，人们又发明了一种高速计算机——阵列式计算机。这种计算机根据并行计算的原则，设计了一些数组或阵列运算（利用矩阵运算理论）。它采用多个处理单元同时处理数组的各个分量，也是用一条指令并行执行多个数据处理操作，如一条数组加法指令可以同时完成两个矩阵所有元素的加法运算，只不过此时一条指令意味着数组的各个分量同时进行加法计算。这样做由于减少了指令的译码和辅助计算量，可以获得很高的运算速度。典型的阵列式计算机中，每一个处理单元都带有自己的存储器，用一个总的阵列控制器控制各处理单元的工作。当然，控制器同时也可以进行常规的数据运算处理。在设计这类计算机时，由于处理单元互连的方式不同，控制数据处理的方式不同，发展了多种技术，研制出一批种类各异的计算机，如流水线向量计算机、心动阵列计算机（Systolic 机器，也叫脉动阵列机）等。

值得指出的是,除了操作系统之外,新型计算机的设计在 70 年代已开始受到高级语言、编译技术和各种应用软件开发的影响。不难想象,如果一台计算机系统今后主要用于科学计算中解高阶方程组,需要对高阶矩阵进行大量计算,那么,这种计算机从一开始就可能采用向量技术设计成向量计算机。而在向量计算机上配置的高级语言编译程序等软件必然会有一些特殊的要求,如对寄存器的数量和类型的要求等。于是,如果从一开始设计计算机时就考虑这些问题,并在设计中体现出来,将有助于今后软件的开发和计算机系统的使用。类似地,主要用于图形处理的计算机的设计,必然要考虑各种与图形处理有关的问题。

20 世纪 70 年代后期,微处理器的快速发展促使人们研究对称多处理机 SMP 系统,这是一种由数目相对较少的微处理器通过共享物理内存和 I/O 总线形成的计算机系统。和随后的大规模并行处理 MPP 系统相比, SMP 扩展能力一般比较有限,当微处理器数量较多时共享内存和 I/O 总线在技术上过于复杂,机器为实现控制各处理器协调计算,需要将大量的时间消耗在软件控制方面,这会导致机器不具有很好的计算性能,这条发展高性能计算机系统的道路后来终于偃旗息鼓。但由于 SMP 与单机系统兼容性比较好,是单机系统的升级与增强,依然被广泛应用于商业计算领域。

20 世纪 80 年代初,随着 VLSI 技术和微处理器技术的发展,传统的研制巨型机沿用向量机的发展道路的情况出现了转机,人们开始采用多个廉价的微处理器来构建并行化程度较高的超级计算机系统,并取得了重要进展。计算机应用的逐步普及也使“性能价格比”而非单一机器性能成为衡量高性能计算机系统的重要指标。到 90 年代初期,大规模并行处理(MPP)系统已经开始成为高性能计算机发展的主流。MPP 主要由多个微处理器通过高速互连网络构成,每个处理器之间通过消息传递的方式进行通信和协调。我国的曙光 1000 计算机系统就是其中的一个代表。

20 世纪 80 年代起,随着并行处理机系统的发展,由于进程运行时面对的已不再是一个处理机,而是多个处理机,因此,必须将并行处理引入进程的內部,通过动态对应分配的方式确定进程运行所占的处理器。由此,产生了类似于进程的线程概念(Thread)。线程是并行或分布式计算机操作系统调度程序的最小单位,是进程内部的一个执行控制流。一个进程内可以包括一个或多个线程,各线程共享该进程的全部系统资源,各自拥有自己的记录系统数据的数据结构和系统入口。当进程处于运行状态时,其线程可以独立地被多个处理机调度运行,也具有就绪、运行和挂起等系统状态。一个进程中的所有线程全部运行结束,该进程就可以结束。引入线程的概念后,多个处理机可以同时处理一个进程,大大加快了计算的速度。

网络计算机系统的出现,支持了人们久已期待的分布式信息处理。在计算机

网络上进行信息处理的计算活动被称作分布式计算,为实现这样一种计算而进行的算法设计和程序设计分别被称为分布式算法设计和分布式程序设计。在现实世界中,大量科学问题的计算和事务处理的任务靠传统的顺序或并行计算机系统并不能解决问题。特别,对于那些需要从地域上分布的多个数据采集点获得数据并进行实时数据交换和实时处理的问题而言,没有网络计算机系统的支持是难以想象的。例如,全球天气预报、空间飞行器的定位与控制、远程医疗会诊等,都需要计算机网络和分布式计算技术的支撑。从上一节的内容可知,计算机网络是一种分布式多计算机系统,在系统结构的分类上近似于 MIMD 类型。

20 世纪 90 年代中后期的一段时间里,为了很好地解决分布式并行计算问题,人们将 SMP 的优点和 MPP 的扩展能力结合起来,产生了后来的 CC-NUMA 结构,即分布式共享内存。所谓分布式共享内存就是让每个处理器结点都可以访问到所有其他结点的内存(通过软件控制技术实现),但访问远程内存需要的通信延迟相对较大,当计算任务所需的通信量很大,通信次数频繁时,必然制约机器整体性能的发挥。这类机器代表性的有 SGI-Cray Origin 等,国内的神威与银河系列也属于这一类。CC-NUMA 结构本身并没有对提高机器的性能做出贡献,其主要优点是便于程序的开发和与 SMP 的兼容,而对科学计算任务 CC-NUMA 结构是否优于 MPP 系统仍存在争议,比较一致的看法是取决于问题的种类和问题本身的特点。

在发展 CC-NUMA 机型的同时,机群系统(Cluster)也得到了迅速的发展。所谓机群系统是由多个微处理器构成的计算机结点通过高速网络互连而成,类似于 MPP 结构。结点一般是可以单独运行的多处理器计算机系统。机群系统与 MPP 的区别在于前者结点是可独立运行的计算机系统,后者是由处理器按照大规模并行化的方式构建。前者属于多机系统,后者是多处理器并行计算机系统。一般认为,机群系统具有比 MPP 更高的性能价格比优势。有代表性的系统是 IBM SP2;国内有曙光 3000, 4000 等系列。20 世纪 90 年代,微型计算机系统的价格一降再降,促使更多的国家和科研机构研制机群系统,到 2000 年初,机群系统实际上已经成为发展高性能计算机系统的主流。据 2003 年的统计,全世界性能最高的前 500 台计算机系统中, MPP (含 CC-NUMA) 占 42%, Cluster 占了 29.8%。

目前高性能计算机体系结构的主流仍然是 MPP 和机群技术的进一步发展,通过将更多的处理器连接起来构建更大规模的并行系统。其中最具代表性的就是美国能源部的 ASCI 计划,分别由 Intel, SGI, IBM, HP 等构建超大规模的机群系统,如 HP ASCI Q 共有 8192 个处理器,可谓规模巨大。

既然高性能计算机系统的发展具有分布式互连结构的特点和支撑分布式并行计算的考虑,那么,分布式计算与并行计算是一种什么关系呢?根据并行计算的定义,不难得知,分布式计算是一种并行计算,但又与并行计算机系统上的并行

计算有所不同。通常，我们所说的并行计算，是指在并行计算机系统的支持下进行的计算。对用户而言，这样一种并行计算机系统不仅处理器的数目、处理器与处理器之间的互连方式是清楚的，而且，并行计算机系统通道结构和通信控制方式也是清楚的。由于 MIMD 类型中常见的共享存储并行计算机系统提供了为各个处理器共享的存储器，其不同级别上的并行可以通过软件和控制部件来完成，加之相互之间的通信线路很短，在软件和控制器的协调下可将通信延迟时间忽略不计。但是，与多处理器（机）的分布式网络计算机系统不同，网络计算机系统不存在为各个计算机所直接共享（可随机访问）的存储设备，一般也没有统一的硬件控制部件，网络上计算机与计算机之间的相互通信、访问其他计算机系统软硬件资源等操作都必须通过网络上的软件和网络通信机构由网上其他结点的计算机协作完成。因此，分布式计算实际上是一种分布式程序运行环境支持下的分布式并行计算。这里提出了一个问题，要在网络上顺利地开展分布式并行计算，就必须开发能模拟 MIMD 系统结构，支持分布式并行计算的分布式并行计算环境或分布式并行程序设计环境。

理论上，如果不考虑计算的实时响应，一个问题在并行计算机系统上可并行计算的话，这个问题在网络计算机系统上也可以分布式并行计算；反之，一个问题在网络计算机系统上可分布式并行计算，那么，它也可以在并行计算机系统上并行计算。然而，事实上，由于许多用户都在共享使用网络计算机系统的资源，网上两个结点计算机之间不存在一条点对点直接连接的通信线路而仅存在路径（经其他结点连接，可能不止一条），网络通信时通信双方交换的数据常常需要经第三者转送，网络上通信双方在进行数据交换时并不能按照事先设想的时间完成通信，很可能发生应该发送或到达的信息实际上并未发送或接收，即通信被延迟，甚至有可能发生信息丢失或被篡改的情况。更糟糕的是，这种通信延迟将导致其他相关结点也发生通信延迟，即群延迟。为了保证网络上的可靠通信，必须有一种保证可靠通信的机制，这样就产生了网络协议。可是，当网上通信双方真正进行通信时，因为通信涉及到需要交换的数据量可能很大，数据必须分组；因为涉及到交换的数据与实际发送的电子信号在表示形式上不同，需要转换；因为涉及到网上信息交换可经过的路线不止一条，各条路线的成本或效率不同，必须对候选路线进行选择；等等。如果将所有这些工作都捆绑在一起完成通信，不仅技术上更为复杂，效率更低，而且不利于网络及时响应不同用户的请求，也不利于网络系统对不同的用户开放。于是，在计算机网络的发展中产生了网络通信结构分层协议技术。网络通信由于网络通信信息延迟将给系统带来不确定性，而且并行计算机系统和网络计算机系统所采用的通信机制和方式不同，根本的原因涉及到两者的体系结构不同，许多分布式并行计算程序并不能通过简单地修改就可将程

序从并行计算机系统移植到网络计算机系统上进行运行和计算。

实际上，网络计算机系统上的分布式计算比并行计算机系统上的并行计算更为复杂，网络上通信延迟带来的不确定性因素和通信的相对不可靠性都决定了分布式计算如果是处理“安全至上”（Safe-Critical System）的一类问题，没有容错功能的系统开发是用户不能接受的。

在计算机发展经历了四代变革的几十年中，尽管电子器件的发展使计算机的运算速度不断提高，如在计算机结构和功能部件的设计中采用高速缓存、多寄存器、I/O 通道、阵列式运算部件、软件固化技术等，但基本设计思想和工作原理仍然囿于冯·诺依曼型计算机的结构原理。虽然在 20 世纪 70 年代和 80 年代有一些人试图突破这一传统的设计思想，如发展直接执行高级程序设计语言计算机等，但一直没有取得真正的成功。于是，从 80 年代初开始，在超大规模集成电路技术发展和各种应用背景的强劲支持下，非冯·诺依曼型计算机的研究和所谓第五代计算机——人工智能计算机的研究开始了。与此同时，在计算科学的基础研究中，非图灵计算模型的研究也开始受到了重视。

传统的图灵计算模型是定义在非负整数集合上的一种数学机器。非图灵计算模型的研究有多种探索途径，但基本目的只有一个，构造一种功能比图灵机更强的数学机器。在功能上不能超过图灵机的情况下，起码应用于某一方面时比图灵机要方便一些。在许多各种各样的研究中，其中一条途径是将机器扩展定义到实数集合或更大的数域上，另一条途径是对图灵机进行结构上的改造，使其具有并发或并行执行的功能。也有一些研究是将所谓的人工智能技术引入到机器的定义之中，如自动推理技术、神经网络技术等。值得一提的研究工作是 1989 年，布卢姆（L. Blum）等人发展了一种实数域上的计算模型，称之为 BSS 机器。

这里存在一个问题，图灵机是定义在非负整数集合之上的计算模型，而我们现在的计算机既可处理整数，也可处理实数，还可以处理复数乃至其他数域上的计算，二者之间究竟是一种什么关系？既然 BSS 机器是定义在实数域上的计算模型，那它与现代存储程序式电子数字计算机和图灵机又分别是一种什么样的关系呢？我们希望读者能在不久的将来，自己回答这个问题。

日本是第五代计算机研制的积极倡导者和实践的开路先锋。进入 20 世纪 80 年代以后，由于综合国力的强大以及在超大规模集成电路研究和生产等方面保持的领先地位，日本通产省于 1981 年制定了实施研制第五代计算机的宏伟计划。日本政府希望通过这项计划的实施，使日本计算机科学技术水平达到世界领先地位，并带动日本整个社会经济的发展。也有人认为，日本的第五代计算机计划“醉翁之意不在酒”，他们是想通过先声夺人的方式，影响和打乱其他国家的整体发展计划和部署，牵着人家的鼻子走。然而，由于第二次世界大战之后日本在重视技术

和经济发展的同时，对基础理论的研究与发展不够重视，以至在计算机制造这一涉及诸多学科且理论远远落后于技术发展的领域内，因缺乏强有力的理论指导，计划的实施最终失败了。不过，第五代计算机的研制过程中提出的许多问题，积累的经验教训对计算科学的发展仍然有着重要的参考价值。

1980年初，日本在研制第五代计算机的时候，提出了将当时非常时髦的一种逻辑程序设计语言 Prolog 作为人工智能计算机的核心语言，为该语言建立一个小的执行机器，在此基础上开发计算机系统的各种智能部件的设想。由于 Prolog 语言的理论基础是建立在一阶逻辑系统的一个更小的子系统——Horn 逻辑之上的，尽管这个子逻辑系统不仅在语法上而且在语义（即模型）上的研究表明其理论基础是坚实的，但是，数理逻辑的研究早已指出了这样一个事实，一阶逻辑在表达能力上是有限的，而且在表达方式上是不方便的，更不用说作为子系统的 Horn 逻辑了。此外，当时非单调逻辑和各种非经典逻辑在人工智能的研究中开始受到广泛重视，对人工智能研究中存在的问题学术界也已经有了比较深入的认识和深刻的反省，自然有许多人并不相信日本这样一种仅具有单一推理机制的设想能够造出智能计算机系统。

在整个 20 世纪 80 年代，计算机处理速度和性能价格比有了很大改善，产生了精简指令系统计算技术（RISC），使多处理器商品化，出现了第一批大容量并行计算机系统。这是一个激动人心的时代，在这一时期，体系结构的研究中出现了精简指令系统计算机（RISC），基于并发程序设计理论的共享存储结构并行计算机，以 CSP 理论为基础设计的通信计算机（Transputer 计算机），心动阵列计算机等多种计算机系统，工作站和网络服务器技术也日臻成熟，同时，也推动了计算机软件和各种计算机应用技术的深入发展。然而，计算机的速度能否在 90 年代继续保持高速增长呢？

20 世纪 80 年代特别是从中后期起，超大规模集成电路的研究使人们认识到集成电路技术的发展已进入极限状态，要想提高计算机的运算速度，必须寻求别的出路。曾经获得过图灵奖的美国 IBM 公司华盛顿研究中心的研究员约翰·柯克（J. Cocke）教授是一位计算机硬件与体系结构设计方面的权威，他在分析、展望 90 年代计算机体系结构的论文中指出：影响计算机技术持续发展的最根本的障碍是金属互连电路的速度极限问题。一般认为，硅微电子技术无论从材料科学的角度还是从当今的制造工艺的角度来看，在硅片上制造晶体管会受到密度的限制，这个极限是 0.05 微米（这个数字是指集成电路中的线宽，或集成电路中两条线之间的最短间距，这两种说法对于集成电路的生产工艺技术来说是同一个概念。1 毫米等于 1000 微米）。我们知道，采用硅片作主要材料的集成电路在制造的过程中，涉及到计算机绘图，刻膜，激光制版，光刻（用直线性相当好的光束在硅片

上刻线), 金属渗透, 镀膜、器件封装等一系列复杂的工艺程序。要保证这些工艺程序的质量, 必须依靠相应的高精尖设备, 如光刻设备、图像发生器、精密封装机械手等, 而这些设备由于主要是光机电一体化设备, 其精度在可预见的将来也存在着精度极限的问题。正是因为辅助设备的制造是一项十分困难的工作, 所以, 可以预见, 体积更小、集成度更高、运算速度更快的计算机芯片的生产, 将不可能保持像 70 年代、80 年代那样的性能增长速度。目前, 采用金属互连结构电路的计算机的时钟速度已经达到或者接近这一技术的速度极限, 因此, 如果速度上不能突破时钟速度极限的障碍, 那么, 90 年代将会结束通用计算机性能持续增长的历史。由于当今最先进的高性能计算机系统基本上都是采用一大批处理器按照某种互连方式和体系结构设计的并行计算机系统, 因此, 影响机器性能提高的因素不仅涉及处理器芯片的性能、计算机体系结构和算法, 而且涉及到通信机构和数据通信的速率。今天, 历史的发展已经基本印证了约翰·柯克教授的断言。于是, 面对许多领域科学技术和数据处理对计算提出的更高要求, 面对许多问题具有天然的并行与分布式处理的特性, 以并行计算机系统和网络计算机系统为基础的高性能计算机系统的研究得到进一步的重视, 推动了计算机体系结构技术、并行与分布式算法、计算机网络与通信等方向的研究与发展。

尽管并行计算机系统的研制因各种各样的理由受到各国的普遍重视, 但是, 发展并行计算技术并不是提高计算机系统性能的根本性出路。在并行计算机系统性能分析的研究中, 阿达尔 (G Amdahl) 定律具有很好的借鉴作用。

为了使读者易于理解, 我们先给出并行计算机系统所产生的加速的概念。

直觉上, 我们都会猜想: 并行计算机系统求解问题的速度随处理器数目的增加而提高。这种提高, 可以用加速的概念来刻画。所谓加速, 定义为

$$\text{加速} = \frac{\text{最佳串行计算时间}}{\text{最坏情况下并行计算时间}}$$

然而, 闵斯基 (M. Minsky) 曾指出, $\square$ 并行计算机系统所可能达到的计算加速随处理器的数目呈对数增长。这实际上指出了并行计算机系统的加速是有限的, 人们不应期望太高。我们知道, 从函数曲线可以看出, 线性加速是比较慢的加速, 能够实现对数加速已经很不错了, 至于超线性加速是否可能, 目前还是尚未解决的猜想。

尽管我们有各种各样的理由期望通过增加并行计算机系统中处理器的数目来提高计算机的处理速度, 但是, 阿达尔已经指出, 一个问题的求解算法中少量必须串行执行的操作能大大地限制并行计算机系统的加速。这是为什么呢? 我们作如下分析:

设 f 为某一计算中必须串行执行的操作部分 ($0 \leq f \leq 1$), 那么, 一台具有 p 个

处理器的并行计算机系统最大可能的加速 S_p 为

$$S_p \leq \frac{1}{f + \frac{1-f}{p}}$$

由阿达尔定律的定量形式可知, 如果某一计算中所含的必须串行执行的操作占 10%, 那么, 不管一台并行计算机系统有多少个处理器, 其最大可能的加速只能是 10。这说明, 单纯以增加处理器数目来提高计算机系统的运行速度的方法是不行的, 必须寻找别的途径。

事实上, 科学家们一直在致力于研究光互连技术。光互连技术的主要优点是高密度, 有较好的信号完整性和信号隔离性。当互连线上的信号传输时间成为延迟的主要因素时, 这一技术能解决金属互连电路所产生的一些问题。遗憾的是, 在计算机内部采用光技术实现互连的费用太高了, 21 世纪初的 10 年内恐怕进入不了商业化产品开发的阶段。于是, 在目前的情况下, 科学家和工程师将当前微电子器件产业升级技术研究与开发的重点放在了高速处理器、多处理器的集成技术, 高速、高密度、高性能随机访问存储器, 智能存储器技术, 处理器与内存储器芯片的集成; 以及多功能部件、多芯片模块的封装技术方面。同时, 将体系结构技术研究与开发的重点放在了指令级并行化、互连拓扑、可缩放超高速缓冲存储相干性、高速通信机构等方面, 并着眼于动态可变结构的体系结构设计思想的研究。今天, 已经有越来越多的人认识到, 实现高性能计算的一个重要的方面是算法设计, 人们不能单纯指望计算机器件速度的继续不断增长, 高性能计算机系统的设计必须针对不同的应用领域和处理的问题, 在设计中对体系结构、部件构成和应用算法(模型与算法)三个技术因素进行综合考虑, 才能使机器系统获得较高的性能价格比, 才能最大限度地发挥机器系统的计算性能和数据处理性能。

在许多对计算机有着高可靠性要求的应用领域, 如军事、航天、通信、核能等领域, 计算机故障或停机将会带来严重损失, 而实际应用中由于计算机常处于恶劣环境之中, 机器故障难于避免。为此, 提出了“容错计算”的概念。所谓“容错计算”是指容许计算机在某些故障存在的情况下仍然能保持计算结果正确的计算。“容错计算”主要包括硬件容错和软件容错两个方面, 具体研究内容有容错计算机, 容错计算机体系结构, 容错软件系统, 测试与故障诊断, 避错技术, 计算机可靠性评估等。

比如, 在硬件容错技术中, 有一种途径主要是通过增加备用部件来提高设备的可靠性, 进而提高计算的可靠性。在软件容错技术中, 如果用户是在分布式环境中开发系统, 那么, 有一种技术是采用定时快照与回退重复计算的方法, 许多情况下可以保证计算结果的正确性。当然, 在分布式计算机系统的研究中, 近年

来容错计算的发展是很快的，除了引入避错技术外，大量研究集中在对于故障检测器的研制。因为，在一个分布式计算环境中，容错系统需要首先弄清楚哪些结点是正常的，哪些结点发生了故障，然后才能有针对性地进行避错计算或带故障运行。将容错技术应用于实时容错计算是一个重要的、急迫的发展方向，该方向近年来获得了很快的发展，在实际应用中取得了很好的效果。

此外，最近几年出现了一个十分热门的研究领域——网格计算（Grid Computing），大量关于网格的文章铺天盖地而来，引起学术界的关注，各国都争相调集力量开展网格计算的理论和应用研究。读者可能会问：什么是网格计算呢？所谓网格是构筑在互联网上的一组新技术，它将高速互联网、高性能计算机、大型数据库系统、传感器、远程设备和各种软件资源融为一体，为用户和普通人提供更多的资源、功能和交互性，提供使用更强大的计算机系统的一个更加友好的环境，实现同类资源集中，异种资源分离，资源调用服务化，资源使用按需分配等。传统的互联网实现了计算机硬件的互连和连通，Web 技术实现了网页之间的连通，而网格试图实现互联网上所有资源的全面连通，包括计算、存储、通信、软件、信息、知识等各种资源。对高性能计算机系统来说，资源的网格化是一种先进的技术，将为新型高性能计算机系统的设计提供重要支持。但是，需要指出的是：网格的实现并不能取代高性能计算机系统。作为一种共享技术，网格能够支持现有的高性能计算资源更好地被共享使用，改善高性能计算机系统性能的发 挥，而不能增加高性能计算资源。可见，网格实际上是一种广义的分布式计算形态，它依然属于计算的平台和环境这一基本问题的研究。

总结计算模型与计算机这条发展主线，我们可以看出，社会的需求是计算机得以发展的外在推动力。不断地追求新型计算模型，根据科学技术发展的现状，采用先进的科学理论和技术工艺，制造先进的计算机系统始终是这条主线发展的目标。要想制造出新型的计算机系统，除了经费的支持之外，必须要有两个基本条件：一是要有成熟的理论指导，二是要有成熟的技术包括相关产业技术的支持。在计算机发展的早期，整体上技术落后于理论的发展，而 20 世纪 40 年代之后，理论开始渐渐落后于技术的发展，到今天已影响和制约了计算机向具有较强思维能力的人工智能计算机方向的发展。可见，在这一主线上，发展新型计算模型和计算理论，为新一代计算机的设计提供新思想是多么重要，其重要性决不亚于发明具体计算机的研制技术。

3.3.3 计算模型、语言与软件开发方法学

随着 20 世纪 40 年代第一台存储程序式通用电子计算机的研制成功，进入 50 年代后，计算机的发展步入了实用化的阶段。然而，在最初的应用中，人们普遍

感到使用机器指令编制程序不仅效率低下，而且十分别扭，也不利于交流和软件维护，调试程序时查找错误尤其困难，软件开发急需一种高级的类似于自然语言那样的程序设计语言，开始认识到语言是人与机器沟通的一种最基本的媒介。1952年，第一个程序设计语言 Short Code 出现。两年后，Fortran I 问世。作为一种面向科学计算的高级程序设计语言，Fortran 的最大功绩在于牢固地树立了高级语言的地位，并使之成为世界通用的程序设计语言。Algol 60 的诞生是计算机语言的研究成为一门科学的标志。该语言的文本中提出了一整套的新概念，如分程序结构，变量的类型说明和作用域规则，过程的递归性及参数传递机制等，而且，它是第一个用严格的语法规则（巴科斯范式或巴科斯-瑙尔范式（BNF））定义语言文法的高级语言。

众所周知，每一种语言都有它的词法、文法、语义和语用等四个方面的内容。计算机程序设计语言也是一种语言，因而也具有一般语言所具有的语言的词法、文法、语义和语用的内容。所谓语言的词法就是构词法，它规定了什么样的符号组成的字符序列或空间排列是这个语言中合法的单词（或字）。所谓文法是关于用语言的单词构成短语、句子和篇章的规则，在此基础上可以形成各种句子和文章。词法和文法构成了语言的语法。借助于语言的语法，可以产生句型和文章的范型（Paradigm）。所谓语义就是指单词、短语、句子和文章的意义或含义。读者可以对照自己学习中文、英文或其他外文的经历作对照理解。所谓语用是指在一定的语言环境中语言和它的使用者之间的关系。

例如，同样是使用中文，上马这个单词的含义具有多个词义项的选择，当用于一般的野外放牧的场景，人们对上马的理解自然会倾向于是骑上马匹。但如果某（些）个骑马人同时也在进行着一项工程项目的开工，那么，对上马的理解就可能理解为工程项目的开工。还有，如果说话人是针对一盘象棋的对奕，那么，上马可能是指某一方的下一步棋应该挪动马，多半可能是上马后马离自己的老将更远。因此，当一个人说出可能带有双关语的一句话时，对这句话的准确理解需要弄清楚在当时的语境中，使用（说这句话和理解这句话）者与这句话的关系。这样的实例是很多的，譬如，几个在草原上遛马的工程项目的决策和管理者，因为对于工程项目是否马上开工有不同的意见，当董事长包尔汗发现总经理乌兰格琪第一次因为没有把马镫子系好，上马时踩空摔下来时，随口说了一句，“没有准备好怎么能匆忙上马？”不难想象，在旁边的局外人和当事人对这句话的理解应该是不同的。

20 世纪 50 年代后期至 60 年代中期，不同领域计算机具体应用的广泛需求和应用特点产生了一大批程序设计语言。除了 Fortran 和 Algol 之外，最具代表性的有 1959 年出现的面向商业领域的 COBOL 语言；用于计算机普及性教育的 BASIC

语言。60年代出现的面向符号处理与人工智能的 LISP 语言;用于算法设计的 APL 语言;用于字符串处理,特别适合于电信领域的 SNOBOL 语言;支持大型软件开发构件式程序设计的 PL/1 语言;集抽象数据类型、并发进程和分层模拟思想于一体的 SIMULA 67 语言等。从另一个角度看,早期的高级语言主要是面向过程的语言,这与当时应用计算机开展工作人们面对的首先是大量的科学计算问题有关,也与当时对大量应用问题的算法研究比较发达有关。但是,面向过程的程序设计语言在编制程序时,用户的负担较重,程序员不仅要关心程序设计的整体构思,而且还要了解具体计算每一部分的程序细节。如何将这种负担中的一部分交给语言或系统来分担,就成为之后的研究课题。在这一指导思想的引导下,以后又陆续出现了面向问题的程序设计语言,面向说明的程序设计语言,子程序库与程序包,等等。

在高级程序设计语言开发群雄逐鹿的 20 世纪 60 年代,曾产生过两种截然不同的设计观点。一种是针对具体应用领域待处理问题的特点和要求,设计面向某一领域的通用或专用程序设计语言,这是一种比较务实的设计观点;另一种观点企图设计一种大而全的通用程序设计语言,面向所有的领域,使用者根据自己的需要,选取语言中的一部分。这是一种求大、求新、求全的设计思想,其代表作主要有 PL/1 和 Algol 68。在 60 年代,大多数人都认为程序设计语言大而全的设计思想是正确的,因为层出不穷的高级程序设计语言已经让人应接不暇。特别是在欧洲,Algol 68 一度受到广泛的重视,不少大学、研究机构已经使用 Algol 68 语言编写程序。然而,由于当时发生了“软件危机”,学术界提出了软件工程的概念,试图用软件工程的方法来解决“软件危机”,因此,计算机界急需一种能够支持软件工程开发思想的高级程序设计语言。此时,关于 Algol 68 的许多语言成分还处于争论之中,加之 Algol 68 的语言描述十分复杂,编译系统的开发漏洞很多,而根据学科发展的规律,新型语言的生命力在很大程度上取决于是否被用户广泛接受,至于语言中存在的一些不足可以随着语言的使用不断完善,结果,当小巧、灵活、融合结构化程序设计思想的 Pascal 语言推出后,竞争的结果是第二种观点所作的努力不太成功。

有了高级语言,就得为其开发相应的编译程序。人们从最初高级语言解释程序的研制开始,在逐步深化语言理论研究的基础上渐渐形成了编译原理与技术的方向。要对语言使用计算机进行准确的翻译,就必须提供语言的语法和语义的严格的定义与描述。早期,编译程序并不是基于严格的语法定义和语义描述设计的,实现技术上主要是采用状态矩阵和递归子程序法,但随着研究的深入以及编译程序使用和维护中出现的问题,语法和语义成为一个语言文本必须要解决的问题。巴科斯和瑙尔在 Algol 60 语言的语法描述中提出了一种新的表示法,简称巴科斯

范式或巴科斯-瑙尔范式，引起学术界的关注。可是，后来人们发现，巴科斯范式虽然是独立地提出的，但本质上等价于乔姆斯基提出的上下文无关文法。

顺便，我们要提到这样一个事实，高级程序设计语言 Algol 68 的设计和推广虽然不成功，但是，研究工作并不等于没有意义。实际上，设计 Algol 68 时，研究小组首先发展了一种新型的描述语言的文法，叫做 W-文法。这是一个双级文法，它由两级上下文无关文法组成，第一级文法产生第二级文法的语法生成规则，第二级文法才生成真正的 Algol 68 语言，通过这种方式，在描述能力上等价于短语结构文法。使用 W-文法，在设计 Algol 68 时，研究者还提出了一些程序设计语言的新概念。所有这些，都对以后的研究工作有积极的参考价值。

乔姆斯基是一个语言学家，他在研究人与语言之间是否存在一种天然关系的问题时，于 20 世纪 50 年代后期创立了转换生成语言学理论。该语言学理论的核心思想在数学上表示为语言可以通过字母表和表上定义的一组语法规则（含词法和文法规则）经演绎过程生成。而且，语言可以按其文法规则的复杂程度进行分级分类，并通过对对应复杂程度不同的文法规则表现出来。于是，文法按照其文法规则的复杂程度被分成四个类，即短语结构文法、上下文有关文法、上下文无关文法和正则文法（或正规文法）。之后，60 年代进一步的研究表明，不同的文法类实际上分别同当时已建立的一些自动计算机模型是等价的，这其中包括图灵机，线性界限自动机，下推自动机和有穷状态自动机。尽管如此，今天许多语言在设计时依旧采用巴科斯范式描述其文法而不是使用自动机，原因是巴科斯范式比较简洁，并已为计算机界大多数人所接受。

上述成果极大地影响了后来计算科学的研究与发展。首先，乔姆斯基的理论使语言学的研究从结构主义语言学进入了一个新的时期——转换生成语言学时期。使用乔氏理论对计算机程序设计语言进行应用研究，很快为编译程序中的语法分析提供了严格的数学方法和计算技术。其次，语言与计算机之间建立的联系扩展了人们对计算模型的认识。许多人开始认识到，计算模型不应简单地理解为描述科学计算（早期为函数计算）过程的一种抽象机器，而是能对一类构造性的能行过程进行描述的一种抽象的形式系统。这种抽象的形式系统可以是一种数学机器，也可以不是；可以是静态的，也可以是动态的。而且，关于语言语义的研究和其他许多问题的研究都可归结为对计算模型的研究。实际工作中，从使用的角度出发，有必要发展多种计算模型。

例如，1962 年，德国人佩特里（C. A. Petri）针对并发现象的描述和处理问题，在他的博士论文中发展了一种数学理论——网论，亦称 Petri 网。这是一种计算模型，它使用一个可以在网上流动的叫做信息流标识（Token）的概念，可用静态的数学定义方式构造和描述动态的并发现象。1965 年，其博士论文的主要内

容刊在“Information and Control”杂志上,至70年代初受到美国军方和欧洲一些国家的重视,到现在已在理论与应用中取得了很大的进展。Petri网特别适合描述各种具有并发现象的系统,如分布式系统等。作为一种计算模型和计算科学理论的一个研究方向,今天仍有不少人在继续进行其基础和应用研究。

在语法问题基本解决之后,随着高级语言中各种新语言成分或机构的出现,以及对高级语言一般性理论和程序理论研究的深化,在开发高质量的高级语言编译程序的工作中,高级语言的语义处理成了当时一个棘手的问题。下面这个例子就是高级程序设计语言中的一个典型的语义问题的示例。

例1 条件子句与条件语句嵌套的语义问题。

在许多程序设计语言的控制语句中设计了条件子句与条件语句,它们的嵌套在语法上没有问题,但在语义上有二义性。我们先给出条件子句与条件语句的格式,然后引出问题。

条件子句: if B then S; 其中, B 是布尔表达式 (一种取值为真假的公式), S 是任何语句;

条件语句: if B then S₁ else S₂; 其中, B 如上, S₁, S₂ 都是任意语句;
如果我们给出这样一个由条件子句与条件语句嵌套构成的语句:

if B₁ then if B₂ then S₁ else S₂;

那么,对这个语句,语义上我们应该作下面的哪一种理解呢?

if B₁ then (if B₂ then S₁ else S₂);

if B₁ then (if B₂ then S₁) else S₂;

这的确是一个不能忽视的问题。

为解决语义问题,从1964年兰丁(P. J. Landin)发表“表达式的机械化求值”一文起,计算科学家陆续发展了若干种语义描述方法,并形成了形式语义学研究方向。各种形式语义学理论都是将计算机程序设计语言的语义描述,如表达式的求值,各种语句的执行内涵,过程调用时各种变量值的传递等语言各种成分的功能的语义用数学的方法进行严格的描述。由于对程序执行中各种程序变量的变化的观察角度不同,使用的数学思想和工具不同,对语言语义的认识观点不同,产生了不同的语义理论,如操作语义、指称语义、公理语义、代数语义等。形式语义学最初是从程序设计语言语义的研究开始的,但不久就被拓展应用到软件工程、通信描述、计算语言学、数据理论等与高级语言密切相关的许多分支学科,触及到了学科比较深层次的内容。程序设计语言的语义描述常常是与该语言描述的程序联系在一起的。从上面语言成分二义性的例子可以联想到语言的语义也可能与上下文有关。因为描述语言成分的语义时,不可能通过穷举语言成分出现的各种可能的程序来刻画语义,而且程序设计语言语义的形式化描述方法不易为一

般人所理解,因此,至今为止,所有语言第一次公开发表时,大多采用自然语言描述语言的语义,或使用语义清楚的元语言描述语义。正是由于语义学涉及到学科比较深层次的内容,在发展的早期还不是很成熟,掌握语义学方法对大多数人来说不是一件容易的事情,于是,当新的高级程序设计语言第一次公开发表而没有采用形式语义学方法刻画语言的语义时,致使一些学者误认为语义学的研究多少有点故弄玄虚,大量关于语义学研究的论文不过是为了语义学而语义学,为了形式化而形式化。但是,这样一种看法所下的结论太早了点,很快形式语义学方法在自然语言处理、形式化软件开发方法学、编译系统构造、计算机网络及其软件开发、数据表示理论、人工智能的基础等领域的研究中取得了重要的应用研究成果,发挥着重要的作用,并在语言的语义模型与逻辑系统的语义模型之间殊途同归。

程序设计语言语义的形式化描述常常是与计算模型联系在一起的。例如,在过去发展的合理的、功能足够强大的与图灵机在计算表达能力上等价的计算模型中,就有刻画电路计算的计算模型。电路可以表示计算可能谁也不会有异议,但是电路可作为基本的计算模型可能许多人就不曾注意到。实际上,如果我们从语言学的角度考察电路,单个分离元器件可以看成是语言的符号,单一功能基本(单元)电路可以看成是语言的单词,具有多功能的组合电路可以看成是语言的句子,由单元电路和组合电路等通过各种合理的方式组成的一个复杂的系统,可以看成是该语言写成的一篇文章。元器件和元器件之间的连接方式和约束条件,单元电路、组合电路相互之间的连接方式和约束条件实际上就是语言的文法。于是,单元电路通过输入、输出表现出来的功能就是该单词的语义,同样,组合电路、系统的功能就分别是语言句子和文章的语义。从这个意义上说,关于电路分析、设计、应用的研究,可归结为语言学的研究,完全可以运用语言理论的方法进行处理。在过去的十几年里,有关 VHDL 语言和(超)大规模集成电路设计的数学理论的研究实际上反映的就是这个认识观点。这个认识明确地告诉我们,在涉及电路设计与分析的研究中,从计算机科学的角度的观察,电路不应该是设计出来的,而是根据用户的需求描述通过多级理解计算得出来的;电路的功能不应该是通过测试得出来的,而是根据描述语言对其语义进行计算之后得到的。

下面,我们简要介绍形式语义学中四种语义学研究方法的主要思想和内容。

操作语义的基本思想是通过指明程序语言中每一成分在执行过程中“值”的变化来刻画语言的语义,以避免描述的语义依赖于该语言实现时所用的具体计算机。通常的做法是先设计一个抽象机,定义一组抽象状态,语言的成分用抽象的符号表达并建立语言成分执行前与初始状态之间的对应,然后,一步一步地指明抽象机每加工一个语言成分时状态作何改变,以此方式确定语言成分的“值”,

也就是语义。抽象机可以是类似于图灵机那样的带状态的数学机器，也可以是自身语义明确的一个可描述语言成分执行的另一个语言（通常称为元语言）。这种语义描述方法与语言实现的关系比较紧密，比较难于用数学方法处理，而且对语义描述者具体使用的实现方法依赖很大。操作语义是许多从事计算科学工作的专业人员比较容易接受和掌握的语义学方法，原因是在 20 世纪 60 年代，许多人由于从事程序设计语言的编译研究，对形式语言和自动机理论比较熟悉。在目前许多编译程序的设计和实现中，采用语法制导的技术方法处理语义常常是典型的操作语义方法，具体实现时，采取在一旦判明某个语言成分符合语法规则时，转而调用语义处理子程序对其进行代码生成（即语义处理）。

操作语义一般被认为是一种过程性描述语义。操作语义思想虽然简单，但是，用操作语义方法刻画程序设计语言语义也存在不足之处，当被处理的程序比较复杂时，抽象机状态数量的增加必然带来抽象机本身复杂程度的提高，也会带来从高级语言到抽象机影射复杂程度的提高。所以，如何设计结构简单而又功能强的抽象机就成为一个关键问题。

指称语义的基本思想是使语言的每一成分对应于一个数学对象，描述方法并不关心语言成分的执行过程，而只关心语言成分执行后的效果。这样就可以通过不依赖于语言成分实现方法和实现过程，仅通过对语言成分所指称的对象执行的效果来刻画语言成分的“值”。

由于指称语义学的思想是引入数学对象来表达语言成分，而语言的许多成分中具有常量、变量、递归、迭代、复合、分情形定义等数学表示形式，于是，通常实现这一设想的途径是引入一组递归定义的定值函数，设法将语言成分与定值函数建立对应，这些函数把语法域影射到语义域，用定值函数来刻画语言成分的语义。当然，对这样一组定值函数的语法和语义，事先应有严格的定义，语义上没有二义性。指称语义方法研究的关键是是否存在这样的一种函数及其体系，能够表述所有的计算？如果这样的函数及其演算确实存在，那么，理论上是否有严格的数学基础（逻辑模型基础）？幸好，函数体系的存在问题被解决了，指称语义得到了长足的发展。

指称语义的思想在计算科学哲学思想的研究中具有典型的代表性，主张对语义的研究应该将语言的定义描述（包括语法和语义）和语言的实现方法相分离。这样一种思想应该说具有合理性，但是，指称语义毕竟是用静态的方法来刻画语言的语义，一旦语言描述的程序具有不确定性，语义的描述也就随之复杂起来，要准确描述决非易事。

公理语义是在程序正确性公理化证明方法的基础上发展起来的。公理语义的基本思想与操作语义和指称语义的思想不同，它首先将语言的每一种成分与一种

数学变换法则建立联系，然后将语言成分的执行看成是某一数据集合上的一组操作，而且，在程序执行的每一个时刻，程序中任一语言成分执行的前后，程序各个量之间的关系或应满足的性质均可用一个（前）逻辑谓词和一个（后）逻辑谓词组成的一个谓词对表示。这样，语言成分的语义就可以从（前）谓词出发，通过运用该语言成分对应的数学变换法则和逻辑推理规则按逻辑演算得出（后）逻辑谓词来刻画。由于无法设定所有可能的各种（前）逻辑谓词，因此，语义的刻画只能通过对各种语言成分提供一组数学变换法则并借助于逻辑系统的演算特征和演算规则用抽象谓词及其演算来定义语言成分的“值”。读者可能会问，指称语义和公理语义似乎在思想上相近，它们都不关心程序中语言成分的执行过程，而只关心执行以后的结果。但是，这两种语义是不同的，公理语义中关于程序中某一语言成分执行后语义是否正确的判定，是通过一个谓词公式是否能够满足来刻画的，所涉及的程序变量不必像指称语义和操作语义中的状态那样，涉及到程序中的所有变量的取值，而只涉及到该程序片段中与前后谓词有关的变量取值就可以了，这正是采用逻辑推理方法的优点。而在指称语义描述中，关于程序中某一语言成分执行后语义是否正确是由定值函数来保证的。

代数语义的提出源于抽象数据类型的研究，其基本思想和方法源于模型论，它将描述语言成分语义的逻辑体系与满足这个逻辑体系的模型区分开来。任何操作语义或指称语义对语言成分的语义描述只给出了该语言描述的某个包含该语言成分的程序的语义模型，而公理语义的描述只给出了该语言描述的包含语言成分的一个逻辑系统，两者都没有解决在满足程序语义的诸多模型中，究竟哪些是设计者想要的，那些是设计者不想要的。不解决这个问题，程序设计的自动化仍然是困难的。由于采用操作语义、指称语义和公理语义在对语言成分进行语义描述时，符合操作语义、指称语义和公理语义的某个语言成分的执行过程（对应为语言成分不同的实现方法）可能不止一种，因此，用户不清楚哪个语义模型是需要的，哪个是应该删除的。换言之，用户希望对语言成分的语义解释通过筛选是惟一的。要做到这一点，就必须研究逻辑体系的模型，为每一种语言的成分确定其在该逻辑体系中如何解释，并用代数方法处理各种模型。

无论是哪一种语义方法，任何语言严格意义下的语义应该是建立在模型论基础上的语义。由于四种语义所采用的数学工具从计算模型角度看都具有很强的计算能力，而且，合理的、功能足够强的计算模型都与图灵机模型在计算能力上等价，而图灵机与（部分）可计算谓词是等价的，因此，不少人猜测：四种不同的语义方法在描述语言的语义时，在描述能力上应该是等价的，而且，在计算模型的层面上，不同的描述相互之间可以转换。由此可见，这里有可能再次出现类似于相似性与对偶性的问题。

程序设计语言的研究与发展在产生了一批成功的高级语言之后,其进一步的发展开始受到程序设计思想、方法和技术的影 响,也开始受到科学计算、程序理论、软件工程、人工智能等许多方面特别是实用化方面的影响。在“软件危机”的争论日渐平息的同时,一些设计准则开始为大多数人所接受,并在后续出现的各种高级语言中得到体现。例如,用于支持结构化程序设计的 Pascal 语言;适合于军队各方面应用的大型通用程序设计语言 Ada;支持并发程序设计的 Modula-2、并发 Pascal 和 Java;支持逻辑程序设计的 Prolog 语言和 Gödel 语言;支持人工智能程序设计的 LISP 语言;支持面向对象程序设计的 Forth, Beta, Smalltalk, C⁺⁺;支持软件开发和程序验证的 XYZ 语言等。而且,伴随着这些语言的出现和发展,产生了一大批为解决语言的编译和应用中所出现的问题而发展的理论、方法和技术,如 CSP 和 CCS。CSP 是一项非常有创造性的研究成果。从计算模型的角度可将 CSP 看成是一种计算模型,它通过抽象程序设计语言中的各种运算操作,提供一组抽象运算操作,包括通信操作,可以对进程给出运行描述。从程序设计语言的角度又可将 CSP 看成是一种程序设计语言,因为 CSP 中的各种抽象运算操作本身包含语法和语义,又能描述分布式程序。类似地,CCS 也是一种刻画分布式程序语义的计算模型,该模型以表达式为基础,用表达式表示进程,通过在表达式之间建立一组等价关系,研究和观察程序的语义。要注意的是 CCS 同时也是一种语言,具有自身的语法和语义。当初设计 CSP 和 CCS 的主要目的是用于并发程序中进程之间通信的描述,并未考虑实用,因此,语言本身的设计比较抽象和简洁。有大量的学术论文可以证明,由高级语言的发展派生的各种思想、方法、理论和技术触及到了计算科学的大多数学科方向,但内容上仍相对集中在语言、计算模型和软件开发方法学方面。

近十多年来,特别是并行计算机系统及其应用软件的开发和发展,极大地刺激了软件开发方法学的研究与发展。这是因为一方面,许多领域的计算机应用对开发的软件提出了高可靠性和安全性的要求。例如,加拿大一核电站的建设因电站自动化控制系统的软件设计的可靠性原因而迟迟得不到政府签发的准运证。1996 年欧共体的欧洲航天局花费 40 亿美元巨资历时 10 年研制、建造的航天飞机,由于软件系统的故障而在上天的刹那间发生爆炸,多年的努力转眼之间付之东流,损失惨重;后经事故调查分析,原因仅仅是航天飞机上控制飞行的软件系统是从以前的系统上移植的,未充分考虑新型号航天飞机增加的功能。由于计算机软件设计中存在的隐含的错误而导致航天发射试验出故障的事决非仅此一例,前后就有多次的记录。2000 年 12 月 6 日,上海浦东的电话系统因故障停用达 12.5 个小时,后经查明原因是由于电话系统使用的美国贝尔实验室开发的交换机通信软件出错引起的故障。可见,随着软件开发复杂程度的提高,软件的可靠性成为

继程序结构与效率之后的又一个十分重要的问题。另一方面，计算机在各个具体应用领域的深入应用又产生了一些新的软件开发模式和程序设计风格，导致软件开发方法学研究出现了一些新的特点，主要表现在：对程序设计语言描述能力的要求不断提高，许多过去容易检验的程序操作正在转而交由计算机系统软件来完成，从而导致用户对软件正确性和安全性的要求不断提高；过去被认为不实用的形式化的软件开发方法已在一些应用开发中取得成功，并有望在不久之后逐步走向成熟；对各种程序设计方法的研究更注重同具体应用领域相结合，向实用化靠拢；各种软件系统复杂程度的不断提高带来了软件体系结构的问题，软件的开发需要像计算机系统一样充分考虑各部分相互之间的结构关系与任务分担问题，并与计算机系统（包括硬件、系统软件与（网络）开发环境）形成合理的分工和协调。这方面的研究在融合面向对象程序设计方法之后产生了统一建模的软件开发方法和技术。由于上述特点，高级语言的研究从各方面考虑都需要与较为成熟的一些软件开发方法相结合，与程序理论相结合，体现一种或几种程序设计方法和风格，并为其建立严格的理论基础。

结合上面所阐述的语义学的概念和软件系统可靠性问题，我们可以从下面的内容获得启发和进一步的感悟。

例2 关于加法计算程序的语义

加法是一种最简单的计算，任何具有小学文化水准的人都不难理解下面的三种加法的计算方法：

第一种方法，传统的加法，运用加法的规则；

第二种方法，基于自然数定义的皮亚诺公理系统。只使用每次加1的一条规则，但是要使用一个前驱函数 $\text{pr}(n)$ 。例如， $\text{pr}(6)=5$ 。以后，对于任意两个数 m ， n 的加法，可以用下面的式子计算：

$$m+n = m + \text{pr}(n) + 1 = m + \text{pr}(\text{pr}(n)) + 1 + 1 \cdots$$

第三种方法，造一张 100×100 的二维表，用矩阵表示表中的每一个元素，这个元素的值是矩阵下标的和。计算 $m+n$ 的值，实际上是从第 m 行，第 n 列的表中去获取元素的值。

现在的问题是：对100以内的加法，三种方法对应的程序哪一个是正确的？如果扩大到100以上的加法呢？如果扩大到实数域呢？

显然，对100以内的加法，三种加法对应的程序都正确。但如果扩大到100以上的加法，第三种加法对应的程序就会出错，如对于 $m=56$ ， $n=71$ ，查表计算 $m+n$ 不会有正确的结果，因为表中根本就没有超出100的内容！进一步，如果加法扩大到实数域，那么第二种加法对应的程序也会出错，因为对于 $m=5.6$ ， $n=7.1$ ，程序根本就不认识带小数点的实数（在计算机内叫做浮点数），更不用说

计算 $m+n$ 了。

这就提出了一个问题：一个程序的正确性是由程序的内涵决定还是由程序的外延决定？

为了回答这个问题，我们可以从刻画这个程序的四种语义的方法来考察和分析。当然，这超出了本书的范围，读者今后可以在语文学课程中学习相关知识。

从外部观察一个程序，可以使用断言的方式。例如，对于程序 P ，我们有：

$\{Q\} P \{R\}$

其中， Q, R 是逻辑（断言）公式，分别为程序 P 的前断言、后断言。 Q 表示程序 P 执行前程序的状态（指影响程序运行的所有变量数据构成的数据状态）必须满足的条件， R 表示程序运行后所有与输入关联的输出数据构成的数据状态必须满足的条件。正确地选择 (Q, R) 这样一对断言，可以从程序的外部准确地刻画程序的功能。例如：

Q : 得到 10 元钱; R : 打回一瓶酒;

$\{Q\} P: S_1; S_2; \dots; S_n; \{R\}$ 其中 $S_1, S_2, \dots, S_n$ 是程序 P 的语句

于是，读者可以对照上面介绍的四种语义学方法的表述形象地进行如下描述理解：

操作语义：

$\{d_0 \mid Q(d_0) = \text{true}\} \quad P: S_1; S_2; \dots; S_n; \{d_n \mid R(d_n) = \text{true}\}$
 $\uparrow \quad \uparrow \quad \uparrow \quad \quad \quad$ 抽象机

用一台每一个操作在语义上完全清楚的抽象机跟踪、模拟程序中语句对程序的数据集合的操作，通过对程序状态的刻画来表述程序的语义。

指称语义：

$\{d_0 \mid Q(d_0) = \text{true}\} \quad P: S_1; S_2; \dots; S_n; \{d_n \mid R(d_n) = \text{true}\}$
 $\uparrow \quad \uparrow \quad \uparrow$
 $f_1 \quad f_2 \quad f_n \quad \quad \quad$ 定值函数

引入严格的语义函数，针对程序语言的每一条语句，给出对应的函数演算规则，这样，就可以根据程序的输入和每一条依次执行的语句，确定对应的函数的值，以此确定程序的语义。

公理语义：

$\{Q\} P: \{Q_1\} S_1 \{R_1\}; \{Q_2\} S_2 \{R_2\}; \dots; \{Q_n\} S_n \{R_n\}; \{R\}$

设法根据程序的每一条语句，由前断言和语言语句对应的推导规则，以及逻辑推理规则，包括代换推导规则导出后断言。

代数语义：

$$\begin{array}{c}
 \{d_0 \mid Q(d_0) = \text{true}\} \quad P: S_1 ; S_2 ; \dots ; S_n ; \{d_n \mid R(d_n) = \text{true}\} \\
 \qquad \qquad \qquad \uparrow \qquad \qquad \uparrow \qquad \qquad \uparrow \\
 \qquad \qquad \qquad \{R_1, R_2, \dots, R_m\} \qquad \qquad \text{代数操作规则}
 \end{array}$$

需要给出对应语言语句定义的代数操作规则，并且要证明这样一套规则在逻辑上语法和语义是一致的，也就是要给出代数操作系统的逻辑模型（证明可靠性与完备性）。

特别是代数语义，从 $\{Q: \text{得到 10 元钱}\}$ 到 $\{R: \text{打回一瓶酒}\}$ 的方式方法是很多的，例如，一个人完全可以走路去，走路回来，用 5 元钱买一瓶酒，另外 5 元钱自己吃了零食；也可以是来回都走路，买了一瓶 10 元的好酒；也可以是坐车去，坐车回，买一瓶 8 元钱的稍好一点的酒；……。问题是我们到底想要哪一种打酒的操作？显然这涉及到模型，也就是能够使得从 Q 为真变动到 R 为真的一系列代数操作规则必须符合我们关于打酒人的行为准则的解释，代数语义要保证代数操作规则和行为准则之间在解释意义下的完全一致。当这个行为准则比较宽松时，很多种打酒的做法（一种做法对应于一组代数操作规则）都可以认为是我们能够接受的，但如果行为准则比较严格，那么，很可能最后剩下的符合要求的做法很少，惟一，甚至没有。如果没有，就等于说在现有的行为准则解释下，该问题的求解没有代数语义模型。但只要找到了代数语义模型，自然如何打酒，以及哪一种打酒的方式方法是我们需要的也就全解决了。

注意，由于程序设计语言的种类很多，如递归程序设计语言，函数式程序设计语言，构件式程序设计语言，面向对象程序设计语言等，上述内容的深入展开表述将涉及到更多的内容，如针对各种各样的程序设计语言及其丰富多彩的语言成分和结构，具体怎么刻画语义？怎么找到一组断言，保证在前断言满足的条件下程序运行必定终止并输出满足后断言的正确结果？代数语义的其他形式，多种语义学方法的结合，等等。

从上面这个例子及其分析中我们可以看到前面提到的构造性数学、计算机科学理论基础的重要性。

20 世纪 60 年代和 70 年代是计算科学大发展的时期。各种类型计算机的出现，不仅需要为其配置汇编语言翻译程序，高级语言的编译程序，而且操作管理系统也从最初的操作管理程序向功能齐全的操作系统发展，形成了操作系统方向。这些研究与发展加上各种计算机应用领域的不断拓展和应用深化，极大地推动了程序设计语言理论（包括形式语言文法理论，形式语义学），编译程序，操作系统，应用软件程序包等方向的发展，特别是编译程序正在从语法制导的技术发展过渡到部分语义制导的编译技术，同时，也产生和促进了数据库系统，人工智能，软件开发方法学方向的发展。例如，在逻辑程序设计语言 Gödel 中引入了模块、延

迟计算、剪枝等语言成分；在数据库系统的研究中，发展了 SQL 查询语言和以 SQL 数据库查询语言为基础的各种派生查询语言；在人工智能的研究中出现了以 Prolog 和 LISP 为代表的逻辑程序设计语言和函数式程序设计语言；在软件工程及其环境的研究中设计出了各种用于描述不同层面软件和程序的语言，等等。值得一提的是，人们曾希望有朝一日设计出一种非常高级的程序设计语言，这种语言允许对算法的描述不必给出详细的数据结构，即算法与数据结构相分离，由编译程序在翻译时依据程序的控制结构来自动确定数据结构。SETL 就是这样一种程序设计语言，不过，至今还没有形成气候。

所有这些都说明高级程序设计语言中确实包含了非常多的内容，难怪对程序设计语言的研究被认为是从事计算科学研究的基本功之一。这就是说，如果一个计算科学工作者从事涉及高级程序设计语言的研究而又仅仅停留在一些具体高级语言内容的范畴里，无论如何是难以将研究工作引向深入的。现实要求计算科学工作者不仅要了解程序设计语言中重要的基本概念、语言理论、实现原理、程序设计方法与技术等内容，而且要了解程序设计对计算科学发展产生的重要影响和作用。

计算语言学是计算科学与语言学的一个交叉学科，其主要任务是运用数学和计算科学的理论与技术研究语言学的基本规律。它的一个直接应用是开发自然语言理解系统和机器翻译软件。形式语言与自动机理论，数理逻辑和形式语义学是计算语言学研究最主要的数学和计算科学理论工具，已有不少工作对西方语言学中的一些规律作了很好的描述。然而，中文自然语言的复杂性使得该方向很长时间未能取得真正的突破，其根本原因是在汉语的分析中，对词义项的选择，句型的分析，语义的理解，以及它们与语境和语用的关系等几方面缺乏完整的理论。例如，即使像如下一些看起来十分显而易见的句子，在处理上也常常是很困难的。

例3 他的老师当得好。

这个句子起码有四种以上解释的含义。比较合适的解释有几种，下面是其中的三种：

- A. 他的传授文化、技术的身份担任得好。
- B. 他的（某位）老师把知识、技能传授得好。
- C. 他的（所有）老师都把知识、技能传授得好。

实际翻译时，采用哪一个句子呢？怎么知道应该用哪一个呢？显然，这与上下文有关。如当上下文中出现“他的老师当得好，比我们学校的老师强多了”的句子时，其中“他的老师当得好”应该译成第三句。又如，“他的老师当得好，今天表现很出色”，显然是说老师今天的在某一方面发挥突出。在这种情况下，我们要找到准确的译句还不是一个简单的事情。如果我们不看上下文，常倾向于译

成前两句。

可见，语言的理解与上下文和语言的背景有密切的联系。如果我们不限于解释成上面三个句子所属的范围，也可能解释成啼笑皆非的含义。例如，分别将解释的范围设定在家庭的角色和国家的政治事务决策的参加者方面，我们何尝不能分别解释为“他的（某位）老师家庭主妇的身份担任得好”，“他的（某位）老师全国人大代表的身份担任得好”呢？

例4 选自语言学家吕叔湘自选文集的一个例子，“调个过儿还是一样”。

- A. 在家里，我对儿媳像闺女一样，儿媳对我也像亲妈一样。
- B. 在家里，我对儿媳像亲妈一样，儿媳对我也像闺女一样。
- C. 在家里，我对儿媳像亲妈对闺女一样，儿媳对我也像闺女对亲妈一样。
- D. 在家里，我对儿媳像亲妈对闺女一样，儿媳对我也像闺女对亲妈一样。

形式上，A和B是两个不同的句子，但语义是相同的。象这样的句子，一个机器翻译系统能保证将A和B都译成同一个英文句子吗？事实上，在句子A中添加“亲妈对”和“闺女对”可得到句子C，在句子B中添加“对闺女”和“对亲妈”就可得到句子D。很明显，A和B都省略了句中的某些语言成分，一旦添加而得到C和D同形，就知道C和D同义。这就涉及到如何建立上述句子的数学表示方法和变换规则，需要有一套科学方法。

有时候，中央电视台的新闻节目中也会报道国外在计算语言学方面研究的一些新进展。例如，我们大家曾不止一次地从电视新闻中看到这样一种场面：一个人正在用说话的命令方式指挥一台计算机操作，或操作一台机器人工作。这类由科学家开发的语音处理系统在今天已不是科学幻想，而是现实。但是，语音的理解在中西文的自动处理中都是更为困难的领域，其困难主要在于对重音字、多义字词、上下文、语境和语用的处理缺乏严格的理论。我们仅从下面一例就可以明白，所谓电视上介绍的语音处理系统，或者功能有限，或者离实用尚远。

例5 有一则广告词，是南京一家出产加佳牌洗衣粉的生产厂家打出的。“家家爱加佳，加佳进家家”。

这一洗衣粉广告词的书面含义不难理解。但是，如果不看中文字的书面表述，仅仅知道加佳洗衣粉这一品牌和听到这句广告词，即使每一字的读音都不听错，也难免会理解成下面的句子：

- A. “加佳爱家家，家家进加佳”；
- B. “加佳爱家家，加佳进家家”；
- C. “家家爱加佳，家家进加佳”。

从语义上看，A和B的句子与原句是不同的，它们都使人联想到厂家的优惠销售，或免费试用的意思。C和原句比较接近，但C含有因喜欢而导致家家主动

购买的意思，而原句不仅包含主动购买的意思，还兼有被动接受（当然是乐于接受）的意思，它使人联想到厂家也许有送货上门的服务举措。在译成英语时，不同的句子，意思不一样，语态也可能是不同的。进一步，如果我们不限定原句是在洗衣粉广告词的范围内，就可能将加佳理解成是一个人见人爱，他（她）的事迹和精神进入了千家万户的英雄人物呢！不过，现在的中国人谁也不会作这样的理解，因为，每个人的头脑中有一些常识，那样的英雄人物是雷锋而不是加佳。

因此，离开了理解者的背景知识，离开了特定的环境（语境），离开了上下文，不限定语用范围，理解将是十分困难的。

从这几个例子可以看出，自然语言的理解和翻译是非常困难的。特别是汉语，类似这样的句型可以说是五花八门，种类繁多，不胜枚举。何况自然语言是不断发展变化的，它使得建立自然语言的计算模型变得相当困难。面对这些一时难以解决的困难，目前一些实用系统的开发基本上是采用缩小应用范围或简化句型的结构。例如，开发外贸专用自然语言处理系统，开发科技论文自动文摘系统等。

我们曾经提到，计算科学这个学科的一个重要的特点是面对困难的问题，研究一旦走向深入，首先是发展计算模型，自然语言处理也不例外。对于语言的研究，早期大多数语言学家都是从分析实例的角度来分析语言的，很少有人能够运用数学工具对语言进行研究，更不用说建立计算语言学的概念了。实际上，语言处理的困难在于缺乏一整套计算语义学的模型。由于自然语言是一个发展的概念，在民间的使用中语言本身处于不断发展和变化的状态，是一个有着大量例外情形无法用统一的、现成的语言规则刻画的研究对象。特别是像汉语这样的自然语言，在相同语法和语义规则的约束下例外太多，这就决定了要找到一种计算模型刻画语言完整的语义是非常困难的。按照现有的语言学理论，更多的学者相信：在语言的发展过程中，理解语言的过程是语义先导的，而不是语法制导的，没有人在日常的交流中为了理解一句话而先将这句话是否符合语法规则的问题先检验一遍，然后再作语义理解。少数高水平的学者相信：既然人本身可以看成是一个能够处理语言的计算模型，那么，他在理解语言时，应该是将各种语言的句子根据自己的判断作分类理解处理，语言的语义应该是这个计算模型计算的结果，只不过对一类句子的处理他采用了一种更专用的计算模型而已；此外，真正的语义学应该是模型论意义上的语义学，语言中最核心的语义理解的要素应该是逻辑，这就使学者们有理由认为，如果一个语言真的存在计算语义学模型，那么，这个模型应该到语言的逻辑基础中寻找，运用与计算科学存在密切关系的数理逻辑的方法去寻找，运用代数的方法建立语言的计算语义学模型。

近年来，国外在这方面的研究主要是采用数理逻辑和语义学理论建立自然语言的计算语义学模型，依靠严格的理论指导开发相应的自然语言处理软件系统。

具体说来,针对自然语言的处理,从计算科学的观点来看,语言的语义是计算出来的,而不是通过语法制导分析出来的。应该努力设法对一类句子的理解建立相应的计算语义学模型,将语言句子的语义作为该模型对句子的计算结果,将该类句子的语法作为理解句子语义的一种辅助信息。当然,在研究中困难很多,第一个困难就是如何对语言句子中的单词给出抽象的数学表示?如何选择单词合适的词义项?例如,“我马上就来。”这句话的“马上”如何用数学的方法表示?这些问题不解决,要实现对语言语义的计算是不可能的。我国知名学者上海交通大学的陆汝占教授在汉语的计算语义学方面按照上述构想开展了具有重要科学意义的开创性研究,从内涵语义的角度采用数理逻辑方法解决了一系列著名汉语理解疑难句型的计算语义学模型,开创了汉语计算语义学研究方向,提出了一条一揽子解决汉语自然语言理解的研究规划和发展道路,初步建立了汉语计算语义学理论的雏形,其研究成果和研究方法得到国内汉语语言学界众多权威学者的高度评价,初步形成了独立的中国计算语义学学派。

在计算语言学的研究中,目前有一种研究路线是使用数理统计方法对大量语言现象作统计,如句子中指称对象的判定,省略成分的补充等,试图从中发现和找出规律,建立语料库以支持机器翻译系统的开发。但是,这样一种研究路线实际上是一条纯实验技术的路线,而自然语言是不断发展变化的,特别是像汉语这样的自然语言,在相同语法和语义规则的约束下例外太多,使用数理统计的方法来处理必然导致对大量例外语言现象的处理无能为力,这也在一定程度上解释了现有的自然语言理解和机器翻译系统的准确率难以有实质性提高的原因。数理统计学方法本身也存在局限性。当被统计对象的属性信息项较多时,统计的结果就容易产生模糊化的问题。这就好比关于什么是高水平大学的统计学概念问题,如果能够成为高水平大学的属性信息项很多,如把学校的科研经费很多是一个高水平大学的属性信息项,运动水平高也是高水平的属性信息项,校园环境优美,教学质量高,科研论文多等等都是,那么,统计的结果哪所学校是真正的高水平大学这个概念必然模糊化。同样的道理,语言的每一个单词有多个(字)词义属性信息项,如果要通过统计学的方法发现这些个(字)词在什么情况下该怎么用,该做什么解释,从中发现语言学更一般的规律并抽取出新的概念和规则,那么,这些概念和规则本身必然存在模糊化的问题,因为数理统计学的方法本身是一种外延的方法,并不是一种刻画语言内在规律的内涵方法。由此不难推测,大量特殊语言现象和统计方法的不精确性都注定了单纯的实验技术路线成功的可能性甚小,更不用说去处理像语感、行文风格之类的问题了。当然,作为一种辅助性的技术,对大量语言现象作统计和据此建立语料库还是需要的,它有可能帮助系统缩短理解和翻译的计算时间。

大多数计算科学工作者熟悉的语言主要是自然语言和程序设计语言。事实上，除了这两类语言外，还有数理逻辑中的一阶或高阶语言，以及用于数学内容陈述（定义、定理及其证明等）的数学语言。在计算科学的发展史上，有相当大一批计算科学家是从语言研究起步的，或曾在语言的研究中作出过贡献。如 Chomsky, Knuth, Wirth, Wijngaarden, Hoare, Dijkstra, Hansen, McCarthy, Landin, Wegner, Gries, Backus 等人。不仅国外如此，国内也一样。虽然，自然语言、逻辑语言、数学语言、程序语言等各不相同，但作为语言，它们都具有语言的某些共同的系统特征，而这些系统特征常常反映出程序的性质，推理的性质，语言成分值的确定（公式的解释），等等。不仅如此，语言的研究在方法论上也有助于计算科学的研究与发展。语言学研究中首先提出的许多概念、方法和技术在计算科学的研究中被广泛使用。针对不同的用途，在各个不同的时期，先后发展了多种程序设计语言及其语法和语义的表示方法，如各种程序设计语言、BNF，语法图，Horn 子句，兼有语法和部分语义描述功能的属性文法、描述和管理数据库中数据的数据定义语言（DDL），支持形式化软件开发方法的维也纳定义语言（Vienna Definition Language, VDL）等。近年来，支持多种程序设计风格的程序设计语言的研究方兴未艾，像面向对象的语言，约束程序设计语言，实时程序设计语言等都有大量的文献发表。可以预见，包括上述语言在内的泛语言学研究仍将是一个长期活跃的研究方向。

20 世纪 80 年代是计算机网络、分布式处理和多媒体大发展的时期。在各种高级程序设计语言中增加并发机构以支持分布式程序设计，在语言中通过扩展绘图子程序库以支持计算机图形学程序设计成为当时程序设计语言的一种时尚。之后，在模/数、数/模转换等接口技术和数据库技术的支持下，通过扩展高级语言的子程序库又实现了多媒体程序设计的构想。进入 90 年代之后，并行计算机和分布式大规模异质计算机网络的发展又将并行的程序设计语言，并行编译程序，并行操作系统，并行与分布式数据库管理系统等并行软件的开发任务提了出来。在实际的研究工作中人们发现，这些软件开发的关键技术依然与高级语言和计算模型密切相关，如各种并行、并发程序设计语言，进程代数，分布式事件代数，Petri 网等，它们正是软件开发方法和技术的研发中支持不同阶段软件开发的程序设计语言和支持这些软件开发方法和技术的理论基础——计算模型。

例 6 生产与消费问题。

这个问题可以结合面包生产和销售这样叙述：面包公司有两个（以后可一般化为 m ）生产面包的作坊和五个（以后可一般化为 n ）分布在城市里的面包专卖点。由于面包的保质期比较短，公司生产的面包产量必须根据产品的销售情况动态地调整作坊的产量。为此，公司首先建立了一个（以后可一般化为 t ）仓库，

用于面包产品的暂存。为了防止出现差错，公司规定，任一时刻至多只能有公司的一个专卖点进仓库取货，或者有一个公司的面包作坊进仓库送货。现在的问题是，我们怎样建立一个系统，使得公司的生产和销售能够正常地自动进行，不会出现仓库没有货时某个专卖点在里面取货，仓库装满货时某个作坊还往里面送货？而且，我们还要保证每一个面包作坊都有机会将自己生产的面包送入仓库，每一个专卖点也都有机会在需要时从仓库中取到货。

对于这样一个问题，可以有多种解决的办法。利用 Petri 网，可以很容易地描述这个问题的求解办法，表达十分简洁。当然，使用并发程序设计语言、面向对象的程序设计语言也都能处理这个问题。这个例子说明，在学科的发展中，从使用方便的角度出发，发展多种理论、技术工具具有重要的意义。

早期，软件开发工作的一个深刻的经验和教训是不重视软件开发工具的研究，许多软件的设计都是一切从头开始，从未考虑过借助软件开发各个阶段的开发工具，采用软件描述技术、程序验证与测试技术、程序调试与代码优化技术、软件重用技术等。值得一提的是，20 世纪 70 年代中期以后，随着计算机软件和计算机应用发展的不断深化，一方面，大量工具软件不断出现，另一方面，软件开发不断呈现出大型化、高投入、高成本的特点，传统的手工作坊式的软件开发方式已不能适应产业发展的需要，软件的生产急需一整套开发方法、支持技术和管理手段的支撑。于是，软件开发方法学应运而生，其典型的应用之一便是用工程的思想方法支持软件的开发，即软件工程。然而，软件开发方法学是一个非常复杂的研究课题，不同的哲学思想和对软件及其开发过程的不同认识会导致截然不同的研究与开发方式和路线。虽然，到今天为止，还没有一个成熟、实用的支持软件生存全周期的系统的软件开发方法论模型及其产品真正有效地投入到各种软件的开发过程中，但在研究工作中确实发展了一大批软件开发模型、方法、技术和工具，其中许多是计算模型和以模型为基础的产物，主要是高级程序设计语言及其开发工具和环境。目前，计算机软件开发环境的研究主要集中在计算机辅助软件工程（CASE）技术方法和程序理论支持下的形式化软件开发方法上，其他一些主要以软件开发经验、程序设计规范和标准化技术支持下的软件开发方法也还在一些公司实行，但不占研究的主导地位。

在计算机发展的早期，软件的开发基本上是逐个进行的，程序设计和软件的开发基本上是一种个人技艺占主导地位的个人行为，其突出地表现在对同一个问题，不同的人设计的程序和软件系统差异甚大。在那时，程序设计方法和软件开发方法是几乎是同义词，两者是混淆在一起的。一般地认为，从 20 世纪 60 年代起，软件开发方法学的研究开始起步，它是从程序设计方法的角度自然地发展起来的。最初，由于软件的代码量比较小，程序设计因受机器资源的约束，先后发

展了虚拟程序设计技术方法、子程序设计技术方法、自顶向下设计方法、自底向上设计方法、结构化程序设计方法、程序推导方法、程序变化方法，并发程序设计方法、面向对象的程序设计方法、函数式程序设计方法、逻辑程序设计方法，等等。虽然，60年代关于程序好坏的讨论在70年代中期趋于一致，程序设计方法学和程序设计规范的概念被广泛接受，但是，如何从宏观上认识软件开发的全过程，怎样建立起一套完整的软件开发的理论、技术、方式和方法，解决诸如像如何保证程序的正确性，如何缩短软件开发的周期，如何将不同的人设计的程序集成在一起，如何开发分布式和并行计算机上的软件这样的问题，使软件开发不再成为一种个人技艺，而是能够适应工业化大规模软件生产要求的一种工业生产方式、方法和技术，确实需要进行细致的研究。这些，正是软件开发方法学研究的范畴。以后，随着机器各种性能的不断改善，软件开发方法开始从微观的程序级别转到宏观的以程序和进程为单位的软件开发方法的研究，在开发了一大批软件开发工具的同时，先后发展了构件式软件开发方法、计算机辅助软件工程开发方法，原型化软件开发方法、软件移植方法、嵌入式软件开发方法、各种形式化程序设计方法，并进一步注意到软件开发过程中的工程化管理问题和质量控制问题，产生了软件开发规范和质量管理认证（CMM），等等。

例如，计算机各种软件系统的人机界面随着图形学和窗口技术的成熟，以及多媒体技术的发展，现在越做越复杂，其程序代码所占整个程序系统代码量的比例也越来越高。为了缩短软件开发周期，使人机界面更友好，应考虑在开发中将系统的核心程序（即反映该软件系统本质功能的程序部分）与人机界面的开发分开进行。对人机界面的开发，人们希望通过对其功能描述，由一个事先设计好了的人机界面生成系统来自动生成。当然，这种生成过程应完成与软件系统核心的处理程序的自动连接。显然，这样一种开发软件的思路与传统的做法不同，它更多地体现的是一种软件开发的**思想方法**。

又如，并行与分布式计算机系统的发展，急需开发其上的各种软件系统。我们知道，经过几十年的努力，顺序计算机系统上的各种软件的开发已经相当富有成效，从系统软件到应用软件，已有大量产品。但是，并行与分布式计算机系统上的软件开发工作却十分薄弱。即使像并行程序设计语言这样最基本的软件开发工具，也还不是很成熟。目前能够看到的并行程序设计语言，有的是从顺序程序设计语言改造而来，即使新设计的功能也很有限。原因是一方面，并行机系统的用户不多，软件开发以后的经济收益相对较低；另一个原因是并行机系统上的软件多与计算机体系结构有关，不同并行机系统上的软件常不能通用，而且开发周期更长。导致并行机系统上软件开发更为困难的主要原因是由于程序并行、并发、通信和体系结构的日益复杂而引起计算的不确定性，使得系统的开发不仅在控制

上,更主要的是在语义的正确处理上困难加大。

在过去的二十多年里,计算机网络的快速发展和大量分布式(应用)软件系统的开发实践,使人们开始意识到,从分布式软件开发可移植性、安全性、资源共享、鲁棒性、透明性、可扩展性、容错计算能力等多个方面考虑,软件开发有必要认真考虑软件的体系结构,合理分工和平衡负载软件系统各子系统的任务,形成良好的系统结构,同时,支撑网络上大量用户的透明使用。经过近二十年的努力,以网络计算机系统为背景的分布式计算机系统上的软件开发在基础研究方面取得了重要的进展,从分布式人工智能(DAI)研究中发展起来的多 Agent 与智能 Agent 技术,从互联网上发展起来的语义 Web 和 Web Service 三个重要的技术概念支撑的分布式软件开发方法得到广泛应用,逐渐成熟,在它们的背后,有着客户端-服务器体系结构技术(Client-Server Architecture)、多线程、软件体系结构等一系列软硬件技术的有力支撑。但是,在涉及网络信息安全、容错计算、通信的可靠性方面还存在很多问题。可以预见,分布式与并行软件开发方法学的研究将是一个需要长期努力的方向。

这些都从另一个侧面证实了计算模型、语言和软件开发方法学确实构成了计算科学发展的一条主线。

3.3.4 应用数学与计算机应用

当第一台电子数字计算机诞生后,计算机应用的大幕便开启了。计算机应用是一个范畴很大的应用领域概念。广义地说,凡是与计算机使用相关联的领域都可纳入计算机应用的范畴。但是,任何学科都有其基本的研究范畴和支持整个学科赖以发展的核心内容,正好像打拳一样,各种流派虽然套路很多,但自成一体的门派必是万变不离其宗。计算科学也一样,支持计算科学向各个学科渗透、应用和发展的正是一些最基本的共性理论、方法和技术。对许多领域的计算机具体应用所使用的理论和技术作深入的分析,其结果反复证明了这一点。也正是从这一认识开始,在计算科学界,人们开始将计算机应用于各行各业的计算机具体应用与研究计算机应用于具体领域的共性理论、方法和技术学问区分开来。前者叫做计算机具体应用,它们应该也已经划入具体应用的学科,后者称为计算机应用或计算机基本应用技术,属于计算科学范畴。

本书今后所说的计算机应用除非另有说明,均指计算机基本应用技术。下面,我们对这一领域的发展历史和现状作一些简要介绍。

要使用计算机来代替人进行计算,就得首先研究计算方法和相应的计算机算法,进而编制计算机程序。由于早期计算机的应用主要集中在科学计算领域,因此,数值计算方法就成为最早的应用数学分支与计算机应用建立了联系。最初的

时候, 由于计算机的存储器容量很小, 速度也不快, 为了计算一些稍稍大一点的问题, 人们常常要挖空心思研究怎样节省存储单元, 怎样减少不需要的操作。为此, 发展了稀疏矩阵计算理论来进行方程组的求根; 发展了杂凑函数来动态地存储、访问数据; 发展了虚拟程序设计思想和程序覆盖技术在内存较小的计算机上运行较大的程序; 在子程序和程序包的概念提出之后, 许多人开始将数学中的一些通用计算公式和计算方法写成子程序, 并进一步开发成程序包, 通过更为简洁的调用命令向用户开放。请读者注意, 子程序的提出是今日软件重用思想的开端。

在计算机应用领域, 科学计算是一个长久不衰的方向。该方向主要依赖于应用数学中的数值计算的发展, 而数值计算的发展也受到来自计算机系统结构的影响。早期, 科学计算主要在单机上进行, 经历了从小规模数值分析到中大规模数值分析的阶段。随着并行计算机和分布式并行计算机的出现, 并行数值计算开始成为科学计算的热点, 处理的问题也从中大规模数值分析进入到中大规模复杂问题的计算。所谓中大规模复杂问题并不是由于数据(如方程组的阶)的增大而使计算变得困难, 使问题变得复杂, 而主要是由于计算中需要考虑的因素太多, 这些因素相互之间的关系特别复杂, 尤其是一些因素具有不确定性使计算变得困难, 使问题变得复杂, 其结果往往是在算法的研究中精度与复杂性的矛盾难于克服。

为了比较准确地划分计算问题的复杂程度, 人们从问题计算过程中所涉及到的数据总量, 多种计算公式和计算方法相结合的复杂程度, 以及并行与分布式计算活动中各个计算结点之间所需要进行的数据通信量的角度进行观察, 将各类问题划分成了数据密集性、计算密集性和通信密集性三种基本的复杂问题类。于是, 面对现代自然科学和工程科学研究中提出的大量问题, 科学计算和非数值计算研究的重点集中在具(兼)有计算密集性、数据密集性和通信密集性的大规模复杂问题的计算方面。这方面的研究特别重要, 其研究覆盖了中长期天气预报, 地震监测, 航空与航天飞行器计算流体力学研究, 地质深部找矿, 新药研制, 受控核能反应堆试验, 尖端武器研制, 图像处理, 自动推理, 符号计算, 数据挖掘与知识发现等许多领域, 需要有高性能计算机系统的支撑, 需要有先进的科学思想和计算方法的支撑。

几何学是数学的一个分支, 它实现了人类思维方式中的形数结合。在计算机发明之后, 人们自然很容易联想到用计算机来处理图形的问题, 由此产生了计算机图形学。计算机图形学是使用计算机辅助产生图形并对图形进行处理的科学。最初, 图形学的研究只是讨论怎样用计算机来绘图, 图形软件的开发集中在标准绘图程序库和绘图程序包方面, 这主要与当时终端显示设备和图形输入/输出设备不发达有关。随着计算机存储容量的进一步发展和与图形有关的设备的进步, 到 20 世纪 60 年代后期, 图形学已经有了很大的发展, 并应用到许多行业。与此同

时，图像设备的发展和图形学的发展也使计算机的应用在 60 年代涉及到模式识别和图像处理领域。计算机图形学由于过去几何学的发展比较成熟，因此，它成为计算科学应用分支学科中发展最快，最见成效，长期活跃的学科方向。相比之下，模式识别和图像处理就困难一些。图像处理中有一项关键的技术是模式识别，最初采用的是统计的识别方法，首先提取模式的特征统计参数，然后通过建立模板与识别对象之间误差的距离值来进行识别。不过，在形式语言和自动机理论提出后，由于美籍华裔学者付京孙教授基于语言的文法理论建立了后来成为主要方法的结构识别法，使模式识别和图像处理的研究取得了长足的发展，并由此推动了计算机辅助设计（CAD），计算机辅助教学（CAI）等方向的发展。

计算机辅助信息处理是一个更为广泛的概念。由于图形学和图像处理技术的发展，计算机辅助设计（CAD）发展很快。在建筑工程、飞机制造、机械制造、服装工业、电子工业等许多领域的产品设计中，都已广泛使用计算机辅助设计系统，结果是极大地提高了生产效率和工作质量。而且，在计算机辅助设计发展的带动下，计算机辅助测试、计算机（辅助）自动控制、计算机辅助制造、计算机辅助教学等计算机辅助工程技术都得到了快速发展。

例如，今天的大型水电站、核电站均采用了计算机自动控制系统；电网的运行，城市的供水系统，城乡住宅小区的设计，也使用了计算机辅助系统；一些机械制造厂家产品的生产过程，通过计算机来控制投料、加工、检测、装配等生产环节可望高度自动化。这就是人们现在常常听到的计算机综合制造系统的研究内容；就连计算机生产过程中对集成电路元器件的检测，也常常使用计算机进行测试。

用计算机进行辅助集成电路的测试和其他元器件的测试是 20 世纪 60 年代末发展起来的一个叫做计算机辅助测试（CAT）的方向。目前，在集成电路开发的同时，人们专门研究开发了相应的集成电路测试设备。早期，集成电路的测试主要是采用老化试验，确定其使用寿命。然而，随着集成电路集成度的不断提高，性能参数的日渐扩展以及被广泛地应用于军事工业，在工作中使用者不满足于仅仅知道元器件合格，而是更希望了解其各项性能的具体实测参数。因此，急需对集成电路进行精确的测试。特别，从经济的角度考虑，一个元器件的某项性能参数不合要求，并不等于它就完全没有用了，只要别的参数合格，它完全可以用在对这项性能参数要求不高的其他产品中。但是，计算机辅助测试的代价是很高的。由于测试设备价格昂贵，而且具体测试时，测试者要根据元器件的电路与原理，先开发测试程序而后才能测试。这是一项高智力的劳动，所以，测试费用常常开销很大。以一台普通个人微型计算机为例，元器件测试的费用大约要占到整台机器售价的 25%—30%。不过，元器件经过测试以后才装配的计算机，如果没有其

他原因,质量是十分可靠的。读者从这里不难理解为什么原装的名牌产品价格如此之高了。

20 世纪 60 年代初非数值计算开始起步后,随着计算机进入商业领域,一个迫切需要解决的问题是怎样对大量的数据进行有效的存储、加工和访问,这导致了数据库理论和技术出现。单处理机上的数据库理论和技术大致只经过了十多年便趋于成熟,到 70 年代中期,已有若干比较成熟的产品进入市场。以后,虽然数据库技术作为一个独立的分支学科仍在不断发展,但主要研究工作集中在完善技术,从研究并行与分布式数据库技术,进而发展到今天增加了多媒体的研究内容。尽管如此,数据库技术中关系数据模型理论的基本框架没有什么改变。

近二十年来,由于各行各业多年来在实际的数据库应用研究、开发与使用中积累了数以亿计的大量数据,以及多媒体数据库的发展和应用,开始出现将各种数据库管理系统管辖下的数据集成在一个系统中的研究,出现了数据仓库的概念,并进而引出了在数据库(仓库)中采用逻辑、统计等数学工具对数据(仓库)中的数据进行统计、分析、推导,从中找出某些带有规律性的知识和信息的研究工作,亦即所谓的数据挖掘,并与人工智能的研究结合起来。这个方向的研究工作尽管已经发表了大量的文章,但是从工作的深度看仍存在不足。与机器学习一样,数据挖掘同样面临着新近获得的数据或知识按照什么标准加入以保持数据或知识集合在逻辑上的一致性问题。而且,这个领域由于采用的数据关联度的研究方法本质上是一种数理统计学的方法,因此,在数据挖掘领域内的研究虽然已经发表了大量的文章,但取得的成就就不大,目前还只能作为一种辅助技术帮助人们对大量数据的信息化处理提供一些有益的启示,如在物流配送领域和超级市场的进货预测方面提供预测信息等。

随着电子信息技术与设备制造的发展,对多种媒体数据与信息进行处理的广泛的社会需求不仅极大地促进了数据库技术的发展,而且,在网络分布式软件开发与应用的推波助澜下,多媒体信息处理技术引起学术界的广泛重视,获得了快速发展。在今天的社会,人们经常在网上传递着各种各样的信息,包括文字、声音、图像等各种数据信息,为了使网络的传输提高速度和安全性,在选定的通信协议基础上,必须考虑各种传输数据信息的数据压缩和解压缩的问题,数据加密和解密的问题。例如,对于占用存储字节数很大的图像数据信息,如何进行数据的压缩和解压缩以保证不仅传输速度快,图像失真小,而且能大幅度减少存储占用的空间是一个值得研究的问题。早期的研究方法主要是在对图像数据信息进行编码的基础上,采用傅立叶变换方法进行数据的压缩,但效果不理想。小波理论的发展使得通过小波变换进行数据信息压缩的效果大大改善,从而推动了该方向取得重要的进展。

不少人认为数据库技术应归入系统软件的范畴，理由是它已成为扩展计算机系统功能的一部分。也有人不以为然，认为数据库管理系统可作为系统软件的一部分，但与数据库系统有关的技术，特别是数据表示理论，数据库管理系统实现技术，数据库查询语言等有不少是计算机应用于各个学科、行业的共性技术，并未在方法论上为系统软件的研究积累新的内容，应划入计算机应用。作者比较赞同后一种观点，但是，并不希望读者在这种问题上花费太多时间。因为，也许我们从一开始就不应该将学科的各项内容严格地划分成若干个不相交的分支学科。

在数据库技术的支持下，20 世纪 60 年代非数值计算领域的一个重要的应用方向是在许多行业开发各种信息管理系统，如各种人事、财务、档案、教学等信息管理系统，包括对生产过程的自动控制和仿真系统等。经过多年的实践，一般认为，这个方向在理论和技术上已经比较成熟，主要的问题是与实际背景相结合，应用各种新技术开发具体的实用系统。

近年来，世界经济和国际贸易的发展使得各国之间的商务往来日益增加，各国经济的发展愈来愈多地依赖于世界经济的整体发展，出现了经济发展全球化的特点。在这种情况下，随着商务活动和贸易量的快速增长，竞争日趋激烈。原来那些商务和贸易方式不仅成本高，效率低，资金周转慢，而且风险大。为了提高效率，降低成本，许多上网企业呼唤着一种电子商务软件，希冀通过这样一种软件可以足不出户，坐在家中与分布在世界各地的厂家和产品推销商联系，了解各国的产品，查询市场行情，买卖交易（期货）商品，并通过与银行的联网实现资金划账交割。这个问题已经引起计算机界的重视，并由一些计算机公司提出了若干种解决方案，也已有若干软件产品问世，应用于实际的服务系统中。从技术上看，电子商务软件总体上仍然是一种信息管理系统，实现技术上也没有特别的困难之处。但是，因为涉及到商务往来和买卖交易，要保证这样的交易安全、可靠，必须解决网上用户身份识别和确认，交易数据的安全、公平、迅速、备查追索机制以及司法预防犯罪等一系列问题，必须在深入掌握网络协议的基础上才能开发可靠的电子商务系统，否则，用户将难于接受这类系统。当然，其中有些问题已经超出了计算科学的学术范畴。

在这个方向的发展中，以信息系统为基础，又发展起来了一个新的方向，它就是决策系统。所谓决策系统是针对各种各样的决策问题，以决策科学理论为指导，以信息系统为基础，开发的一类能进行某一方面问题自动或半自动决策，并进行相应的简单处理的系统。有人认为，决策系统应该划入人工智能的范畴，主要理由是决策的过程可看作为一个智力的活动过程。实际上，人工智能方法和技术只是目前决策系统经常使用的方法和技术中的一部分，其他一些理论、方法和技术像线性规划、非线性规划、概率与统计、模糊数学的方法和技术也是决策系

统经常使用的方法和技术。决策系统是一个重要的、可能会长期存在并获得发展的一个方向。原因主要有两点:

其一是社会的发展不断要求各行各业的决策过程科学化,而对大量日益复杂的问题的科学决策必须依靠计算机辅助决策系统进行才可能提高决策的效率和科学性;

其二是决策科学中的许多参数反映了一定历史时期人们的价值观念,对决策模型的建立有直接的影响作用。不同的用途,不同的出发点所考虑的决策模型也不一样。一般地说,不存在较大范围内通用的决策模型和决策系统。

正是由于上述原因,在一些西方发达国家,有相当多的计算科学专业人员正在从事计算机辅助决策模型与系统的研究、开发与应用工作。

在各种实际应用系统的开发中,有一个重要的方向值得注意,即实时系统的开发。所谓实时系统是指这样一些系统,它们的运行及系统行为与时间有关,系统在运行过程中需要对数据进行实时响应和处理。实时系统由于数据通信的时间延迟引起的系统的不确定性,系统调试困难等多种因素,如何保证系统开发的正确性将是一个大问题。这方面近十多年来国内外正在研究各种实时系统开发模型,如区段演算(During Calculus)理论、实时容错计算模型等。

在电子计算机诞生后不久,关于机器能否思维的话题就引起了人们的关心。1952年1月14日,英国BBC广播电台在第三套节目里安排了一个别开生面的内容,吸引了许多听众的注意。广播的内容是几位著名科学家进行辩论的实况。论题是:可以说自动计算机能思维吗?参加讨论的科学家有杰斐逊(G. Jefferson)、纽曼(M. H. A. Newman)、布雷斯韦特(R. B. Braithwaite)和图灵。当然,正像大家今天看到的情况一样,争论的结果自然是胜负难分。

事实上,这次讨论是BBC广播公司半年多以前举行的题目为“自动计算机”系列讲座的一个结尾。图灵也受邀参加了系列讲座,并于1951年5月15日在第三套节目里做了他的首次广播讲话,题目就是“数字计算机能够思维吗?”参加那套系列讲座的还有一些英国著名的科学家。图灵始终坚持计算机能思维。还在这场讨论之前,他就于1947年为一次计算机会议写下了“机器智能”的研究报告,并于1950年发表了著名的“图灵试验”的文章,对人工智能作了开创性的研究。

“图灵试验”是一个“模仿游戏”。该游戏由三个人来玩:一个男人(A),一个女人(B),以及一个同他们分隔开的询问人(C)。C为了弄清楚A和B两人中谁是男人可以通过电传打字机提出各种问题,而A为了使C作出错误判断可以随意回答。试验时要求由一台计算机来担任A的角色,看它能否像人那样使C经常判断失误,并以此来确定机器是否具有思维能力。图灵的机智使他巧妙地绕

开了哲学的陷阱。从科学哲学的角度来看，图灵用“图灵试验”的方式定义机器的思维是不够严谨的，但他关于机器思维定义的开创性对后人的研究具有重要意义，它使人们认识到人工智能的研究必须要对机器思维有一个明确的定义，这样才能使其后的研究有一个共同的基础。可惜，要给出机器思维的定义是一件十分困难的事，其难度决不亚于给出“可计算性”概念的定义。

尽管直到今天也没有谁给出了一个机器思维的严格的定义，但人工智能的研究从那时以来一直在不断地进行探索。人工智能作为一门独立的学科出现一般被认为是 1956 年。那一年的夏天，麦卡锡 (J. McCarthy) 和其他一些学者在美国的达特茅斯 (Dartmouth) 大学联合发起了第一次人工智能学术大会，参加者有许多正在从事人工智能探索研究的著名科学家。经麦卡锡的提议，在会上正式决定使用人工智能一词来概括这一研究方向。

人工智能是计算机应用的一个重要方向，也是一个长期活跃的分支学科。人工智能早期的研究问题有许多是很有趣的，这些问题初看起来一时难于马上给出解答。下面几个例子是典型的人工智能的问题。

例 7 克里特岛上真假话的判别问题。

克里特岛是传说中的—个海洋小岛，岛上居住着许多人。这个岛上有一个奇怪的现象，一部分人说假话，另一部分人说真话，令游客大伤脑筋。不过，好在岛上的居民有一个特点，说真话的人从来不说假话，说假话的人从来不说真话。—次，—个游客分别向三个岛上的居民问路后，为了弄清楚谁说的是真话，游客向三个人中的每一个人都提了一个共同的问题：谁是说谎者？结果 A 答：“B 和 C 都是说谎者”，B 答：“A 和 C 都是说谎者”，C 答：“A 和 B 至少有一个是说谎者”。试问，游客有没有办法弄清楚谁是说谎者？

例 8 传教士与野人过河问题。

三个传教士和三个野人来到—条河边准备渡河。他们仅找到了一条只能供—人或两人乘渡的船。现在，如果河的任—岸边野人的数目超过了传教士的数目，传教士就可能有危险。试问，他们应该怎样渡河，才能使传教士始终处于—种安全的状态并最终全部渡过河去？

例 9 部队过独木桥问题。

战争年代，—支深入敌后的侦察小分队在通过—个封锁区的时候遇到了麻烦。当时部队来到—个山梁上，—条鸿沟挡住了去路。跨过鸿沟到达对面山梁只有两条路，—条是经过眼前的一—条独木桥，另—条是绕原路先退到远处的山脚下，再通过鸿沟爬到对面的山梁上。由于时间的原因，部队决定走独木桥直插对面山梁。队长仔细查看了独木桥面，发现上面布满青苔，似乎很久没人走过了。为慎重起见，他让体重最轻的几名战士依次腰间系上绳子—个—个地通过独木桥。但是，

这样做太费时间，队长很着急。当队长看到几名战士顺利地通过后，他根据归纳推理的方式，判断其他人通过没有问题，于是，果断下令余下的人不系绳子一个一个地通过。正在这时，当地的一个老农赶来告知，这座桥年久失修，过桥一定要格外小心！在老农的提醒下，队长认识到这样的桥经过的人越多越危险，于是，重新命令所有人系上绳子一个一个地过桥。恰巧，当最后一人过桥时，独木桥从中间断成两截。幸好，战士系上了绳子，没有掉下去。试问，人工智能程序在这样一种情况下，究竟是按照什么逻辑进行推理，对独木桥的可通过性（即安全性）作出正确的判断？

现在人们已经知道，上述三个问题中，第一个问题是一个逻辑问题，用消解法就能解决问题。第二个问题可以化归为状态空间的搜索问题，通过搜索算法不难得出结果。第三个问题是一个选择逻辑推理方式的问题。对第三个问题，如果队长头脑中没有独木桥年久失修的概念，用归纳推理（一种单调推理）的方式，一般不会有问题。但若有人告诉他桥已年久失修，那么，再坚持使用归纳推理方式，可能要失败。如果用后一种推理方式（一种非单调推理方式），尽管过桥比较慢，但不会出差错，比较安全。可是，时间呢？这里的问题是，在类似的情况下，若部队过桥可能桥不断也有可能桥断的情况下，那么，我们该选择哪一种推理方式，才能求得更好的解呢？进一步，我们根据什么来选择推理方式呢？

这样一种认识在日常生活中具有相当广泛的大量生活中的实例支持。下面的两个例子足以说明问题。

例 10 张翼德长坂坡单骑喝退百万大军。

在著名的中国古典文学小说《三国演义》中，刘备手下有一位骁勇善战的将军张飞，字翼德。他虽然武艺超群，英勇无比，但粗心暴戾，缺乏智谋，被普遍认为是一位不太可靠，不能独当一面的武夫。但在曹操大败刘备，穷追不舍的危急关头，他大智大勇，在长坂坡小木桥背后的树林里命士兵用马拉树枝扬起尘土作疑兵，自己单骑横矛立马站在长坂坡桥头喝退曹操的百万雄兵。后来，刘备的军师诸葛亮说：翼德虽然平时粗心，但关键时候有智谋，粗中有细，不可小视。这个例子说明，一个被认为是最不可靠的人在某种情况下往往是最可靠的人。我们每个有一定生活阅历的人对这种思维逻辑并不陌生。

可是，令人奇怪的是我们还有另外一种对称的思维逻辑，下面的例子说明了这一点。

例 11 诸葛亮大意失街亭。

这是蜀国丞相诸葛亮六出祁山中的一个故事。诸葛亮按照常人的思维逻辑，在战役要地街亭的守备中起用了一个普遍被人认为是熟读兵书，大智大勇的将军马谡领军驻守，谁知马谡是一个夸夸其谈，不可大用的人物，结果丢了街亭，害

得诸葛亮兵败祁山，唱了一曲空城计，险些被敌军俘虏。这个例子与上面的例子正好相反，它说明了最可靠的人往往是最不可靠的人。这样的思维逻辑我们同样在生活中不陌生。

显然，在常人一般的思维逻辑中，正常情况下不会使用这两种逻辑来判断事物和进行推理，但传统的经典逻辑又远不能处理各种情况，因此，我们确实需要各种不同的逻辑系统，以支持对各种复杂问题的处理。从上面两个例子中读者不难联想和感受到人工智能的困难，因为我们很难说清楚在什么情况下使用例 10 正确处理问题的思维逻辑，在什么情况下又应该使用例 11 正确处理问题的思维逻辑。

人工智能是研究机器智能的学科，是研究如何让计算机系统来模拟人的智能行为。在过去的几十年里，人工智能在知识表示、自动推理、机器学习、智能系统、分布式人工智能、神经元计算、自然语言理解、机器翻译、计算机视觉等方向上开展了大量的研究工作，然而并未取得真正的突破。原因之一是我们对人脑的结构和应激机制尚未弄明白，对思维的方式缺乏完整的计算模型。对前者的认识引发了神经元计算的研究，企图通过大量处理单元组成非线性大规模自适应神经网络系统，参照生物学对人脑结构研究的发现，来模拟人的大脑的组织结构和思维方式；对后者的认识目前主要体现在人工智能的逻辑基础方面，坚信人的思维方式和过程是可以形式化的，其主要的理论基础应该是逻辑。但是，传统的经典逻辑和单调推理还远远不足以概括人类的思维推理方式，必须发展各种新型逻辑系统和推理方式。如非单调逻辑、时态逻辑、模态逻辑、概率逻辑、常识推理、归纳推理等。原因之二是事物发生的随机性，客观世界认识对象的无穷性，人类知识的不完全性、不可靠性、不精确性和不一致性，描述语言的二义性，都会带来不确定性和不精确性，仅靠经典逻辑是不够的。为此，在人工智能逻辑基础的研究中，近二十多年来非单调推理和各种非经典逻辑的研究十分活跃。我国在非单调逻辑与自动推理研究方面最出色的学者为贵州大学的张明义教授和北京大学的林作铨教授。他们先后围绕非单调逻辑系统及其模型的系统的研究成果在国内外重要杂志发表了一批研究论文。其中，张明义是中国大陆第一个在国际人工智能领头刊物“Artificial Intelligence”和计算科学领头刊物“Information and Computation”上发表论文的学者，其学术成就获得国内外同行的高度评价，后担任了国际重要学术期刊杂志的编委，并受国际人工智能联合会的委托负责中国区 2003 年 IJCAI 会议的论文组稿工作。

自然语言的机器翻译是人工智能研究最早的方向之一。在计算机问世后不久，机器翻译的问题就被提了出来。到现在为止，西语系的一些语言之间的互译技术已经相当成熟，产生了一些比较实用的辅助翻译系统，这主要是由于对西语

系中各种语言理论的研究比较成熟所致。如同我们在前面的例子中已经看到的，相比之下，汉语就没有那么走运，至今尚缺乏严格的形式化的数学理论对其进行描述，故机器翻译系统的开发困难很多。

用计算机来下棋是一个很有趣的事情。1956年，塞缪尔（A. L. Samuel）研制了跳棋程序，该程序具有学习功能，能够从棋谱中学习，也能够在实践中总结经验，提高棋艺。1959年，这个程序击败了塞缪尔本人，又在1962年打败了美国一个州的冠军，成为当时轰动一时的新闻。事实上，对于像跳棋、象棋、拾火柴棍、围棋之类的博弈游戏过程，完全可以用一棵博弈树来表示，利用人工智能最基本的状态空间搜索技术来寻找一条通向胜利的行棋路线。只要这棵博弈树的存储计算机能“装得下”，那么，在规定的行棋规则下，只要双方不走错棋，先行者是胜，是负，还是和是一种客观存在。既然如此，为什么至今仍开发不出百战百胜的系统呢？原因在于像围棋、象棋这样的博弈游戏行棋过程中产生的棋局状态数太大，现有的计算机存储不了这么多的数据，或即使能够存储，所需搜索的计算时间也不能容忍。不然的话，世界上的各种棋类游戏就不会像今天那么吸引人了。因此，现有的计算机下棋程序的设计技术主要是在状态空间搜索技术的基础上，通过记忆和修改行棋过程中对中间棋局状态获胜可能性的估计值来判断下一步应该怎么走，这样可以大大减少棋局状态数据的存储。这种思想的具体实现常常通过计算机算法设计思想和方法反映出来。但是，当状态空间很大时，如果我们不能有效地找出引导正确搜索的优先函数，那么，这种技术的实用性仍将是一个有待解决的问题。

尽管说起来不难理解，但是，每前进一步都是困难的。计算机界有人原指望在20世纪80年代末用计算机下棋程序战胜国际象棋冠军，现在看来计划的实现已经延期。美国的一个计算机公司（IBM）投入人力和经费这些年来开发了若干下棋程序，直至1997年前的两次仍然没有取得成功。一次是公司设计的名为“深思”的程序输给了俄罗斯的世界国际象棋冠军卡斯帕罗夫。几年以后，1995年，经过改进的程序“深蓝”还是败在国际象棋冠军卡斯帕罗夫手下，而且是同一人。不过，对手承认，与上一次相比，“深蓝”比“深思”有了明显的进步，不太容易对付。从报道看，第一局国际象棋冠军故意走出一些不符合棋理的行棋法以试探对手，结果，“深蓝”“上当”了。虽然“深蓝”赢了这一局，但它的致命弱点已经暴露，最终连输几盘，失败了。从比赛可以看出，“深蓝”尚不具备较高的动态分析棋局，总结下棋规则的学习能力，这似乎与常人对机器具有智能的评判标准相差很大。几年前，有记者采访了这位国际象棋冠军，他断言：“2000年之前，他将仍然是世界国际象棋冠军。”言下之意是即使开发出新的下棋程序，也不是他的对手。这表明，他对现有的人工智能技术处在初级阶段有一种基本的估

计。可惜，他的估计稍稍乐观了一些。1997年5月，经过再次改进的“深蓝”又与卡斯帕罗夫对弈，前两盘双方下成了一比一平。之后，双方连下了三盘和棋。在关键的第六盘比赛中“深蓝”取得了胜利。最后，“深蓝”以二胜三平一负的成绩终于战胜世界国际象棋冠军卡斯帕罗夫。

虽然如此，我们仍然不能说人工智能已经取得了突破性的进展。首先，正如卡斯帕罗夫所说的，他与“深蓝”的对弈存在不公平之处。理由是他从未获得过“深蓝”的棋谱，他是在完全不了解对手的情况下与之比赛的，而“深蓝”掌握了他大量与其他人对弈的棋谱。其次，如果“深蓝”在计算和推理的思想、方法和技术上没有重大突破，仍然使用状态空间搜索、逻辑演算、模式匹配等传统的技术，那么，倘若人工智能在计算机下棋中的应用仅仅只是用计算速度已经相当高的并行计算机采用常规技术开发下棋程序，人类就会因计算机硬件制造极限速度的限制而看到人工智能研究暗淡的前景。

根据新闻报道，此次“深蓝”使用一台拥有32个处理器的并行计算机，其运算速度达到每秒2亿次。然而，正如我们在前面并行计算机加速问题的讨论中已经知道的，少量必须串行执行的操作或运算将大大限制并行计算的加速。由于国际象棋的行棋法每走一步前的思考具有分情形计算判断的特点，因此决定了计算机下棋程序的设计使用并行计算技术符合行棋思考的特点。而且，与围棋相比，国际象棋状态空间较小，对各种行棋可能性计算判断结果的汇总处理比围棋要简单得多，估计所用操作（这些是必须串行执行操作的一部分）占整个系统的比例不会太大。所以，使用并行计算机的“深蓝”程序即使采用了常规人工智能技术，战胜国际象棋冠军卡斯帕罗夫将是可能的。但是，如果计算机程序下的是围棋，状态空间的加大将使可能出现的棋局更复杂，每走一步前对各种行棋可能性计算判断结果的汇总处理显然要比国际象棋大得多。如果确实如此，那么，只要其必须串行执行的操作占整个系统的比例超过10%，即使采用并行计算机技术，根据阿达尔定律，计算加速也不会大于10。

据此，我们有理由相信，采用人工智能现有的这些技术开发的软件系统在“智能”方面将是十分有限的。更进一步，虽然现在还没有公认的关于“智能”的定义，但是在许多专业人员看来，一个智能系统必须具有一定的学习功能，在演绎推理方面具有单调和非单调推理能力，在模式匹配和模式识别方面具有误差匹配和容错识别的功能，等等。那些以状态空间搜索、单调逻辑演算推导和模式匹配等传统的技术为基础开发的软件系统，实在不能被称之为智能系统。我们仍然以围棋为例通过一个设想中的科学试验进行说明，相信读者会从中悟出智能的涵义。

众所周知，现在我们使用的围棋棋盘是 19×19 网格。如果一个计算机下围

棋的程序具有很高的“智能”，那么，我们如何来评价它所使用的智能技术的优劣呢？不妨设想将棋盘增加四倍，改为 37×37 网格， 74×37 网格，……，扩大状态空间，其他下棋规则不变，计算机下棋程序会怎么样呢？显然，棋盘增大四倍，八倍，……，对围棋高手来说，他仍能从容应付（当然随棋盘的增大其从容应对的能力会减弱）并表现出较高的水准。如果这个计算机下棋程序真具有很高的智能，能够在普通围棋比赛中战胜围棋高手，那么，使用并行计算机，对原下棋程序进行少量的修改，增加不太多的对各种行棋可能性计算判断结果的汇总处理操作，下棋程序理应表现出相当的“智能”。此时，根据这台计算机的下棋程序的“表现”，我们有可能能够判断出它是否真的具有智能或较高的智能。

尼尔森（N. J. Nilsson）坚持对人工智能的研究演绎推理是核心，耐威尔和西蒙强调思维规律是研究的核心，费根鲍姆（E. A. Feigenbaum）则认为对知识的研究是核心。麦卡锡认为这样的核心尚未形成，但他认为人工智能的突破在于非单调逻辑推理。甚至有一些极端的观点认为我们应该退回到对感官触觉、条件反射等最基本的动物行为的研究上去，从中发现和认识更为基本的智能行为应激机制。实际上，思维方式及其表示基础是人工智能基础研究的主要内容，其目的在于揭示智能计算的本质。非单调逻辑推理只是人类思维方式研究中一个目前尚未被认识清楚的重要的思维推理方式，但思维方式中还有一些不是以推理的形式出现的。例如，直觉、感知、感情等。我们常常会遇到这样的人，认为这件艺术作品好，就给予充分肯定，不容任何人提出质疑。但如果问为什么，却说不出理由来。下面的对话就反映了这种思维逻辑：

甲：这件作品非常好，绝对是超一流水平的。

乙：为什么？有什么理由吗？

甲：不为什么，没什么理由，就是觉得她好，感觉是超一流水平的。

乙：你这是什么逻辑？

甲：我就是这个逻辑！

类似的事情每个人平时是经常遇到的。可以说，每个人都在自觉或不自觉地使用自己的逻辑。但如果各种希奇古怪、违反常理的逻辑推理发生在人的身上，我们一般并不能直接武断地给这个人下一个缺乏思维逻辑或逻辑思维混乱的结论，顶多说他（她）感情用事，是一个性情中人。然而，要是机器表现出了类似于人这样的逻辑推理行为，那么，很多人就会认为机器没有思维能力，没有智能，整个系统是乱套的。可见，机器有没有智能和思维能力，首先在于看待机器的人具有怎样的思维结构，这正好说明了“图灵实验”带给我们的启示。如果我们认同对外界的输入数据机器有可能表现出某些人的行为，可以把这一点视为机器是否具有智能的判定标准的话，那么，关于机器能够思维的人工智能的研究早已取

得了许多重要的研究进展和成就，但如果人工智能是要系统整理人类所可能拥有一切（包括尚未开发的）思维结构和推理方式，而不是代替人做一些“简单的、辛苦的、疲劳的、机械的”技术活，死板的苦力活（如状态空间上的搜索）的话，那么，人工智能的研究确实离真正取得突破还十分遥远。因为在后面一种认识中，如何让机器学习、积累知识是一个尚未解决的关键问题，尽管机器学习被认为是机器是否具有智能的最重要的标志之一。可见判断机器是否具有智能，决不能从外延的方法去观察、分析和下结论，必须要从内涵的角度去分析和判断。判断机器是否具有智能，这个问题恐怕是一个永远无解的问题，因为它取决于看待机器的人具有怎样的思维结构，而人的思维结构是一个发展的概念，永远不会有终极、完备的一天。

人工智能的逻辑基础是目前人工智能研究的主流发展方向之一，但从目前来看也并非是唯一、绝对正确的发展道路。因为，在人工智能发展的方式方法和技术层面上还存在着其他的研究与发展道路，如认知心理学、演化计算等。对于赞同人工智能逻辑学派主张的学者，对于各种各样的逻辑系统，确实需要对其进行语法、语义或逻辑系统及其模型的研究，真正弄清楚各种逻辑系统的内涵和适用范围。正如卡尔纳普（R. Carnap）所说：“对逻辑来说不存在清规戒律，每个人只要他愿意，都可以使用自己的逻辑，即他自己表达思想的语言形式。但他如果想讨论这种逻辑，那么对他的惟一要求是：他必须说清楚他的语义解释方法，并给出语法规则，而不是给出哲学论据。”因为，卡尔纳普坚信：通过语言的逻辑分析可以清除所有形而上学的东西。

利用计算机证明数学定理被认为是人工智能的一个方向。1956年耐威尔等人使用演绎推理的方法首先取得了初步成功，他们发表了自动定理证明的第一篇论文：“逻辑理论机”。该论文报告了他们编制的程序证明了《数学原理》一书第二章中的38条定理，开创了自动定理证明的方向。七年以后，他们再接再厉又证明了该书第二章中的全部52条定理。美籍华裔数理逻辑学家王浩教授运用演绎方法，用一种相当原始的汇编语言写了两个证明程序，在1958年证明了《数学原理》中有关命题演算的全部220条定理和谓词逻辑中带等词的一部分定理。一年以后，王浩教授又证明了该书中所列的一阶谓词逻辑的全部定理，其运算用时在速度不及今日586微机的计算机上仅为8分多钟。定理证明在20世纪60年代取得了新的重要进展，罗宾森（J. A. Robinson）提出了与传统的演绎思想完全不同的消解法（或称归结原理），掀起了自动定理证明研究的新高潮。以后，许多人在这一方向上开展研究工作，发表了大量的学术论文。我国在归结原理这一方向上最出色的学者是刘叙华教授。

20世纪70年代中期，我国学者吴文俊教授在初等几何定理的证明中引入了

完全不同于前人的代数方法。该方法在证明中通过引入坐标，将几何图形及待证定理通过几何图形中点线之间的关系用代数方程组来表示，然后把对定理的证明转化为对方程组解的性质的判定。他的工作经出国留学生介绍到国外后，在学术界引起重大反响。国际“Artificial Intelligence”杂志曾刊登专集讨论他的研究工作，并将其创立的证明初等几何定理的代数方法叫做“吴方法”。吴文俊教授的工作使我国在初等几何定理机器证明方向上处于领先地位。但吴方法也存在不能令人满意之处，主要是吴方法给出的定理“证明”是关于大多项式的繁复计算，人难于理解其几何意义，也难于检验其正确性。那么，能否在自动定理证明的过程中生成所谓的可读证明呢？

我国学者张景中教授系统地研究了前人的工作，以其多年来发展的几何新方法为基本工具，提出了消点思想，与周咸青等人一起，于20世纪90年代初实现了初等几何定理可读证明的自动生成。他们的工作在国内外引起重大反响，被认为是自动推理领域30年来最重要的工作。

人工智能的另一个方向是研究一种不依赖于任何领域的通用解题程序或通用解题系统，称为GPS（General Problem Solver or General Problem-Solving System）。这方面的工作以耐维尔、肖（J. C. Shaw）和西蒙等人为代表。在他们看来，要使机器具有智能，关键是建立一个通用的、功能强大的符号逻辑运算体系，也叫物理符号系统假设。该体系应能对输入的用抽象的符号表示的智能问题，转换成该系统的抽象的内部表示并进行处理，一般是在问题的模型表示级进行抽象的转换，最终给出一个解答。

有一个和GPS密切相关的方向是机器人规划。该方向主要研究如何在计算机视觉技术的支持下，对机器人的行为进行规划或安排。该方向迄今已取得很大进展，成功地应用到一些机器人系统中。但是，正如大家已经在电视中看到的表演，机器人的能力还是十分初级的，那些所谓有着超强本领，无所不能的机器人，目前还只能出现在戏剧故事作品中。

人工智能研究在20世纪50年代和60年代虽然取得了不少技术研究成果，但本质上并没有取得重大突破性的进展。人工智能所使用的基本技术主要是逻辑演算技术、状态空间的搜索技术和模式匹配技术，功能强大的（近乎万能的）符号逻辑运算体系没有建立起来，机器思维的定义也没人说得清，这使得传统的对人类思维规律的研究受到严肃的批评。在这样的背景下，美国斯坦福大学的学者费根鲍姆提出了新的观点，他认为对人工智能的研究应该以知识为中心。一个智能系统，只有弄清其拥有的知识，才能把握它的行为规律。为此，他首先提出并于1968年主持研究成功了第一个专家系统DENDRAL，它能根据质谱仪的数据推知物质的分子结构。从那时起，全世界陆续开发了各种各样的数以千计的专家系统。

所谓专家系统是指这样一类系统，这种系统拥有某些领域专家的一部分知识，能够模拟人类专家的思维方式进行分析、计算、推导、归纳、学习等数据处理，使用户感受到该系统对数据处理的反馈就功能或效果而言具有一定的类似于专家的能力。

很明显，要设计这样的系统，必须首先将领域专家的知识总结、整理出来，以某种表示形式存放在计算机中，然后，设法设计一个模拟专家使用这些知识分析、计算、推导的推理机（程序）。但是，这样做也会遇到一些困难。首先，专家系统中所有知识都是由人类专家和知识工程师进行整理的，难免会出现遗漏、差错、不准确、前后不一致等情况，而且，至今为止，由专家整理出来的知识大多属于领域的浅层知识，形式上也主要是条件规则型知识。其次，因为知识种类、数量可能很多，难于详细检验，待到专家系统具体使用这些知识时，机械、死板的计算机程序就有可能推导出一些错误的结论。第三，目前各种专家系统在收集、整理领域专家知识时并没有把专家是如何学习、获得这些知识的知识整理出来，也即这样开发的专家系统并不具有真正的学习能力，结果导致系统的表现只能处理人类专家见过的各种情况，不能“随机应变”。例如，人类专家没有见过或不会处理的情况，一般也不可能提炼出知识，碰到这种情况，因专家系统不具备学习能力自然也不会处理。又如，人类专家某些只在限定范围有效的知识，在提炼中有可能被扩大了使用范围，导致将来应用推导时出错。所以，通过总结多年来开发专家系统的实践经验，一般认为，专家系统还应该具有学习新知识的能力，包括修正已有的知识。更为高级的智能系统，还必须具有不同的推理方式和正确选择推理方式的能力。显然，这将涉及到学科的本体论和学习的方式方法等更困难的领域的内容。

不过，专家系统的研究也并非一团糟。在许多简单的领域，专家系统已能代替人类专家完成一些需要专家知识才能完成的任务。特别值得一提的是在专家系统的开发中发展了一批新的技术，如知识表示方法、知识获取技术、知识求精技术、不精确性推理技术等，积累了经验，加深了对人工智能的认识。然而，由于对各个领域的专业知识的整理是一个十分困难的问题，再加上知识自动获取的重要技术——机器学习并没有取得实质性的重大进展，研制的专家系统大多不实用，躺在实验室里睡大觉，开发工作无法达到与经费投入相匹配的预期目标。人工智能再一次面临严峻的考验。

例如，像上面说到的计算机下国际象棋，如果单纯考虑使用状态空间搜索技术和少量的逻辑演算推理，那么，即使战胜了卡斯帕罗夫，也不能就此简单地认定这种基于并行计算机的人工智能系统具有很强的智能，还必须考察其采用的人工智能的理论和方法。因为，国际象棋相对是一种状态空间较小，棋局状态相对

较为简单的棋。如果计算机与围棋的九段高手对弈,恐怕计算机输的可能性是百分之百。因为在围棋中,状态空间的增大使得对棋局的判断更为困难,有关的推理既涉及到单调推理,又涉及到非单调推理,高手经常都处于很难准确判断的境地。如果计算机还是采用状态空间搜索技术,向前计算十多步(九段高手的能力标志之一)后通过判断决定下一步的行棋,那么,即使能够缩小计算的范围,也将需要更高速的并行计算机。显然,一味依赖计算机速度增长的程序决不是真正的人工智能程序。懂得围棋的人都知道,用计算机准确判断某一棋局的“势”究竟更有利于哪一方都将是十分困难的。

20 世纪 70 年代末期,一部分学者认识到了人工智能过去研究工作基础的薄弱,开始转而重视人工智能的逻辑基础研究,试图从总结和研究人类推理思维的一般规律出发去研究机器思维,并于 1980 年在“Artificial Intelligence”发表了一组非单调逻辑的研究论文。他们的工作立即得到一大批计算机科学家的响应,非单调逻辑的研究很快热火朝天地开展了起来,并由此深入到常识的表示和常识推理等更深的层面。10 年以后,“Artificial Intelligence”于 1991 年出版了一个专集,邀请一批著名的人工智能专家对过去的工作进行总结性讨论,各抒己见。由于许多专家的学术观点很不相同,相去甚远,因此,论文就一些重要的焦点进行了针锋相对的争论。虽然,这些不同的观点一时难辨高低,但是,从那以后,人工智能的逻辑基础研究成为了人工智能方向发展的主流。

作者认为,问题研究的焦点还是应该回到机器思维的定义上来。未来智能机系统研究的突破点应该是机器思维定义的给出,因为这种可能存在的“思维计算模型”中一定蕴含着智能计算机原理的思想,就像当年表示“可计算性”的图灵机概念的提出导致存储程序式电子计算机的诞生一样。

弄清楚逻辑系统的语法和语义只是逻辑系统研究的一个方面,弄清楚一个逻辑系统解决问题的能力范围和范围则是另外一件事情。弄清楚了各种逻辑系统并不等于就能够解决世界上的所有问题,更不等于解决了思维的一般规律。千百年来,伴随着人类社会物质文明的不断进步,人类的思维科学理论研究也在不断地发展,对自然界和自身的认识也在不断地向深度和广度进军。也许,按照卡尔纳普的说法和“图灵试验”的潜台词,机器早已有了智能,不过,机器目前拥有的智能程度还不能令人满意,尤其是还不能让大多数人工智能的专家满意,只是许多人不愿意承认这一点罢了。因为如果认同这一点,那很可能给自己设下一个永远无法解脱的自锁装置,机器智能或者可以表现得非常平凡,或者永远没有尽头,谁都可以成为智能研究专家,谁都不可能成为真正的智能研究专家!

从 20 世纪 80 年代中后期开始,随着计算科学中一些分支学科方向的发展,人们在实际应用中已不满足于计算机只对数值、字符等数据的处理,要将计算机

应用推向更深、更广的社会生活的几乎所有的领域，必须设法让计算机能同时处理数值、字符、文字、声音、图形、图像、各种信号等多种信息媒体，于是，多媒体信息处理或多媒体数据处理应运而生。多媒体信息处理是以数据库技术为基础的。作为计算机应用的一个新方向，它首先需要研究的问题是怎样对来自各个方面的多媒体数据进行有效的表示、识别、存储、加工、维护、通信、输入和输出，其中，包括数据编码，数据压缩与还原，数据一致性检测，数据传输与通信，数据变换与转换（加密与解密），等等。

数据库技术、多媒体技术、图形学技术等方向的发展产生了两个新方向，即计算可视化技术与虚拟现实技术。这两个方向产生的背景是这样的。随着计算机技术的发展，人们在科学计算和计算机实时辅助处理各种问题的应用中已不能满足将数据交给计算机，然后等待输出结果这样一种工作方式，有时候甚至达到难于容忍的地步。用户迫切希望了解和参与整个计算工作的全过程，也就是在用户的实时参与和监督下完成计算工作。另一种情况是许多教学与训练性质的实验费用花销很大，过程繁琐，人们希望利用计算机来模拟实验过程，节省开支。为了使这种计算机模拟的实验过程逼近真实的情景，就需要开发能模拟真实环境的软件。由于 20 世纪 70 年代以后集成电路技术的发展，计算机的运算速度越来越快，主存的容量已能达到相当大的量级，这样，就为计算可视化和虚拟现实的发展创造了条件。于是，计算可视化和虚拟现实技术被提了出来。

科学计算可视化是 20 世纪 80 年代末提出并发展起来的一门新技术。它将科学计算过程中的数据，包括最终的计算结果数据转换为几何图形和图像信息，以某种形式在计算机屏幕上动态地显示出来，并采用交互方式进行处理。科学计算可视化有助于发现和理解科学计算的过程，其主要的支持技术是计算几何学、图像处理、计算机视觉、数据库技术、数据压缩技术、计算机图形学 and 用户界面技术。

所谓虚拟现实技术是用计算机生成逼真的三维视觉、听觉、触觉等真实感觉形成的虚拟世界，集多种媒体的表现技术于一体，使用户可以在这样一种虚拟环境中通过与计算机的交互感受真实的世界和活动过程。不难想象，虚拟现实技术在军事、科技、工农业、交通、教学等许多领域有着广泛的应用。利用虚拟现实技术，人们可以模拟再现各种复杂、高成本的科学实验，如核裂变试验、大型风洞实验、模具设计与制造实验、地质钻井勘探试验、大气环流模拟实验，飞机、汽车驾驶实验，机械与仪器设备操作模拟练习等。

随着计算机网络的发展，分布在全世界的各种计算机正在以惊人的速度相互连接起来。网络上每天都在进行着大量政治、经济、军事、外交、商贸、科学研究与艺术信息的交换与交流。由此，计算机信息安全受到各国政府的高度重视。除了下大力气研究对付计算机病毒的软硬件技术外，由于各种工作中保密的需

要，计算机密码学的研究更多地受到各国的重视。

我们知道，将信息（特别是文字）经过一种有规则的变换而得到的新的文字就叫做码，有关的一组转换规则和变换过程被称为编码。例如，每一个汉字用一个4位数代表，就形成了今天的电报码。我们也可以规定一句中文话中除掉所有占着素数位置的字才是真正要发送给接受方的隐藏着的一个命令：

三月二十八号早晨七点不发一起总攻。

去掉所有占着素数位置的字，这个命令实际上是：

三十号晨七点发起总攻。

这当然是一个十分简单的加密方法，聪明人一眼就能识破。

由于现代化的通信设备必须将信息转换成数字（这种数字就是一种码）才能传送，于是，对码和编码的研究就显得尤为重要。通信中码的设计主要有两个目的：一是保护不受天然的损害，如纠错码可以纠正传送时发生的失误；另一个是保护不被人为的窃取，即所谓的密码。在我们生活的周围，存在着各种电话线、网络数据线、微波塔和人造卫星，它们每天都在接受、发送和传递着各种信息。用简单的设备，谁都可能收听信息、谁都可能发送信息。问题是，我们如何才能知道，我们发出的信息没有被第三者窃取？我们收到的信息不是第三者的伪造？因此，密码在信息传递上的应用就变得十分紧迫。目前，计算机信息安全研究的重点是密码学。

怎样才能保证信息传递时的安全呢？要是密码不被破译，首先编码的变换规则必须设计得相当精巧，使得局外人不易通过概率与统计的方法摸索、找出规律。其次是变换规则必须确实地保密，只有收发双方知道。然而，密码被破译往往出在第二点上。因为，一套密码如果使用的时间长了，第三者完全可以通过概率与统计的方法进行破译，密码的底细迟早会被别人摸清。但是，如果经常更换，那么，怎样安全地将新的密码变换规则送到对方手中就是一个大问题。

1976年，狄菲（Diffie）和海曼（Hellman）首先提出一种密码体系的构想，兼有网络公开和信息保密的优点，称为公钥密码体制。其基本思想是：一套信息与密码间的变换规则可以分成把信息编码和把码译回信息两部分。编码法是公开的，它只能编码而不能译码。译码规则是保密的，是独家拥有的秘密。钥是指一个编码方法的代号，有公（开）钥和秘（密）钥之分。譬如，我们可以将所有已知可能的编码法都指定一个代号，规定哪一个用户使用那一个代号的编码法，然后编一本代号簿的索引。谁要和某个用户通信，就查索引找到这个用户的代号，把信息按照代号对应的变换规则（编码法）将信息编成密码发送。因为编码法是公开的，所以称公钥密码。

读者可能会问：既然每个人都能查到代号，知道公钥，一个人收到了一段密

文是否可立即知道传递的信息呢？答案必须是否定的，否则公钥密码体制就不灵了。要使别人难以破译，不仅要保护好密钥，关键是编码法要有一种称作“陷阱”的性质，知道了编码法也只能编码而不能译码，如果有人想破译密码，“陷阱”会导致破译者进去后在相当长的时间内被束缚住。

设想我们得到了一本世界上独一无二的甲骨文/中文字典。我们首先将中文译成甲骨文的部分公开，于是，每一个想与我们通信的人都可以把中文译成甲骨文发送给我们，即使别人看到了也不知是什么信息。这时，只有我们自己可以利用中文译成甲骨文的规则将其转换成原文，明白是什么信息。由于我们掌握了无人知道的密钥，因此，就实现了利用公钥密码传送信息。也许，有人说，既然公开了中文译成甲骨文的编码法，只要投下人力和物力，完全可能破译密钥，即从编码法猜出译码法。这个想法在理论上是可行的，但是，如果这样的破译所要花费的人力和物力超出了实际可能，那么，这样的公钥密码体制就是成功的。以破译时间作为重要参数，若一个公钥密码体制在现有的实际可能的人力物力的条件下起码要花费三个月才能破译，那么，用于战争中的某一次战役将是可靠的。因为即使对方能够破译，等到密码被破译时，战斗早已经打完了。

是否有这样的编码法呢？请看下面的例子。

例 12 RSA 编码法。

选定两个很大的素数 p 和 q ，令 $n=pq$ 。再随意选定一个整数 d ，满足 $d>p$ ， $d>q$ ，但 d 小于且互质于 $\psi(n)=(p-1)(q-1)$ 。用欧几里德算法可以算出 d 按 $\psi(n)$ 模乘法的逆元素 e ，也就是

$$e \cdot d = 1 \quad [\text{mod } \psi(n)]$$

此时， e 也是一个小于 $\psi(n)$ 的正整数。

用户现在公布他的编码法如下：信息发送者首先把要发送的信息分段，每段所代表的数字（如一个汉字用四个数字代表）不得大于 n 。设 M 为某一段的信息，则发送者按照用户公布的 n ， e 两个数将 M 编成暗码：

$$E(M) = M^e \quad (\text{mod } n)$$

设 C 为密码，则用户的译码法是

$$D(C) = C^d \quad (\text{mod } n)$$

这样，当用户收到 $E(M)$ 时，按照译码法得到：

$$\begin{aligned} D(E(M)) &= (M^e)^d \quad (\text{mod } n) \\ &= M^{ed} \quad (\text{mod } n) \\ &= M \quad (\text{mod } n) \end{aligned}$$

其中， $M^{ed} = M$ 引用的是费马-欧拉定理 $X^{\psi(n)} = X \quad (\text{mod } X)$ 。

然而, 第三者截取了 $E(M)$ 后, 由于不知道 d , 故无法破译密码。第三者要想破译密码, 必须首先求出 d 。假定能把 n 分解成 p 和 q , 就可以计算得出 $\psi(n)$, 再从 $\psi(n)$ 和 e 用欧几里德算法计算得到 d 。可是, 到目前为止, 还没有多项式计算时间复杂性算法来计算因子分解问题, 所以, 当一开始取的素数 p 和 q 很大时, 第三者要破译的难度是很大的。当然, 高性能并行计算机的出现已使这一问题的计算变得相对容易。

迄今为止, 科学家已经研究出了基于不同编码法的许多公钥密码体制。这些工作, 因为需要更多的数学知识, 主要研究者多数都是数学家。

一个今天为许多人所重视的计算机应用研究方向是所谓的并行与分布式计算。当与体系结构密切联系, 研究工作不仅涉及并行与分布式算法、程序的研究工作, 而是从问题出发, 统一地考虑体系结构的设计、算法与程序的设计, 以及性能价格比等因素时, 也可以用意义更广泛的术语并行与分布式处理加以概括。

由于并行计算机系统和网络计算机系统的出现, 许多学者从并行计算模型和分布式计算模型出发, 借助和参考顺序、并发程序设计的研究思想和研究内容, 有针对性地开展并行与分布式算法、程序设计的内容。最初, 首先开展的研究工作主要是并行数值分析和并行非数值分析研究, 主要通过挖掘各种数值和非数值顺序计算中的并发事件, 将各种成熟的顺序数值和非数值算法改造成为各种体系结构的并行计算机系统上的算法, 这类工作随着数值和非数值计算算法研究的不断深入, 至今仍有不少人在继续开展研究工作。另外, 针对各种并行计算机系统和网络计算机系统上大量非数值数据密集型计算和计算密集型复杂问题计算, 特别是实时计算问题中提出的各种各样的难题, 必须首先研究计算模型和基本算法。例如, 问题分解与分布算法、实现可靠通信的协议算法、快速加密和解密算法、并行与分布式计算容错算法、并行计算机与网络计算机上的分布数据与文件系统的一致性算法、并行与分布式计算稳定与安全算法。

在相当长的一个时期内, 学术界有不少人一直是将程序和算法, 并行与分布式算法和并行与分布式程序视为同一研究对象。这可能是由于早期许多从事并行与分布式算法的研究人员是从研究程序设计、操作系统、编译系统和数据库管理系统方向转向而来所致。从软件开发的角度看, 并行与分布式软件开发方法学的研究还属于刚起步不久的研究方向, 原因是对于这类软件开发从程序设计语言及其理论基础, 程序设计技术和程序理论的研究仍存在许多需要解决的问题, 如并行与分布式程序中具有不确定性特点的语言成分的动态语义问题等, 有关这方面算法的研究也存在大量问题需要解决, 如怎样保证计算结果是可靠的、安全的问题等。

因此, 在属于计算机应用范畴的并行算法与分布式算法的研究中, 今后相当

长一个时期的研究工作将集中在抽象的并行与分布式计算模型,各种互连结构的可靠通信的算法研究,以及容错算法的研究。至于由并行与分布式计算派生的分布式人工智能、分布式协同计算、并行编译技术、并行检索等所谓比较新颖的方向,因并没有提出什么特别的研究问题,也没有发展特殊的、有重大创新的新理论、新技术,故不太可能成为今后影响学科全局发展的重点方向。

最后,我们介绍一个近年来发展起来且引起广泛重视,特别重要的学科发展方向——演化计算(Evolutionary Computation)。

著名的数学家、计算科学家武汉大学的康立山教授 2002 年在全国计算机科学与技术暑期高级研讨班(贵州大学)“并行演化计算”课程讲授的开场白中说道:“谁是最伟大的设计师?不是世界上最伟大的建筑设计师,也不是世界上最伟大的机械自动装置的设计师,更不是世界上最伟大的服装设计师。最伟大的设计师是大自然,她设计了自然界的万物生灵,她创造了世界上的一切,设计出了各种各样最优秀的作品。演化计算,这是一个思想、灵感直接来自于大自然演化过程和生物进化过程的科学领域。”这是一个非常重要的学术领域。由于近年来人们运用演化计算思想和方法解决了许多其他传统的思想和方法没有能够解决的问题,受到学术界的广泛关注。康立山教授在演讲中大胆预言:演化计算是智能计算的基础,智能计算是计算科学的未来¹⁾。演化计算,也叫进化计算,是研究使用模拟大自然的演化过程,特别是生物进化过程的方法来求解复杂问题的一类计算模型。演化计算的起源可以追溯到 20 世纪 50 年代,瑞臣博格(I. Rechenberg)和西威菲尔(H. P. Schwefel)等人系统地研究了演化策略(Evolutionary Strategies),福格尔(L. J. Fogel)等人发展了演化规划(Evolutionary Programming),但在随后的近二十至三十年里并未受到普遍重视,其主要原因是当时这些方法本身还不够成熟,存在着一些缺陷,且演化计算工作量较大,那时计算机系统的性能也还不能满足需求。70 年代以后,由于霍兰德(J. H. Holland)、瑞臣博格、西威菲尔、福格尔等人的工作,演化计算逐渐地为人们所知晓。此后,随着演化计算在科学计算、机器学习、过程控制、经济预测、工程优化等领域所取得的成功,引起了包括数学、物理学、化学、生物学、计算科学、经济学以及工程应用等领域科学家的极大兴趣。自 80 年代中期以来,世界上许多国家都掀起了演化计算的研究热潮,有多种以演化计算为主题的国际会议在世界各地定期或不定期召开,目前演化计算已形成一个多学科交叉的重要的科学研究领域。

在过去的半个多世纪里,尽管计算机使人们能够完成一些过去无法想象的任务,但仍然有许多复杂的实际问题得不到很好的解决,人们期望能够研制出具有

1) 此处计算科学指广义计算科学。

某种智能的程序来解决复杂问题。人工智能学者曾相信, 通过将表示知识的规则加入到程序中, 可使程序具有智能。然而, 当专家系统和知识工程的研究在深层遇到困难之后, 一部分专家认为应该回到人工智能的基础上去研究问题, 建立人工智能的逻辑基础, 而另一部分学者认为研究所以受挫是因为表示知识的规则过于复杂, 以至于很难通过人以自上而下的方式将表示知识的规则抽取出来加入到程序中, 相反他们相信达到人工智能以及求解复杂问题的最佳途径是通过一种自下而上的方式, 让人们先写出一些非常简单的规则 (知识), 然后提供一种使系统适应的机制, 让系统可以通过对规则的并行应用和交互作用而表现出智能这样的复杂行为。演化计算就是体现这种思想的一个实例。

生物进化使人们获得求解复杂困难问题的灵感。自然界存在各种各样的生物, 这些生物与自然环境相互作用, 表现出了复杂的行为。进化使得生物能够更好地适应环境, 最复杂的生物就是由简单的生物经历了漫长的岁月后进化而来。虽然进化的结果非常复杂, 但进化的过程却很简单: 繁殖、变异、竞争和选择。现在, 演化计算就是要将这样一种进化的过程通过计算机的程序来模拟、再现, 从而解决大量的复杂问题。

演化计算正是基于生物进化思想而发展起来的一种通用问题求解方法。有多种方法实现模拟生物进化的演化计算, 这些方法我们称之为演化算法。常见的演化算法有: 遗传算法, 演化策略, 演化规划和遗传程序设计。

在演化计算的研究中, 最关键之处在于从大自然的演化和生物进化过程中发现规律, 获得灵感, 通过设计或建立演化算子 (一种数学变换) 来进行算法研究与演化计算。显然, 谁的数学基础好, 悟性高, 谁在演化计算研究领域就可能获得更多的机会。此外, 并行计算机的发展进一步促进了演化计算的发展, 在软硬件技术手段上提供了更强有力的支撑。我国学者康立山教授和以他为首的科研小组, 在过去的十多年里于并行演化计算领域取得了一系列高水平的研究成果。

康立山教授是我国计算科学界一位成就突出, 著作等身的杰出学者, 他长期默默无闻地工作在科学研究的第一线, 不图虚名, 不为浮云遮眼望, 在辛勤的耕耘中先后开创了解一类大规模偏微分方程组的区域分裂法¹⁾ (非演化计算领域) 研究方向和弹性混乱松弛法 (演化计算领域) 研究方向, 并在多目标整体优化研究方向上解决了一些重要的世界性难题, 先后在国内外高水平刊物上发表了一系列系统的研究论文, 出版过多部学术专著, 担任国际重要学术期刊杂志的编委,

1) 康立山教授创立的区域分裂法在科学计算和国防科研领域具有特别重要的科学意义, 在世界上引起广泛关注和重视, 特别是引起了美国军方科研机构的重视。现在全世界每年都召开区域分裂法专题的国际学术会议。

在并行计算领域的国际学术界享有很高的威望，被国外学者誉为康方法、康学派的领头人。

3.3.5 学科发展带给我们的启示

综上所述，应用数学与计算机应用学科发展主线各方向上的研究工作，都与应用数学有着密切的联系。从学科的发展可以总结得到一条定律：理论上，凡是可以计算机来处理的问题及其处理过程，都可以用应用数学来描述；凡是可以以离散数学为代表的构造性数学描述的问题及其处理过程，只要所涉及的论域是有穷的，或虽为无穷但存在有穷表示，也一定可以用计算机来实现。至于现实是否能行，则主要取决于计算的复杂性。这里所讲的实现，是指以某种技术形式来表示和反映计算模型和计算过程。实现计算的技术形式可以是程序技术，也可以是微电子技术，甚至还可以是光电子技术。但不管是什么技术形式，计算模型与计算过程的“严密性”和“精确性”都决定了所采用的技术与理论之间，特别是技术相对于理论而言偏差是很小的。否则，这种技术就不适合作为计算科学中的实现技术。例如，早期制造计算机所使用的机械技术。所谓计算复杂性，通俗地讲就是一个可计算问题通过算法在某计算模型上计算所需要花费的机器时间和机器的存储空间。但是，细心的读者马上就会提出一个问题：由于各种计算机的运算速度和存储容量都是不同的，这会导致同一个算法在不同的计算机上运行所需要花费的机器时间和机器的存储空间不同，那怎么来评价需要在不同的机器上运行的算法呢？令人欣慰的是，正如上一章中已经介绍的，计算科学理论已经采用数学的形式给出了严格的形式化的计算复杂性定义。很明显，在计算科学的研究内容中，计算是更基础、更本质的研究对象，它具有很强的规律性。计算的表现形式和实现技术是相对次要的，但它们却常常最具实用价值。

通过上面内容的介绍，我们已经知道，从现有的研究工作看，人工智能领域所讨论的智能计算明显存在三条完全不同的认识理念指导下的发展道路，第一条是人工智能的认知心理学派的发展道路，第二条是人工智能逻辑学派的发展道路，第三条是演化智能计算学派的发展道路。我们暂时还看不清三条发展道路中究竟哪一条发展道路更有希望和更有发展前途。但如果它们都属于计算科学的范畴，那么，从学科的三个基本问题尤其是计算的正确性问题来看，认知心理学派和演化计算学派在保证计算的正确性方面还有很多的工作需要开展，逻辑学派在处理非逻辑智能计算方面还有很多的工作需要开展。作者猜测：如果认知心理学派的计算语义学模型和演化智能计算学派的语义学模型真的能够建立，或许保证认知心理学计算和演化智能计算的正确性的语义学模型并非真的一定要是个构造性数学模型。至于这三条发展道路以外的人工智能发展道路，目前还没有成

为主流。

许多从事计算科学工作的人,往往将计算科学中的实现技术和计算模型与计算理论放在同等重要的地位,甚至将一些计算机操作技术看得比基础理论更重要,这不仅限制了自己将研究工作不断推向深入,而且有可能使自己在学科发展中处于被动地位。例如,在50年代和60年代,我国随着计算机研制工作和软件开发工作的开展,陆续培养了一些在计算机制造和维护中对计算机某一方面设备十分精通的专家,他们能准确地弄清楚磁芯存储器、磁鼓、运算器、控制器。以及整机线路中哪一部分有问题并进行修理和故障排除,能够编制出使用最少存储单元而运算速度很快的程序,对机器代码相当熟悉。但是,当体积大、容量小的磁芯存储器、磁鼓,速度慢的运算器、控制器很快被集成电路替代后,当程序设计和软件开发广泛使用高级语言、软件开发工具和新型软件开发方法后,这批技术精湛的专家,除少量具有坚实的数学基础,在工作中已有针对性地将研究工作转向其他方向的人之外,相当一部分专家伴随着新技术的出现,在替代原有技术的发展过程中而被淘汰。这方面的教训是十分深刻的,它常常令计算科学专业人员对自己过去一段时间以来的工作进行反思。

现在,我们已经知道了在计算科学中,计算比实现计算的技术更重要。正是计算科学的这一学科属性,决定了技术在整个学科中与理论相比只占有相对次要的地位,决定了计算科学主要不是以经验科学占主要成分的学科,而是一门以数学为主要基础的学科,它是一门科学技术,或技术科学。我们作这样的相对比较分析,丝毫没有降低技术重要性的意思,而是要提醒读者,只有打下坚实的理论基础,学习计算科学技术才能事半功倍,只有建立在高起点理论基础之上的计算科学技术,才是真正高水平的高技术。

有人可能觉得,上述方向中有一些研究工作并不需要高深的数学,这是一种错觉。造成这种错觉的原因之一是某个方向的研究可能刚刚起步,学术上还不够深入。例如,当年专家系统热起来的时候,曾经吸引了大量的科技工作者投入到专家系统的研究与开发工作中。由于最初的时候专家系统使用的技术十分简单,理解也不困难,许多人都认为自己有能力从事这项开发工作,纷纷申请相应的科研课题。在美国和中国等国家曾出现过这样的情形,一时间各种科研申请项目都想方设法带上专家系统或智能系统的名词,生怕自己落伍。但是,没过多久,当研究工作的成效与当初的期望存在较大距离,一些深入的研究指出了专家系统开发所面临的真正困难,非单调逻辑等高深理论和技术的研究受到重视并取得阶段成果时,却很少有人据此理论和技术开发专家系统,而是在专家系统开发转入低潮时,不少人就像躲避瘟疫一样惟恐避之不及。另一个原因是某一方向前人已做了大量的基础工作,很多人都是在一些开发工具和开发环境的基础上开展工作,

从而使一般的具体应用变得较为简单。这方面的一个典型例子是美国微软公司董事长盖茨的成功之路。

盖茨并没有很高的专业学历，但他在操作系统的开发中因成功地推出了操作系统的换代产品 Windows 系统而创建了微软公司，并取得商业上的巨大成功。然而，我们却不能也不应该将其个人成功的经历作为今日专业人才成长的一种基本模式加以普遍推广。就是盖茨本人也不赞成这样做。众所周知，早期字符命令式的操作系统的缺陷早已为许多人所认识，可是由于当时计算机的内存容量太小，图形技术和窗口技术不成熟，限制了基于图形和多窗口技术的操作系统的开发。一旦计算机存储容量取得突破，图形技术和多窗口技术走向成熟，那么，类似于 Windows 这样的操作系统的出现就只是一个时间问题，因为其在软件的设计技术上并没有什么特别的困难和阻力。这方面的一个有力的证据是 20 世纪 90 年代初 IBM 公司推出的可与 Windows 系统竞争的 OS/2 Warp 系统。

随着计算科学发展的不断深化，许多需要高深数学知识和计算机专业知识的软硬件开发却不是什么人都能胜任的，这也正是高科技发展及其产品开发的显著特点和诱人之处。又如，计算机网络现已成为趋之若鹜的热门方向，许多研究生争先恐后地加入到这一方向上。目前，凡是从事计算机网络方向的专业技术人员都知道网络上有网络通信协议，也知道七层协议等基本概念，对此也有兴趣。但是，绝大部分人对网络协议究竟怎样描述和实现却知之甚少。虽然国外早已有大量科学期刊、会议录、技术报告等文献报道如何描述协议，怎么实现协议，但由于需要比较深入的计算机基础知识和数学知识，多数人心有余而力不足，只能“隔着窗户看花”。因为对网络协议的内容不了解，研究工作无法深入，只能停留在网络的一般性应用层面上，利用网络上提供的开发工具开展工作，如网络的配置设计，网络站点的网页设计等，难于深入开展网络的研究工作。

因此，每一个计算科学的学生都应明白，那些基础工作正是大家毕业后所要从从事的主要工作，而那些大量的计算机具体应用则主要应由具体学科的专业人员去完成。

在计算机软硬件产品的开发道路中，目前比较普遍的途径主要是三条，第一条是通过剖析其他公司的产品，仿制并加以改进；第二条是购买其他国家的专利技术，生产同类产品；第三条是从公开出版的刊物上通过分析、挖掘、研究、创新，掌握新技术，以此为基础开发新产品。第三条路是最有希望后来居上的，并产生巨大经济效益的光明之路，当然也更为艰难，需要科技人员有良好的素质。

现代科学技术的发展有一个显著的特点，从基础理论研究到应用基础研究再到技术和产品开发的研究周期在不断缩短，高科技的飞速发展和激烈的竞争已经不允许发展中国家再走过去那种引进、消化、仿制、改进的老路，科学研究和技

术开发必须建立在全面创新的认识和机制上。在西方国家的软硬件开发公司中，科研开发工作者是要经常通过阅读科技期刊学习和掌握新知识的。正是依靠这样一批高素质的科研开发人员，公司才能在激烈的竞争中不断开发出采用新技术且技术含量高的新产品。如果等到别的公司的产品进入市场，然后买来分析、改进、仿制，那么，等到自己的产品开发出来，技术上早已经落后了。如果公司的科研开发人员不具备这样的素质，那么，公司就难以参加激烈的竞争，所开发的产品也常常是其他公司都能开发的产品。这说明，在高科技领域，创新体系的建立首先必须建立在深入（应用）基础研究工作之上，没有深入，何以创新？这也是为什么美国的 IBM, AT&T, Microsoft, Intel 等国际著名大公司除了开发机构外，还聘请并支持一批从事（应用）基础研究的科学家开展研究工作的原因。

3.4 计算科学的分类与分支学科简介

学习一门新的学科，首先对这门学科的学科分类与各分支学科的内容作一些简要的了解是十分必要的。这样做，有助于由浅入深地、系统地学好这门科学。

对计算科学的不同认识可能导致不同的学科分类。我们从知识组织结构、学科方法论和学科知识所属的基础、专业基础与专业属性出发，对这一学科进行了学科分类。下面，对各分支学科作一简要介绍。

3.4.1 构造性数学基础（数理逻辑、代数系统、图论、集合论等）

在数学发展史上，有三项重要发现对数学科学的发展产生了深远的影响。一是解析几何的发明，它使数学思维实现了形状与数量之间的沟通；二是微积分的发明，它使数学思维进入了无限小分析领域；三是群论的发明，开创了代数研究的新纪元，即从局部性的字母与结构的计算研究转向更为抽象的系统结构的整体性分析研究。注重对被研究对象系统结构的研究是现代数学的一个重要特征。

20 世纪初数学研究中最深入的活动是关于数学基础问题的讨论。这场讨论不仅关系到演绎数学的正确性，而且关系到整个数学学科的发展。在这场讨论之前，有几项研究工作将数学基础问题推向讨论的高潮。

首先，在集合论的研究中发现了悖论。罗素把悖论通俗化，给出了一个“理发师悖论”。

从前，有一个理发师，他声称自己愿意而且只为所有不给自己理发的人理发。一日，他突然发现自己陷入了矛盾之中，走入了一个怪圈。因为，如果有人问他：你给不给自己理发？他将面临两难的尴尬境地。如果他给自己理发，那他显然违反了誓言；可是，倘若他不给自己理发，那他就违背了自己的承诺。该悖论可用

下面的一个简单的式子来表示。

例 13 定义集合 $S = \{x \mid x \notin S\}$ 。

显然，按照集合 S 的定义，任意元素 x ，如果 x 属于 S ，那么，可以导出 x 不属于 S ；另一方面，如果任意元素 x 不属于 S ，那么，据此性质，就知 x 在 S 中。显然，这里出现了矛盾。

例 12 的说法并不是一个严格的表述，但大致上能够说明问题。严格的说明可以如下进行叙述。

依照二值逻辑的二分法，可以把集合分为两类：第一类是非常规集合，第二类是常规集合。所谓常规集合是这样的集合，不允许集合本身作为自己的一个元素。而非常规集合则不同，它允许集合本身作为自己的一个元素。于是，我们可以从下面给出的定义的基础上导出悖论。

定义 1 如果 S 是一个非常规集合，当且仅当 S 可以包括它自己作为一个元素。

通常把 S 满足非常规集合的条件记为 $S \in S$ 。

定义 2 如果 S 是一个常规集合，当且仅当集合 S 本身不是自己的一个元素。

通常把 S 满足常规集合的条件记为 $S \notin S$ 。例如，全体北京人组成一个集合 S ，这个集合的一个特点显然是自己（北京人集合 S ）不能作为自己的一个元素。

现在假定： T 是包括所有常规集合组成的一个集合，即

$$T = \{S \mid S \notin S\}$$

那么，如果有人问集合 T 包不包含自身，我们将如何回答？

(1) 如果说 T 不包含 T ，那么， T 满足定义 2，于是可得 T 可以作为 T 的一个元素，因为我们已经假定 T 包含所有满足定义 2 的集合为元素。所以，假设 T 不包含在 T 中得出了矛盾。

(2) 如果说 T 包含在 T 中，那么， T 满足定义 1，于是由 T 的定义， T 就不能包含在 T 中了，因为我们已经假定 T 是所有满足定义 2 的集合为元素构成的集合。所以，假设 T 包含在 T 中同样得出矛盾。

这就是说，当人们设想把集合分成两类，并设想满足定义 2 的“所有集合”能构成一个集合后，那就一定可以导出集合 T 包含在 T 中等价于集合 T 不包含在 T 中这样一个矛盾，即

$$T \in T \equiv T \notin T$$

这就是罗素悖论要义。

另一项引起重视的研究工作是数学系统的相容性问题。许多数学家都相信，数学的每一个分支是可以建立自己的公理体系的。但是，一个公理体系必须要保证不会导出矛盾的结论（定理），即不会在该体系中同时证明一个定理和该定理的

否定都成立。这就是相容性问题。

实际上,从19世纪后期起,就有一些人已经开始重新考虑数学的基础问题,特别是数学和逻辑的关系,试图弄清或建立数学的哲学基础或数学哲学理论。当时,数学悖论和相容性问题提出后,考虑这些问题主要是针对数学的可靠性和严格性的根据这样一个问题。一些数学家们开始对传统的数学证明中一些逻辑原则无限制使用的合理性产生怀疑,如对反证法的使用。他们否认排中率的普遍有效,并对某些存在性证明是否有意义表示怀疑。如对闭区间上连续函数的零点问题,当函数在区间两端的值互异时,就零点作出的“存在性”证明而言,如果零点是无理数时,这种证明是否合理表示怀疑。这实际上涉及到更深刻的整个数学基础的问题。罗素、希尔伯特(D. Hilbert)等一大批科学家先后参加到了这场争论中,并提出了自己的观点。

按照鲁宾逊(A. Robinson)的观点,数学基础的一个重要问题是关于无穷总体的存在性,或真实性,或可知性,或客观性的问题。他认为那些据说是研究无穷总体的数学理论是有所指的,但这与这些理论是否应该发展是无关的,而且有很强的理由说明我们应该继续以古典的方式进行数学研究。

在这场争论中,大多数数学家承认或相信数学可以建立在逻辑基础之上,但因对逻辑学的认识不同逐步形成了三大数学流派。他们是逻辑主义学派,形式主义学派和直觉主义学派。

逻辑主义学派以罗素和怀特海等人为代表,他们认为数学可以从逻辑推导出来,因而是逻辑的一种扩展。数学与逻辑之间没有分界线。逻辑与数学是相互关联的,但不是作为两个不同的主题,而是作为同一主题的先后部分相互关联的,逻辑先于数学。这种相互关联是说数学能够不用引入外加的原始词项和外加的假定而完全从纯逻辑得到。这一学派的许多思想体现在罗素和怀特海的名著《数学原理》中。

罗素最初有这样一种认识得益于皮亚诺(G. Peano)。1900年7月国际哲学家大会在巴黎召开,罗素在会上见到了皮亚诺。会议之前,他已经读过一些皮亚诺的著作,但并没有完全掌握皮亚诺发展的符号逻辑体系。在会议期间的讨论中,罗素发现皮亚诺比别人都更加严格,辩论时始终占上风,于是逐渐认识到这必定是由于他的符号逻辑体系在起作用。回去以后,罗素潜心攻读皮亚诺的著作,很快发现他的体系正好是他梦寐以求的可进行逻辑分析的工具。他很快掌握了这个工具,并把这种方法推广到关系逻辑上。那年的秋天,他天天和怀特海讨论数学问题,研究如何从逻辑推导出全部数学来。从那时起,他们一起合作,在19世纪的最后一天,终于完成了20万字的《数学原理》这本名著。

形式主义学派的创始人是希尔伯特。希尔伯特为了解决悖论引起的数学危机,

对数学体系的基础进行了认真的思考，他开始认识到：除非把数学命题首先还原成公式，使数学命题本身不成为数学研究的对象，给无矛盾性一个绝对的证明，才能真正修复数学大厦。他要求建立一个形式体系，不是去直接关心具体数学命题的真假，而是整个体系的无矛盾性。一个科学体系如果按照推演规则永远不能推出公式 $0 \neq 0$ 来，那么这个体系就没有矛盾。他认为数学知识的真理性就在于数学体系的无矛盾性，而不在于这个体系和其中的数学对象是否能够构造出来。这就是他的形式主义的哲学思想。形式主义学派认为数学本身是一堆形式系统，即形式系统的集合，各自建立自己的逻辑体系（概念、公理和推演规则），同时建立自己的数学。数学的主要任务就是发展一个一个这样的演绎系统。每一个数学分支都有自己的概念、公理和推导定理的法则，以及自己的定理。把这些演绎系统通过形式化的（逻辑）推理方式开展起来，就可以建立一门一门的数学分支，而且，用这种方式建立的各个数学分支将更为严密，不会出现悖论。需要指出的是，希尔伯特当时的主张和后人所说的形式主义有区别，不完全是一回事。希尔伯特最初主要是考虑给数系提供一个非集合论的基础，并确立算术的相容性。他并不认为无理数是“罪魁祸首”，也不排除排中率的使用，认为只要避免对无穷论域公开使用全体（All）就可以了。

我们想要简单介绍的是与计算科学关系密切的直觉主义学派，其他的学派的主张和内容读者可参考有关的数学著作，如数学史，数学哲学、古今数学思想等方面的书。

直觉主义学派形成于 19 世纪末，当时的主要活动是对数系和几何的严密化。悖论的出现刺激了它的进一步发展。一群被称为直觉主义者的数学家，对数学采取了根本不同的研究途径，他们不满意 19 世纪在分析的研究中引入的概念和证明方法，强调数学概念和证明过程的能行性，主张数学中的所有概念和证明都必须是构造性的，都可以从最基本的直观的无须作进一步说明的本原概念出发，一步一步地通过构造的方式给出。这些数学家认为，过去的数学研究中所以产生悖论，是因为许多数学证明是非构造性的。人们平素与有穷集合打交道，直觉表明他们平时常用的逻辑规则对于这种集合的推理是正当的。但是，这些规则曾被当成绝对的规则，并毫无直觉根据地应用到无穷集合，自然产生了错误。他们认为，排中律只在有限论域中是正确的，对无穷论域就不一定正确。因为，在无穷论域中，一个数学命题除了其正面、反面之外，还至少存在一种可能，即不可解。由此，甚至怀疑到无理数存在定义的合理性。这就是说，数学中的反证法不能无限制地使用，命题双重否定的消去率并不总是有效的。例如，对数学证明中普遍使用的反证法，直觉主义学派只限于证明否定命题，即数理逻辑中的归谬律，它比反证法要弱。直觉主义的基本想法是，在数学中应当寻求只接受构造性证明的可靠保

障。其代表人物布劳威尔 (L. E. J. Brouwer) 把数学理解成为一种构造性的程序, 试图将数学理论知识的真理性建立在“心智的直觉”的基础之上, 要求数学的基本概念受到直观的限制, 主张数学的定理和证明应该从直觉和直接感知的合理性出发, 不能设想为公理系统理论中那些不定义的概念。从最直观的概念, 概念的构造性定义和数学定理证明的构造性方法出发, 运用直觉和感知合理的构造性证明规则所发展起来的数学就形成了所谓的构造性数学。1907 年, 布劳威尔在他的博士学位论文中初步拟订了直觉主义纲领。1908 年, 他发表的“论逻辑原理的不可靠性”一文, 对亚里士多德时代以来传统逻辑的有效性进行了一次认真的批判, 以后数年他又发表了一系列文章, 用许多例子驳斥了经典数学的论证方式和结果。从 1918 年起, 他着手在严格的直觉主义观点的指导下重新构造分析学并取得成功, 赢得了一批同路人和追随者, 从那时起, 直觉主义学派开始形成。

直觉主义学派的一个重要特点是对于数学概念和定理的直觉感知性和数学证明的构造性观点, 这一学派与数学发展史上所谓的构造主义学派保持了高度的一致, 但又有所不同。直觉主义学派早期的观点与以布劳威尔为代表的直觉主义学派又有一定的区别, 有关早期直觉主义学派和构造主义数学学派的内容, 读者可以参考有关的文献。

从科学哲学的角度看, 布劳威尔试图把数学的真理性建立在心智的直觉的基础上, 并把数学中所有非构造性证明的那部分内容统统排除在数学王国之外, 宣布为非数学, 视为另类。布劳威尔甚至认为逻辑充其量只能算作数学的一个分支而不能作为数学的基础, 但这样一种极端的认识在数学真理观上是错误的。因为, 即使是直觉主义学派在数学的研究中也在自觉或不自觉地使用逻辑来说明他们发展的数学的真理性, 只是他们使用的逻辑不是古典逻辑罢了。而且, 从语义学研究中关于语义模型和模型论研究中关于数学系统模型的发展看, 模型并非一定要是构造性的。尽管如此, 直觉主义学派提出的科学观对后人的研究具有重要的意义, 并对计算科学的形成与发展产生了深远的影响。下面的这个例子可以帮助我们理解直觉主义学派的观点和思想。

例 14 为什么说反证法比归谬律强, 而归谬律比反证法弱。

所谓反证法和归谬律用数理逻辑中命题逻辑的语言来表述就是:

反证法: 若 $\Gamma, \neg A \vdash B, \neg B$, 则 $\Gamma \vdash A$

归谬律: 若 $\Gamma, A \vdash B, \neg B$, 则 $\Gamma \vdash \neg A$

为了说明反证法比归谬律强, 而归谬律比反证法弱, 从数学的角度可以这样说明: 将反证法和归谬律分别看成是两个命题, 设法从反证法命题推导出归谬律命题, 并证明从归谬律命题出发无法推导得出反证法命题。实际上, 要从归谬律导出反证法, 必须要加上“否定之否定等于肯定” ($\neg\neg A \vdash A$) 这样一条定律或

推理规则。显然，这就从数学上证明了反证法比归谬律强。但是，如何从直觉主义逻辑的角度说明反证法比归谬律强？事实上，从直觉主义逻辑的角度可以用一个非常简单的直觉实例来说明反证法比归谬律强。当然，读者可以暂时不作过多的长时间思考，有兴趣的读者可以参考本书的教学辅导书。。

此外，直觉主义者坚持数学先于逻辑的观点，认为逻辑是从科学思维特别是数学思维中总结和概括出来的。按照直觉主义学派关于数学对象必须是可构造的原则，直觉主义逻辑也应该是一种构造性的逻辑。例如，数理逻辑中基于构造性证明的自然演绎的逻辑系统就是直觉主义逻辑的一个范例。

尽管数学科学的发展指出了这三种流派都存在这样或那样的问题，但是，与计算科学在科学方法论上形成一致的是构造性数学。这就是直觉主义所以受到计算科学家欢迎的原因。可以这么说：历史上，对计算的能行性和可构造性研究的最著名的产物要数图灵机。如果没有 19 世纪末 20 世纪初关于数学基础问题的讨论，没有直觉主义学派对数学的贡献，计算科学可能要推迟出现。

肯定直觉主义逻辑对计算科学的贡献，并不等于要否定逻辑主义和形式主义。实际上，公理化方法秉承的是形式主义的数学思想方法。在计算科学的发展中，公理化方法曾在一些分支学科的发展中得到广泛应用并产生重要影响。例如，在形式语义学和程序理论的研究中，公理语义学方法一直占有重要的地位，一些软件开发工具也是按照这种思想进行的。当然，我们应该认识到，公理化方法本身应该建立更为严格的理论基础。特别是在计算科学界，目前主要是靠构造性逻辑来建立它的理论基础。

20 世纪上叶，以法国一批年轻的数学家形成的布尔巴基学派曾计划对传统的数学作全面的改造。这些数学家以布尔巴基作为自己的笔名，用结构主义的观点计划对数学的所有分支进行重新整理。他们从数学中找出几种基本的结构，像代数结构、序关系结构、拓扑结构等，并在这些基本的结构上重新整理全部的数学，在数学研究中注重和强调对各种结构的研究，得到许多数学家的支持。从那以后，现代数学，特别是纯粹数学的发展基本上沿着两个方面发展：一个方面是数学形式系统的研究，即对数学理论的研究转化为对形式系统的研究，像公理化系统、数学机械化等，都属于这一方面；另一个方面是对各种数学结构的研究，产生和推动了一些新兴数学分支学科的发展和形成，像拓扑学、泛函分析、巴拿赫空间等，都属于后者。尽管从目前来看，这些数学分支在计算科学中的应用还不明显，但由于其更多的是对数学基本结构的研究，有迹象表明有可能未来在计算科学中会得到广泛的应用。

数理逻辑与抽象代数是计算科学最重要的两项数学基础，它们的研究思想和研究方法在计算科学许多有深度的领域得到了最广泛的应用。我们常说，在今天，

世界上还没有一门科学像计算科学那样与数学的联系如此紧密, 因为, 计算科学从根本上来说是研究如何让计算机来代替人从事复杂、繁琐的计算任务, 这不仅包括非智能计算, 也包括智能计算, 而数理逻辑与抽象代数在计算科学中最广泛的应用不仅解决了大量计算科学的问题, 并且反映和融入了现代数学研究的主流。

数理逻辑是研究推理的科学, 特别, 在过去主要是研究数学中推理的科学。数理逻辑与哲学有着密切的联系, 其哲学方面是形式逻辑, 而形式逻辑的数学化方面构成了数理逻辑的研究内容。数理逻辑的研究最早起源于莱布尼兹关于排列理论概念的演算。经过几百年的研究, 现今, 人们通常认为数理逻辑由下列领域所组成:

- (1) 逻辑演算;
- (2) 集合论;
- (3) 模型论;
- (4) 递归论;
- (5) 构造主义和证明论。

此外, 按照美籍华裔数理逻辑学家王浩教授的观点, 还有两个不太确定的领域:

- (1) 逻辑与计算机;
- (2) 数学基础。

数学与计算科学之间存在密切联系, 读者在前面的章节中已有所知。但是, 在计算科学的研究和发展中, 应该接受什么样的数学理论呢? 鲁宾逊认为, 如果大家不对一个数学理论的可解释性提出非议, 那么, 有几条通常被看作是基本的可接受性的准则:

(1) 一个数学理论, 仅当它是协调的, 或称无矛盾的, 才能被看作是可接受的;

(2) 一个数学理论是可接受的, 要是它能够作为自然科学的一个基础;

(3) 一个数学理论要由美学标准来判断, 比如它的美或内在的适当性。

(3) 中提及的标准, 迄今还无从进行任何科学的研究。不过, 对于 (1) 中提出的标准, 足可以使我们认识到数理逻辑对计算科学的重要性。众所周知, 在计算科学的各个分支学科中, 使用了各种数学理论, 包括数理逻辑。要保证一个数学理论是协调的, 或称无矛盾的, 实际上是要保证该数学理论对客观世界或应用范畴的 (语义) 解释是无矛盾的, 用逻辑学的术语来说就是该数学理论必须存在模型。这在方法论上是靠逻辑学中的模型理论提供的。而要学习模型论的内容, 仅有命题演算、一阶谓词演算的知识是不够的。对于 (2) 中提及的标准, 读者在今后的学习中会逐步体会到其涵义。在这里, 我们只对个别方向指出如下事实:

程序设计语言的语法基础是形式语言与自动机理论，研究方法主要是代数的方法；程序设计语言的有些语义描述方法主要来自于模型论的思想和方法；在顺序程序设计和并发程序设计中，像子程序调用时的参数传递，并发程序之间的通信，以及程序变换和程序推导等程序设计技术，实际上在数理逻辑的逻辑系统演算特征中早已作了方法上的理论准备。许多深入的研究已经表明，程序设计的基础是逻辑。

目前，许多国内外大学的计算机科学系数理逻辑的教学内容只涉及到命题演算、一阶谓词演算和少量的逻辑系统演算特征，这在今后深入学习计算科学理论时常常是不够的。例如，人工智能的研究，程序理论的研究，数据库理论的研究都必须解决公式、数据、操作和符号在论域中的涵义问题。要使学生在今后的学习和大学毕业以后的工程师再教育中能比较顺利地继续深入掌握新知识，必须加强数理逻辑、抽象代数、集合论、图论、理论计算机科学的教学。因为，理论计算机科学的内容中包含了许许多多本学科最基本的方法，它们可以在学科的深层统一学科的知识。

所谓代数，是对字母和由字母构成的结构的计算。从中学到大学一二年级，读者已经学习和将要学习一些代数系统的基本内容，这其中包括解代数方程与方程组，多项式理论，行列式理论，矩阵理论，向量空间与线性变换等内容。然后，进一步还要学习对计算科学有重要实用价值的抽象代数（近世代数）。

抽象代数是关于运算的学问，是关于计算规则的学说。与古典代数不一样，抽象代数不是局限在字母的运算性质上，而是研究更具有一般性的元素（在表达的形式上仍可看成是字母）上的运算及其性质。

现在，有经验和成熟的计算科学家都知道，除了数理逻辑以外，对计算科学最重要的数学分支是代数，特别是抽象代数。在计算科学中，代数方法被广泛应用于许多分支学科。例如，可计算性与计算复杂性，形式语言与自动机理论，密码学，算法理论，数据表示理论，网络与通信理论，Petri 网理论，程序理论，形式语义学等许多方面都离不开代数。

当然，离散数学中的其他一些内容也是重要的。

人们常常要考虑各种各样事物的集合，于是产生了集合的概念。研究集合不依赖于组成它的事物的非数学特性的性质，即不研究构成集合的事物的物理学、化学、生物学等特性，而仅仅研究集合的一般性质，其研究内容就形成了集合论。

图论是数学的一个古老的分支。图论最初起源于欧拉（L. Euler）对著名的哥尼斯堡七桥问题的研究。哥尼斯堡七桥问题是这样的。有七条桥将普莱格尔河中的两个岛及岛与河岸连接起来，其中，一个岛与河的两岸各有一座桥，另一个岛与河的两岸各有两座桥。问题是要从这四块陆地（含两个岛）中的任何一块开始，

通过每一条桥正好一次,再回到起点。读者很容易据此画一个图,然后用尝试法去摸索解决这个问题。然而,无论怎么试,都不可能找到一个可行的路线作解答。欧拉研究了这个问题,他证明了这个问题无解。欧拉将每一块地用点来表示,将每两个点之间的一条桥用一条线来代替,得到了一个只用点和线(也叫边)构成的形式上十分简单的图。于是,哥尼斯堡七桥问题就等价于这个图是否存在这样一个性质:从图中任何一个结点出发,经过每一条边正好一次,最后回到起点的路径存在。

许多现实中的问题常可以用图来表示,抽象成为图论的一个问题。例如,网络结点互连的拓扑结构实际上可以抽象成一个图,一个工程各阶段任务的计划安排和调度也可以抽象成一个图,等等。研究各种一般的图的性质,就构成了图论的研究范畴。

组合数学也叫做组合分析、组合论,主要研究与按照一定的规则来安排一些物件有关的问题,研究这种安排的存在性问题、安排的计数问题、安排的构造性问题以及最优化问题。

集合论、图论和组合数学的内容在计算科学中有着广泛的应用。首先,它们为数据结构和数据表示理论奠定了数学基础;其次,它们为许多问题从算法的角度如何加以解决提供了进行抽象和描述的一些重要方法的数学基础。

需要指出的是,在学习数理逻辑、集合论和计算科学理论与技术的内容时,读者可能会初次接触到公理化方法。然而,什么是公理化方法,为什么要引入公理化方法,像这样的一些问题,许多已毕业的数学系和计算机科学系的学生却不甚明白。因此,没有较高起点的数学修养,没有数学上的某种成熟性,要想深入学习计算科学是有困难的。我们将在典型方法与典型实例那一节作简单介绍。

这样粗略地考察一下构造性数学基础足以使我们感到计算科学专业教学中数学教育所占的份量。

3.4.2 计算的数学理论(计算理论、高等逻辑、形式语言与自动机、形式语义学等)

所谓计算的数学理论是指一切关于能行性问题的数学理论的总和。也有一种更具体的定义是指一切关于计算与计算模型问题的数学理论的总和。随着计算科学研究的不断深化,计算的数学理论的内容日益丰富,主要涉及计算理论(可计算性与计算复杂性)、高等逻辑(模型论与各种非经典逻辑)、形式语言与自动机、形式语义学、Petri 网理论、通信顺序进程(CSP)、通信系统演算(CCS)、 λ 演算、进程代数(PA)、分布式事件代数(DEA),等等。这里,我们只对最基本的分支学科内容作简单介绍。

广义的计算理论可以看作同计算的数学理论。狭义的计算理论主要指算法理

论、可计算理论、计算复杂性理论。算法理论主要研究在各种抽象的计算模型上的算法的设计方法和算法复杂性分析。可计算性理论主要围绕计算这一概念展开,研究抽象的计算模型及其性质,可计算函数,以及两者之间的关系。计算复杂性除了包含算法复杂性的内容之外,还包括研究基于抽象的公理基础之上的可计算函数的计算复杂性。

高等逻辑主要由两部分内容所构成。一部分是模型论,既要研究逻辑系统在论域上的解释,即逻辑系统的语义,它构成了各种逻辑系统深入研究的基础,同时,又要研究各种形式系统主要是形式数学系统的解释,构造形式系统的模型;另一部分是各种非经典逻辑。由于命题逻辑系统和一阶谓词逻辑系统是成熟的,所以通常将它们归入基础数学的范畴。习惯上,人们把其他的各种逻辑系统归入高等逻辑之中。

模型论是研究形式语言及其解释之间的关系的理论。具体地说,给定一个问题的论域,要研究的是对形式语言给出的合法公式(理论)或形式系统在论域中是否存在合理解释的问题,以及这种解释的内在规律与性质。对形式逻辑系统来说,通过解释,人们可以对一些抽象的逻辑结论和逻辑系统在某一论域中是否合适作出判断。例如,在经典的逻辑系统中,我们说“没有一个人能跳过 2.43 米”,这句话常常是有所指的。在上下文或语言情景中,往往涉及到一个具体的论域。如果这个论域是讲话人所在的某个单位的全体职工,或者是所在城市的全体市民,而这个论域中确实没有人能跳过 2.43 米,那么,这句话是一句真话,而且可以看成是一条真理。但是,如果将论域扩大到全世界的人类,这就错了,因为目前男子跳高的世界记录已超过 2.43 米。显然,后一种情况下这句话是一个相对真理。不难理解,如果某个结论在任意的论域条件下都为真,那么,这个结论就是一条绝对真理。现在我们只涉及到一句断言,如果考虑几句断言呢?如果考虑的结论是一种抽象的符号表达式或一组抽象的符号表达式呢?它们在某一个解释下会出现矛盾或内含矛盾吗?什么样的解释是合理的?解释能否比较快地构造出来?这是模型论要研究的问题。应该注意到,模型论是一门高度抽象的、重要的分支学科,它从根本上奠定了计算机程序设计语言形式语义学的逻辑基础。

不少人认为模型论对计算科学的研究作用不大,事实并非如此。模型论在人工智能中有着广泛的应用前景。例如,人工智能中的机器学习和知识获取就与模型论有关。我们知道,一个人在学习知识时,总是在不断地获得新知识的同时,将新知识与原有的知识进行一致性检测处理,在保持自身知识体系无矛盾的前提下,扩展知识体系。一旦发现新知识与原有的知识存在矛盾,就要作适当的修正。那么,当这样一个扩展知识的工作由机器来完成时,有什么方法能够保证原有体系在加入新知识后是无矛盾的呢?用模型论的语言来说,就是一个具有模型的逻

辑系统如何扩张。可惜，今日从事机器学习和知识获取研究的学者，目前还很少使用模型论。

非经典逻辑的内容比较多，主要包括时序逻辑、模态逻辑、概率逻辑、非单调逻辑、归纳逻辑、模糊逻辑，等等。

这里简要地介绍一下与当前人工智能研究密切相关的非单调推理的概念。我们知道，逻辑是研究推理的科学，任何一个人应用某个逻辑系统进行推理，总是希望在一个无矛盾的前提下进行。以数理逻辑中的一阶逻辑为例，如果有一个无矛盾的公理系统作为它的基础，那么，只要公理系统是合理的，推理的结论相对于公理系统而言，正确与否应该仅取决于推理的规则是否正确。现在假设这些都没有问题，于是，加入新的事实，有可能推出新的结论，而且至少保持原来能够推出的结论仍然能够推导出来，即原有的结论不变。这种推理称为单调的，相应的逻辑系统称为单调逻辑系统。然而，类似于一阶逻辑这样的单调逻辑并不是无懈可击的，因为它不能完整地反映人类的思维过程和推理方式。我们在日常的思维活动中常常会遇到这样的情况：根据某些前提 A 推出某一结论 P ，当人们又获得一些新的事实 B 后，由 A 和 B 却可能不再能推出结论 P 。这种推理称为非单调的，相应的逻辑系统称为非单调逻辑系统。

怎么会出现这种情况的呢？原因是单调逻辑在某些前提 A 下推出的某一结论 P 后，就不能再改变。此时， P 可以加入到前提 A 中而不会产生矛盾，因为 A 中隐含了结论 P 。 P 的加入对逻辑系统导出新的结论本质上不起作用。但是，如果在 A 中加入新的事实 B 后能导出 $\neg P$ ，那么，由于逻辑系统的单调性，从 A 和 B 出发仍能推导出 P ，这说明 A 加入 B 后隐含了某种矛盾，这在传统的经典逻辑系统中是不允许的。怎么办？

解决问题的思想是基于如下假设：在没有事实证明一个结论 P 不成立之前，我们总是假定 P 是成立的，并允许运用 P 进行推理。一旦推理得出与结论 P 矛盾的事实 $\neg P$ ，我们就设法消除 P 以及根据 P 推理得出的其他结论。按照这种思想建立起来的逻辑系统就是一种非单调逻辑系统。

形式语言与自动机理论涉及两方面的内容。形式语言理论是指用数学的方法研究语言的语法（词法和文法），研究语言的构造性结构的学问。自动机理论主要研究各种能自动处理符号的数学机器，即自动机。当把自动机看成是一种符号接受器或识别器的时候，研究表明，文法与自动机本质上是等价的。这也是习惯上把形式语言与自动机相联系的原因。

形式语义学，顾名思义，是用数学的方法研究语言的语义或语言的含义。在计算科学范围内，形式语义学主要指用数学的方法研究计算机程序设计语言的语义问题。由于对语义的认识和描述方法的不同，形式语义学研究目前主要分成四

个流派。它们是：操作语义、指称语义、公理语义和代数语义。作为语言学研究的工具与方法，形式语义学已应用于自然语言的研究和开发之中，而作为计算科学的一个理论，其方法已广泛地应用于软件工程、计算机体系结构模型、人工智能等方向的研究与开发之中。

计算的数学理论远不止这些内容。实际上，像项重写系统、图文法、 λ 演算、类型理论、程序的不动点理论、框图理论、有序类别代数等都可以归入计算的数学理论的范畴。

3.4.3 计算机组成原理、器件与体系结构（计算机原理与设计、体系结构等）

计算机组成原理与设计是计算机发展的一个主流方向。这一方向的主要任务是根据各种计算模型研究计算机的工作原理，并按照器件、设备和工艺条件设计、制造具体的计算机硬件系统。

早期，由于计算机的设计是建立在分离元器件的基础之上，这方面的工作更多的是集中在对各个部件微观的精细分析。然而，随着集成电路技术的进步，虽然学习计算机组成原理免不了要掌握计算机各部件的工作原理，但重点已开始转到计算机的组织结构。在计算机硬件系统的设计中，国外早已普遍使用大规模集成电路设计的描述语言 VHDL 来进行功能电路的设计，这不仅大大提高了设计的效率，而且，从根本上改变了设计师的工作方式和劳动强度。此外，大规模集成电路对电路和功能部件的高集成度和计算机设计与软件开发之间建立的密切关系，使这一方向的发展逐步形成了一个计算机体系结构方向。

计算机设计和计算机体系结构一直是计算科学的一个主流方向。由于硬件设计中可以预见的极限，使这一方向的研究更趋于向分布式网络计算机系统和并行计算机系统发展。要开发分布式和并行计算机系统，基础研究的重心便从计算机的设计转向了体系结构的研究。虽然 20 世纪 80 年代发展了若干有代表性的分布式与并行计算机系统，如剑桥分布计算系统和克雷并行计算机，在体系结构的研究中也出现了如共享存贮结构、超立方体结构、环型结构、星型结构、树型结构、分层互连结构等各种体系结构（例如，Transputer 机即是在 CSP 理论的基础上发展起来的一种机器），但因为支持分布式程序设计和并程序设计的计算机系统必须具有向用户提供各进程或并程序之间相互通信的功能，而随着分布式算法和并行算法及其应用的发展，目前算法的设计和研究一般均与通信通道的结构或体系结构有关，从而使研究者很快认识到在体系结构的研究中，硬件的设计和软件的开发必须统一加以考虑。在基础研究即体系结构模型的研究中，要努力实现分布式或并行算法的设计与体系结构无关，程序的开发与具体机器无关的原则，才能保证将来应用和开发的软件可以移植、交流和比较。这一思想直接影响了并发

与并行计算模型、体系结构的研究。为了实现分布式或并行算法的设计与体系结构无关, 程序设计与具体机器无关, 有些研究工作开始集中在开放系统、可扩展的系统结构和可变结构方面。开放系统是一条途径, 但不一定是惟一的途径。可扩展的计算机系统结构也是一个受到广泛关注和重视的方向。采用模型论的方法研究可变结构的计算模型也许是一条有希望的道路。

计算机体系结构既不是单纯的硬件方向, 也不属于软件方向, 而是一个既包含硬件内容, 又包含软件内容的研究方向。一个现代的计算机系统实际上是由处理器、存储器、功能部件、互联网络、编译程序、操作系统、外围设备、通信通道等内容组合而成。这些内容怎么组织? 部件之间怎么互联? 软件设计提出什么样的要求和思想? 如何与硬件系统之间进行合理的分工? 怎样提高计算机的性能价格比? 等等, 都是计算机体系结构所要研究的内容。所谓新一代计算机体系结构是相对于过去的体系结构而言的。目前, 对这类体系结构的研究内容很多, 主要是各种新型并行计算机的体系结构, 机群式计算机体系结构, 体系结构的可扩展性、任务级并性、指令级并行性、动态可改变结构等方面的内容, 也有一些内容还不成熟, 正在发展之中。

当然, 与传统的计算机研究一样, 对计算机器件和系统的故障诊断与检测始终是附属在这一方向上的一项研究工作, 尽管很重要, 但不是重点方向。

3.4.4 计算机应用基础 (算法基础、程序设计、数据结构、数据库基础、微机原理与接口技术等)

要开展各个领域的各种计算机具体应用, 就必须首先要有一些计算机应用基础知识。对非计算科学专业的学生来说, 大致要有如下一些计算机应用基础知识, 才可能在各自的领域独立开展一般性的计算机具体应用工作而不至于遇到太大的困难。而对于自动化学科、电子工程学科等专业的学生, 如果学习了计算机组成原理, 那么, 还可以从事与硬件有关的计算机具体应用。

计算机应用基础知识包括算法基础、程序设计、数据结构、数据库基础、微机原理与接口技术等。这些内容的简要介绍可参看第一章和 3.4.6 节。

所谓算法基础, 主要是研究数值算法和非数值算法的基本设计方法。具体地说, 是从给定的问题出发, 通过研究该问题的已知条件与解之间的内在联系或内在规律, 为算法的设计提供理论方法。进一步, 使用算法设计的技术方法给出求解该问题的算法设计, 并对算法的性能进行分析。我们说, 算法的设计常常是与计算方法联系在一起的, 但算法不等于计算方法。例如, 求解一个线性方程组, 可以利用线性方程组理论, 采用矩阵计算的方法求解。像消元法, 它实际上已给出了算法的主要思想。又如, 对多段图的动态规划问题, 根据递推计算公式, 可

以采用向前计算的算法设计思想，也可以采用向后计算的算法设计思想。

数据结构主要研究数据表示与存储的方法、抽象的逻辑结构及其上定义的各种基本操作。数据的逻辑结构常常采用数学描述的抽象符号和有关的理论。如使用串、表、数组、树、图等结构和理论来表示数据在存储时的逻辑结构，研究这些结构上定义的各种操作。

微型计算机系统是在计算机产业发展的过程中独立发展起来的一类重要的产品，也是目前应用最广泛的一类计算机系统。其基本原理虽然与大、中、小型计算机相同，但在具体设计方面与大、中、小型计算机有较大差别。另外，微型计算机的接口有不同的几种类型，如串行接口、并行接口等，它们可以分别与不同的外部设备和仪器相连，用计算机和程序来控制其他设备的操作和进行数据处理，是计算机应用于许多行业时的重要技术。因此，微机原理与接口技术形成了一个相对独立的方向，属于计算机应用基础的范畴。

3.4.5 计算机基本应用技术（数值计算、图形学与图像处理、网络、多媒体、计算可视化与虚拟现实、人工智能等）

我们知道，计算机应用和计算机基本应用技术是一回事，涉及到的分支学科很多。然而，从计算机应用的定义和科学方法论的角度出发，大致可以将计算机应用分支学科的范畴确定下来，即它所研究的内容在方法和技术上为计算机在各个领域的具体应用奠定了基础。

按照这样的定义，计算机应用主要包括数值计算，信号处理技术，图形学与图像处理技术，网络技术，多媒体技术，计算可视化与虚拟现实技术，人工智能技术，信息系统设计技术，决策支持系统技术，办公自动化技术，计算机仿真技术，计算机辅助设计、测试、制造、教学等辅助系统。这些内容中，一些比较重要的技术内容已经介绍，其他的技术内容是比较普及性的概念，大多由非计算科学专业工程师在开展工作时，在此就不赘述了。有兴趣的读者，将来可以在选修课程中学习，也可以在具备较好的基础的情况下自学提高。

3.4.6 软件基础（高级语言、数据结构、程序设计、编译原理、数据库原理、操作系统原理、软件工程等）

软件是计算科学一个较大的学科门类，包括众多的分支学科方向，主要有高级程序设计语言、数据结构理论、程序设计原理、编译程序原理与编译系统实现技术、数据库原理与数据库管理系统、操作系统原理与实现技术、软件工程技术、程序设计方法学、各种应用软件等。

数据结构是计算机软件和计算机应用的重要基础，它研究数据的组织形式。

数据的组织形式有抽象的逻辑数据结构和数据在计算机中的内部存储结构之分。研究数据结构的目的是为各种计算机应用提供处理对象的数据表示和存储结构。为了支持各类数据结构的合理性，在数据结构的研究中，人们总是设法将各种逻辑数据结构的研究和其上的基本操作通过算法及其复杂性分析联系起来。需要指出的是，数据结构的内容有许多实际上是图论、集合论、组合数学、向量代数等应用数学在数据表示及其运算方面的扩展。

程序设计方法学也是计算科学中软件的一个方向。众所周知，转向语句（Goto 语句）是许多程序设计语言中共有的语言成分，也是一条对初学者十分好用的语句。但是，转向语句在程序中无限制地使用较多时，确实对程序的质量有影响。1968 年，“Communication of ACM”杂志上刊登了著名计算科学家戴克斯特拉（E. H. Dijkstra）在 1967 年给编辑部的一封信，题目是“转向语句是有害的”。戴克斯特拉在信中抨击了转向语句对程序的害处，主张把它从程序设计语言中取消。由此，在计算科学界引起了一场热烈的争论。这场争论虽然在七年之后由克努特作了结论性的总结而告结束，但它引发的关于什么是好结构的程序的讨论，对计算机程序设计语言和程序设计的发展产生了重大影响，并推动了结构化程序设计和程序设计方法的研究。程序设计方法学正是在这一背景下产生的。

程序设计方法学主要从好结构的程序定义出发，通过对构成程序的基本结构的研究，讨论设计和保证高质量程序的各种程序设计方法。对转向语句有害问题的讨论在一定程度上引发了计算科学界对“软件危机”问题的考虑，所有这些都促使软件工程作为计算科学一个方向的形成。应该说，正是戴克斯特拉的这封信，催生了软件工程这个分支学科方向。

通俗地讲，所谓软件工程是用工程的观点和方法来对软件开发与维护的过程进行研究的活动和学说的总称。

软件工程是计算科学的一个比较大的研究方向，属于软件开发方法学研究的范畴。沿着这条主线，几十年里形成了两个主要的研究分支路线和一大批较小的研究方向。这两个分支是计算机辅助软件工程（CASE）技术和形式化软件开发方法。前者是一种由一批较为成熟的技术支持下的工程技术方法，而后者是计算科学理论支持下的工程技术方法。根据软件生存周期的定义，软件工程的研究可划分为若干阶段分别研究。

计算机辅助软件工程技术的核心思想是基于各种较为成熟的软件技术，首先研制出适用于软件开发各个阶段所需要的各种工具系统，然后将这些工具系统集成在一起，形成一个环境，支持软件开发和生存周期的全过程。由于形式化软件开发方法在理论上尚不成熟，也由于理论方法支持下的技术不易为一般人所掌握，以及由于从事计算机软件工程研究的队伍由来自不同知识背景的人组成，CASE

技术应运而生。CASE 技术是在软件开发和发展的过程中由朴素的思想 and 经验逐步形成的，它对于规范软件开发者的开发过程和行为，提高工业化软件生产的质量和效率，具有重要实用价值。然而，单纯作为一种技术发展，迄今为止，CASE 技术中理性的成分较少，经验的成分居多。不过，在软件工程的发展中，将一些较为成熟的形式化技术用于软件工具的开发中并支持 CASE 技术已是一个趋势。

随着计算机应用的深化和并行与分布式网络计算机系统的出现，计算机软件开发遇到了不少困难，特别是在并发、并行、实时软件 and 一类诸如核电站控制系统、航天飞行器自动控制系统等称之为“安全至上”的软件系统开发中，对软件系统高可靠性的要求使得如何保证程序的正确性成为一个令人头痛的大问题。这个问题不能靠经验技术加以解决，而只能走程序正确性证明之路或从一开始就依靠某种在开发过程中能确保软件开发正确的技术道路，并贯彻在整个软件开发的始终，即所谓的形式化软件开发方法之路。

一般地说，工业化开发和生产软件，离不开高级程序设计语言和软件开发环境。从用户需求和软件规格说明出发，直至软件的维护，要经过多个阶段，软件的质量在很大程度上取决于对语义问题的处理。特别，在并发和并行软件的开发中，由于并发程序和并行政程序的动态语义定值问题一直是一个尚未彻底解决的困难问题，从而导致证明这类程序的正确性便成为一个悬而未决的问题。此外，研究人员认识到，在形式化软件开发的各个不同阶段，因为处理对象的目的和特点不同，有时也由于软件的性质不同，需要发展多种数学模型、方法或计算科学理论来支持软件开发各个阶段的工作，包括软件规格描述技术，程序规范说明技术，（并行）算法设计方法，计算模型，语义理论和程序证明技术等。正是在这一背景下，由计算模型，形式语义学，程序理论，程序设计语言及其逻辑基础支持下的形式化软件开发方法学获得了大发展，并在并行与分布式计算机系统出现之后展现出广泛的学科发展前景。

3.4.7 软件开发方法学（并行与分布式计算机系统、智能计算机系统、软件开发方法学等）

软件开发方法学我们已经在前面的内容中介绍了其主要内容，这里要指出的是，高起点的软件开发方法学的主要基础是新一代计算机体系结构、高等逻辑、形式语义学、计算模型理论以及算法基础，许多逐步成熟起来的形式化软件开发方法将陆续进入实用阶段，以软件开发工程方法和软件开发工具的形式出现。这实际上也指出了新一代计算机体系结构与软件开发方法学（包括并行与分布式计算机系统、人工智能计算机系统、并行与分布式软件开发方法学等）是计算科学界需要长期努力奋斗的工作重点。当然，正如我们在前面已经介绍的，近 20 年来，

软件开发技术有了很大的变化,多 Agent、语义 Web 和 Web Service 连成一体,支撑了 WWW 网络层面上全新的软件开发。

3.5 计算科学与数学和其他相关学科的关系

计算科学与数学的关系我们已在前面的讨论中有了比较清楚的认识。总的说来,数学是计算科学的主要基础,数学与电子科学构成了今天计算机系统的基础,也构成了计算科学的基础。但是,与数学相比,电子技术基础地位的重要性不及数学,原因是数学提供了计算科学最重要的学科思想和学科的方法论基础,而电子技术主要是提供了今日计算机的实现技术,它仅仅是对计算科学许多数学思想和方法的一种当前最现实、最有效的实现技术。试问,假如有一天,其他的某一项技术可以更有效地取代电子技术,那么,计算科学的基础是否会发生变化呢?相比之下,作为数学和计算科学理论的内容,由于其为计算科学提供的是最重要的学科思想、技术理论和方法,因此,它们的生命力就更长,影响也更深远。

在数学中具体地分析,要算数理逻辑和代数对计算科学影响最大,这是为什么呢?我们需要理性地回答这个问题。

3.5.1 为什么说数理逻辑和代数是计算科学的主要基础

数理逻辑和代数是计算科学最主要的基础,这是为什么呢?

我们所以这样说是基于以下几点:

(1) 首先,从计算模型和可计算性的研究来看,可计算函数和可计算谓词(一种能够判定其真值的断言或逻辑公式)是等价的,相互之间可以转化。这就是说,计算可以用函数演算来表达,也可以用逻辑推理来表达。作为计算模型可以计算的函数恰好与可计算谓词是等价的,而且,数理逻辑本身的研究也广泛使用代数方法,同时,逻辑系统又能通过自身的无矛盾性保证这样一种计算模型是合理的。由此可见,作为一种数学形式系统,图灵机及其与它等价的计算模型的逻辑基础是坚实的。

(2) 实际计算机的设计与制造中,使用数字逻辑技术实现计算机的各种运算的理论基础是代数和布尔代数。布尔代数只是在形式演算方面使用了代数的方法,其内容的实质仍然是命题逻辑。依靠代数操作实现的指令系统具有(原始)递归性,而数字逻辑技术和集成电路技术只是计算机系统的一种产品或实现的技术形式。

(3) 从计算机程序设计语言方面考察,语言的理论基础是形式语言、自动机与形式语义学。而形式语言、自动机和形式语义学所采用的主要研究思想和方法

来源于数理逻辑和代数。程序设计语言中的许多机制和方法,如子程序调用中的参数代换、赋值等都出自数理逻辑的方法。此外,在语言的语义研究中,四种语义方法最终可归结为代数和逻辑的方法。这就是说,数理逻辑和代数为语言学提供了方法论的基础。

(4) 在计算机体系结构的研究中,像容错计算机系统、Transputer 计算机、阵列式向量计算机、可变结构的计算机系统的结构及其计算模型等都直接或间接同逻辑与代数密不可分。如容错计算机的重要基础之一是多值逻辑,Transputer 计算机的理论基础是 CSP 理论,阵列式向量计算机必须以向量运算为基础,可变结构的计算机系统的结构及其计算模型主要采用逻辑与代数的方法。

(5) 从计算机各种应用的程序设计方面考察,任何一个可在存储程序式电子数字计算机上运行的程序,其对应的计算方法首先都必须是构造性的,数据表示必须离散化,计算操作必须使用逻辑或代数的方法进行,这些,都体现在算法和程序之中。此外,到现在为止,程序的语义及其正确性的理论基础仍然是数理逻辑,或进一步的模型论。因为,真正的程序语义应该是模型论意义上的语义。

(6) 高等代数和一般抽象代数只解决了个体对象为简单个体的论域上的大量运算问题,但是对具有结构特征和属性成分的复杂个体的论域上的运算问题,表达和处理是不方便的,常常是有困难的。针对这类对象的运算操作及其正确性等语义学问题,有必要发展泛代数和高阶逻辑理论。

当然,这样一种解释或许是粗略的,不全面的,但基本上能说明问题。至于其他的数学分支内容,我们仅需要从它们往往只在有限的几个领域内有重要作用,而在其他许多重要的领域或方向上较少用到,或对整个学科的知识结构不起核心的支撑作用这样一个事实便不难看出它们相对次要的作用和地位。

在国外许多著名的大学中,计算理论是许多教授开列的研究方向之一,即使该学者根本未从事这一方向的研究,为什么?可见计算理论在计算科学中的地位。就本科教学而言,可计算性理论,形式语言与自动机理论对计算科学专业的学生很重要,它通过重要概念和典型方法的教学,有助于学生从理论上深化所学习的各门课程的内容,纲举目张,并掌握计算科学的一些初步的研究方法。有了这一基础,毕业生就能对一些问题的能行性作出初步的估计,也为将来进一步学习和深造打下较好的基础。

认清了计算科学的专业理论基础,就会坚定重视基础数学教学的认识。众所周知,像数理逻辑这样的专业基础课,其内容并不涉及很深的数学分析,函数论或高等代数的知识,更不涉及到微分方程理论等后续数学课程。然而,若单独作为一门 72—76 学时的课程开设,它的内容就比较抽象,学习起来就比较困难。许多数理逻辑名著在前言中都强调,要求读者具有较好的大学数学训练。难道是作

者故弄玄虚?不是。实践告诉我们,要学好这门课程,实在是要求学生必须具有良好的大学基础数学训练和某种数学上的成熟性。然而,由于大多数学校开设的数理逻辑或离散数学课程只涉及较为简单的命题演算与一阶谓词演算,也由于大多数学校包括招收研究生的学校在本科生、研究生教学计划课程中因种种原因没能开设算法设计与分析、可计算性理论、计算复杂性理论、形式语言与自动机理论、高等计算机体系结构、形式语义学、高等逻辑、并行算法设计与分析等课程,更不可能开设计算语言学、程序理论、软件工程(形式化软件开发方法为主)、容错理论及其应用、人工智能的逻辑基础等课程,其内容和学术深度的不足掩盖了对学生前期课程的深度要求,从而导致计算科学界对这一学科的教学在认识上出现偏差。

计算科学作为一门年轻的学科,由于其研究与开发中思维方式的特殊性而使得它与数学建立了密切的联系,决定了一代又一代计算科学工作者的思想方法与工作方式。现在的问题不是我们愿意不愿意承认这个事实,而是我们有没有勇气面对这个现实,按照科学的方式方法和正确的方向开展工作,乘风破浪,扬帆远航,去探索未知世界,去开发新的知识王国。

3.5.2 计算科学与其他相关学科的关系

计算科学还与其他学科有着各种各样的联系,这是需要读者了解的。

计算科学可以在几乎所有的学科领域,甚至我们日常生活的各个方面找到应用,原因是计算(作广义理解)确实是人类最基本的智力活动之一。如果把每一个人看成是一台计算模型或机器,那么,人的思想是一种计算,相应的行为就是计算的外在表现形式。一旦离开了计算,人类将一事无成。在各个比较大一些的领域范围内,计算科学已或多或少地找到了一些应用,这是计算机具体应用研究与开发同各个学科之间建立的联系。需要指出的是,在大多数计算机具体应用中,由于所采用的计算机基本应用技术都是比较成熟的技术,实际应用中对系统的开发和数据的处理客观上要求研究人员具备具体专业领域的知识,因此,计算科学专业人员在计算机具体应用中不具有特别的优势。

计算科学的发展也必然受制于其他科学技术的发展,这早已为计算机发展的历史所证实。为了将计算科学推向更高的层次和水平,学科的发展近年来正在更多地依赖其他学科的发展和进步,参照和利用其他学科的思想、方法和成果。例如,在新一代计算机系统的研制中,人们正在研究和考虑使用光电子技术应用于计算机的设计和制造;医学中脑细胞结构、脑神经应激机制的研究,认知心理学的研究,甚至蜜蜂蜂窝结构的研究都在影响着计算科学一些方向的发展,如体系结构、神经元网络计算,等等。从现在大量的计算科学资料中不难得知,要断言

一个学科对计算科学的发展没有丝毫的影响是难于想象的。就目前可以预见的对计算科学可能产生较大影响的学科而言，物理学中的光学，物理和化学中的精细材料科学，哲学中的科学哲学，生物科学中的生物化学、脑科学与神经生理学、行为科学等对计算科学可能会产生较大影响。

尽管如此，我们应该清楚地认识到，与计算科学联系最紧密的学科是哲学中的逻辑学，数学中的构造性数学，电学中的（微）电子科学。在不远的将来可能是光电子科学，生物科学中的遗传学和神经生理学，物理和化学科学中的精细材料科学，其影响的切入点主要集中在信息存储、信息传递、认知过程、大规模信息传输的介质和机理方面。

有一个问题需要指出，在不少学校计算机科学系的教师中，特别是过去从事硬件教学和研究的一些教师，始终坚持将有关硬件及其基础的教学放在过分重要的位置上，而实际上，国内除了极少数有条件从事硬件研究的学校和国家需要少量硬件制造人才之外，大多数学校不可能也没有必要将教学的重点放在硬件方面。随着超大规模集成电路技术和体系结构技术的发展，在计算机设计和半导体集成电路设计制造之间早已进行了严格的专业分工，即计算机的整机设计和体系结构由计算科学专业人员负责，而计算机系统各硬件部件的制造则由半导体集成电路或微电子专业的人员负责。当然，这样认识不是说硬件不重要，而是应该在硬件基础教学的基础上，加强计算机体系结构、数字系统和微机接口的教学，努力为机械、电子、光学、探矿、汽车、飞机、火车等工业的产品升级换代培养应用技术人员，并设法在大学本科开设计算机体系结构实验单元。遗憾的是，计算机体系结构的实验开支太大了，国内几乎没有学校有经济实力能支持这类实验单元的开设。

计算科学与通信科学之间的关系的重要性需要引起学术界的广泛重视，实际上不仅仅是计算科学与通信，3C（Computer, Control, Communication）作为时代的标志符号，反映了计算机、控制与通信三者之间的结合代表了今后一个时期新兴工业科学技术发展的一个方向。计算技术与控制技术如何很好地结合以支持产业升级、自动机器人、智能控制等，计算技术与通信技术如何很好地结合以支持分布式计算、分布式移动计算、分布式容错计算、分布式信息安全等，控制技术与通信技术如何很好地结合以支持远程遥控、自动目标跟踪等，都需要人们投入更多的人力、物力去探索，去开展研究工作。其研究与开发前景光明且重要，它们将在技术上真正为未来高度智能化的计算提供重要的技术支撑。

3.6 范型及其科学意义

科学家在科学的研究过程中是怎样发现新知识的? 更具体地说, 他们是如何获得和接受定律或理论的? 这实际上涉及到科学发现、科学推理和工作的方式方法问题, 是科学方法论的中心问题。这个问题有三个方面: 如何看待研究的对象? 如何发现新的知识和如何证明新知识的真理性? 发现和证明新知识应该依据什么推理规则? 在 19 世纪中叶以前, 从亚里士多德到穆勒 (J. S. Mill) 都认为科学家依据固定的推理规则既能够发现也能够证明科学真理, 科学是得到证明的确实可靠的知识, 只是他们对于科学推理的规则存在不同的看法, 它们构成了逻辑实证主义最核心的内容。亚里士多德认为三段论是科学推理的标准模式, 因而科学推理是演绎的; 穆勒则继承了培根的归纳法, 认为实验研究方法是科学研究所依据的推理方法, 从实验结果中运用归纳法可以发现和证明新的科学知识, 并系统阐述了归纳逻辑系统, 发表了著作《逻辑系统》。

实际上, 亚里士多德和穆勒等人许多哲学研究的工作说明了两个问题: 第一, 如果从评价科学知识的角度看问题, 我们需要对新知识的真理性进行科学的评价; 第二, 如果是从发现知识的角度认识世界, 那么, 我们需要的是建立一整套的方式和方法, 解决如何去发现新知识。

围绕第一个问题, 人们可以从科学知识系统的结构, 科学知识系统的逻辑形态及其结构, 科学知识系统逻辑结构的系统特征等几个方面进行考察, 研究科学知识系统的逻辑体系结构。这样, 就可以把科学知识的真理性真正建立在严密的逻辑基础之上, 能为绝大多数科学家所接受。

研究知识系统的结构, 可以将科学知识分成经验 (科学) 知识、理论 (科学) 知识和计算 (科学)¹⁾ 知识三部分。经验知识是在科学观察和科学实验活动中获得的对自然界现象和事物发展与变化的客观规律的认识, 理论知识是认识主体在对经验知识进行的理性思维活动中所取得的对自然界事物发展与变化的客观规律的认识, 而计算知识是在经验知识和理论知识的基础上, 通过某种自动装置的计算运行揭示的对自然界事物发展与变化的客观规律的认识。经验知识、理论知识、计算知识三者之间存在着相互作用, 相互参照等复杂的关系。当从科学知识的性质来考察科学知识系统的结构时, 必然涉及科学知识的形式与内容, 也就是科学知识具有内容—形式的结构。经验知识、理论知识、计算知识包含了大量的具体内容, 但同时也常常以多种形式出现在世人的面前。任何事物都是有形式的

1) 此处的计算 (科学) 知识指以广义计算科学作为科学类形态得到的科学知识。

内容。例如，自然数是一个所有小学生都知道的概念，但是，它的形式可以有多种，如十进制形式、二进制形式等。其实，如果用皮亚诺公理系统来定义，用集合来表示，自然数的形式不同于任何进制的阿拉伯数字的表示形式。将自然数归结为经验知识，那么，可以建立阿拉伯数字；将自然数归结为理论知识，自然数可以用皮亚诺公理系统加以定义，每一个自然数都可以推导出来，或证明它是某个自然数的后继；将自然数归结为计算知识，每一个自然数可以在皮亚诺公理系统定义的基础上通过计算得到，因为，我们总能从 0 出发，计算得到它是某个自然数的后继。显然，表示形式不同，将来的变化、发展、应用的前景就可能不同。

一门科学作为一个知识系统，是对其对象领域中事物的本质、发展与变化规律的揭示，也即最一般的联系的把握和表达。这种认识过程正是通过科学知识系统的形式—内容结构的形式方面来保证的。要将科学知识系统地建立在逻辑基础之上，这种形式只能是逻辑的形式，即我们所说的逻辑形态。为了运用抽象的原则精密地阐述科学知识系统，科学知识系统的逻辑形态常采用形式系统，包括概念和命题两种逻辑成分。概念是逻辑形态的“原子”，作为概念形式通常称为词项；命题由词项组成，是对事物本质属性和事物发展与变化规律的陈述。形式系统由词项、命题以及它们之间的逻辑关系组成，而逻辑关系包括构成命题的规则和逻辑推理两部分。人们总是试图在形式系统中用命题表达被研究的对象，通过推理证明命题或否定命题，由此发现和揭示科学知识。

所谓科学知识系统的逻辑结构，是指“逻辑部件”及其联系。“逻辑部件”指基本原理、逻辑演绎、经验结论这样一些表达为形式系统中词项和命题这样的一些东西。基本原理包含基本概念和基本定律，逻辑演绎是从基本原理出发推出定律和定理的逻辑手段，经验结论是从基本原理推出的命题中那些直接反映经验的命题。历史上，命题逻辑、一阶谓词逻辑都是著名的形式系统，可以作为许多科学知识系统的逻辑结构。

随着科学知识总量的不断增加和科学知识系统的日益复杂，除了需要研究科学知识系统的结构，研究科学知识系统的逻辑形态及其结构外，还需要对科学知识系统逻辑结构的系统特征进行考察，以此确定科学知识系统的逻辑结构的合理性。科学知识系统的逻辑结构具有下列诸多系统特征，这些系统特征从不同的角度反映了这个逻辑结构的合理性。

(1) 一元性

一个科学知识系统的逻辑结构是对该系统的对象领域最一般规律性的把握和反映，这就决定了逻辑结构的一元性。这就是说，对于一个客观的实在对象，只有惟一的一种逻辑结构作为对它的认识。当然，在这个领域内可以有多个不同的理论，但它们具有相同的逻辑结构。不同的理论只是该逻辑结构的不同表达形式。

当然,关于一元性学术界存在着不同的认识。因为在计算科学中,不同的逻辑结构可以支撑同一个方向的科学理论。

(2) 简单性

科学知识系统是对其对象领域中事物的本质、发展与变化规律的揭示,而本质对于现象来说是简单的,否则科学研究就失去了意义,所以,科学知识系统的逻辑结构应该是简单的。逻辑结构的简单性是建立在对象和事物本质的属性与内在规律的基础之上,具有客观的依据,同时又是科学思维的逻辑要求。

(3) 内部一致性

逻辑结构是由逻辑成分通过逻辑联系而形成的一个整体,是对对象领域中事物的本质、发展与变化规律的揭示,也即最一般的联系的把握和表达。逻辑联系的一个重要特征是一致性,即无矛盾性,否则,这样的逻辑结构是不能保证科学知识的真理性的。所以,逻辑结构必须具有内部一致性的特征。

(4) 外部相容性

一个科学知识系统的逻辑结构还必须具有外部相容性的特征。所谓外部相容性,是指人们获得的经验知识同这个已经确立的科学知识系统的逻辑结构之对应部分之间没有矛盾。一个科学知识系统总有它一定的对象领域和适用范围。当科学的发展导致形成包含原来科学知识体系的时候,外部的相容性将保证原来的科学知识体系被包容和同化到新的理论之中,而不是相互更迭或推倒重来。

(5) 稳定性

一个系统的稳定存在必然维系在其结构稳定的基础之上,这是系统的重要特征。任何科学知识体系的发展,当其具有很好的稳定性时,预示着它的逻辑结构具有很好的稳定性。也只有当它的逻辑结构具有很好的稳定性时,才可能在引入科学知识成分时获得进一步的深入发展,才可能获得在更广阔的经验领域内给出精密语言和解释。

上面所述的内容主要体现了演绎逻辑主义和逻辑经验主义对于认识世界的哲学观。

我们知道,科学知识的形式有着特殊重要的意义,科学知识以其抽象性、确定性和精密性而产生了巨大的认识力量。如果离开了特定的形式,任何科学实验、理论思维、计算过程所获得的知识内容便不成其为科学知识,这就是说科学知识必须保证内容和形式的统一。实际上,正如库恩后来所指出的,人类的认识历史表明:“必须先研究事物,而后才能研究过程。”人们可以从具体科学理论中抽象出抽象理论,从抽象理论中揭示理论的具体性。

围绕第二个问题,需要研究科学探索的方法。科学探索在它漫长的历史进程中,大大扩展了人们对世界的认识,它所带来的科学预见性和技术应用的巨大成

功,说明了科学本身具有很强的洞察力和很高的可靠性。科学的发展使人们相信,科学所以取得这样的成功,源于它特有的科学探索方法。一切科学理论只有在赋予哲学解释之后才能获得升华。所谓升华是指一个科学理论不仅在内容上更趋完备,而且其基本思想、方法、原理和技术已不单纯只在这一科学理论中有效,而且在方法论上已能对其他科学理论的发展产生示范、参考和借鉴作用,在人类的认知过程中具有哲学上的指导意义。于是,科学方法论被认为是一门系统探究科学研究所特有的方法,分析、检查它成功或合理的依据。

将科学方法描述为一套解决科学问题的一般的规则系统,是一个很好的想法,而且,我们也已经知道,对科学研究所使用的这种规则的不同,产生了不同的科学哲学流派和思潮,科学也分化成了经验科学和非经验科学两大类。所谓经验科学是对在科学研究中具有这样特点的一类科学的总称,它主张知识的可靠性必须参照用以检验经验判断的观察或实验结果材料来进行评估。而非经验科学并不一定需要通过观察和实验来进行,例如,可以通过逻辑和数学的方法进行演绎推导出,或通过其他方式方法进行。

然而,科学的发展和社会的进步并不总是偏爱演绎逻辑主义和逻辑经验主义。众所周知,为了保证科学探索的合理性,需要有一套更一般性的原则和标准。可是,在一种科学主义理念的约束下,一种科学研究过程不可能自己规定自己的行动是否合乎理性,因为它究竟是否合乎理性还需要从这个过程之外来观察,通过与此过程行为有关的事物来综合判断。例如,一个衣冠整洁的人向别人伸出请求施舍之手,然后过了一会转而拿出食品巧克力分发给行人中有大人带的儿童,合乎理性吗?这要看他的行为想要达到什么目的。如果这个人本身是一个普通人而非科学工作者,那么,他的行为显然有点问题,可能属于病态。但如果这个人是一个心理学家,他想通过这个试验测试陌生人与人之间的互信程度,那么,他的研究过程是合理的。特别,心理学家还可以在儿童福利院、大街上进行不同环境下的试验,他本人可以分别用佩带慈善机构的标牌和不佩带标牌等是否亮明身份的方式进行组合试验,这一切似乎都是理性的。但是,如果他带上标牌在一些精神病人中进行试验,并口头说明自己的身份,或者在一些医院的住院部未经医生同意就在病人中进行试验,那么,他的行为是非理性的。可见,一个科学工作者的科学研究过程和行为是否合乎理性,不仅与行为的目的有关,而且和研究对象及其环境有关。

科学整体上是一种理性的事业。虽然我们现在已经知道,科学研究中获得的知识并非都具有真理性,但科学研究确实是我们探求知识的过程中所拥有的最好的方式。因此,有必要探求科学研究更一般化的“套路”。人们相信,就像我们日常生活中做事,总是能够把事情处理得比较好的人,必然有一种“套路”。不仅

日常生活中有“套路”，而且每一个学科都应该有科学研究的“套路”。

20 世纪 40 年代后期，美国科学哲学家库恩在研究理论物理时发现，他所接触的科学史和实际文献史料与自己以前受逻辑实证主义影响而形成的科学史观很不相同，他开始意识到逻辑实证主义对科学发现的局限性。其时，第二次世界大战已经结束，人类社会再次迎来了一个相对和平的发展时期，现代科学技术的发展正处于日益抽象化，加速发展的前夜。此时，库恩深刻地认识到，人类对自然的认识从古希腊时代到欧洲文艺复兴时期再到现代，经历了一个从原始综合到分支深入再到全面综合的过程。在过去的半个多世纪里，现代科学技术的发展不断地向纵深和广度发展，在既高度分化又高度综合的发展过程中，形成了以高度综合为主的发展趋势，验证了库恩的预见和观点，有力地支持了库恩提出的以科学哲学、科学史和社会科学相联系的长期综合观点去重新认识和研究科学历史过程，从中发现认识自然规律的科学观。换言之，库恩提出的以长期综合的观点为指导思想的科学观，体现了从事现代科学技术需要以多层次、多视角、立体地审视研究对象的客观要求，反映了时代的要求。

按照传统的看法，科学是现行著作和课本中事实、理论和方法的总汇，是一堆现成的知识，来自经验或合乎逻辑，因而是比较可靠的或较少谬误的。科学的发展就是对这个知识大厦的不断添加。库恩描绘了科学的另外一个形象：科学是一定社会集团按照一套公认的信念所进行的“专业活动”，除了非常短暂的时期以外，通常科学主要表现为一种在一定思想指导下的高度定向的研究活动。它不仅仅是现成知识的堆砌，而且是人类探求知识的活动。

今天，众所周知，这不过是科学本身所具有的两个不同的侧面。在漫长的历史进程中，科学主要表现出第一个作为知识的侧面，而隐蔽着后一个作为活动的侧面。库恩的独创性在于他认识到，无论是学习、分析知识的认知活动，还是发现知识、发展科学的认知活动，都有着一种“套路”，这种“套路”就是范型。经过库恩和后人的工作，范型的概念被逐步定型。一般认为，范型包含作为从事科学活动的精神价值观，作为文化活动的工作形态，作为探索世界奥秘和解决实际问题的方法工具，作为检验和构建科学理论体系真理性的本体论基础。

对于计算科学来说，从事科学活动的精神价值观包括如何看待计算科学，比如，计算科学应该建立在构造性数学基础之上，还是其他科学技术之上；作为文化活动的工作形态，计算科学包含了理论、抽象、设计三个学科形态；作为方法工具，学科基本问题、学科发展主线、核心概念、典型方法、典型实例、学科基本工作流程方式等内容构成了它的主要内容；作为检验和构建科学理论体系真理性的本体论基础，范型包含了对于各种元数学、元概念、元方法的讨论。

每一个学科都有其自身的基本问题、学科发展主线、学科知识组织结构、学

科形态、核心概念、典型方法、典型实例和学科基本工作流程方式，这些内容可以在很大程度上帮助科学家发现知识，认知世界，探索未来，建立比较可靠的科学体系。

3.7 计算科学的学科形态与核心概念

理论、抽象和设计是计算科学三个基本的学科形态。所谓学科形态，是指从事该领域工作的文化方式。对计算科学的深入研究使我们已知该学科存在三种主要的学科形态，即理论、抽象和设计。对这三种学科形态，我们作如下解释：

第一种形态是理论，基于计算科学的数学基础和计算科学理论，广泛采用数学的研究方法。按照统一的、合理的理论发展过程，包含以下四个步骤：

- (1) 对研究对象的概念抽象（定义）；
- (2) 假设对象的基本性质和对象之间可能存在的关系（定理）；
- (3) 确定这些性质和关系是否正确（证明）；
- (4) 解释结果（与计算机系统或研究对象形成对应）。

这个学科形态的基本特征是其研究内容的构造性数学特征，是区别于更广泛的数学科学学科形态的典型特征。

第二种形态是抽象，或称模型化，基于计算科学的实验科学方法，广泛采用实验物理学的研究方法。按照对客观现象和规律的实验研究过程，包含以下四个步骤：

- (1) 确定可能世界（环境）并形成假设；
- (2) 构造模型并做出预言；
- (3) 设计实验并收集数据；
- (4) 分析结果。

这个学科形态主要出现在计算科学中与硬件设计和实验有关的研究之中。当计算科学理论比较深奥，理解较为困难时，不少科研人员在大致了解理论、方法和技术的情况下，基于经验和技能常以这种学科形态方式开展工作。

第三种形态是设计，基于工程，广泛采用工程科学（如建筑工程）的研究方法。按照为解决某一个问题的构造系统或装置的过程，包含以下四个步骤：

- (1) 叙述要求；
- (2) 给定技术条件；
- (3) 设计并实现该系统或装置；
- (4) 测试和分析该系统。

这个学科形态广泛出现在计算科学中与硬件、软件、应用有关的设计和实现

之中。当计算科学理论(包括技术理论)已解决某一问题后,科研人员在正确理解理论、方法和技术的条件下,可以十分有效地以这种学科形态方式开展工作。

第二种学科形态抽象,是对客观现象和规律的抽象,是对客观现象和规律的描述和刻画。在这一过程中,与具体客观现象和规律无关或对具体客观现象和规律影响不大的因素被去掉了,其内容构成了计算科学理论的一部分,即技术理论中其内容比较难于用简洁的数学形式描述的那一部分。第一种学科形态理论,也是一种抽象,是对客观现象和规律的数学化、形式化描述,是对能行问题及其求解过程的刻画,其内容构成了计算科学的数学基础和理论计算机科学。本质上,这两种学科形态都是对客观现象和规律的抽象,只是方式的不同。第三种学科形态设计是对计算科学理论的工程化实现。

理论和抽象(或称模型化)在本质上的统一,以及计算科学理论的构造性特征与全部计算科学的核心问题,即能行问题一起决定了计算机科学和计算机工程在本质上并没有区别。

在计算科学的研究与开发中,三种学科形态并不是孤立出现的,它们常常交织在一起。经验告诉我们,没有抽象形态的支持,理论研究就失去了背景参照;没有理论的指导,工程开发往往会误入歧途。有许多例子表明,抽象和设计阶段出现了理论;理论和设计阶段需要模型化;而理论和抽象阶段离不开设计,它们必须考虑到现实是否能行。

三种不同的学科形态实际上反映了在计算科学领域内从事工作的三种文化方式。理论关心的是以形式化方式揭示对象的性质和相互之间的关系,这是一种按照某种科学规律构筑人工科学的典型模式。抽象关心的是以实验方式揭示对象的性质和相互之间的关系。而设计关心的是以生产方式对这些性质和关系的一些特定的实现,完成具体而有用的任务。

在计算科学的发展中,有一批在各个分支学科中重复出现的概念。它们虽然在各学科中的具体解释在形式上有差异,但相互之间存在着重要的联系。核心概念是计算科学重要思想、原则、方法、技术过程的集中体现,有助于在学科的深层统一认识计算机科学。对核心概念的深入理解和正确拓展与应用的能力,是计算科学家和工程师成熟的标志之一。

核心概念是方法论的重要组成部分,一般具有如下特点:

- (1) 在本学科的不少分支学科中经常出现,甚至在学科中普遍出现;
- (2) 在计算科学理论、抽象和设计这三个过程的各个层面上都有许多示例;
- (3) 在理论上具有可延展和变形的作用,在技术上具有高度的独立性。

下面,我们把计算科学的核心概念大致作一个筛选和分类介绍,这样或许能更好地便于读者理解。当然,需要指出的是,某一概念划入一个类并不说明它只

在一些分支方向有效。本书中我们对核心概念的介绍只列出了名称而未作详细解释，这主要是考虑到一年级的学生在理解上还不具备基础。我们希望，读者在今后的学习中注意这些概念在不同课程中的具体解释，最好能够通过思考，将这些概念串联起来，把自己所学的专业知识尽可能地系统化。

(1) 计算模型与能行性

计算模型 (Computational Model)，可计算性 (Computability)，计算复杂性 (Computational Complexity)，最优性 (Optimum)，相似性与对偶性 (Similarity and Duality)；

(2) 抽象与构造性描述

论域与计算对象 (Domain and Computing Object)，枚举 (Enumeration) 与有穷表示 (Finite Representation)，分层与抽象的级 (Hierarchy and Levels of Abstraction)，内涵与外延 (Intension and Extension)，递归 (Recursion)，归纳 (Induction)，自由与约束 (Freedom and Restriction)；

(3) 系统特征

相容性 (Consistency)，完备性 (Completeness)，单调性 (Monotoneity)，透明性 (Transparence)，容错与安全性 (Fault-Tolerant and Security)，开放性 (Openness)，稳定性 (Stability)，健壮性 (Robustness)；

(4) 计算方法

折衷 (Compromise)，分解 (Decomposition)，集成 (Integration)，类比 (Analogy)，推导 (Inference or Reasoning)，变换 (Transformation)，扩展 (Extension and Expansion)；

(5) 实现技术

类型 (Type)，进程与线程 (Process and Thread)，顺序与并发 (Sequence and Concurrent)，软计算结点 (Agent)；关联 (Binding) 与实例化 (Instantiation)，现役 (的) (Active)，虚拟 (的) (Virtual)，编码 (Coding)，模式匹配 (Pattern Matching)，分权 (Branching)，合一 (或通代) (Unification)，循环与迭代 (Loop and Iteration)，重用 (Reuse)，协议 (Protocol)，规范与标准化 (Standardization)。

3.8 计算科学的典型方法与典型实例

对于上面所讨论的计算科学中的三种学科形态和在学科中重复出现的核心概念，在将三个过程 (学科形态) 贯穿在整个学科各门课程教学的始终，用核心概念将各分支学科的内在联系串联起来时，事实上，学科形态与核心概念并不能完全解决问题，还需要于教学中介绍在这门学科中反复出现的具有哲学意义的典型

方法, 并通过一批典型实例给学生以较深刻的感性认识。

典型方法与典型实例是属于方法论的内容。就目前我们对学科的认识, 已知有下列几种典型的方法和一大批典型实例。它们是内涵与外延的方法, 以递归、归纳和迭代技术形式为代表的构造性方法, 公理化方法, 快速原型方法, 演化方法, 以及展开与规约方法。

内涵与外延是哲学的两个基本概念。所谓内涵是指一个概念所反映的事物的本质属性的总和, 也就是概念的内容。外延是指概念所界定的所有对象的集合, 即所有满足概念定义属性的对象的集合。内涵与外延的方法广泛出现在计算科学的许多分支学科中。这是一个能够对无穷对象的集合作分类处理的方法。

下面我们举一个例子。

例 15 奇数与偶数的定义。

奇数与偶数的外延定义:

$$\{1, 3, 5, 7, 9, \dots\}$$

$$\{2, 4, 6, 8, 10, \dots\}$$

奇数与偶数的内涵定义:

$$\{x \mid x \in N \ \& \ (x \bmod 2) = 1\}$$

$$\{x \mid x \in N \ \& \ (x \bmod 2) = 0\}$$

为了对被研究的客观世界的对象作概念上的抽象, 我们需要内涵与外延的方法。例如, 构造通用专家系统, 即开发专家系统工具(外壳)的思想是一种内涵与外延的方法; 指称语义和操作语义使用的是外延的方法。不过, 人们已经认识到外延方法在语义学研究中的局限性, 开始从方法上探讨内涵的途径; 非单调逻辑系统的模型(语义)研究中所以遇到困难, 看来以前逻辑学中使用指派(或解释)的方法(一种外延的方法)并不适合可能世界(环境)会发生改变的逻辑系统的模型研究, 应该探索内涵的方法, 或其他方法。可以考虑对逻辑系统的公式解释时先不确定值, 而是确定一个取值范围, 然后通过约束条件的改变(反映环境的变化)逐步缩小取值范围, 最后确定值。此外, 还有像操作系统中的虚拟存储技术, 体系结构研究中的(并行)虚拟机器技术等本质上也是一种内涵与外延的方法。

构造性方法是整个计算科学学科最本质的方法。这是一种能够对论域为无穷的客观事物按其有限构造特征进行处理的方法。

例 16 谓词逻辑系统中合式公式的定义。

(1) $F(a_1, \dots, a_n)$ 是合式公式。其中, a_i 是表示不空论域中个体的形式符号, 即个体词;

(2) 如果 A 是合式公式, 则 $\neg A$ 是合式公式;

(3) 如果 A 和 B 是合式公式, 则 $[A \wedge B]$, $[A \vee B]$, $[A \rightarrow B]$, $[A \leftrightarrow B]$ 是合式公式;

(4) 如果 $A(a)$ 是合式公式, a 在其中出现, x 不在其中出现, 则 $\forall x A(x)$, $\exists x A(x)$ 是合式公式。

在人们的日常生活中, 命题是用来表示现实世界中各种断言的。如“1997年6月30日上海地区是晴天。”显然, 我们可以凭直觉想象到存在无穷多的命题。那么, 面对各种各样的命题, 我们如何用计算机自动地进行处理呢? 考虑命题处理的工作, 有几个问题是我们必须考虑的。首先是命题的识别, 即确定一个符号序列的式子是否构成命题, 这也是对命题作进一步处理的基础; 其次, 如何从已知的命题出发, 构造或推导出更复杂的命题, 包括形式上的推导和逻辑上的推导, 从命题等价的角度如何简化命题 (注意, 这也是一种推导); 第三, 命题的语义问题, 即如何在我们所讨论的范围内对命题作出合理解释的问题, 并保证这种解释不会自相矛盾。于是, 问题归结为能否找出命题的构造性定义描述, 以及如何对命题进行识别、推导, 如何判别两个命题等价。例 16 实际上为解决命题的识别、命题的构造奠定了基础。

例 17 (斐波那契序列) 设有一个数的集合 $S = \{0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots\}$, 对任意给定的正整数 x , 试设计一个判别 x 是否在 S 中的算法。

由于给定的集合 S 是一个无穷集合, 无论在任何计算机上都不可能将这个集合完整地存放, 而且, 用事先存放斐波那契序列数的方法, 根本就无法在计算机上设计出判别算法 (这一点请读者想一想)。因此, 必须寻找别的方法。注意到 S 中的元素恰好形成一个斐波那契序列, 利用斐波那契序列数的递归表示, 实际上并不要存放所有的集合元素。下面, 先给出递归表示:

$$f_0 = 0$$

$$f_1 = 1$$

$$f_n = f_{n-1} + f_{n-2}, \quad n \geq 2$$

由上式可以看出, 任何一个斐波那契序列中的数都是排在它前面且紧挨着它的两个斐波那契序列数的和。现在, 有了这样一个递归表示, 实际上无穷多个斐波那契序列数已被有穷表示所完全确定。于是, 关于要求的判别算法就可以很容易地设计出来。一个直观的判别方法是从 f_0 和 f_1 出发, 不停地求后面的斐波那契序列数。每得到一个斐波那契序列数, 就与给定的待判别数进行比较, 直到相等时输出“真”, 或者当得到一个斐波那契序列数大于这个待判别数时输出“假”, 算法结束。

读者可以思考一下, 能否设计更快, 也就是效率更高的算法。

这一方法在不同的分支学科的运用中具体形式可能不同, 但其本质在哲学意

义上是一致的。构造性方法以递归、归纳和迭代技术形式为其代表形式之一。除了众所周知的递归函数论（或可计算性理论）中使用递归定义和归纳证明技术，方程求根和函数计算中使用迭代技术外，在程序设计语言的文法定义和自然演绎逻辑系统的构造中，在关系数据理论模型和对象模型的研究中，以及在编译方法、软件工程、计算机原理、算法设计和程序设计中均大量使用了递归、归纳和迭代等构造性方法。例如，考察自然演绎的逻辑系统、程序设计语言和自然语言可知，它们都是一种语言。既然是一种语言，那么，语言的词法、语法、语义、语用、语境（情景）就成为研究的内容。在对这些内容进行研究的过程中，虽然在这三个方向上创造了形式上不同的概念和方法，也有相同的概念和方法，但本质上这些概念和方法在方法论上是相同的或相近的。如它们都具有构造语言成分的形成规则，进行演算或推导的运算规则，以及语义解释的定值方法。构造性方法为计算科学中大量问题的能行性研究奠定了坚实的方法和技术的基礎。

当然，我们不应该由于递归、归纳和迭代技术形式为构造性方法的代表形式之一而将自己的思维方式囿于一种固定的模式。实际上，对无穷对象的有穷表示处理完全可以采用其他方法，请看下面的例子。

例 18 试设计一个处理宇宙星球的信息系统。

根据天文学知识，宇宙中的星球数量非常多，多到可以认为有无穷多个，包括已经发现的和尚未被发现的。虽然，我们可以将所有的星球按照发现的顺序给它们编上号，将其有关的信息单独存放，但是，如果割裂了它与其他星球的关系，对研究工作是很不方便的。事实上，工作中常常需要对星球作比较研究、分类研究，这就给我们出了一个难题，如何表示星球的有关信息。

中国有句老话，叫做“物以类聚”，意思是同类的东西常常聚在一起。何为同类？广义地说，就是有着共同的，或相近的属性的事物。按照这种思路，我们可以针对处理的对象，引入关系。关系的设计是以区分对象的属性为背景设计的。两个对象之间存在某个关系，当且仅当这两个对象都具有或都不具有该关系所指称的（背景）属性。于是，就像人与人之间依据亲疏关系划分成多个类一样，或者分别按照一个人的年龄、学历、性格、收入等属性划分为多个类一样，也可以对宇宙中星球的各种属性分别将所有星球划分成一个一个的类。这样，当我们需要对宇宙中的星球信息进行处理时，可以很方便地利用事先设计好的，建立在这一数据信息表示基础之上的，具有多种操作功能的宇宙星球信息系统为我们的需要服务。对于同一类的星球，当我们需要研究它们的某项共同的性质或属性时，我们只要研究其中的一个就行了。至于宇宙星球的信息系统怎么设计，只要具体的设计目标和要求给出，对具有本科计算机毕业水准的工程师来说，已不是困难问题。于是，无穷多个星球可分为有限个类。

无独有偶，在近世代数和集合论中，对关系和由关系确定的类的表示已建立了严格的数学理论，甚至早已为今后计算科学的发展提前作了一些数学基础的准备。当然，这类数学的发展，数学家们不一定料到会在计算科学中得到应用，而是要靠计算科学家去挖掘。近年来，小波理论在信息压缩方面的应用就是一例。

公理化方法也是计算科学的一种典型方法。它能帮助学生认识一个系统如何严格表述，认识到完备性和无矛盾性对一个公理系统的重要性，认识每一条公理深刻的背景、独立性和它的作用。可惜，其深刻的哲学意义、学术深度和理解上的困难性使得它在本科的课程中较少出现。近年来，这一方法出现在一些学科的前沿研究中。除了形式语义学的研究中使用公理化方法外，开放信息系统的思想和设计，自定义逻辑框架系统的研究，以及分布式代数系统的研究都采用了公理化方法或吸取了公理化方法的思想。随着学科发展的深化，预计这一方法还将在一些分支方向上得到运用，并推动学科进一步深入发展。

下面，我们对公理化方法的意义和作用作简单介绍，以备大家在学习时有一个虽然初步但是正确的认识。本书的第五章中，我们介绍了布尔代数系统，并给出了布尔代数的一个公理化系统，可以作为本节内容的一个实例。

所谓公理化方法是从尽可能少的无需定义的原始概念（基本概念）和一组公认的命题（基本公理）出发，使用可反映科学推理的逻辑推理规则（即推理法则。注意，这种逻辑推理规则常常具有语法的推理规则和语义的推理规则），用演绎推理来对一门学科进行科学研究的方法。显然，要构建一门完整的科学理论体系，所建立的公理系统必须具有无矛盾性，即在该系统中不能推出自相矛盾的结论。进一步考察，所建立的公理系统应尽可能多地推出这门科学中客观存在的结论，最好是能推出全部的结论。这就是形式系统的完备性。不过，令人遗憾的是哥德尔已指出了有一定复杂程度的完备的形式数学系统是不存在的。此外，对公理的设计和选择，人们也希望公理的条数应尽可能不多不少，正正好好，最起码一条公理不应由其他公理可以推出。这就带来了公理的独立性问题，即一条公理是独立的，当且仅当它不能由其他几条公理推出或表达出来。

对一门科学公理化的目的在于把该门学科表述为一个演绎系统，并通过基本概念的选取，一组公理的设计，以及严格的演绎推理规则、推理过程和形式系统特征的研究，揭示这门学科深刻的内在规律，研究这门学科中的形式证明，同时，也确保这一学科的表述更加严格、准确。

1889年皮亚诺提出了自然数公理系统。下面我们用公理化方法来定义自然数。这个公理系统一共有五条公理。令 N 表示自然数的集合，于是有：

- (1) 0是自然数，即 $0 \in N$ ；
- (2) 每个自然数 n 均有一个确定的后继 n^+ ；

- (3) 没有以 0 为后继的自然数;
- (4) 若 $n^+ = m^+$, 则 $m = n$, 即每个自然数有且只有一个自然数作为其后继;
- (5) 若子集 $S \subseteq N$, 且具有如下性质:
 - $0 \in S$;
 - 若 $n \in S$, 那么 n 的后继 $n^+ \in S$;

则 $S = N$ 。

其中, (5) 断言满足上述两个条件的极小集合就是自然数集合 N , 它是数学归纳法的基础。

显然, 从这个公理系统可以推导出所有的自然数, 并逐步建立数论。

公理化方法的主要作用有以下几条:

(1) 具有分析总结学科知识的作用。一旦某一门科学有了公理化的表述形式, 由于前提和结论, 结论和结论之间已按照逻辑演绎关系串联起来, 有时会方便使用;

(2) 可把一门科学的基础分析得清清楚楚。因为它们可以归结为少量的基本概念和公理 (注意: 此处不要求按公理的独立性要求理解), 有助于在学科的深层次比较和统一不同的学科, 启发、促进和推动新理论的创立;

(3) 在科学方法论上有示范作用。它对各门学科新方法的建立具有很好的借鉴作用。

公理化方法的引入, 确实在研究工作中带来了方便。但是, 作为一个系统, 也存在必须认真对待和加以研究的许多问题。如给定一组公理和若干形式推导规则, 要保证今后的推理是正确的, 必定涉及到这样一些问题:

第一, 这组公理的选择本身是否合理, 并足以概括研究领域的最基本的知识? 选择公理的依据是什么?

第二, 这组公理是否本身存在矛盾? 针对公理系统将来研究的范围, 在合理的解释下, 这些公理是否存在矛盾? 能否保证这组公理在合理的解释下不存在矛盾? 怎么保证?

第三, 若干推导规则本身是否充分地反映了该领域中事物变化的更一般的规律, 它们确实是可以从该领域事物变化规律的进一步抽象中得到的规律? 是否存在某种不合理性? 怎么检验?

第四, 形式上的推导是否与逻辑上的推理保持一致?

第五, 公理系统对客观规律的描述是基于二值逻辑还是其他非二值逻辑?

.....

这些, 都需要我们对原来被研究的对象在整体上具有更深入的了解。

在学科的基础和应用基础的理论研究中，上述三种典型方法是比较常见的学科研究方法。下面，我们介绍在学科的应用研究的工程技术开发研究中使用较多的另外三种典型方法。当然，读者切不可由此认为，某种典型方法仅仅适用于某一类研究而不适用于其他类研究，典型方法在学科的各个层面具有广泛的例示，只是在不同的类别中运用时有的多一些，有的少一些。

快速原型方法是计算科学的典型方法。最初，快速原型的思想出现在软件工程的研究之中，其主要内涵是：在软件的开发中，随着程序代码量的日渐庞大，开发费用和周期的不断增长，人们迫切需要对软件开发中引入的新思想、新原理和采用的新方法、新技术的可行性进行验证，通过验证过程提出改进意见，为实际产品的工程技术开发提供原理性的指导。此时，Prolog 语言刚刚出现，研究人员惊喜地发现，该语言特别适合充当快速原型实验的验证工具。对于软件开发中系统设计的一种新的构思，采用 Prolog 语言只对涉及新构思的内容“去粗取精”地开发原理性的验证程序，在证实了新构思确实可行之后，再按照规范的、工程化的要求进行软件的开发，并且在系统的开发中，基本上按照快速原型验证开发中设计的程序设计思想进行细化。这样做，不仅可以避免大型软件系统的开发出现原理性的错误，而且能够提高大型软件开发的速度和质量。

由此不难看出，快速原型方法事实上是一种低成本系统原理验证性实验方法。这种方法是从其他学科的研究方法发展而来。例如，在许多工业行业，如机械、电子、光学仪器等，新产品的设计和制造广泛采用先研制样机，然后再进行批量生产的方式方法。许多光、机、电产品的开发，为了尽快采用新思想、新原理、新方法、新技术，开发公司首先是组织力量开发新一代样机，这样，不仅验证了产品设计和生产采用的新思想、新原理、新方法、新技术是否可行，而且，在样机研制过程中，积累了对未来批量生产有益的生产与开发经验。不同之处是光、机、电产品采取的这种方式方法因研制的样机接近于未来批量生产的产品而成本较高，而软件开发过程中采用的快速原型方法因开发工具的发展和程序设计技术的支撑使得快速原型方法在实际使用中与最终产品之间存在较大距离但成本低廉。快速原型方法在计算科学的许多方向上都可以找到应用，可惜，能够使用软件开发工具，掌握快速原型方法和技术的专业技术人员目前太少了。

演化方法也是计算科学的典型方法。所谓演化 (Evolution) 方法，在一些文献上也叫进化方法。演化方法的主要思想是，针对具体的问题，首先找到解决该问题的办法（或算法、程序、电路等）或一个初步的解，然后通过各种有效的技术方法改进解决问题的办法（或算法、程序、电路等）或解。近年来在演化算法基础上发展起来的演化计算是该方法最具代表性的研究领域之一（见本章前面对演化计算的介绍）。演化方法在计算科学中有着广泛的应用实例。例如，算法设

计与分析中的代数化简方法; 程序设计方法学中的程序变换方法; 电路设计与分析中首先通过对电路对应的布尔值函数进行化简, 然后简化电路设计的方法; 软件维护与升级; 遗传算法等都是演化思想和方法的具体应用实例。

演化方法在使用时常常与其他典型方法联系在一起。例如, 与快速原型和展开方法合用, 可以开发一类特殊软件的程序自动生成系统, 如人机界面自动生成系统。

尽管演化方法实际上在计算科学的发展中早就出现了, 其他学科也有类似的方法, 但是, 学术界在相当长的一个时期内没有注意到该方法的方法学意义。作者认为, 演化方法是一种特别适合计算科学的技术方法, 因为大量复杂问题的形成过程很可能直接反映了背景对象长期演变的过程, 具有大自然演变、进化的特点, 解决这些问题需要通过重现演进过程来使问题获得解决。随着学科研究与发展的不断深化, 人们不可能在软硬件开发和应用中将所有的研究与开发工作都做得十分圆满, 万无一失。学科自身知识体系的复杂性决定了今后在研究与开发中需要不断地对先前进行的工作进行改进, 不断地完善, 这不仅仅是工程技术开发中一个具有普遍规律性的问题, 而且, 演化方法本身还可能具有重大的、酝酿着学术研究突破价值的科学意义。从近年来演化计算的研究 (主要是算法演化计算研究) 中出现广泛使用逻辑与代数方法的特点考察和获得联想: 如果在计算模型的研究中引入演化计算的方法改造传统的计算模型, 或设计新型的计算模型, 那么, 新计算模型有可能变得具有演化计算能力, 表现出某些健壮性的特征, 这是否暗示了智能计算系统的研究不必一定要沿着首先发展人工智能的逻辑基础, 然后发展智能计算机、智能计算系统的“演化”路线, 而存在着一条崭新的, 以前从未有人涉足过的研究路线呢?

另一种典型方法是所谓的展开与归约方法。展开与归约是一对技术概念, 是在处理实际事务的过程中对两个相向的处理活动所作的一般化的方法学概括。展开的内涵是从一个较为抽象的目标 (对象) 出发, 通过一系列的过程操作或变换, 将抽象的目标 (对象) 转换为具体的细节描述。例如, 在程序设计方法学中, 从一个程序的规范出发, 运用程序推导技术, 可以将一个程序一步一步地自动设计出来, 这样就实现了对该程序的展开。理论上, 一个具体的程序与该程序的规范指称的是同一个对象, 语义完全相同, 只是表现形式不同。从描述与理解的角度看, 前者较为具体, 后者较为抽象。规约方法可以视为展开过程的逆过程。

不难理解, “自顶向下”的程序设计方法是一种展开方法, 从一个电路系统布尔函数表达式出发到对应的电路系统的实际完成所进行的工作方法也是一种展开方法。“自底向上”的语法分析方法, 语义约化方法都是规约方法的实例。读者今后在学习计算科学许多后续课程时, 将会遇到和发现更多具体的展开与规约方法。

典型方法之所以重要，不仅在于它能深刻地揭示学科的基本规律和各主科目之间内在的联系，而且在于它能启发人们更深入地思考学科的一些基本问题，培养学生的创造性能力和科学研究能力。因此，在本科高年级以上的专业教学中，如果教师能够超越于具体某一科目方向的内容向学生介绍一些对整个学科具有普遍意义的思想、方法和技术，无疑将有助于培养学生正确的思想方法和科学研究的能力。

当然。计算科学中还有许多方法也是重要的，如类比推理，证伪发现，统计推断与分析，各种测试方法与测试数据筛选的方法等。但相对于上述三种方法来说，这些方法应用范围比较单一、狭窄，或是哲学意义比较浅显。至于其他的典型方法，还有待人们去整理和总结。

所谓典型实例是指那些反映学科某一方面内在规律和典型问题本质内容的实例。典型实例也是方法论的重要组成部分，由于它们往往以简化形式深入浅出地表达学科深奥的科学规律和学科典型问题的本质内容，因此，在学科研究中常常被用来辅助说明思想、原理、方法和技术，用来比较思想、理论、方法和技术的优劣。显然，典型实例在计算科学发展中具有重要的作用。

在计算机科学的发展中，经过几十年的研究与积累，人们构思和设计了一大批能够反映各分支学科有代表性的、具有问题本质特性的典型实例。

例 19 哲学家共餐问题。

设有五个哲学家，围坐在一张圆桌旁。圆桌中心放有一盘可自动添加的面条，每两个哲学家之间放有一根筷子，共五根筷子。现在，我们进一步假设每一个哲学家的活动均为一个进程：

重复下面的动作：

- (1) 思考问题；
- (2) 饿了；
- (3) 拿左手的筷子；
- (4) 拿右手的筷子；
- (5) 吃面条；
- (6) 放右手的筷子；
- (7) 放左手的筷子；
- (8) 重新转到思考问题状态 (1)。

问：怎样协调五个哲学家的活动，才能保证每一个哲学家都能按照他们的进程正常地活动，不致发生某个哲学家因吃不到面条而被饿死。

考虑这样一种情况：五个哲学家的进程是并发的。如果每一个哲学家当前都恰好拿到了位于左手边的筷子，等待拿右手边的筷子准备吃面条，那么，由于谁

也拿不到右手边的筷子，又不愿意放下左手中的一根筷子，所以，这个系统就处于一种僵死的状态（也叫系统处于死锁状态），最终将导致所有的哲学家饿死。

哲学家共餐问题实际上反映了计算机程序中多进程共享单个处理机资源时的并发控制问题。要防止这种情况发生，就必须建立一种机制，既要让每一个哲学家都能吃到面条，又不能让任何一个哲学家始终拿着一根筷子不放。采用并发程序语言、Petri 网、CSP 等工具，都能很容易地解决这个问题。

典型实例在计算机软硬件的开发中具有重要的应用价值。我们知道，一个系统开发出来之后，面临着用户的确认和验收。对硬件中的程序而言，测试程序只能发现程序有错，但不能证明程序无错。然而，采用形式化的方法证明程序的正确性又是一个很困难的问题，不仅成本高、周期长，而且，对并发程序和并行程序的验证还有一些关键问题尚未解决，怎么办？

众所周知，带有程序的软硬件系统，特别是软件系统与其他产品的一个显著区别是允许产品在销售时带有一些尚未被发现的错误，它们可以在今后的使用过程中，经过开发商不断的售后服务和产品的更新换代加以完善。为此，欲证明系统的性能和状态良好，许多带程序的软硬件系统的确认和验收常采用对典型实例的测试来完成。例如，美国国防部专门准备了上千个系统软件测试实例，在 Ada 语言的编译系统开发出来后，要求研制的软件系统必须通过四百至一千多个典型实例的测试才能投入运行。系统通过四百个典型实例的验证，可以获得一级测试证书，通过一千多个典型实例，可以获得二级测试证书。由于典型实例是在长期的实践中不断积累，根据各类问题的特点经抽象、分类和总结得来的，因此，只要系统能够通过各种典型实例的测试，就在很大程度上有理由使人相信该系统的质量是有基本保证的。

下面，我们列出一些典型实例。读者可以在今后的学习中自己去发现、认识更多的典型实例。

例 20 一组典型实例。

停机问题，哲学家共餐问题，饮料问题，最小费用流问题，可满足性问题，货郎担问题，生产与消费问题，文法的二义性问题，上下文语义问题，最小生成树问题，阿克曼函数，鞍点问题，子集和数问题，确定性自动机与不确定性自动机的等价问题，八皇后问题，九宫排定问题，三个中国人算法，证比求易算法，0/1 背包问题，作业调度问题，路由选择问题，图的最小覆盖问题，荷兰国旗问题，等等。

3.9 学科基本工作流程方式及其科学意义

每一个学科都有其基本问题、学科发展（主流）方向、学科发展主线、学科知识组织结构、学科形态、核心概念、典型方法、典型实例，这些内容不仅能够帮助我们很好地认识一个学科，而且，也能帮助我们在学科研究与开发中较好地开展工作。然而，如何较好地开展工作，又直接归结为是否能够借助上述内容，总结得到一套从事这个学科工作更为具体一些的方式方法，也就是基本工作流程方式。

我们认为，对现实世界中被研究的对象进行抽象，建立必要的基本概念，运用数学工具和方法对其进行基础和应用基础研究，研究（对象）概念的基本性质、概念与概念之间的关系，由此揭示对象发展变化的内在规律，为实验设计和工程设计实现提供方法和技术，并开展实验和工程设计与实现工作是计算科学学科的基本工作流程方式。其中，抽象（主要指抽象化过程）是理论、抽象和设计三个基本学科形态中最重要的一個形态，它是连接学科科学研究与工程应用开发研究的重要环节。

有了学科的基本问题、学科发展（主流）方向、学科发展主线、学科知识组织结构、学科形态、核心概念、典型方法、典型实例、学科基本工作流程方式，认识学科和开展学科研究与开发的“套路”变得清晰起来，库恩所提出的范型的概念在计算科学学科获得比较准确的定位。按照范型概念内涵的多层含义，一个有专业训练的基础，从事计算科学工作的工作者，首先需要对从事计算科学树立一种哲学上的价值观，即更多地是提倡直觉主义逻辑作为整个学科的逻辑基础呢？还是秉承其他逻辑学派的主张。当然，对整个学科认识的价值观还包括其他一些方面，如机器的智能能否超过人的智能？选择哪一个自己更感兴趣的方向？等等。其次，当他开展工作时，完全可以根据学科发展的主线、主流方向、一般方向的内涵和发展现状进行选择，在确定了自己的主攻方向和熟练地掌握了本方向的相关信息之后，借助学科基本问题寻找需要解决的具体问题，或者直接选择别人提出但尚未解决的具体问题，然后，可以根据具体问题的特点和特性对照学科形态，选择解决这个问题拟沿用的学科形态开展工作。在实际解决具体问题的过程中，根据前人和自己工作的基础、当前已经掌握的最新的有关文献资料 and 目前具备的工作条件，按照学科基本工作流程方式开展工作，弄清楚解决这个具体问题是否需要完成全部的基本工作流程方式，还是只需要完成其中的一部分工作流程，并在确定之后，借助典型方法、典型实例、核心概念等内容开展深入、细致的具体研究工作，直至问题的解决。其中，这样一种工作的过程可能存在着多次的反复，存在着深入之后发现新的需要优先解决的问题，存在着需要更新知识

以适应工作需要的问题,等等,但开展学科研究与开发的基本“套路”并没有也不可能发生大的、根本性的变化。显然,按照上述内容,从事计算科学研究与开发不再“神秘”,而且确实有“章”可循,它可以帮助人们认识学科,发展学科,应用学科,这就是学科范型的科学意义。从中,我们也可以深切地感受到学科基本工作流程方式所起的重要作用。

下面,我们通过两个实际的例子来进一步认识学科基本工作流程方式及其科学意义。

例 21 找出用于 RSA 编码法中的一个合适的大整数。

假设现在没有素数或质数的概念,按照 RSA 编码法的要求,我们希望找到一个 32 位左右的大整数 n ,用于编码。我们希望 n 是两个很大的自然数 p 和 q 的乘积,即 $n=pq$ 。显然, p 和 q 分别都不能再含有因子,否则密码很容易被别人破译。 p 和 q 如果符合要求的话,它们分别具有只能被 1 和自身整除的性质,于是,可以抽象得到素数或质数的定义。现在,进一步的研究可以发现,如果 p 和 q 都是素数,那么, p 和 q 的乘积值 n 除了 p 和 q 这两个素数因子之外将不再包含任何大于 1,小于 n 的自然数因子。此外,要使密码不易被对方破译, p 和 q 这两个数应该比较相近,即位数比较一致。弄清楚了这些,找到 p 和 q 应该是没有什么问题的。素数的定义、性质、两素数乘积值的结果分布早已折射出我们应该在什么范围内寻找,而找素数的算法方法也是很多的。当然,好的算法涉及到对素数性质更深入的研究结果及其证明方法。

例 22 以进程为基础的操作系统的发展历程。

历史上,当计算机发明之后,由于用户直接使用机器指令编写程序很不方便,使用外部设备不仅要用机器指令,还要带上“对号入座”的设备编号参数,显然,这样的使用效率很低。随着机器速度和性能的提高,快速的 CPU 与低速的外部设备之间的不匹配引起的 CPU 和内存等资源不能充分利用的矛盾暴露了出来,研制管理程序成为当时一个急待解决的问题。经过若干年的发展,开发计算机系统又从管理程序过渡到了操作系统。早期,由于 CPU 的速度仍然相对较慢,操作系统的主要部分是一种顺序程序,其运行每次只能为一个机器用户提供服务。然而,当 CPU 速度和内存容量大幅度提高之后,这种操作系统显然已经不能适应硬件发展的要求和提高资源有效利用的需要,更不能为多个用户同时使用计算机系统提供服务,操作系统的研究与开发面临重大转折,怎么办?

此时,研究者设想,如果操作系统要能够为多个用户同时使用计算机系统提供服务,必须首先对同时进入机器运行的系统程序、用户程序建立某种运行的最小单位,弄清楚各程序单位的特性和相互之间的关系,然后才能进行系统控制和管理。当他们对操作系统的运行程序进行了某种抽象之后,产生了进程的概念,

紧接着他们对进程的性质进行了研究，弄清楚了一个进程在生成之后于退出前可以有三种状态，弄清楚了三种状态在什么情况下可以相互转换。进一步，研究者又弄清楚了进程（包括系统进程和用户进程）之间的相互关系，弄清楚了进程与计算机系统的其他各部分部件之间的关系，这实际上是系统地研究了进程的性质和进程与进程之间的关系，由此形成了一整套的进程理论。当所有这些基础研究工作完成之后，由于描述进程理论的数学工具属于构造性数学的范畴，有关进程理论的大量具体内容早已折射出操作系统实现的技术思想，于是，以进程为基础的操作系统很快诞生，难题得到彻底的解决。

读者看了上述找大整数和研制操作系统过程的阐述虽可能有事后诸葛亮的感觉，但这些内容清楚地表明了计算科学工作者如何沿用学科基本工作流程方式开展工作，解决问题。毋庸置疑，学科基本工作流程方式的科学示范意义是深远的，因为它是学科范型的核心内容。实际上，类似的例子可以找到很多很多，读者不妨在研读后仔细思考，在今后的学习中注意分析、归纳和总结，这对一个人专业素养的养成十分重要。

3.10 计算科学学科特点、发展规律、趋势及其社会影响

计算科学是在数学和电子科学基础上发展起来的一门新兴学科。它既是一门理论性很强的学科，又是一门实践性很强的学科。几十年来学科自身发展的实践表明，一方面，围绕着一些重大的背景问题，各分支学科和方向上取得了一系列重要的理论和技术成果，推动了计算科学向深度和广度发展；另一方面，由于发展了一大批成熟的技术并成功地应用于各行各业，更多的人是把这门科学看成是一种技术。事实上，理论和技术是计算科学两个互为依托的侧面。计算科学的理论绝大多数属于技术理论。学科的基本问题和本质属性决定了计算科学理论、技术与工程相互之间常常界限模糊。从理论探索，技术开发到工程开发应用和生产的周期很短，许多实验室产品和最终投向市场的产品之间几乎没有太大的差别，这在软硬件产品中是屡见不鲜的。虽然，目前整体上理论研究滞后于技术开发，但随着学科研究和应用的不断深化，理论的重要性地位将愈来愈突出，而技术则渐渐退居为次要的位置。因为大量的研究已经表明，像新一代计算机体系结构、NP 完全性问题、软件工程、并行计算与处理、计算语言学、人工智能等许多方面的难题，并不是技术问题，而恰恰是理论问题。这些难题基本上都与某些数学理论或工具之间存在着密切的联系，它们的解决将对计算科学的发展产生极其深远的影响。

数学是计算科学的主要基础，以离散数学为代表的数学是描述学科理论、

方法和技术的主要工具,而微电子技术和程序技术则是反映学科产品的主要技术形式。在计算科学中,无论是理论研究还是技术研究的成果,最终目标要体现在计算机软硬件系统产品上。然而,作为硬件产品的计算机系统和作为软件产品的程序指令系统应能机械地、严格地按照程序指令执行,决不能无故出错。计算机系统的这一客观属性和特点决定了计算机的设计、制造以及各种软件系统开发的每一步都应该是严密的、精确无误的。就目前基于图灵机这一理论计算模型和存储程序式思想设计制造的计算机系统而言,它们只能处理离散问题或可用构造性方式描述的问题,而且这些问题必须对给定的论域存在有穷表示。至于非离散的连续性问题,如实数域上的函数计算,方程求根等还只能用近似的逼近方法。于是,由于计算模型的非连续性(或称离散化)特点,使得以严密、精确著称的数学尤其是离散数学被首选作为描述计算科学的主要工具。理论与实践相结合,理论与实践的统一是计算科学发展中最重要的特点之一。在这一学科中,数学与电子技术的结合是理论与技术完美结合的一个成功范例。从事计算科学的人都知道,计算科学中不仅许多理论是用数学描述的,而且许多技术也是用数学描述的。大多数计算科学理论不仅仅是对研究对象变化规律的陈述,而且由于能行性这一本质的核心问题和特点的作用,理论描述中常通过构造性方法折射出技术的思想和步骤,而从理论通过方法跨越到技术则完全取决于对理论的深刻认识和理解。一个人如果看懂了以形式化方法描述的技术文献,自然明白技术上应该怎样去做,否则往往误以为是一种理论,离实现尚远。由于离散数学的构造性特征与反映计算科学本质的能行性之间形成了天然一致,从而使离散数学的构造性特征决定了计算科学的许多理论同时具有理论、技术、工程等多重属性,决定了其许多理论、技术和工程的内容是相互渗透在一起的,是不可分的。与大多数工程科学的工作方式不同,在几乎所有高起点的,有学术深度的计算科学的研究与开发中,企图参照经验科学的工作方式,通过反复实验获得数据,经分析后指导下一步的工作从而推进科研与开发工作的方式是行不通的。原因是有学术深度的问题其复杂性早已大大超出了专家们的直觉和经验所能及的范围。

例如,在制造计算机和各种控制系统硬件的过程中有一个电路布线的问题。当电子元器件数量很大时,如何根据电路图设计布线是一个十分繁琐而又辛苦的工作。这在许多人看来是一个技术性很强的工作。从20世纪60年代比较先进的计算机系统出现后,人们就开始研究使用计算机进行自动布线。然而,由于电路的设计中有许多限制参数,如某些元器件不能离得太近以防相互干扰,因此,这一看似简单的问题实际上很不简单。电路布线的目标不仅要在限制条件下在线路板上布得通、布得下,而且还要布得“好看”。尽管一个有经验的工程技术人员能够胜任一般复杂程度的电路布线工作,但是,效率太低了!采用目前先进的自动

布线系统，许多非常复杂的电路布线工作只需专业人员的少量“干预”就能很快正确完成，而且，效率往往提高几十倍。今天，有兴趣的读者可以看一看各种微机的主板。凡是名牌厂家生产的主板，线路常常布得十分漂亮，而杂牌产品则要差一些。这个例子充分说明了电路布线本质上不是一个单纯的技术问题。

当然，并非所有的计算科学理论研究都具有应用价值。实际上，在计算科学发展的几十年中，有不少计算科学理论研究后来并没有得到继续发展。例如，程序理论的研究中，早期曾出现过框图理论和程序的不动点理论，尽管也取得了一些有趣的结果，但后来发现使用价值比较小。又如，在计算模型的研究中，也曾经产生了各种各样的计算模型，但由于后来陆续被证明它们的计算能力没有超出图灵机的能力，又因不具备突出的使用方便的描述能力而不常使用。

应用计算机技术来解决问题，可以采用硬件的方法，也可以采用软件的方法，甚至还可以采用电子线路（非集成电路）和机械的方法，只是后者因成本太高，精度难于保证而早已弃之不用。但无论采用哪一种方法，都必须提供处理该问题的计算过程描述，即详细地刻画计算的每一步应该怎么做。这在计算科学中称之为算法。于是，算法成为计算科学的一个重要研究内容，也有人称算法为计算科学的首要问题或核心问题。在计算科学的研究中，发现或创立一个新算法将是对计算科学的一个实质性的贡献。经验表明，算法研究的基础是数学和与计算科学有关的专业背景知识。

计算科学并不完全排斥工程的方法。相反，计算科学在发展中广泛采用了其他学科行之有效的工程方法。如在软件开发中认识到并采用首先开发工具和环境，进而开发软件的方法；在计算机的设计中，目前广泛使用标准组件的方法；在软件的设计和质量检查中广泛使用软件测试方法和技术、标准化技术，等等。事实上，从软硬件开发的经济效益和专业技术人才的局限性两方面考虑，完全采用形式化的方法来解决计算科学中的所有问题是不现实的。

长期以来，数学，特别是以离散数学为代表的与应用数学与计算科学之间建立的密切联系不仅使计算科学具备了坚实的理论基础和科学基础，而且使计算科学从一开始就以一种与其他学科发展方式很不相同的方式发展。这种方式就是抽象描述与具体实现相分离的方式，即在计算科学的发展中，对大量的、各种有深度的问题的处理采用了将理论上解决问题的抽象描述计算方法、算法和技术的内容与具体解决问题的细节、具体实现计算的技术内容相分离。这样做的好处是由于可计算问题能行性的特点和数学方法的构造性特点，也由于计算科学理论的发展落后于技术发展的特点，人们可以更深入地探讨一些已经出现的技术问题的内在规律。一般说来，采用抽象描述方法从理论上解决的问题，实际工作中也是可以解决的。能否根据抽象描述解决具体的实际问题，关键在于能否正确地理解抽象

描述的内容,这在计算科学各个分支学科的发展中不胜枚举。例如,对数据结构中各种表的数据处理我们已经建立了一套理论方法。在实际工作中,结合具体问题,一个表究竟是用数组表示,还是用链表等其他数据结构表示,在抽象描述阶段是不必关心的。而在实现阶段,我们虽然可以自己选定具体的表示形式,但是表上的具体操作则完全是在理论方法的指导下完成的。又如,针对一个复杂的可化简的逻辑电路,我们并不是先将这个电路设计、制造出来,然后通过反复试验减少电路的元器件实现化简,而是从一开始就设法先通过抽象这个电路的逻辑表达式,然后按照逻辑公式的化简方法化简这个电路,最后根据化简结果设计、制造电路。

抽象描述与具体实现相分离是计算科学发展过程中一个十分重要的特点,它不仅决定了一大批计算科学工作者的工作方式,而且使计算科学的研究与开发在很短的几十年里就已进入计算科学比较深层的阶段。特别,读者应该注意这样一个事实,计算科学正是在将抽象描述与具体实现相分离的过程中,经过对各种计算模型的深入研究和应用使学科很快走向深入的。由此形成了计算科学发展的另一个重要的特点,也使我们窥见到计算模型在整个学科发展中所起的不可替代的独特作用。

计算科学发展的另一个重要特点是几乎在学科各个方向和各个层面,一旦研究工作走向深入,研究内容具有比较复杂的特点,人们首先是发展相应的计算模型和数学工具,然后依靠计算模型和数学工具将研究工作推向深入。例如,在网络协议、程序设计语言、程序语义、并发控制、计算机系统结构、人工智能等领域的研究中都是如此。应该指出的是,这里所讲的计算模型不是指计算数学中的数值分析方法或计算方法。

随着计算机科学技术的不断发展和学科研究的深化,一方面,对计算模型和各种新型计算机体系结构、人工智能的研究将不断进行下去;另一方面,围绕着各种科学计算和数据处理的计算机应用问题,软件开发方法学,特别是并行与分布式软件开发方法学研究以及某些计算机基本应用技术,如演化计算技术将成为未来学科发展的主线。

数据与信息表示是计算科学研究一个非常重要的方面,过去人们对这个问题的重视程度不够,后来在实际工作中才慢慢感受到它的重要性。例如,对于汉字的信息处理,如果当需要处理的汉字超过 10 000 个以后,存储汉字字形的字库必然需要很大的存储空间,即使采用一种称为矢量存储的技术设计字库,依然需要很大的存储空间。于是,谁使用的存储技术好,谁设计的字库所需要的存储空间就小,必然对今后文字处理程序的运行速度会产生很大的影响。人们认识计算机科学与技术,往往是从一些具体的分支学科出发去认识。由于知识结构存在缺失,

难免缺乏整体性的观点和意识。如果我们把计算机科学与技术放在一个更大的信息科学与信息处理的背景下来审视和认识,那么,就不难发现学科发展的一个内在特点,这就是近几十年来随着学科发展的不断深化,研究对象的数据或信息表示的地位明显突出,一种方法求解一个问题的质量和效率,往往更多地取决于处理对象的数据与信息表示而不是施加在对象之上的运算或操作过程本身。这也说明了为什么数据与信息表示理论一直受到理论计算机科学家的重视。因此,像“数字逻辑”、“算法设计与分析”、“编译原理”、“数据库系统原理”、“程序设计方法学”、“人工智能”、“形式语义学”等课程都应该在阐述具体内容时反映学科发展的这个特点。从整个学科发展的趋势看,针对大量信息处理问题,问题如何表示已经成为能否较好地处理问题的关键,处理方法的好坏往往更多地取决于表示方法而不是其他。例如,对于子集和数问题的处理,传统的一些方法效率很低,但是,如果采用二进制数据表示,使用移位等运算操作,可以在位运算级别上给出快速的解题算法。

学科日渐深化的特点决定了计算机科学与技术领域专利技术科学含量的不断上升,也决定了计算机科学与技术领域新产品技术含量的不断上升。过去那种依靠个人的灵性和出人意料的浅显的新思想、新技术、小发明和新产品异军突起,独领风骚的时代已经过去,相反,产业的竞争更多地已经逐步转化为企业背后高学历、高智力的优秀专业人才之间的竞争,甚至已转移到高等学校人才培养质量的竞争。此外,总结人类社会发展与进步的历史,今后社会的发展和国与国之间的竞争将会越来越多地表现为文化的竞争,社会发展与进步不仅需要大量各学科的专门技术人才,而且,在人才培养中,包括科学理念、人文精神和文化修养等对人才综合素质的要求也会不断提高。

计算科学虽然从出现到现在不过半个多世纪,但它对人类社会的影响是极其深刻的。计算科学不仅促进了各行各业的发展,影响和改变着人类的生存方式和生活习惯,而且,使其他学科不同程度地出现加速发展的态势,极大地影响和改变着人们的价值观念和对许多事物的认识。例如,当网络技术日益成熟,互联网络走入千家万户的今天,我们应该意识到联网意味着走向统一。这种统一,首先不是指政治上的统一,而是指技术、文化观念等方面的统一和趋同。由于互联网络的存在,一个国家要想参与世界竞争,就必须融入国际社会,了解国际社会的发展变化动向,在产业分工、经济运行方式、产品技术标准、传统价值观念、生态环境保护、人口控制等方面主动适应国际社会,或通过国际交流取长补短,同时还必须防止自身受到伤害;由于互联网络的存在,各国的文化传统、生活理念、民俗风情、人文精神和国家安全都将面临相互碰撞、渗透、同化、变异、发展的考验;由于互联网络的存在,地球变得很小,地域上的差别在很大程度上消失。

一个国家在科学技术、经济技术、社会形态方面的落后很可能在随着传统文化受到冲击和逐渐消失的过程中引发深刻的社会危机。中国是一个有着五千多年人类文明史的古国,中华民族是一个有着灿烂文化的民族,如何使中华民族文化在现代科学技术条件下,在融入世界民族文化的过程中,继承、发扬和光大,是摆在每一个中国人面前值得思考的问题。

3.11 计算科学知识组织结构及其演变

虽然计算科学的发展在时间上很难划定几个独立的阶段,但粗略地分阶段考察学科的发展及其知识组织结构的演变确能给我们一些启示。

在计算科学发展的早期,大约在20世纪30年代至50年代末,对计算科学研究的主流方向主要集中在计算模型、计算机设计、高级语言和科学计算方面。由于应用主要是大量的科学计算,与数学关系密切,加之在设计计算机的过程中,对逻辑和布尔代数的基本要求,导致大量从事数学研究的人员转行进入计算科学领域。他们不仅在数量上占有绝对优势,而且在工作中也处于主动地位。就当时的情况来看,具有坚实的数学基础,懂得一些电子学,逻辑和布尔代数,很容易掌握计算机原理和设计的方法。如果还掌握了一些程序设计的技术,那他完全可以进入学科前沿。在学科发展的早期,数学、电子学、高级语言和程序设计是支撑计算科学发展的主要专业知识基础。

20世纪60年代、70年代是计算科学蓬勃发展的时期,面对学科发展中遇到的许多重大问题,如怎样实现高级语言的编译系统,如何设计各种新语言,如何提高计算机运算速度和存储容量,如何设计操作系统,如何设计和实现数据库管理系统,如何保证软件的质量等问题,发展了一大批理论、方法和技术。如形式语言与自动机理论,形式语义学(主要是外延的方法),软件开发方法学(程序设计方法学和软件工程等),算法理论,高级语言理论,并发程序设计,大、中、小型计算机与微型计算机技术,程序理论, Petri 网, CSP, CCS 等。这一时期的发展有两个显著特点:

其一是学科研究和开发渗透到社会生活的各个方面,广泛的应用需求推动了学科持续高速发展;

其二是经过大量的实践,人们开始认识到软件和硬件之间有一个相互依托,互为借鉴以推动计算机设计和软件发展的问题;

与此同时,人们也开始认识到计算机理论和工程技术方法两者缺一不可,且常常是紧密地结合在一起的。许多复杂而困难的硬件与软件设计,离不开计算机理论的支持,而大系统的实现也应广泛采用工程方法。对大系统实现中困难性的

认识促进了软件理论、开发环境和工具的研究。大约在此 20 年里, 计算机原理, 编译技术, 操作系统, 高级语言与程序设计, 数据库原理, 数据结构与算法设计, 以及逻辑成为学科的主要专业知识基础。

从 20 世纪 80 年代起, 针对集成电路芯片可预见的设计极限和一些深入研究所遇到的困难, 如软件工程, 计算模型, 计算语言学, 大规模复杂问题的计算与处理, 大规模数据存储与检索, 人工智能, 计算可视化等方面出现的问题, 人们开始认识到学科正在走向深化。除了寄希望于物理学中光电子计算技术研究取得突破, 成倍提高机器运算速度外, 面对现实, 基于当前的条件, 人们更加重视理论和技术的研究。这方面的努力推动了计算机体系结构, 并行与分布式算法, 形式语义学, 计算机基本应用技术, 各种非经典逻辑及计算模型的发展, 从而推出了并行计算机, 计算机网络和各种工作站, 并带动了软件开发水平和程序设计方法、技术的提高。尤其值得一提的是, 在图形学和图像处理这两个相对独立的方向上, 不仅科学研究而且实际应用均取得了长足的进步。这两个方向的迅速发展不仅使计算机的各种应用变得更易于为社会接受, 而且随着计算机硬件和数据库技术的进步, 计算机应用触及到了一些以前被认为是较为困难的领域, 并引发了计算几何、多媒体技术、虚拟现实等计算可视化方向的发展。

正当学科处于高速发展的时候, 许多有识之士却对前景改变了看法, 不像以前那么乐观了。基于并行软件开发方法学, 计算语言学, 人工智能, 超大规模计算机网络的控制与信息安全, 以及硬件芯片设计中遇到的困难和极限, 人们开始对一些基本问题进行反思。如计算概念能突破图灵计算模型定义的范围吗? 什么是智能计算机的理论基础? 逻辑能成为智能计算机的基础吗? 如果能, 那么什么逻辑能成为智能计算机的基础? 软件开发方法学中遇到的程序语言的动态语义问题和计算语言学中的语义、语用问题本质上是相同的问题, 它们都是关于语言的成分如何定值和解释的问题。由于定值的环境(即可能世界和环境)是变化的, 因此, 前人的工作已经表明, 采用指称语义、操作语义等外延的方法不能解决这个问题。那么, 采用内涵的方法能行吗? 基于目前的芯片技术, 如何设计高性能的计算机系统? 如何将并行算法的研究与体系结构相分离? 于是, 基础理论研究重新引起人们更多的重视。围绕着学科遇到的问题, 新一代计算机体系结构, 高性能计算与通信系统, 各种计算模型(包括非图灵计算模型), 形式语义学(主要是内涵的方法), 并行算法设计与分析, 以及各种非经典逻辑系统成为专家关注的重点。然而, 由于长期以来理论的研究滞后于技术的发展, 技术和工程应用的发展速度开始受到制约。在研究工作的方法上甚至还出现了广泛借鉴其他学科的现象, 特别是在研究与人及其行为有关联的学科方向上, 如从神经系统的生理结构与思维功能得到启发产生了神经网络与神经元计算, 试图从别的学科的进展来

确定下一步工作的思路。

经过一段时间的探索之后,人们开始认识到:许多情况下确定一个取值的范围比直接确定一个值的方法更重要;发现一个不能(有效地)自动进行的问题甚至比发现解决一个问题的方法更重要,原因是证实各种不能(有效地)自动进行的问题常常从不同的侧面,以不同的形式揭示了计算的极限,它有可能帮助我们更深刻地认识计算的本质,推动基础研究,特别是计算模型研究的发展;新型计算机研究能否取得突破的关键在于是否能发展新型计算模型并从中提炼出新的计算机设计思想,而目前高性能计算机设计的主要问题仍然是计算机体系结构新技术;在体系结构的设计和开发中,必须针对用户需求的特点和要求,同时考虑软件和硬件的具体配置,平衡软硬件的复杂性;在软件的开发中,算法(特别是并行算法)的设计应该与体系结构相分离,而程序的设计应该与具体机器无关;在并行计算机系统和网络计算机系统中开展应用的关键是首先开发各种并行算法和分布式算法;软件研究的重点是软件开发方法学;在软件开发方法学的研究中,软件开发各阶段的研究和不同用途、不同性质的软件开发需要各种计算模型和数学理论的支持,这正是近年来各种计算模型如分布式代数系统、类型理论、区段演算等出现的背景;大规模工业化软件生产必须要有先进的软件开发环境和产业规范的支持,软件开发环境的核心是软件开发工具、高素质的软件开发工程师,以及先进的软件工程思想、方法、技术和规范;所有想要参与最高层次软件产业竞争的软件开发商,应特别重视形式化软件开发技术的发展和能够从事形式化软件开发的员工的储备;计算机在各行各业的具体应用应该由计算科学专业人员与具体学科的专业人员合作开展工作,两者之间既有分工,又有合作,才能更好地将先进的计算机技术引入到应用中去;人工智能并不遥远,但也不像原先想像的那么简单。人工智能的理论基础应该是逻辑,但传统的经典数理逻辑不能有效地解决许多问题,必须发展新型的逻辑系统,目前主要是各种非经典逻辑系统。而且,人工智能的研究还必须注意其他相关学科的发展所取得的进展,从中吸取新思想;特别需要重视的是有必要研究什么是智能计算,提出智能计算模型。人工智能所以一再受挫,问题在于我们对智能的认识还十分肤浅,对什么是人工智能的理论基础仍处在探索之中;演化计算是一个非常值得重视和有发展前途的研究方向,演化计算技术未来有可能成为智能计算重要的支撑技术之一。

在计算科学的研究与发展中,虽然具体地形成了一大批更为细小的方向,有些理论性较强,有些技术性较强,有些则与其他学科产生了密切的联系。但就核心的,在学术上有较深刻内涵,其概念、方法和技术对整个计算科学影响较大的主流方向而言,我们认为,在未来的20年或更长一点的时间里国内外重要的计算科学学术研究机构将会逐步把研究重点集中在以下几个新的综合方向上:

(1) 新一代计算机体系结构。该方向包括神经元计算, 计算机设计与制造, 网络与通信技术(含信息安全技术), 大容量存储设备的研究, 容错理论, 算法理论, 计算模型内容等;

(2) 并行与分布式软件开发方法学研究。该方向包括数理逻辑, 计算理论(包括算法理论), 形式语义学, 高级语言与程序设计理论(包括程序设计方法学), 系统软件设计, 软件工程, 容错理论等内容;

(3) 人工智能理论及其应用。该方向包括数理逻辑, 高等逻辑, 算法理论, 知识工程, 神经元计算, 人工智能高级语言与人工智能程序设计等内容;

(4) 计算机应用的关键技术。将主要围绕计算可视化与虚拟现实, 演化计算, 计算几何, 科学计算这几个重点方向开展工作, 并带动数据库技术, 计算机图形学, 自然语言处理与机器翻译, 模式识别与图像处理等方向的发展。在这一综合方向上研究内容将几乎覆盖所有计算科学应用技术方向的内容。

无论计算科学在发展中产生了多少新的研究与发展方向, 我们应该清醒地认识到, 推动计算科学发展的主要动力在于社会广泛的应用需求。发展计算科学各个分支学科方向主要的目的有两个: 一个是针对各种实际问题应用背景的特点, 希望能够不断地制造出性能更好的计算机系统, 包括硬件和软件。为此, 必须首先发展软硬件开发的思想、理论、方法、技术和工具; 另一个是针对计算机在各行各业中的各类具体应用, 为了在计算机系统上更有效地进行科学计算和事务处理, 也必须首先发展支持计算机具体应用的各种共性思想、共性理论、共性方法和共性技术。我们还应该清醒地认识到, 无论是哪一方面, 发展一个学科的思想、理论、方法和技术必定有赖于支持该学科发展的公共知识体系。

对这四个综合方向研究的重点内容作分析, 结合国外顶尖学术刊物发表的与上述方向研究内容有关的学术论文, 不难得出, 它们共同的基础都趋向集中在数理逻辑(指高等逻辑), 形式语义学, 新一代计算机体系结构, 算法设计与分析, 计算模型与计算理论这五个专业基础之上。具体地说, 软件工程的主要基础是程序设计语言理论(主要涉及形式语言与自动机理论和形式语义学), 形式语义学, 高等逻辑(主要涉及一阶逻辑、模型论和非经典逻辑, 如时序逻辑、非单调推理等)和程序设计方法学与程序理论; 人工智能的主要基础是数理逻辑与高等逻辑, 算法设计与分析; 计算语言学的主要基础是数理逻辑与形式语义学, 形式语言与自动机理论; 而计算机体系结构的主要基础是计算模型与计算理论, 算法设计与分析, 以及形式语义学; 其他方向在学术上有深度的内容从方法论的角度考察, 基本上可以在这些内容的基础上发展。限于大学一、二年级读者的基础, 我们不可能在本书中对这一观点作全面的论证, 而且它所涉及的专业内容不易为一般读者所理解。

研究工作重点的转移实际上揭示了学科的发展和重大突破离不开学科核心知识组织结构中各分支学科的发展和支持, 它们是现代计算科学人员所必需的专业基础知识。从知识组织的层次结构和方法论的角度观察, 据对计算科学国际领头和重点刊物的分析, 当前学科的重点研究几乎都与计算理论 (包括算法理论)、体系结构、高等逻辑和形式语义学的内容密切相关。从目前的学科的整体发展情况来看, 计算模型与体系结构、软件开发方法学与计算机应用技术是学科未来发展的主要方向, 而计算理论 (包括算法理论)、体系结构、高等逻辑与形式语义学是支撑学科未来主要方向发展的四大核心专业知识基础。

如同 20 世纪 50 年代数学研究所需的核心专业基础知识从老三基 (高等代数、数学分析 (含几何、分析和函数论) 与微分方程) 向新三基 (抽象代数、泛函分析与拓扑学) 过渡转移发生变化一样, 计算科学未来发展所需要的核心专业基础也正在发生转移。这是一个不以人的意志转移的客观规律, 是学科发展的大趋势, 应该引起计算科学教育界和师生们严肃认真的对待。

3.12 计算机产业发展前景

计算机产业作为工业革命的产物, 在 20 世纪的出现已经极大地改变了整个世界的面貌, 深刻影响并仍将继续影响世界各国政治、经济、军事、文化、环境格局, 人类的生存前景和生活质量。由于计算机产业和由其引发的更为广泛的信息产业能够对提高一个国家的综合国力产生巨大的影响, 因此, 发达国家中的一些国家经过若干年的努力, 在传统产业淘汰、升级、结构调整的过程中, 计算机和信息产业在国家经济总量中已经占到相当大的比重, 并给本国带来了巨大的好处。随着资本的不断积累和经济的高速增长, 有力地促进了高科技的研究和开发, 使西方发达国家在许多领域处于垄断和领先的地位。在世界计算机产业的发展中, 无污染、投入/产出率高的软件产业尤其引人注目。

面对这样一种态势, 如何加快民族计算机产业特别是软件产业发展的步伐, 推进其他产业的进步、升级, 推进产业结构的调整, 促进社会的全面进步, 就成为摆在各国政府面前的一个紧迫的任务。中国政府在目前的竞争格局中, 针对新型计算机系统的研制, 采取了集中优势力量, 重点投向, 定点研制的策略, 先后在高性能计算机系统、高性能网络服务器的研制方面取得了重要进展。我国已成为世界上少数几个能够独立研制高性能计算机系统的国家, 而微型计算机系统的生产已经成为大国。但是, 我们还应该看到, 目前国内研制的高性能计算机系统的超大规模集成电路器件主要来自西方发达国家, 也就是核心部件如高速处理器、大容量内存存储器等集成电路主要依靠进口, 整个产业依然处于受制于人的状态。

为此，国家正在进一步加大投融资与科技研发的力度，大力发展自己的超大规模集成电路芯片制造业，为今后的竞争创造条件。我们高兴地看到，在国家的大力扶持下，我国在计算机集成电路芯片的设计与制造方面已经取得了重大突破，可以预见，随着芯片生产国产化水平的提高，我国计算机软硬件的制造和生产必将进入一个快速发展的新周期，前景非常光明。

下面我们重点讨论软件产业在中国的发展前景。

由于以美、英、法、德等为代表的西语系的国家是最先研制计算机并发展计算机产业的国家，一些人从中国与这些国家在软件产业方面存在的较大差距出发分析中国软件产业所以落后的原因，认为硬件的落后，汉语与西语系语言不同系，协作精神和团队意识差，资金投入不足等是制约中国软件产业发展的主要因素。

固然上述因素对软件产业的发展有影响，但作者认为并不是主要的原因。中国软件产业所以没有能够得到迅速的发展，形成强大的产业，应该说既有历史原因，也有现实原因。历史原因是针对 20 世纪 80 年代中期之前而言的。在 80 年代中期之前，由于软件开发主要集中在系统软件方面，与硬件的联系比较紧密，而且当时我国计算机人才特别是软件人才奇缺，软件开发工程师中大都没有经过系统的专业学习和培训，加上国家投入的资金很少，主要集中在中国科学院、重点高等学校等学术研究机构，投入的方式方法和管理运作机制也与软件产业发展的规律不协调，所以，投资收益不高。现实的原因主要是针对 80 年代中期之后而言的。80 年代中期以后，由于微机在国内外的普及，计算机应用技术的发展，软件开发呈现出多元化的格局，系统软件在全部软件市场份额中所占的比重日益降低，各种应用软件和游戏软件所占的比重不断上升。其时，国内计算机软件和应用人才的培养开始大规模展开，计算机设备不断增加，微机开始进入各行各业甚至家庭，国有资本和民间资本的投入也逐年增加，资金和硬件设备都已不再成为影响软件产业发展的主要因素。然而，为什么我国软件产业仍然没有出现高速发展的情况呢？这确实值得我们深思。作者认为，关键是我们对软件产业发展的内在规律和特点的认识不清楚、不准确，导致我们在政策的制定和运作管理上缺乏正确的指导思想。

众所周知，软件的开发首先是一项高智力的活动，软件产业的发展既有生产成本低，产品高附加值，高收益的特点，也有产品寿命短，升级换代快，市场变化快，投资风险大的特点。这些行业特点决定了要参与软件产业的市场竞争，开发商必须特别重视两个方面的问题。第一个问题是企业要有专业技术素质高的软件工程师。所谓高素质的软件开发工程师主要是指具有坚实的专业基础和科学素养，具有较强的应用基础研究能力和技术开发能力，能够通过高水平学术刊物与学术会议交流跟踪世界最先进的软件理论、开发方法和开发技术，善于与人合作

开展工作。第二个问题是企业要有良好的投资管理机制。这种投资机制主要包括三方面的内容：

（1）根据自身的经济条件，有能力选择大小适宜，前景良好的投资项目迅速组织实施。在软件开发中，争取时间往往意味着争取市场的主动权；

（2）能够通过分散投资风险，包括通过调整利益分配政策将开发者的利益和风险捆绑在一起，以留住核心技术人才，同时降低主要开发商的投资风险；

（3）要重视市场的开发和用户信息的收集、整理和分析，为软件升级提供重要的用户信息。

总结过去我们在发展软件产业方面的经验和教训，对今后更好地发展软件产业是十分有益的。我们过去的主要问题是按照软件产业发展的规律行事，过多地依赖科研机构特别是以（应用）基础研究为主的学术机构开发软件，没有注意依托企业进行软件开发，并在发展中培养大批软件产业所需要的各类人才，有目的地培育企业的科技研发队伍形成科技研发中心，成为技术创新和产品创新的市场主体。例如，从20世纪70年代中后期起很长一段时间内我们是通过联合攻关的方式方法开发系统软件和少数应用软件，由于选题太大，项目从选题、酝酿、申报、审批、立项到组织实施，往往花费了大量的时间、精力和资金。等到项目开始实施时，因为参加攻关单位有若干个，按照分工，每个单位首先要统一对整个系统结构的认识，以后各自在开发中还需要经常相互讨论各部分之间的接口。这样，虽然整个系统最初提出的技术路线，依据的理论方法和采用的技术也许比较先进，可是，等到立项、投入开发到研制开发成功、推广，产品早已落在国外的后面。再加上承担软件项目开发的单位并不是项目的投资人，而是主要从事基础和基础研究的学术研究机构 and 高等学校，开发单位和许多技术人员更关心的是他们在研究与开发中是否具有创新和高水平的论文发表，成果是否能够获得奖励，而在产品的开发中常常忽视了市场的需要和行业的技术规范，这就决定了产品最终难于参与市场竞争。这说明我们对软件的管理运作机制缺乏了解，把本该属于企业的产品开发任务委托给了以（应用）基础研究为己任的学术研究机构。

又如，从20世纪80年代中后期起，随着国内一批民营企业的崛起和一大批计算机专业博士、硕士、学士进入计算机公司，许多公司开始把软件的开发作为企业今后发展的重点，希望产生新的利润增长点，给公司带来丰厚的投资回报。然而，尽管软件开发可选的项目很多，但是，国内大多数公司的软件开发项目主要集中在信息系统和生产自动化方面。事实上，信息系统作为应用软件的一个子类，技术上已经比较成熟，技术含量也不高，自然其收益也十分有限。而生产自动化的开发研究，往往是对现有设备的改造，研究与开发工作的关键和工作量并

不取决于软件而是硬件。若设备本身不够先进，即使开发成功，软件产品也没有市场；若设备比较先进，开发工作一般也属于二次开发，由于缺乏资料和源代码，开发周期很长。可是，面对大量可以参与竞争的项目，例如系统软件，（分布式）应用软件（如电子商务、（分布式）决策支持系统）等，许多方面我们仍然拿不出能参与国际竞争的产品。

还有一个问题，由于从 20 世纪 80 年代起，国内高等学校在计算机专业的人才培养中普遍忽视了数学基础和计算科学理论基础的严格训练，比较注重一般操作技术的训练，人才培养偏向普通工科教育，加上许多优秀毕业生出国未归和进入用非所学的非计算机产业部门，结果导致大多数毕业生在这一理论和实践并重的学科内，因难于看懂国内外高水平计算机科学刊物上发表的大量新思想、新概念、新方法、新技术而无法参与这个学科的创新竞争。这是我国计算机软件产业不能迅速崛起的深层原因。

软件产业的发展特点决定了专业技术人才的数量并不是决定一个软件企业成功与失败的关键因素。决定一个软件企业兴衰的关键是企业获得优秀人才后，能否选择合适的开发项目，迅速地拿出技术上相对先进的产品占领市场，然后在获得资本收益的同时，不断加大产品维护、升级换代和新项目、新产品开发的投入，实行滚动发展，保持技术领先的地位和市场主导的作用。中小企业，尤其是资本不足的企业，千万不能一开始就投资大型软件产品开发。从国内外大量成功的经验可以看出，独一无二的、有市场竞争力的产品是个人或中小软件企业快速发展形成产业的关键。例如，盖茨首先开发了视窗操作系统形成了微软公司，杨志远因建立受欢迎的网络站点而创建了雅虎公司，刘旭开发的防病毒卡成就了瑞星公司，还有像王选等人发起成立的方正电脑（初期主营激光照相排版系统）、王江民的江民公司（KV-300）、王永民的汉王公司（五笔字型汉卡）、以及国外的 Oracle 公司（Oracle 数据库）、网景公司（网络浏览器），等等，无一不是从一个王牌产品起家的，尽管有的软件最初还是很不起眼的产品。

显然，问题不在于是否有足够的资金，而在于对产业规律和机会的把握。当然，我们还要注意这样一个事实，随着产品开发采用的技术日渐深化，今后这样的机会对一个普通的软硬件开发工程师和计算机爱好者会越来越少。更多的机会将会留给那些具有坚实的理论基础，掌握先进的开发技术和行业规范的工程师。

一个公司开发的软件产品，能否取得可观的经济效益，不仅取决于市场份额，还取决于产品的技术含量。例如，电视台现在用来制作广告的三维动画制作软件系统是一种价格昂贵的软件产品，因为其广泛使用计算几何的知识，技术含量较高。尽管该产品销售量不如系统软件，但同样可以获得巨额回报。又如，同样是网络上的数据交换设备 Hub，不同公司的产品速度不一样，价格也相差很大。为

什么？因为网络路由协议的实现技术不一样。

对学科发展和产业发展的相互关系作历史分析，我们不难看出，在软件业，大致十年前正在进行的（应用）基础研究，十年左右将推出新的产品，而且，这样一个从基础研究到产业开发的周期有缩短的趋势。如果我们不能正视这样一个事实，有针对性地在人才培养、（应用）基础研究和产品开发的创新研究中迎头赶上去，我们将始终处于跟踪国外先进技术的被动状况。

现在，越来越多的人已经注意到了这样一种现状，开始着手进行改革。首先，一些学校在人才的培养中加大了基础课程和专业基础课程的改革力度，转变人才培养观念，改革旧的教学模式和教学手段。其次，产业投资主体发生了明显的变化，国家在软件产业方面的投资从过去主要投向科研机构 and 高等学校，开始逐步转到改善投资环境，扶持重点企业和加快建立风险投资机制上来。这是一种正确的决策，相信随着国家软件产业政策的调整，随着对高等学校投入的增加，实验室条件的改善，高等学校教学改革的深入和一批高质量人才的毕业，随着重点企业软件自主研究与开发能力的增强，我国软件产业一定会在不远的将来赶上世界先进水平。

在结束这一章的内容时，我们要提到计算科学的意义。或许有的读者已经发现，本章讨论计算科学：它的内容、意义和方法时，我们并没有专门论述计算科学的意义。这不是作者的疏漏，而是有意将其留给读者作为一个思考问题。我们希望，读者在完成本书的学习后，经过自己的思考和分析，能比较完整地总结出计算科学的意义。

第四章 如何学习计算科学和健康成长

本章我们将围绕计算科学专业培养规格和目标，计算科学教学计划与课程体系，各学期课程的重点，以及如何学好计算科学专业，加强多方面训练，顺利完成学业，使自己逐步成长为一个高素质的专业人才展开讨论。本章的重点在于向读者传达一种科学的思想方法和精神，倡导理解科学，帮助读者塑造人文精神与科学精神。

4.1 计算科学（专业）的培养规格和目标

高等学校计算科学专业本科以上教育主要是为计算机产业，重要部门的计算机应用，中、高等学校教学和研究院所的科研工作培养人才。毕业生的主要流向应该是计算机公司，产品技术含量较高的工业企业，各行各业计算中心，中等以上学校和科研院所。但是，市场经济的发展规律并不能保证毕业生按照预设的目标流向和分布。一方面，由于社会不同行业、不同层次的单位对计算科学人才的需求和市场经济规律的作用，由市场配置人力资源必然导致相当大批毕业生将进入以一般性计算机具体应用为主的岗位，只有少数毕业生会继续深造，或从事科学技术研究、开发和高等学校教学工作；另一方面，由于计算科学发展异常迅速，研究与开发内容的不断深化导致学科的知识组织结构变化很快，特别是学科知识组织结构中构造性数学基础和计算科学理论知识日渐积淀，并正在对学科研究与开发产生重要而又深刻的影响。因此，在高等学校，本科计算科学（专业）教育常存在两种基本的教学计划和办学模式，以适应社会对计算科学人才的不同要求。

首先需要说明的是，本书中关于计算科学博士、硕士研究生教育，本科生教育的培养规格和目标是以我们关于面向 21 世纪教学内容与课程体系改革项目¹⁾研究工作的国家获奖成果为参照的，项目研究成果在学术鉴定中获得了鉴定专家组的充分肯定。由于高等教育法规定了高等学校实行自主办学，教育部今后将不再制定国家统一的学科专业教学计划与课程体系，因此，各高等学校今后必须自己制定学科专业教学计划与课程体系。应该看到，国家教育部以前颁布的计算科

1) 教育部计算机科学类专业面向 21 世纪教学计划与课程体系改革研究（编号：13-22）。

学硕士研究生教育、本科生教育的培养规格和目标只是一种最基本的要求,作为国内具有较高起点的高等学校的计算机科学系,显然不能满足于这样一种要求。我们相信,高起点和高标准的要求不仅能够为读者提供参考,而且对读者今后从事计算科学事业,参与国际竞争是有益的。

高等学校计算科学专业研究生教育培养适应计算科学学科发展,国家社会发展与进步事业实际需要,德、智、体、美全面发展,具有良好的科学素养和文化修养,系统地、较好地掌握本学科公共基础知识,较好地掌握本学科基本概念、基本原理、基本方法、基本技术等基础理论知识,理论联系实际,受到良好的计算科学基本实验技术与技能等实践能力的基本训练,受到科学研究与实际应用训练的专门人才。除了对各级毕业生在德育、体育、外语、文化方面的要求外,对专业业务要求应有如下具体培养规格和目标。

(1) 计算科学硕士研究生教育的培养规格和目标。

① 为未来从事计算科学学科教学、研究、应用与开发提供一个深入开展工作的坚实的理论、方法和技术基础。

② 毕业生应了解整个学科当前的发展现状和未来的发展趋势,了解学科发展的一般规律,掌握学科深入发展所需的研究生一级的核心基础知识和某一专业化方向的基本原理、基本技术和基本方法。

③ 具有在较高的起点上,即能够在阅读和正确理解相当于国际重要学术刊物,包括国内《计算机学报》、《软件学报》、《计算机研究与发展》等同档刊物(以国务院学位委员会评估文件为准)一个方向上若干学术论文和技术报告的起点上,独立开展学术研究或专业技术工作的能力。

④ 具有对一些计算科学技术项目所提出的思想、方法、技术和工程技术路线的能行性作出准确估计的能力。

⑤ 理论联系实际,具有运用所学专业知识和分析、解决中低等难度专业技术问题的能力。

(2) 计算科学博士、研究生教育的培养规格和目标。

博士研究生毕业后,除对一般毕业硕士研究生的要求外,还应达到如下培养规格和目标:

① 在计算科学学科各方向的重要的基本概念、基本原理、基本技术,特别是典型方法、典型实例和学科形态方面,应具有本学科比较广博的专业基础知识,进一步掌握学科深入发展所需的核心基础知识和自己所从事的专业化方向的基本原理、基本方法和基本技术;

② 具有在较高的起点上,即能够在阅读和正确理解相当于国际一流学术刊物一个方向上若干学术论文和技术报告的起点上,独立开展有创造性的学术研究或

专业技术工作的能力,或主持有学术深度的专业技术工作的能力。

计算科学本科专业培养适应计算科学学科发展,国家社会发展与进步事业实际需要,德、智、体、美全面发展,具有良好的科学素养和文化修养,系统地、较好地掌握理工科公共基础知识,较好地掌握本学科基本概念、基本原理、基本方法、基本技术等基础(理论)知识,理论联系实际,受到良好的计算科学基本实验技术与技能等实践能力的基本训练,受到科学研究与实际应用初步训练的计算科学专门人才。

毕业生适宜到科研部门和高、中等学校从事科学研究和教学工作,适宜到计算机产业、重要部门,以及相近学科的有关单位从事计算科学开发研究、应用与管理等工作,可以继续攻读计算科学及其相关学科的硕士学位。

其中,计算科学专业第一类(A类)教学计划是以培养计算科学学术人才和技术人才,着眼于学生未来成为主任工程师、总工程师、教授、学者和高级专业管理部门主管官员为具体目标;第二类(B类)教学计划是以培养专业技术熟练的软硬件生产、开发、经营、维护工程师,以及专业技术管理工程师为具体目标。两种教育模式各有侧重和特点。在教学内容方面,A类教育重在基础理论、基本技术和未来深造,或从事科学研究和专业技术研究与开发能力的培养;B类教育重在基本理论、基本技术、应用技术和实际应用与开发能力的培养。

(1) 计算科学本科生A类教育模式的培养规格和目标。

- 思想政治和德育方面(略,请参看国家统一规定)。
- 体育方面(略,请参看国家统一规定)。
- 业务方面:

① 系统地、较好地掌握理工科公共基础知识,较好地掌握本学科基本概念、基本原理、基本方法、基本技术等基础(理论)知识,理论联系实际,受到良好的科学思维和科学实验的基本训练。

② 毕业生应初步了解整个学科的知识组织结构、学科形态、典型方法、核心概念和学科基本工作流程方式,初步了解学科当前的发展现状和未来的发展趋势,掌握计算科学本科生一级的核心基础知识和某一专业化方向的专业基础知识,为未来在计算科学领域从事一般教学、研究、应用与开发或深造提供一个开展工作的坚实的专业知识基础。

③ 毕业生应初步具有进一步深入掌握学科核心基础知识和某一专业化方向专业基础知识的能力,初步具有对一般计算科学技术项目所提出的思想、方法、技术和工程技术路线的能行性作出基本估计的能力。

④ 能通过文献检索和其他方式阅读中外文书刊,获取专业科技信息;理论联系实际,具有运用所学专业基础知识分析、解决简单的专业技术问题的能力。

- 文化方面:

① 对中国传统文化有一个基本的普及性的了解,对中外文化的某一方面有一定的基础。

② 通过积极参与学校的文化建设,在文艺修养、审美情趣、言谈举止、礼仪风貌等方面达到一定的水平。

(2) 计算科学本科生 B 类教育模式的培养规格和目标。

- 思想政治和德育方面 (略, 请参看国家统一规定)。

- 体育方面 (略, 请参看国家统一规定)。

- 业务方面:

① 为未来从事计算科学学科一般应用、开发、维护、技术服务和技术管理提供一个开展工作的比较坚实的理论、方法和技术基础,为未来在本学科掌握流行新方法和新技术提供一个计算科学核心的专业知识基础。

② 毕业生应初步了解整个学科的知识组织结构、学科形态、典型方法、核心概念和学科基本工作流程方式,初步了解当前的发展现状和未来的发展趋势,掌握计算科学本科一级主要的核心基础知识的基本概念、基本原理、基本技术和基本方法。

③ 熟悉某一种或若干种流行的计算机系统 (包括硬件、软件工具和环境),在操作使用计算机进行数据处理,维护、开发和管理方面具有比较熟练地开展一般性专业技术工作的能力;具有借助专利资料和各种渠道获得的软硬件产品的技术资料,掌握新产品、新技术的操作与使用的能力。

④ 理论联系实际,具有运用所学专业知识和分析、解决简单的专业技术问题的能力。

- 文化方面:

① 对中国传统文化有一个基本的普及性的了解,对中外文化的某一方面有一定的基础。

② 通过积极参与学校的文化建设,在文艺修养、审美情趣、言谈举止、礼仪风貌等方面达到一定的水平。

特别,在一些综合性大学,采用 B 类人才培养模式,教学管理实行学分制,允许学生在选修课程中按照配套课程 (“课程套餐”) 进行选修,利用学校多学科优势,有目的、有针对性地培养适应某一学科专业专门化方向的信息处理专业人才,完全可以在 B 类模式的基础上进行变形,通过设立信息处理专业,支持跨学科人才的培养。今后,学校可以根据毕业生所选修、通过的 “套餐课程” 名录,发放信息处理专业 (某一专门化方向,如机电一体化、国际金融与商务信息管理 etc) 文凭。

分析上面的论述,实际上,我们在计算科学人才的培养目标中已经贯彻了一种理工科高等教育的目标:

使用科学原则去了解科学过程,能够建立和体会科学美和艺术美之间的内在联系,感知和体验因了解自然世界和未知领域而带来的满足感和激动;

提高学生从事科学研究和经济生产的能力,在智力方面有能力参加涉及科学技术问题的社会讨论。

同时,我们又不局限于将计算科学专业作为一种泛科学来对待,而是紧密与学科最重要的公共科学基础知识,专业的基本概念、基本理论、基本技术与基本技能联系起来,让每一个学生在大学阶段打下良好的基础。

4.2 一个计算科学(专业)参考教学计划与课程体系

限于篇幅,本节仅列出计划的核心内容:课程设置与进度表。许多课程内容读者将在今后的课程中学习。课程设置及进度表请参见表 4.1(A 类),表 4.3(B 类),表 4.2 和表 4.4 分别为 A、B 类的选修课程表。

4.2.1 计算科学专业本科生 A 类教学计划与课程设置进度表

由表 4.1 可以看出,在教学计划中我们设置了一门单列的“计算科学实验”课程,其内容分属于各门课程。实验课以单元为单位划分各部分相对独立的内容,各单元分配的学时数视具体内容而定。实验的具体内容包括操作、硬件和计算机软件与应用,属于必须掌握的操作技能和核心实验。第一学期的 54 学时,主要用于指法训练,微机操作系统的基本命令及操作,简单工具软件的使用,上网操作等。第二学期除继续第一学期的有关练习外,开始输入 1—2 个单元的 Pascal 语言程序,上机调试验证。第三学期继续程序设计的实验。从第四个学期起到第七个学期止,实验内容依各门课程的进度交叉进行,大体上要完成与下述内容有关的一批实验:

- 几个有典型代表性的电路。
- 几个典型的数值与非数值问题。
- “编译原理”、“操作系统”、“数据库原理”、“计算机组成原理”以及若干选修课中的基本实验,如语言语法分析,代码生成, I/O 驱动程序代码分析与设计,文件的管理,加法运算器的设计,中断系统的设计等。其中,有些实验要求学生采用指定的语言,有些实验要求使用规定的数据结构与程序技术。

考虑到大多数中学生初入大学时,在学习方法上一时不适应大学的方式,第

续表

课程 编号	必修课程名称	考核 方式	学时	各学期学分（周学时）分布							
25	可计算性与计算复杂性	查	72							4	
26	形式语言与自动机理论	试	72							4	
27	计算科学实验（≥450 学时）	查	500	3	3	3	3	6	6	9	
军事训练（入学教育）（周学时数）			5 周	22	25	27	27	25	26	25	
毕业论文（科研训练）											18
总计			2472	500							

表 4.2 计算科学专业（A 类）选修课程一览表

1. 计算机组织与系统结构	(C (公共), (6, 8))
2. 新一代计算机体系结构	(A (体系结构), S, (7))
3. 微机原理及其应用	(C, A, Ap, S, (6, 7))
4. 计算机图形学	(Ap (应用), S, (7))
5. 计算几何	(Ap, Th (理论), (8))
6. 并发程序设计与分布式计算	(S, Th, (7))
7. 模式识别与图像处理	Ap, (7))
8. 容错理论及其应用	(A, Th, (8))
9. 信息系统分析与设计	(Ap, S, (7))
10. 计算机网络与通信	(Sp, S, (7))
11. 软件工程	(A, S, Th, (7))
12. 软件开发工具与环境	(S, Ap, (7))
13. 并行计算与并行算法	(C, (8))
14. 数据库系统实现技术	(S, (8))
15. 模型论	(S, Th, (7))
16. 程序设计方法学	(Ap, S, (7))
17. 形式语义学引论	(S, Th, (8))
18. 定理证明与程序理论	(S, Th, (8))
19. Petri 网理论及其应用	(S, Th, (7), (8))
20. 实时系统的分析与设计	(Ap, S, (8))
21. 高级程序设计技术	(Ap, S, (7))
22. 程序设计语言概论	(A, S, Th, (7))

续表

23. 人工智能基础	(A, Ap, S, Th, C, (7))
24. 逻辑程序设计基础	(A, S, Th, (8))
25. 计算机密码学	(S, Th, (8))
26. 计算机辅助设计与制造	(A, S, (8))
27. 函数式程序设计基础	(A, S, Th, (8))
28. 计算机自动控制基础	(Ap, (7))

表中, A, AP, S, Th 分别表示计算机体系结构, 计算机应用技术, 计算机软件, 计算机理论专门化方面选修课程的标识, C 为四个专门化方面公共选修课的标识。方括号中的数字表示建议开课学期。

A 类教学计划实行的是非学分制, 学生在完成计划中所有课程的学习之后, 只要再修完 2—3 门选修课程, 完成毕业论文或设计就可以毕业了。要做到这一点并不困难, 计划表中已留出足够的时间和空间。需要指出的是, 如果“计算科学实验”课程实行单列后在培养学生实践能力方面比较成功, 三个课程设计(硬件、软件、应用各一个)质量有保证, 那么, 学生就未必一定要做毕业论文。这样一种观点若得到认真对待, 就可以在第八学期安排学生再学习 2—3 门课程(含讲座性课程, 以 20—54 学时为宜, 考核方式为考查), 同时让学生独立自主地自选一个研究课题, 完成一篇读书报告或设计(不必要求很高), 教师只起少量的指导作用。尽管最后的 2—3 门课程在数量上不多, 但对学生顺利适应工作岗位有积极意义, 有利于学生在较好的基础上拓宽专业知识。学校对这两门课程不作硬性规定, 每一个学生都可以根据自己的情况选学。

我们建议采用 A 类教学计划的学校实行主辅修制, 主修计算科学, 辅修哲学、艺术、文学等其他课程。主辅修制完全不同于国内一些学校实行的“一本一专”制, 双学位制。学生主修的是一个学科专业, 辅修是根据个人的兴趣和爱好, 学习一些科目的专业知识, 促进个人修养的提高和全面发展。实行主辅修制学校并不发放辅修专业或辅修科目的文凭, 但学生辅修的专业课程, 只要通过课程考试, 都应该记录存档, 并入辅修科目成绩单, 并在学生毕业前的个人总结和学校评语中得到体现。

执行 A 类教学计划的学校, 学生在选修课程时, 考虑到学生中多数将走向社会各行各业, 建议优先考虑在“计算机组织与系统结构”、“微机原理及其应用”、“计算机网络与通信”、“计算机图形学”、“软件工程”、“信息系统分析与设计”等课程中进行选择。

为什么在 A 类教学计划中没有将“计算机组织与系统结构”、“微机原理及其应用”、“计算机网络与通信”课程规定为必修课呢? 这主要是因为大多数学校根

本不具备开设“计算机组织与系统结构”课程的条件。我国高等学校很少有教师能够熟练地运用 VHDL 语言来设计计算机系统,对各种先进的计算机系统的软硬件系统设计思想、结构并不熟悉,在这种情况下,开设这类课程常流于照本宣科的形式。即使学校有能够胜任上课的教师,一般也不具备做实验的条件。而“计算机组织与系统结构”没有实验支持,仅仅停留在书本的讲解上,效果不是很好。当然,高年级准备选修“计算机组织与系统结构”课程的学生,必须在此之前选修有关的前驱课程。

“微机原理及其应用”是一门实用性比较强的课程,属于专业课程。由于其在整个学科知识组织结构中的地位不是很重要,因此,没有必要列为必修课程,而可以选修课程的形式出现,让学生结合自己今后的发展目标选修。

“计算机网络与通信”虽然是近年来十分热门的方向,有关的课程已在很多学校开设,但是,到目前为止,关于网络与通信方面的知识,仍然是比较简单的,基本上仅适合于网络配置设计、网络的一般使用和安装等需要。其中,像网络互联的软硬件技术已经相当成熟而且产品已标准化和系列化。但是,与网络软件开发有关的软硬件知识,尚未涉及到网络中比较深层的结构、原理和开发方法学的内容,如网络协议的描述与实现等,还不能支撑其他方向的发展,学术深度一般,作为 A、B 类模式培养的人才,完全有能力自学掌握。所以,目前作为选修课程是合适的。至于网络深层次的应用开发,不仅涉及到分布式基础算法,而且涉及到网络软件的体系结构和开发工具,不是本科生课程所能覆盖的。曾经有专家在没有弄清计算科学专业与通信专业如何分工,以及学生相关先导数学课程还没有学习的情况下,盲目倡导开设通信原理类课程,结果导致教学工作陷于被动,反而弄巧成拙。

也有的读者可能会提出问题,为什么“电子线路基础”、“模拟电子技术”、“集成电路原理与设计”、“计算机设计”、“通信原理及其应用”等与硬件直接相关的课程内容没有列入教学计划中的必修课?这主要是由于今日在专业的分工方面,计算科学专业需要与电子技术专业、半导体专业、自动化专业、通信专业之间有一个比较明确的分工,许多情况下不能越俎代庖。至于将来跨学科研究工作的需要,学生完全可以通过选修相应的课程来为自己今后的发展作准备。需要指出的是:“计算机设计”与“计算机组织与系统结构”之间有相当一部分的内容是重叠的。计算机设计主要运用 VHDL 语言来设计和描述计算机的各个主要硬件部件,配合相应的实验可以完成整个计算机硬件系统的设计与实现。“计算机组织与系统结构”比“计算机设计”包含更多的内容,除了硬件还涉及到一些系统软件的内容。

仔细查看计算科学专业(A类)教学计划表的读者就会发现,我们在教学计

划中设置了两门互换课程,即“概率统计”与“电子线路基础”可以互换,“数据库原理”和“计算机设计”可以互换。按照现在的设计,少数师资和培养目标偏硬件的学校,完全可以通过让学生在互换课程中和三年级(下)以后的选修课程中选修硬件类课程或偏硬件的课程。由于A类模式中几乎所有课程均为72学时以上,不包括实验课时,而且,在物理学课程中已将一部分电学知识作为该课程的重点内容之一,加上B类模式实行真正的学分制,学生选课有很大的自由选择空间,两类模式均能支持各种计算科学人才的培养。例如,一个有志于计算机硬件设计制造的学生,可以根据A类模式学习“物理学”、“电子线路基础”、“数字逻辑”、“计算机组成原理”、“汇编语言程序设计”、“计算机设计”、“计算机组织与系统结构”、“微机原理及其应用”、“计算机网络与通讯”等课程。如果学校允许不做毕业设计,甚至还可以再选修“高等计算机体系结构”课程。根据我们已经提出的《高等理科计算机科学与技术专业一级学科培养方案研究报告》中各门课程的教学内容以及实验内容纲要,无论A类还是B类教学计划都不存在严重偏理论,偏硬件还是偏软件的问题,也不存在不重视实验的问题。按照上面的设计,A类模式必修课程的学时数为2472,实验课程的学时数为460学时左右,在此基础上,学生可以选修108—162学时的选修课程(保留毕业设计)或270学时的选修课程(不做毕业设计)。这样,八个学期包括实验课程在内大致上每周上课学时数为20—26学时,这对一般的同学压力是比较适度的。

需要特别提出的是A类模式的课程总学时数虽然不低,但课程设置与学时数设定比较规范,不会出现一个学期忽紧忽松的局面。另外,实验课程如果每周是四个学时,应该安排在一个下午进行,以提高学生版块时间的利用效率。

这里给出的仅仅是人才培养的一种参考教学模式,各校学生应在完成本校规定的学习任务的前提下,参考本模式,决定自己的学习方式和学习策略。

4.2.2 计算科学专业本科生B类教学计划与课程设置进度表

由表4.3可以看出,“信息处理与程序设计基础”课程是在“高级语言程序设计”课程的基础上增加了近20个学时信息处理的内容,主要介绍常用软件工具的使用和中文信息处理基础知识。

与A类教学计划相比,B类教学计划对数学的要求有相当程度的降低,但仍然保持比一般理工科专业稍高的水准。具体地说,“数学分析”课程减少约110个学时,军训课程不列入正常学期的学校大约减少80学时,教学的侧重点有较大改变;“高等代数”课程我们希望保持与A模式一致;“离散数学”课程一分为二,比A类计划学时数减少72个学时,但内容上仍然比大多数学校现在开设的离散数学要深入一些;“微分方程”、“概率与统计”以及“计算方法”等课程内容合并

为“工程数学”，再补充一些信号处理等方面可能要用到的复变函数论的内容，计划 144 个学时，减少 72 个学时。其教学指导思想是让学生掌握这些科目方面最基本的概念、思想、方法和技术（技巧），剔除一些比较有深度、有一定难度的内容，并使达到这样一种能力，若将来有必要，可以通过自学适当更新这方面的一些知识，能够理解科学文献上与这方面知识有关的一些应用研究成果。

数据结构与算法设计两者内容有紧密的联系，合二为一后，学时数减少了 72 个学时，但不包括实验学时，这对 B 类专业是足够了。当然，在选修课程中不再设置“算法设计与分析（54—72 学时）”课程的学校，可以将“数据结构与算法设计”这门课程设定为 90 学时，适当增加算法设计与分析的内容。

“计算机组成原理”改为“计算机组成原理与设计”，学时数增加了近 20 个学时。在内容的讲授上要求结合计算机组成原理和典型实例介绍计算机设计的方法。这样做，有助于学生毕业后，如果从事硬件设计与应用研究，有能力在主任工程师总体设计的框架下，根据硬件部件的功能要求，设计出符合用户要求的功能部件。

表 4.3 计算科学专业（B 类）教学计划表（学制四年）

课程 编号	必修课程名称	考核 方式	学时		各学期学分（周学时）分布							
			讲 授	实 验	一年级		二年级		三年级		四年级	
					上	下	上	下	上	下	上	下
0	体育	试	144		2	2	2	2				
1	中国革命史	查	36		2							
2	中国社会主义建设	查	36			2						
3	哲学	试	72				2	2				
4	政治经济学	试	72						2	2		
5	英语	试	288		4	4	4	4				
6	数学分析(含空间解析几何)	试	216		6	6	4	4				
7	高等代数	试	144		5	4						
8	计算机科学与技术导论	查	18		1							
9	物理学(含实验)	查	144			4	4					
10	信息处理与程序设计基础	试	90				5					
11	电子线路基础	试	72					4				
12	离散数学(I):逻辑与集合论	试	72					4				
13	工程数学(I):方程与计算	查	72					4				
14	数字逻辑(不含实验)	试	72						4			
15	离散数学(II):代数与图论	试	72						4			

续表

课程 编号	必修课程名称	考核 方式	学时		各学期学分 (周学时) 分布							
16	工程数学(II): 复变函数等	查	72					4				
17	数据结构与算法设计	试	90					4				
18	计算机组成原理与设计	试	72						5			
19	系统软件原理与设计方法	试	72						4			
20	汇编语言程序设计	查	36						2			
21	以下 21—27 课程由学生在	试	72					4				
22	规定的范围内任选。如在	试	72						4			
23	表 4.4 的专业选修课表范围	试	72						4	4		
24	任选。	试	72							4	4	
...		试	72							4	3	
27		试	72							4	3	
28	计算科学实验(≥ 500 学时)	查		500	4	4	4	6	4	4	8	8
军事训练 (入学教育) (周学时数)				5 周	24	24	25	26	25	24	18	
毕业论文 (科研训练)												14
总计			1962	500								

表 4.4 计算科学专业 (B 类) 选修课程一览表

1. 计算机组织与系统结构	(A, Ap, S)
2. 微机原理及其应用	(A, Ap, S)
3. 计算机图形学	(Ap, S)
4. 计算机辅助设计	(Ap, S)
5. 计算机绘图	(Ap, S)
6. 并发程序设计与分布式计算	(S, Th)
7. 模式识别与图像处理	(Ap)
8. 故障诊断与检测	(A, A)
9. 信息系统分析与设计	(Ap, S)
10. 计算机网络与通信	(Ap, S)
11. 软件工程	(A, S)
12. 软件开发工具与环境	(S, Ap)
13. 数据库系统实现技术	(S)
14. 程序设计方法学	(Ap, S)

续表

15. 实时系统的分析与设计	(Ap, S)
16. 高级程序设计技术	(Ap, S)
17. 程序设计语言概论	(A, S, Th)
18. 人工智能基础	(A, Ap, S, Th)
19. 逻辑程序设计基础	(A, S, Th)
20. 计算机自动控制基础	(Ap)
21. 多媒体技术及其应用	(Ap)
22. 计算机辅助测试	(A, Ap)
23. 专家系统与智能软件	(Ap)
24. Prolog 语言程序设计	(A, Ap, S)
25. C++与面向对象程序设计	(Ap, S)
26. LISP 语言程序设计	(Ap, S)
27. 数据库系统及其应用	(Ap)
28. 人机界面与人机工程学	(Ap)
29. 计算机仿真技术及其应用	(Ap)
30. 计算机保密与信息安全技术	(Ap)
31. 分形理论与计算机应用	(Ap)
32. 计算机集成制造系统 (CIMS)	(Ap)

对使用 B 类教学计划的学生, 由于今后主要是从事应用开发, 即使是从事软件开发, 编制程序也是在主任工程师总体设计的框架下和组织下开展工作, 对他们的要求应该是在理解基本要求 (如软件规格说明) 后, 能够很快编制、调试出程序。因此, 对系统软件课程没有必要逐一加以仔细研修。为此, 我们将“操作系统原理”, “编译原理”和“数据库系统原理”课程合并成一门“系统软件原理与设计方法”课程, 简要介绍各种系统软件的原理和有代表性的设计方法。

对专业选修课, 教学计划重点突出的是应用类课程。在选课时, 应允许学生在教师的指导下任意选修, 有条件的学校应发挥综合性大学多学科的优势, 以“课程套餐”的形式让学生根据自己的兴趣进行选择, 以突出 B 类教学计划的优势和特点。这样做, 既能满足大多数学生向专业的各个方向发展, 也能满足部分学生向跨学科的专业方向发展。

B 类教学计划实行的是学分制。学生在完成计划中所有课程的学习并修满学分之后, 只要完成毕业论文或设计就可以毕业了。要做到这一点并不困难, 计划表中已留出足够的时间和空间。需要指出的是, 如果“计算科学实验”课程实行

单列后在培养学生实践能力方面比较成功,六至七个课程设计(硬件、软件、应用至少各两个)质量有保证,那么,学生就未必一定要做毕业论文。这样一种观点若得到认真对待,就可以在第八学期安排学生再学习2—3门课程(含讲座性课程,以20—54学时为宜,考核方式为考查),同时让学生独立自主地自选一个研究课题,完成一篇读书报告或设计(不必要求很高);教师只起少量的指导作用。尽管最后的2—3门课程在数量上不多,但对学生顺利适应工作岗位有积极意义,有利于学生在较好的基础上拓宽专业知识。学校对这两门课程不作硬性规定,每一个学生都可以根据自己的情况选学。B类模式必修课程为1962,实验课程为500学时左右,在此基础上,学生可以选修450—500学时的选修课程(保留毕业设计)或600—620学时的选修课程(不做毕业设计)。这样,八个学期包括实验课程在内大致每周上课学时数为20—26学时,这对一般的同学压力是比较适度的。

需要强调的是,同时实施计算科学专业A类、B类教学计划的学校,应允许学生选择适合自己的教学计划接受教育。

一些学校采用讲座方式开设选修课程是一个值得推荐的好方法。讲座类课程的内容一般应结合学校教师的科学研究选题方向开展,学时数以20左右为宜。讲座类课程排课应该考虑从学期第一周开始至学期约16—18周末结束,给予学生比较宽松的听讲座报告和参加学术研讨的时间跨度。拉长讲座类课程的排课时间区间有助于学生更多地阅读文献,独立思考,以科学研究的工作方式比较深入、全面地掌握课程的内容,独立完成(读书)研究报告。这样做,虽然实际的听课学时数有所减少,但通过这种认真研讨的学习、工作方式,对培养学生较强的自学能力和良好的科研素养有一定的积极意义。

开设讲座类课程是一件严肃的学术工作,讲课者需要有充分的准备。具体地说,一位教师开设诸如面向对象程序设计的讲座类课程,我们希望他按照如下的规范纲要进行授课,讲授内容和教学应该包括下列几个方面:

(1) 什么是面向对象的程序设计?介绍面向对象程序设计的科学意义,面向对象程序设计的来龙去脉,面向对象程序设计需要解决的问题,等等;

(2) 介绍面向对象程序设计的基本概念、术语、方法和技术(包括举一些实例);

(3) 介绍面向对象程序设计研究的当前进展,取得的成果,实际应用(包括举例);

(4) 介绍面向对象程序设计当前研究中存在的需要解决的问题,未来发展的分析与展望;

(5) 介绍自己在该领域所做的工作(可以缺省);

(6) 开列一批参考文献,展开讨论,每个人独立完成一篇读书报告。

讲座课程与同学们以前学习的课程完全不同,不仅进度快,而且给每个人预留的发展空间比较大,学习方式更具有相当的灵活性。因此,每一个参加讲座课程学习的高年级同学,都应该在任课教师规范的讲座报告中,认真听讲,记录内容要点,通过阅读一批指定的文献和自选的参考文献,掌握课程的主要内容。尤其需要引起重视的是,每一个同学都应该独立地立足于自己的能力,通过讲座类课程的学习,学会如何迅速地进入一个新的科学技术领域,如何借助重点文献,通过泛读、精读、思考、分析、演算、推导、类比、设计、质疑、求证、归纳、总结、体会,从实践到理论,从理论到实践这样一个可能需要反复进行的过程,经受锻炼和考验,使自己在综合素养方面得到充实和提高。

我们希望B类模式的执行在选修课程方面能够发挥整个学校多学科的优势,不仅能够支撑培养计算科学专业的专门人才,而且能够支撑培养跨学科的复合型人才,为信息处理领域培养专门人才。这样,我国在培养既有较好的计算科学专业基础,又有某一学科领域一定的专业知识的专门人才方面迈出坚实的步伐,相信毕业生一定会受到社会各行各业的欢迎。B类模式虽然是以培养应用型和技术开发型人才为主,但我们依然希望课程设置与学时数设定比较规范,一般的正规课程应以安排72学时为宜,既保证教学内容的深度,也保证一定的广度。另外,实验课程如果每周是四个学时,应该安排在一个下午进行,以提高学生版块时间的利用效率。B类模式可以实行学分制,也可以不实行学分制。学分制并不是保证教学质量的一剂良药,而仅仅是学校教学管理的一种管理方式。

这里给出的仅仅是计算科学应用型人才培养的一种参考教学模式,各校学生应在完成本校规定的学习任务的前提下,参考本模式,决定自己的学习方式和学习策略。

4.3 如何学习计算科学和顺利完成学业

初入大学校门的计算科学专业的学生,往往对自己能否学好这一学科专业知识,顺利完成学业,怀着一种忐忑不安的心情,这是很自然的。在初步了解了计算科学发展简史和学科的特点、内在规律等一些基本知识后,根据多年的教学实践经验,学生最关心的也是最重要的问题有以下五个:

- (1) 如何实现思维方式的数学化?
- (2) 计算科学专业各学期重点课程有哪些?
- (3) 实验课程在计算科学教学计划中的作用和地位是什么?
- (4) 如何提高专业技术能力?
- (5) 如何形成科学的思想方法?

下面,我们就这几个问题进行详细的讨论。

4.3.1 思维方式的数学化及其实现的途径与方法

在计算科学教育界,高等学校的教师们普遍都承认数学教育对学生学习计算科学专业知识的重要性。在本书第二章的讨论中,我们指出了这样一个事实并提出了一个观点,即数学教育对计算科学专业人才的培养有两个目的:

一是通过教学使学生掌握进一步学习这一学科所需要的数学基础知识;二是通过严格的数学训练,使学生实现思维方式或思维过程的数学化。

围绕第一个问题,学术界根据学科各方向发展的实际情况和需要,结合人才培养目标,以及教学计划与课程体系留给数学教学的时间和空间,按照学以致用原则,可以比较准确地界定数学教学的内容和要求。但是,计算科学和数学都是正在发展的学科,今天的数学教学内容并不能保证满足明天学科发展的需要。从教育学和学科教育学的角度看问题,针对计算科学,学校应该更关心如何通过高质量的数学教学,使每一个学生成为不仅掌握一定的数学基础知识,而且能够运用数学的基本思想、原理和方法来进一步学习数学知识,丰富数学知识,应用数学知识解决计算科学实际问题这样一个教学问题。这就涉及到如何培养学生的数学修养和数学能力的问题,而这个问题的实质是如何实现思维方式数学化这样一个核心问题。

如何来实现思维方式的数学化呢?这是许多学生普遍有疑问而且又感兴趣的问题。下面,我们从思维方式数学化的定义出发,讨论这一观点的内在涵义及其实现途径,进而提出实际操作方法,并结合例子做详细说明。

所谓思维方式的数学化是指从普通人的思维方式转向数学家工作的思维方式。在科学界,数学家的思维方式与其他学科的学者很不相同。他们认识客观事物,对客观事物的观察和分析,一般并不直接关心事物的物理、化学、生物学等特性,而是通过对事物的抽象,运用特殊的符号或语言系统,研究事物在空间中的数量关系、位置关系、结构关系和变换规律,研究具有共同抽象概念、性质的一类事物的某些内在规律,以此指导人们从一个侧面去认识事物。逻辑是严格数学论证和科学论证的主要工具,而数理逻辑则是从数学的角度为数学研究乃至科学研究提供了科学推理的逻辑基础。由于数学对客观事物规律的描述是建立在严格而又抽象的符号推演的基础之上,因而数学家工作的思维方式与其他学科很不一样。数学不仅能够从物理学、化学、生物学等众多学科领域各种各样的科学实验中运用抽象、统计、归纳、演绎、构造、算法计算等科学方法提炼出事物发展的内在规律,而且,也因为使用的科学方法的不同而产生了不同的数学思维方法。由于这些数学思维方法的科学哲学基础不同,产生了不同的数学流派。但是,在

数学学科的研究与发展中,借助一套抽象的符号与数学语言系统,对抽象、统计、归纳、演绎、构造、算法计算等一系列方式方法的正确运用,确实培养了数学家严密的思维方式和思维过程,实现了思维方式或思维过程的数学化。大多数数学家是经过严格的数学训练实现思维方式数学化的,但要将这种思维方式上升为系统的理性思维方式,则主要取决于人们的数理逻辑或形式逻辑的修养,取决于人们对科学哲学的认知。

为了增加读者的感性认识,进一步加深大家对实现思维方式数学化重要性的认同,我们先举几个实例。

例 1 给定 n 根火柴棍,由两人依次交替从中拾取若干根。如果规定每人每次拾取的火柴棍最少 1 根,至多 m 根,谁拾得最后一根为输者,那么,有没有一种方法来判别这种比赛从一开始就是胜负早已“命中注定”的呢?

我们先来看一看 $m=3$ 的情形。设参加比赛的两人为 A 和 B ,每次 A 先拾火柴棍。 A 为了战胜对手 B ,必须在自己走了最后一步之后,留给 B 一根火柴。而由于 B 每次可以拾 1 根,或拾 2 根,或拾 3 根火柴,因此, A 要确保自己获胜,必须设法在走了倒数第二步之后,留给对手 5 根火柴。理由是 $1+4$ (A, B 一次所拾火柴根数之和),即无论 B 拾 1 根,或拾 2 根,还是拾 3 根火柴, A 所拾火柴根数与 B 所拾火柴根数之和为一常数 4。这样,不管 B 的最后第二步是拾 1 根,或拾 2 根,还是拾 3 根火柴,只要 A 不拾错,最后一根火柴还是留给了 B 。依次推导下去, A 只要在走了最后第三步之后,留给 B 的火柴根数为 9 根, A 只要在走了最后第四步之后,留给 B 的火柴根数为 13 根,……,如此推导, A 就能够保证在不走错每一步的情况下,战胜 B 。

这个问题也可以换成是拾金币,拾巧克力豆之类的游戏,但在数学家看来,拾什么东西并不重要,抽象以后都是数字。但是,比赛的规则很重要,它直接决定了博奕双方拾取对象的数字和剩余对象的数字之间变化的规律,决定了谁是胜者,谁是负者。用公式来表示,就是:

$$d = (n-1) \bmod 4$$

其中, n 为一方拾取火柴棍时当前剩余的根数,那么,他在这一步的正确取数应为 d 根。万一 $d=0$,那就说明:

- (1) 当 $n=1$ 时,自己已经输了;
- (2) 当 $n>4$ 时,自己处于“命中注定”失败的状态,只能期待对方出现失误。这时候,他可以随便走一步,只要符合对奕的规则就行;
- (3) 当 $1<n<4$ 时,对方在最后第二步完成后处于劣势,自己只要按照对奕的规则取走 $n-1$ 根就行,最后一根必然留给了对手。

按照上面的分析,由于 B 与 A 的地位是对等的, A 的“如意算盘”或叫策略

B 也可同样效仿。于是, 给定比赛开始前的火柴棍总数 n , 确定了谁先行第一步, 那么, 究竟谁胜谁负显然是一种客观存在, 是一种“命中注定”。进一步, 如果我们今天考虑的是任意给定的 n 和 m , m 不一定等于 3, 允许 m 于比赛前在一定的范围内可以变化, 那么, 可以进一步抽象原问题, 导出求解问题的通式, 用计算机来编制通用程序。这必然要用到数学。

大家可以思考一下, 根据上面的分析, 能否较快地写出判断自己是否能够获胜的计算通式, 并确定如何应对的策略?

上面这个例子反映出数学家考虑问题的思想方法: 通过抽象来研究事物在空间中的数量关系、位置关系、结构关系和变换规律, 研究具有共同抽象概念、性质的一类事物的某些内在规律, 以此指导人们从一个侧面去认识事物, 解决具体的问题。因为通过寻求计算通式这样一种研究, 不仅弄清楚了博弈的双方应该怎么操作, 而且, 弄清楚了在什么情况下谁能够获胜, 如果他不发生操作失误的话。进一步的抽象又帮助我们弄清楚了当操作规则在一定的范围内可变时, 怎么认识这个问题, 解决这个问题。

数学就是这样, 抽象有不同的层次, 可能还可以继续进行下去。抽象的程度越高, 往往揭示的规律和解决的问题更具有一般性。

例 2 给定一个字符串长度不超过 m 的集合, 请将集合中的每一个字符串反转一下 (或称调个个儿)。例如, 对字符串 abc , 反转就是求 cba 。注意, 除最终的输出外不允许使用输入/输出操作。

对这样一个问题, 怎么求解呢? 又怎样设计算法和程序呢? 我们作如下考虑:

设 P 是长度不超过 m 的某个字符串的有穷集合, 即 $P = \{x \mid x \text{ 是具有某个性质的字符串且 } |x| \leq m\}$, 我们所要做的是:

求 $Q = \{x' \mid x' \text{ 是字符串 } x \text{ 的反转且 } x \in P\}$ 。其中, x' 的计算如下:

- (1) 对任意 $x \in P$, 若 $|x| = 1$, 则 $x' = x$;
- (2) 对任意 $x = uv \in P$, 若 $|x| = 2$ 且 $|u| = 1$, 则 $x' = vu$;
- (3) 对任意 $x = uy \in P$, 若 $|x| > 2$ 且 $|u| = 1$, 则 $x' = y'u$ 。

很明显, 按照上面给出的计算方法, Q 的计算对初通程序设计的读者已不再是一个困难的问题。细心观察, 寥寥数语, 我们就将一个初看起来好像挺困难的问题给解决了, 这实在应归功于数学所起的作用。

当然, 这里给出的计算方法不是惟一的。考虑如下的计算方法, 它同样也计算了 Q :

- (1) 对任意 $x \in P$, 若 $|x| = 1$, 则 $x' = x$;
- (2) 对任意 $x = uv \in P$, 若 $|u| = 1$ 且 $|v| = 1$, 则 $x' = vu$;
- (3) 对任意 $x = u y v \in P$, 若 $|x| > 2$ 且 $|u| = 1$ 且 $|v| = 1$, 则 $x' = v y' u$ 。

我们曾经说过,不同的计算方法将导致不同的算法。不难想象,这个例子的两个解法将导致两个计算思想不同的算法。而且,不难证明,第二个算法比第一个算法要好,因为它的计算复杂性度量更好一些。

是否还存在其他的计算方法?答案是肯定的,而且存在不止一种方法。我们给出的仅仅是最简单的计算方法。读者若有兴趣可以简单地思考一下,包括计算的效率问题。这个例子说明了怎么利用抽象来进行非数值的字的计算。

下面再看两个例子。

例 3 设 P 是 n 个实数构成的集合。给定一个实数 m , 求 P 中所有满足其元素的和数等于 m 的子集合。这就是著名的子集和数问题。进一步,设 P 中的 n 个数均为大于 1 的正整数,再求所有满足上面条件的子集中其元素的连乘积最小的子集合。

这个问题的第一问可以有很多种解法,比较简洁的一种计算表示形式如下:

设 $P = \{a_1, a_2, \dots, a_n\}$, $a_i \in R$ (实数集), $1 \leq i \leq n$,

求 $Q = \{S \mid S \subseteq P \text{ 且 } \sum_{x \in S} x = m, m \in R\}$ 。

有了第一问的求解计算公式,很容易给出第二问的求解计算公式,这就是:

设 $P = \{a_1, a_2, \dots, a_n\}$, $a_i \in N - \{1\}$ (大于 1 的正整数集), $1 \leq i \leq n$,

先求 $Q = \{S \mid S \subseteq P \text{ 且 } \sum_{x \in S} x = m, m \in R\}$,

再求 $W = \{T \mid T \in Q \text{ 且 } (\forall T' \in Q, \prod_{x \in T} x \leq \prod_{y \in T'} y)\}$ 。

上面给出的问题求解的静态计算形式或计算公式并不等于给出了算法计算的方法。当然,根据上面给出的计算方法,我们很容易设计求解第一问的算法和程序,只需在算法设计或程序设计中遵从计算方法的思想就能很顺利地完成了。不过,要注意,同是上面对第一问给出的解法,却可以设计不同的算法。但是,如果我们在设计求解第二问的程序时依然采用计算方法的思想,先计算 Q , 然后,逐个计算 W 中的 T , 再将每一个 T 同其他的 T 进行比较,求出其元素的连乘积最小的 T , 那么,这样设计的程序当 P 是一个很大的集合时,程序运行的效率将是很低的。事实上,采用求解第二问的分枝限界法(一种算法设计思想和方法)的算法,可以设计一个效率很高的程序,而类似这样的程序是以高效率的算法为基础的,将会用到更多的数学知识。

第三个实例及其说明给了我们很多启示,读者不妨自己思考、总结一下。

例 4 自动机接受的字的集合的表示问题。

字母表是符号的有穷集合。一串有穷长度的符号序列可以看成是一个字。设

$\Sigma = \{a, b\}$ 是一个只有两个符号的字母表。由字母表上符号的所有有穷长度序列构成的字的集合我们用符号 Σ^* 表示。自动机接受的字的集合的表示问题是对给定的一台数学机器, 如何用形式化的方法表示该机器接受的字的集合。例如, 我们假设有一台自动机 (读者可以设想为一台图灵机), 它接受所有这样的字: 字中所含 a 的个数和 b 的个数一样多。按照这样的文字表述, 下列字都是这台机器所接受的字:

$aabb, abab, aaabbb, abbbba, aabaaabbbbba, bbaaabbaba, \dots$

初看起来, 毫无规律。我们说形式化的表示有时是重要的, 如果给不出这种表示, 那么, 今后关于这些字上的运算和操作都将是困难的。注意到 Σ^* 内所有的字都是有穷长度的符号序列, 而且, 其中出现的符号只能是 Σ 中两个符号 a 和 b 之一, 因此, 如果我们能够设法计算出任意给定的符号串中所含的 a 的个数和 b 的个数, 那么, 哪些字是一台自动机接受的字也就清楚了。为此, 考虑引入两个函数 f 和 g 。 f 和 g 都是 $\Sigma^* \rightarrow N^+$ 的映射 (其中, N^+ 是非负整数集), 定义如下:

$$f(a) = 1, a \in \Sigma;$$

$$f(b) = 0, b \in \Sigma;$$

$$f(xa) = f(x) + f(a), x \in \Sigma, a \in \Sigma^*;$$

$$g(a) = 0, a \in \Sigma;$$

$$g(b) = 1, b \in \Sigma;$$

$$g(xa) = g(x) + g(a), x \in \Sigma, a \in \Sigma^*.$$

于是, 下面的 L 就是所要求的字的集合:

$$L = \{w \mid w \in \Sigma^* \text{ 且 } f(w) = g(w)\}$$

一旦给定了字母表 $\Sigma = \{a, b\}$ 上字的一个集合 $J \subset \Sigma^*$, 那么, $L = \{w \mid w \in J \text{ 且 } f(w) = g(w)\}$ 的计算应该是不困难的。

上面这两个例子都涉及到一个问题, 即如何将自然语言的含义与形式化的数学语言表述进行互换或转换。最初我们给出的问题及其求解要求的描述都是用自然语言表述的, 后来在给出问题抽象表示及其求解计算方法时又采用了抽象的符号和形式化的数学语言进行表述, 两者之间在含义上是等价的。事实上, 这是实现思维方式数学化的基础和途径, 涉及到两种数学表达方式之间的转换, 读者在今后数学课程的学习中, 必须首先习惯于实现这样一种转换。特别要注意的是, 数学的学习和计算科学理论的学习, 一定不能从符号中来, 到符号中去, 将数学描述和定理证明看成是一种文字游戏。数学语言中抽象的符号在表达问题及其求解等内容时, 都有特定的含义和指称的范围, 由此产生了不同的数学理论。如果离开了这个基础, 数学语言也就失去了其存在的意义, 数学理论也就失去了科学

性的基础。对于给定的任何一个问题及其求解方法,当我们使用抽象的数学符号和数学语言对其进行表述时,应该能够用自然语言对这一数学表述给予语义上等价的描述;同样,如果我们使用自然语言对这个问题及其求解方法进行表述时,也应该能够用抽象的数学符号和数学语言对其进行语义上等价的描述。许多情况下,一旦一个人在这两种表达方式之间实现了比较自如的转换,不仅说明了这个人的数学基础和修养达到了很高的水平和境界,而且,反映出他在思维方式数学化方面已成为一个训练有素的专业人员。

但是,也应该认识到,形式化不是目的而只是一种研究工作的方式和方法,我们不能为了形式化而去形式化。许多计算科学的问题采用形式化的方法主要是为了简化问题的表述,将研究工作建立在严格的数学分析与逻辑推理之上,为算法的设计提供科学思想、理论方法和技术方法,而使用非形式化的方式和方法,有时常常难于严密、准确地把问题说清楚,更不用说展开数学的分析和研究了。

近几十年来计算科学的教学实践已经一再证实了这样一个规律,学生数学基础比较好,在高年级的时候专业基础课程和专业课程的教学就比较容易实现加速,反之,不仅不能加速,甚至会影响教学进度。因为,一个人是否具备良好的数学基础,在使用数学语言和自然语言之间实现自如的转换,不仅反映了其在思维方式数学化方面的训练水平,而且也反映了其抽象思维能力与智力的深度。例如,有的学校和专业(基础)课程中开设了数据库系统原理、编译原理、人工智能基础、模式识别、图像处理、面向对象的程序设计、计算机图形学等课程。事实上,这些课程的内容都是从一些更为基础的课程派生、发展而来,有些内容本质上并没有什么区别,只是认识问题和理解问题的角度不同罢了。例如,数据库原理依目前大多数教材的主要内容——关系数据库模型,其基础主要是集合论、关系代数和算法设计;编译原理的主要基础是形式语言与自动机理论、高级程序设计语言、算法设计;人工智能基础包括两部分,基本算法设计基础和知识表示与自动推理,前者主要是状态空间上的各种计算机搜索算法设计,后者主要是离散数学基础(数理逻辑、图论等)以及数理逻辑的形式演绎推理方法,而且,数理逻辑的形式演绎推理的方法还反映了专家系统原理的主要思想,与数理逻辑的差别主要是前者将公式视为知识,而后者将公式视为形式符号或逻辑符号(不考虑解释或考虑解释),其内容并没有超出数理逻辑和计算机算法设计的内容;模式识别目前比较成熟的主要方法有统计识别法、结构识别法、模糊数学识别法,而统计和模糊数学识别法的基础是概率论与数理统计和模糊数学,结构识别法的主要基础是形式语言及其分析算法;图像处理的基础主要是数学分析、函数论、代数和编码理论,并由此发展了大量的图像处理技术;面向对象的程序设计是首先从抽象数据类型发展而来的,如果学生有抽象数据类型理论的基础,又有高级语言程序

设计的基础,那么,面向对象的程序设计在一般深度的意义上学习是不困难的。至于涉及到面向对象程序设计语言的语义问题,现有的教材和教学基本上采取了回避的方式;计算机图形学的基础主要是几何学、数学分析、代数、算法设计等内容。可见,只要学生前期的数学基础知识比较扎实,思维方式数学化的训练水平较高,那么,学习这些专业(基础)课程是不困难的。

有时候,当使用数学语言表述和研究问题时,数学和计算科学从表面上看并没有什么区别,但是,两者之间实际上存在很大区别。除了我们前面指出的计算科学理论的构造性特征外,另一个区别是计算科学理论中几乎所有的符号在现实世界中都是有指的,它们或者是程序中的一个变量,或者是计算机的某个部件,如存储器等,或者代表其他的实体对象;而纯粹数学则不然,有时高度抽象的数学符号难以在现实世界中找到具体的应用,甚至到了一些定理、结论一时给不出一个实例的情形,难于同现实世界建立对应。正是因为在这两点上存在着认识观点的差别,直觉主义学派和逻辑主义学派在对待数学的基本认识上存在重要的区别。当然,关于这方面内容的讨论超出了本书的范围,有兴趣的读者可以参考数学哲学和数学史著作。

既然计算科学理论常常使用数学语言和数学的研究方式和方法,那么,对计算科学的研究也应该继承和发展数学科学已有的方法和成果。有一点是需要指出的,即数学研究中常常要对被研究的对象考察其存在性、惟一性和稳定性。存在性、惟一性和稳定性是数学研究中普遍关注的基本问题。当然,作为自然科学最基础的学科之一,数学还有其他一些重要的基本问题,如可靠性问题、完备性问题、可构造性问题,等等。对计算科学的研究,应该将研究的对象和内容与数学中关心的存在性、惟一性和稳定性等基本问题联系起来。例如,将它们与算法相联系时,存在性就表现为一个问题是否是算法可解的;惟一性就表现为已知的算法是否是最优算法;而稳定性此时则是指设计的算法当问题的规模由小变大时,算法计算所需的时间和空间是否呈稳步增长状态,当输入的数据处于最好和最坏情况时,算法计算分别所需的时间之间和所需的空間之间是否相差较小,也就是算法的平均性态是否比较好。

不论计算科学理论采用数学方法研究计算科学问题时多么抽象、多么形式化、多么千变万化,最终的基本目的只有一个:为计算模型、算法、程序或电路的设计提供严格的理论基础和计算方法,以及相关的技术方法。

我们希望,读者在学习数学基础课程的时候,应当经常进行阅读、练习、思考、体会和反思。阅读、练习和思考是去体会数学的具体研究思想和内容之间的一致性,而思考、体会和反思是对现有的结果、内容和可能存在的问题作各种深入的分析、体会与探讨,真正做到透过数学概念(定义)、基本方法和基本结论(定

理)理解这些内容的基本思想,能够用自己的语言(如数学语言或自然语言)重新进行完整的叙述。这样做,对于培养读者思维方式数学化的能力是有帮助的,也是十分有效的。

下面再看一个十分重要的例子。

例5 图灵机的停机问题。

试设计一个算法,对任意给定的图灵机 T 和任意给定的输入 $(x_1, x_2, \dots, x_n)$ (输入到带上的 x_i 和 x_j 之间用空格分开, $i, j \leq n$), 能够判定 T 是否停机?

对于这样一个问题,许多人都会千方百计去考虑怎么设计一个判定程序。但是,实际上这个问题是不可解的,根本就不存在这样的判定算法!也许有的读者不信,这样一个看来毫无特别之处的问题怎么就设计不出算法呢?

事实上,在可计算理论或递归函数论中,停机问题已经从理论上用数学方法被严格地证明是不可解的。

这个例子引发了一个值得我们思考的问题:如何来确定什么样的问题是可解的,什么样的问题是不可解的?进一步,如果我们在学习计算科学时对不可解的计算科学问题缺乏足够的了解和基本的估计,那么,一种“计算机几乎是万能的”直觉就会像幽灵一样时时占据我们的头脑,其结果必然是在今后的工作中难免不做出浪费资源的事情。这个问题涉及到计算机能力的界限问题。曾获图灵奖的世界著名计算科学大师戴克斯特拉对此深有体会。他曾经说过:我一生从事计算机软件的研究与开发工作,曾经犯过不少错误。由于对数理逻辑了解不够,许多在数学家看来已经解决的问题,而我还在想方设法去努力解决。大师尚且如此,何况计算科学的普通工作者呢?

现在我们知道了停机问题是不可计算的。理论上,不可计算的问题存在无穷多个。一个可计算的问题加上一个不可计算的问题组成的复合问题仍然是不可计算的。从这样一个思考问题的角度出发,我们有理由相信,不可计算问题的总数不会少于可计算问题的总数。那么,当不可计算问题的形式千差万别时,我们如何来判定一个问题究竟是可计算的还是不可计算的呢?随着科学技术的发展,具有计算密集性特点(如受控核聚变、全球气候中长期预测等)和数据密集性特点(如事务处理和综合决策支持系统等)的大规模复杂科学问题的计算任务将会越来越多。这类科学问题的计算成本是很高的,需要投入巨大的人力、财力和物力。如果没有把握,是不能匆忙上马的。欲解决这些问题,非得从理论上进行一番深入的研究不可。

从目前已认识的一些不可计算问题来看,它们往往与先前已知不可计算问题之间存在着相互等价的关系,理论研究中也常常通过证明一个问题与某个已知不可计算的问题等价,或通过某种方法将问题化归为某个已知不可计算的问题来说

明这个问题也是不可计算的。这就需要对典型的不可计算问题的本质特点和含义有深入的了解,能够“看到”不可计算的界限,能够真正透过许多不可计算问题的外观“表象”看出其内在“本质”,逐步培养起具有对一些计算科学技术项目所提出的思想、方法、技术和工程技术路线的能行性作出基本估计的能力。这就要求读者能够更多地了解不同种类计算模型——数学机器的特性和计算能力,在学科的深层次对可计算性进行分析和探讨。下面这个例子是程序理论中的一个不可计算的例子。

例6 通用程序自动设计系统问题。

在程序理论中,一个程序可以分解成一组语法单元 S , 这些语法单元 S 可以看作程序中的语句。 S 单元必须是惟一地可识别的。程序变量的一组可能的取值,称为一个状态。任何一个程序 P 及其语义均可以用一对谓词(或称断言) (φ, ψ) 来表示,记为 $P = (\varphi, \psi)$, 或 $\{\varphi\}P\{\psi\}$ 。其含义是,以满足谓词 φ 的任一状态作为初始状态,执行程序 P 且终结,则终结状态满足谓词 ψ 。由此, φ 和 ψ 分别称为程序 P 的前谓词和后谓词。由于程序 P 可以分解为一个有限长度的 S 单元序列 $S_1; S_2; \dots; S_n$, 因此,如果对每一个 S 单元都能找到一对前后谓词,即 $P = \{\varphi_0\}S_1\{\psi_1\}; \{\varphi_1\}S_2\{\psi_2\}; \dots; \{\varphi_{n-1}\}S_n\{\psi_n\}$, 其中, $\varphi = \varphi_0$, $\psi_n = \psi$, $\varphi_i = \psi_i$, $1 \leq i \leq n$, 那么,这种谓词对序列实际上进一步刻画了程序的语义。显然,前谓词 φ 确定的只是一个充分条件,即以满满足前谓词 φ 的状态作为初始状态,执行程序的 S 单元,若执行终止,则结果状态必然满足 ψ 。但是,程序的某个 S 单元也有可能执行不终止。1975 年,戴克斯特拉引进了一个能使程序执行终结且终结状态满足后谓词的充分必要条件,即对于给定的程序 P (或 S 单元),程序 (或 S 单元) 执行终结且终结状态满足后谓词 ψ , 当且仅当程序 (或 S 单元) 执行的初始状态满足最弱前置谓词 $WP(P, \psi)$ (或 $WP(S, \psi)$)。

在软件开发中,为了提高软件开发的效率,人们根据结构化程序设计的思想和方法(一种仅使用三种基本的程序结构的程序设计方法),提出了一种比较形式化的程序推导方法和技术,允许软件的开发者从程序所要达到的目标(用谓词 ψ 表示)出发,通过一些推导法则,反向一步一步地确定程序 P 的所有 S 单元,即推导出整个程序,并确定能使程序执行终止且终结状态满足目标谓词 ψ 的最弱前置条件(谓词 φ)。现在我们提出问题:能否设计一个软件系统,该系统能够根据用户输入的任意一对谓词 $P = (\varphi, \psi)$, 自动推导出整个程序 P ?

根本就不存在这样的软件系统!如今软件设计自动化的研究都是在某种约束条件下开展研究。例如,要求 ψ 是一个可判定谓词断言公式。这个例子最终可以归结为停机问题,但对许多人来说,不易察觉。我们认为,如果一个人不具备可计算性理论的基础知识,要看出这一点是不容易的。我们所以说可计算性理论与

计算复杂性理论、形式语言与自动机理论对计算科学专业的本科生是重要的，原因是我们遇到的问题必然是种类繁多的，要看清楚这些问题的本质，需要将这些内容与某种计算模型建立联系，通过计算的数学理论课程使读者能够在学科的深层统一以前学过的和将要学习的许多专业（基础）知识，使读者初步达到某种“融会贯通”的程度，而且，计算的数学理论课程中出现的典型方法、核心概念、典型实例还可在方法论的意义下被用于审视其他学科方向的内容。

很明显，要想从根本上真正理解计算机的局限，还得依靠数学，因为，只有在理论计算机科学中依靠数学，才能将这个问题讲得清楚、透彻，让读者体会和感受到计算机能力的边界。这并不是教师或计算科学工作者特别偏爱数学，而是因为计算科学的许多问题从观察者的角度看都具有两个基本特点：一是对问题的处理每一步都必须是能行而又准确的，二是用自然语言或其他方式表达处理的过程常常是不方便的，有时甚至是笨拙的，含混不清的。

计算机执行运算操作时的“机械、死板、严格、精密”的特性是数学与计算科学建立天然联系的主要内在原因。计算理论业已表明，理论上，凡是可以由计算机处理的问题，包括问题描述和处理过程，均可以通过抽象实现数学化或形式化，即用数学符号系统来描述；反之，凡是可以用以离散数学为代表的构造性数学描述的问题及其处理过程，只要论域是有穷的，或虽论域为无穷但存在有穷表示，也一定能够用计算机来处理。至于现实是否能行则取决于计算复杂性和实际需要的计算时间和空间。

由此可以看出学习数学，特别是学习以离散数学为代表的构造性数学对计算科学专业工作者的重要性。今天，虽然许多人能够完成大量计算机应用的任务而并不需要很多的数学训练，但这不等于说计算科学不需要高深的数学，原因是仅凭直觉和经验就能顺利处理的那些问题，不仅说明了这些问题是足够简单的，而且还由于科学家和工程师们所做的大量前期工作使得其中一些原来困难的问题在前人工作的基础上已变得较为容易，计算机界业已开发和使用的海量软件开发工具为大量专业和非专业人员方便地使用计算机开展各种工作提供了便利的条件。当然，在这些足够简单的问题中有许多问题的处理工作在性质上本不属于计算科学专业在社会分工中确定的范畴，它们大都属于计算机具体应用（应归属各具体学科）的范畴，而不属于计算机应用（指计算机应用于各学科的共性技术研究）的范畴。对非计算科学专业从事计算机具体应用的人员来说，数学也许并不重要，他们只需要懂得怎样使用各种计算机软硬件资源，如编译程序、操作系统、数据库管理系统、有关的硬件接口、各种软件工具和应用软件程序包的使用就可以了，但对计算科学专业人员，没有坚实的数学基础，就不可能从事较高起点的，而且是其他学科专业人员不能胜任的计算科学专业技术工作，特别是那些需要专

业人员自己寻找解决问题的途径、理论、方法和技术的問題。由此可见,思维过程的数学化对计算科学专业人员是何等的重要。

学习数学基础课程是实现思维方式数学化的有效途径。然而,由于计算机处理能力上的离散特性,以及受专业学制年限的制约,对于计算科学专业的本科学生,还不能也没有必要像普通数学专业的学生那样学习大多数基础数学专业的数学课程。按照计算科学专业数学教育的两个目的,教学的重点应是离散数学和少量理论计算机科学的内容。但是,计算科学专业后续课程中广泛出现各种应用数学知识的情况和事实上对学习离散数学与理论计算机科学必须具有较好的数学修养的要求,使我们的教学不可能直接从一年级就进入构造性数学的教学环节,实现思维方式数学化的步骤必须分两个阶段来完成。

第一阶段,通过对空间解析几何、数学分析、高等代数、常微分方程、概率统计、计算方法等数学课程的学习,使学生熟悉和习惯于使用数学语言和符号系统对研究的数学对象进行严格的分析、表述、计算和推演,为学习后续课程打下坚实的数学基础,初步实现思维方式的数学化,初步达到数学上的某种成熟性。

第二阶段,数学学习转向以计算科学为背景的离散数学和理论计算机科学的学习,特别是通过对数理逻辑的系统学习,使学生将思维方式逐步上升为系统的理性思维方式,进一步实现思维方式的数学化,最终使学生达到良好的数学上的某种成熟性。

也许有人认为通过其他途径和方式也能实现学生思维方式的数学化。但是,正如上面所述,计算科学与数学的天然联系和数学的学习具有能够培养思维方式数学化的作用这两个基本事实,使几乎所有著名计算机科学系的计算科学专业在教学中选择通过数学教学途径来实现学生思维方式的数学化。

有人可能会认为,计算科学本科学生的大学基础数学教育不必要求很高,比一般工科专业学生的数学稍深一点就够了。理由是将来如果学习后续课程需要更深的数学基础,可以按需补课。其实,这个观点是不正确的,是一种想当然的、不负责任的观点。有经验的教师都知道,学习大学基础数学的最好时机是中学毕业后刚刚步入大学的一、二年级时期,年龄以不超过23岁为宜。在大学的高年级,如果原先的数学基础仅仅停留在普通工科专业的基础数学水平上,那么,无论在主观上还是在客观上,此时深入学习数学是很困难的,更不用说计算科学理论的内容了。因为,一般工科专业的基础数学教学主要是一种以应用为主的教学,基本概念、基本定理和套用现成公式进行应用计算成为教学的主要内容,学生对数学知识的掌握主要停留在浅层面,对大量数学基础知识很少知其然而且知其所以然,更缺乏对数学概念、定理、证明方法、计算方法、演绎推理等多方面的深入分析与系统训练。经验告诉我们,在高等学校,许多在大学一、二年级基础课程

的学习中训练一般的学生,当进入高年级后,对那些需要比较艰深的数学知识的课程往往不感兴趣,即使主观上想努力,也常常不易学好。而在大多数一般工科专业已毕业多年的工程师中,大学毕业之后数学知识的更新很少,除非他原先的数学基础和修养较深,否则,其数学水准远远低于他在校时的水平。这也从一个侧面印证了大多数工程师往往经验知识比较丰富,而创新研究、开发能力较弱的现象。计算科学高起点的研究开发方向对研究开发者的数学基础的要求是具备必要的数学基础,特别是数学上的某种成熟性和思维方式的数学化。而数学上的成熟性和思维方式的数学化不是一般工科专业数学教育所能实现的。

也有一些人认为,数学的一些理论在计算科学中至今尚未找到应用,我们是否有必要学习这样的数学知识。作者认为,首先,对数学知识的学习,就大学生而言,并不是越多越好,毕竟计算科学专业不是数学专业;其次,对待计算科学专业学生学习的数学知识,应该有一个整体性的认识,应注意到数学基础知识的教学对计算科学人才成长的两个基本作用。过分从实用主义的角度出发选择数学基础知识进行教学,不符合教育学基本原则;第三,应该注意到,某些数学理论一时没有找到应用,有可能确实用不上,但也有可能是目前学科在相关方面具体研究的深度不够,有必要深入分析、研究对象的性质和特点。因此,在选择学习的数学课程内容的时候,学校和读者应该根据学科发展的特点和内在规律,有针对性地进行选择,必要时应该有一种超前意识。生物学中保护生物多样性的一个重要的科学意义在于当人们掌握了先进的科学研究方法、技术和手段时,可以对以前不了解的生物运用新的科学思想、方法和技术进行研究。类似地,如果某个人首先掌握了一种新的数学理论,一旦他了解到某个计算科学的问题的本质特点与这一数学理论具有某种一致性时,他将有可能把该数学理论引入计算科学的研究之中,小波理论即是一个有力的例证。当然,对于这种开辟一个新的研究方向的工作,只可能依靠少数才华横溢的科学家来完成,不宜在本科生中推广。本科教学仅仅只是为未来的科学大师提供进一步深入学习数学和计算科学的基础。

在计算科学专业的教学中,为了实现数学教育的两个目的,有关数学课程究竟应该选择什么样的内容,实际如何操作?这也是教师和学生共同关心的问题。

当数学教育的目的明确之后,根据基础数学专业长期教学的实践和经验,要使学生实现思维方式的数学化,达到数学上的某种成熟性,关键在于科学地操作。

在第一阶段的数学教学中,重要的是要在诸如数学分析中的极限理论、闭区间上连续函数的性质、导数与不定积分的关系、定积分理论、级数理论,高等代数中多项式理论、矩阵理论、线性空间与线性变换、二次形理论,常微分方程中

常系数线性方程理论、微分方程一般理论、定性理论等若干数学课程中有深度的内容上下功夫。这不仅因为数学分析、高等代数、常微分方程曾经是早期数学专业最重要的三项基础课程,而且还由于这三门课程所代表的分析方法、代数方法以及微分方程的方法能使学生会和初步掌握使用数学语言或符号系统处理问题的基本方法,熟悉和习惯于这种抽象的符号表示与演算形式,初步学会抽象思维方法。实践告诉我们,采用面向一般理工科专业高等数学和线性代数课程的教材、学时数和大纲组织教学,并不能实现第一阶段的目标。可见在内容的形式上对初步实现思维方式数学化起主要作用的并不是有关课程中的计算及其熟练程度,而是各种反映事物抽象概念的定义,反映事物内在规律的定理及其分析、推演的证明方法。

在这一阶段的教学,应注意以下几点:

(1) 不同的学校由于生源不一样,学生入学时有一些差距。但只要坚持质量第一,宁可慢一些,甚至比名牌大学少学一点,也同样可以实现第一阶段的目标。我国 20 世纪 80 年代培养的计算科学专业毕业生中,有相当一批拔尖的学者是非重点大学培养的本科毕业生,这是一个很有说服力的例子。

作者始终认为:中学阶段的学习和高考入学考试中形成的学生与学生之间的成绩差距,对于一个学生在大学学习有深度的学科专业知识的过程中所产生的影响是微乎其微的。世界上只有笨蛋的教师,没有笨蛋的学生。我们不否认学生个体之间存在的差异,也不否认客观上存在着少数天赋很高的学生,但是,计算科学是一门有深度的学科,有着快速发展和变幻无穷的特点,任何一个学生,无论他(她)的天赋多高,只要没有打好基础,在前进的道路上走偏方向,那么,他(她)的发展前景并不比天赋一般,基础扎实,脚踏实地,循序渐进的学生更光明,更有希望成才,中国和世界计算科学发展的历史已经反复证明了这一点。因此,凡选择了计算科学专业的学生都应当下决心和下苦功夫把数学学好。一旦教师教学进度过快,应及时向教师提出,调整教学进度。

(2) 应建议和要求学校使用国内外优秀教材,不必要也尽可能避免过多地借阅和参考其他教材。因为低年级大多数学生不具备同时接受几个(教材)体系的能力。而且,好的教材具有能使学生准确、完整理解内容足够多的信息。

目前正在中国高等学校宣传的双语教学并不是一个值得普遍推广的模式,中国是否需要推行双语教学的问题是一个有待学术界认真研究、讨论的问题。在双语教学中,一年级学生对于学习有一定难度的数学、物理类等基础课程时,由于存在着要同时弄清楚外文语言的语法和语义,数学语言的语义和数学内容的含义等三方面的内容,加上其他不利因素,许多人感到十分困难,教学效果往往不理想。而实际上,大学中的大多数基础课程的内容都是相当基本的、成熟的、稳定

的,合格的教师对于这些内容的掌握也是娴熟的,中外语言比较学的研究也并没有得出西语系的语言在表述科学技术或计算科学知识时具有比中文更多的优势,因此,完全没有必要采用外语教材和口语讲授,学生更不应该迷信西方发达国家的教材。

(3)教材中的习题原则上学生应全部做,并以中等水平完成此工作量的75%—80%来衡量教师的教学进度。学习有余力的学生还应主动多读几遍教材内容,多做一些教材以外的习题并得到及时的答疑和批改。每一个学生应力求独立自主地学习数学课程,尤其不能抄袭别人的作业,但可以和同学讨论问题,交流学习心得。

数学家华罗庚教授说过:“学数学如果不做习题,就等于入宝山而空返。”学习与数学学科具有密切关系和类似特点的计算科学,也应该如此。做习题是学生学好每一门课程的一个必要而且关键的环节,每一个学生都应该将做一定数量和质量的习题作为自己学好每一门课程的自觉行动。有经验的教师都知道,学习数学,不经过反复阅读、思考、演算、探索、体会、归纳、总结来掌握所学的内容,没有经过一定数量各种类型习题的解题实践,要学好数学是不现实的。

(4)应要求任课教师加强课外辅导和答疑。初入大学校门的学生,由于一时不适应大学的学习,也由于大学的数学在内容上对学生有压力,及时辅导和答疑是极其重要的。一般地,每门课程每周应安排两次辅导和答疑,以便及时发现和解决问题,调整教学进度。

目前,由于高等学校急剧扩张带来的教育资源紧张的问题已经影响到了教育质量的保障,很多学校已经丢掉了过去数学、物理学等基础课程教学中形成的定期辅导、答疑、批改作业等传统,考试要求日渐放松,考试方式也在逐步转向文科化,试题难度大幅度下降,这些无疑对于像计算科学这样有深度、有学问的学科不是一件好事情,对于培养高质量的专业人才是一个冲击。应该看到,中国高等学校近30年来集中了国内最好的高中毕业生,但计算科学的发展与西方发达国家之间存在很大差距,计算机科学系的办学存在比较严重的漏洞!因此,在目前浮躁的社会氛围中,计算科学专业的学生更要保持清醒的头脑,切忌人云亦云,学会独立思考,认清自己目前的有利条件和面对的不利因素,坚定自己的人生信念,牢固树立远大的志向,以学好知识,报效祖国和人民,献身科学与人类进步事业为己任,下决心将各门功课学好。

在第二阶段的数学教育中,数学的教学内容是以计算科学为背景的离散数学和理论计算机科学,目标是进一步实现思维方式的数学化,最终使学生达到良好的数学上的某种成熟性,同时,在计算科学基础理论方面初步打下良好的基础。在这一阶段的教学中,学生要特别注意根据教师的引导,借助计算科学的背景,在

近世代数、集合论、图论、数理逻辑、可计算性理论、形式语言与自动机理论等课程中将科学哲学的思想方法,理论、抽象、设计三个过程,若干学科的核心概念、典型方法与典型实例,学科基本工作流程方式等学科范型的内容贯穿在学习过程的始终,并在某些课程如集合论、数理逻辑的教学中真正理解并能够有针对性地对一些内容赋予哲学解释。

在这一阶段的教学,有以下几点:

(1) 学生应下大力气把离散数学学好。学习中应注意将其内容与计算科学的背景相联系。应该清楚地认识到,计算科学专业的学生学习基础数学和离散数学的目的和作用,更多的是为今后开展计算科学研究和开发工作建立必要的基础,培养科学的思维方式和方法,而不是为了数学学科的发展而学习数学。对计算科学专业的学生来说,数学是基础,是思想方法和工具,要熟练地运用它解决计算科学的问题,必须更多地与计算科学建立背景联系,从中悟出真谛。

(2) 有人认为计算科学专业还应学习一些数论和组合数学的内容,理由是有一些计算科学的文献中出现了这方面的内容。考虑到这些内容在图论和集合论中有一些是相重的,在方法和学术深度方面相近,四年制的学生可以不学。与其泛泛地介绍一些各方面的内容,不如认真地学好一门课程。有了相应的能力,学生在今后的工作中不难看懂这方面的文献,补上相应的知识。

(3) 离散数学的教学不应该采用 CAI 方式。数学的学习是一种思维的训练和智力的活动,许多情况下要靠学生反复阅读、思考、演算、归纳、总结、体会来掌握所学习的内容,而不能依赖于提示和借助图形来学习。因为后者往往使读者停留在知识的表层,对知识的掌握不牢固,并没有真正提高学术水平和工作能力。

当第二阶段的数学教学与计算科学专业课程的教学因内容相联系同步进行时,通过理论与实际背景的对应,以及有相当深度的数学训练(包括理论计算机科学的初步训练),进一步实现思维方式的数学化并使达到数学上的某种成熟性是有保证的,这已为国内外一些成功的办学实践所证实。一旦具有坚实的计算科学数学基础的学生在按部就班地学习了有关计算科学专业课程之后,分析问题和解决问题的能力就比较容易形成。

通过上面的讨论,我们知道思维方式的数学化对学习计算专业知识是重要的,也是可行的。计算科学专业的学生们决不能因为今日不少大学计算科学专业的毕业生大量进入了学非所用,或从事低层次计算机具体应用工作的单位,能够基本适应目前岗位对他们的要求,就以此为根据认为没有必要学习那么多数学课程。

一切科学理论只有在赋予哲学解释之后才能获得升华。计算科学作为一门年轻的学科,由于其研究与开发中思维方式的特殊性而使得它与数学建立了密切的

联系。作为数学和计算科学共同使用的符号系统或语言，它与一般的语言有着相似的系统特征。所谓数学上的某种成熟性实际上同语言学家所说的语感本质上是同一概念。两者虽然说法不同，但都是对自如地运用各自的语言刻画事物的一种属性观察。一个大学计算科学专业的学生，如果不能在基础课程的学习中初步实现思维方式的数学化，那么，他在今后学习一些需要用到较多数学基础知识，有学术深度的课程时，往往事倍功半，效率比较低。这一点已为许多大学过去的教学实践所反复证实。这种现象也就不难解释为什么在有的学校，在排除了其他因素后，有学术深度的课程往往教学进度不能达到理想的进度和目标，而那些重视基础的学校，教学进度不仅正常，有时还可能实现加速。此外，只要细心观察，我们不难发现这样一个现象：总体而言，数学基础比较好的学生对事物的表达往往比其他人更清晰、准确、严谨。为什么？

在高水平大学的中文系或语言文学系，有两门课程学生必须学习其中的一门课程，即“语言学中的逻辑学”，或“形式逻辑”，许多语言文学专家早就指出了逻辑在语言学中的重要作用。我们说一个人的文章写的好，总是包含了他的文章结构合理，叙述有条理，论述严密，修辞水平较高，语感很好，等等。在语言学研究，语感是一个很难严格、准确定义却又客观存在的概念，它与一个人对语言的认识水平和感悟能力有关。长期使用符号系统或数学语言处理问题和事物，也会产生对数学的一种感悟能力，并逐渐形成思维方式的数学化，达到数学上的某种成熟性。但是，要从这样一种感性思维方式上升为理性思维方式，必须依靠哲学或思维科学。特别，与推理有关的是形式逻辑与数理逻辑。显然，将数学上的某种成熟性同语言学中的语感作相应的对照分析，有助于我们加深对思维方式数学化重要性的认识，也使我们进一步认识到数理逻辑在整个学科中的重要性和地位。

4.3.2 计算科学（专业）各学期重点课程

（1）硕士、博士研究生各学期重点课程

硕士研究生课程学习一般安排一年至一年半。在课程学习期间，重点课程有五门，它们是：

英语、高等逻辑（或称数理逻辑高级教程）、高等计算机体系结构、并行算法设计基础（或互换课程分布式算法设计基础）、形式语义学。

博士研究生课程学习的主要任务是在硕士研究生重点课程的基础上有选择地拓广和加深，一般安排两门重点课程。由于高等逻辑、高等计算机体系结构、并行算法设计基础（或互换课程分布式算法设计基础）、形式语义学这四门课程作为研究生学位课程时，加上学生在本科阶段（A类模式）中学习的其他所有必修课

程知识,可以支撑学生未来向计算科学的任何一个方向发展,因此,不难看出,上述本科生和研究生的教学计划与课程体系是真正按照计算科学一级学科开展教学的人才培养方案和模式。但是,由于历史的原因,目前我国绝大多数甚至全部高等学校完全依靠自己的力量还不能同时开设上述四门研究生重点课程,教育部有关的师资培训工作正在加紧进行,所以,各大学暂时还很难真正按照一级学科培养人才。

由于上述四门研究生重点课程有相当的学术深度和难度,目前,一些学校对于安排学生学习这四门研究生重点课程有疑虑,学生也因过去基础比较薄弱,在教学中实际付出的时间和精力代价比较大,为此,我们建议有关学校采取变通的办法:一方面要求学生学习上述四门课程,先培训教师,后带领学生讲授和讨论。另一方面,对于专业方向的选修课程,可以允许学生结合自己的研究方向自学提高,开展学位论文工作,在完成学位论文工作的基础上,结合论文质量和答辩成绩,由研究生指导教师签发两门相关方向课程的学习成绩。

(2) 本科(A类)专业各学期重点课程

第一学期:英语、数学分析、高等代数

第二学期:英语、数学分析、高等代数

第三学期:英语、数学分析、常微分方程、高级语言程序设计

第四学期:英语、数学分析、近世代数、数字逻辑

第五学期:集合论与图论、计算机组成原理、数据结构、数理逻辑

第六学期:汇编语言程序设计、操作系统原理、编译原理、算法设计与分析

第七学期:可计算性与计算复杂性、形式语言与自动机理论

四年学习期间,计算机科学实验课程也是重点课程。

选修课程中就本科生今后服务于社会的大多数应用部门而言,相对比较重要的课程有:

计算机组织与系统结构、微机原理及其应用、计算机图形学、信息系统分析与设计、实时系统的分析与设计、计算机自动控制基础、计算机网络与通信、软件工程

(3) 本科(B类)专业各学期重点课程

第一学期:英语、数学分析、高等代数

第二学期:英语、数学分析、高等代数

第三学期:英语、数学分析、信息处理与程序设计基础

第四学期:英语、离散数学(I)、工程数学(I)、电子线路基础

第五学期:数字逻辑、离散数学(II)、工程数学(II)、数据结构与算法设计

第六学期：汇编语言程序设计、计算机组成原理与设计、系统软件原理与设计方法

四年学习期间，计算科学实验课程也是重点课程。

选修课程中就本科生今后服务于社会的大多数应用部门而言，相对比较重要的课程有：

计算机组织与系统结构、微机原理及其应用、计算机图形学、信息系统分析与设计、实时系统的分析与设计、计算机自动控制基础、计算机网络与通信、软件工程、故障诊断与检测、多媒体技术及其应用、数据库系统及其应用、计算机仿真技术及其应用、C++与面向对象程序设计、高级程序设计技术、程序设计方法学。

就高年级的学生来说，如果所在计算机科学系能够开设一些比较新颖的前沿领域的选修课程或专题讨论班，或者开设一些反映学科各个领域共同基础的选修课程，那是很理想的，但实际情况可能与理想情况相差较大。对于选修课程，如果一时条件不成熟，也可以讲座类课程或专题讨论班的形式进行。在美国和加拿大的许多高等学校，在研究生和本科生中开设讲座类课程和专题讨论班是十分普遍的。以斯坦福大学、麻省理工学院、卡内基·梅隆大学、多伦多大学等著名大学为例，许多 20 世纪 80 年代开设的讨论班的题目和内容，今天已成为一门一门的课程。当然，现在他们又有了新的讨论班和题目。有时候，学有余力的学生也可根据兴趣在高年级自发地组成讨论班，请一些有关的教师担任课外辅导。

作为计算机科学与技术专业的学生，外语的学习十分重要，因为目前大量先进的计算科学文献资料是以英语的形式发表的。为了今后顺利地更新知识，有效地跟踪国内外学科发展的前沿动向，尽快掌握先进的计算技术，参与国际学术交流，本科生一定要下决心学好外语。

4.3.3 实验课程在计算科学教学计划中的作用和地位

在 A 类、B 类模式教学计划的实验教学中，每一单元的实验都是从各门课程中选取的，其内容相对独立，自成体系。这些实验内容集中地反映了各门技术课程的典型方法和技术，对加深和理解课堂学习的内容有重要意义，它能够培养起学生将来毕业后在实际工作中结合具体研究内容，依靠理论指导开展实验工作的能力，逐步形成理论与实践相结合的工作作风。

实验教学是学科教学过程中的重要环节。由于学生在校期间时间有限，各校实验条件普遍不及社会用人单位，而且大学教育重在基础，因此，实验内容主要是从学科知识组织结构出发，选择最能反映本学科基本实验技术的实验内容组织教学，并注意实验题目与内容理论联系实际，相互衔接，防止和减少重复。应该

清楚地认识到,在大学学习期间,实验与基础理论知识的教学相比,毕竟是层次比较低的工作,也是相对比较次要的工作。与物理学、化学、生物学、医学、地球科学等许多学科不同,在计算科学中实验教学和课堂基础理论教学不是并重的,这是由学科的特点、学科形态、学科发展的内在规律和学科方法论的内容决定的。作为一个训练有素的科学人才,计算科学实验工作应该在理论指导下进行,而不是在盲目地摸索中获取知识。任何一个学生均不可能在校期间学会并掌握社会上流行的各种软件和技术,在操作和应用计算机上达到一个相当于高级程序员或工程师的水平,同时又保持在基础课程学习上的良好成绩。许多具体的实际操作技术与技巧,完全可以也应当由学生毕业后在工作中加以提高。

可能有一些学生会产生顾虑,担心自己将来在计算机的实际操作动手能力方面不及其他人,在将来激烈的就业竞争中处于劣势。这种担心是可以理解的,社会上也确实存在着有一部分用人单位注重实际操作技能而忽视科学素养的情况。不过,我们应该注意到这样一个基本事实:计算科学毕竟不是一门经验科学占主导成分的学科,而是一门在学科的深层理论(包括技术理论)占有主导地位,应用技术发展异常迅速,需要深厚功底和较强的实验能力才能参与竞争,属于年轻人的学科。由于学科发展很快,一个人以前积累的实践经验对今后的工作并不像其他学科那么重要,更不能靠今天主要通过实际操作掌握的实践经验和技能一劳永逸。许多非计算机专业的毕业生经过简单的技术培训和一段时间的实践锻炼,也能比较熟练地掌握一般的计算机操作技术和基本的实验技能,这就足以说明各种计算机操作技能和一般性基础实验对计算科学专业的科班毕业生更不会有什么特别困难之处。当然,专业培养计划要求每一个学生在校期间都应该认真完成实验课程中的基本实验,它对培养学生基本的实验能力具有重要意义。实际执行实验的过程中,我们希望学生特别注意实验操作规程和规范性要求,在实践中不断地总结经验,从感性认识上升为理性认识,坚持理论与实践相结合的发展道路。

我们相信,随着学科的深化,行业内部的竞争加剧,计算机技术的更新和产业的升级,以及参与世界竞争,对人才专业素质的要求会越来越高,决不会仅仅看重计算机操作技能。有一个事实可以让学生放心。我国 20 世纪 80 年代初毕业的本科生和硕士研究生中,在计算机应用领域做出突出成绩的几个优秀人才,确实出自以培养理科人才见长的文理科综合性大学或工科院校中偏理的系科。如在计算机地质数据处理的研究中做出一系列高水平成果的陈凛(本科毕业),1982 年毕业于武汉大学计算机科学系,是我国 1977、1978 两级学生中第一个以理论联系实际的成果在《计算机学报》上发表学术论文的青年计算科学家。据了解,他在大学学习时,因不满足于本系开设的数学基础课程的深度而去数学系旁听有关课程,尽管大多数课程考试成绩一般,但其实际基础、专业基础知识水平和能力

高于其他许多同龄人；上海、深圳计算机证券同步交易网络系统的主要设计者邓一辉（本科毕业），1982年毕业于贵州大学数学系计算机专业，因解决了证券交易系统撮合算法等问题而在计算机证券交易系统的开发中取得成功，受到深圳特区政府的重奖；主持开发了三代瑞星防病毒卡的刘旭（本科毕业），1982年毕业于福州大学数学系计算机专业，后在中国科学院数学研究所毕业并获硕士学位，现在是瑞星公司的总工程师；我国“文化大革命”以后培养的计算机专业毕业生中第一个成为中国科学院院士的科学家林惠民教授也出自福州大学数学系计算机专业。尽管在工作中取得成果的年轻人什么学校毕业的都有，但是，当注意到他们的研究开发工作不是一般专业人员能胜任时，注意到他们当年在大学学习时学校的实验条件和开设的实验课程的质量等远远不及今日大学的实验教学时，我们有理由相信，那些成果出自他们之手决不是偶然的。他们在大学曾接受的较多的数学和计算科学理论等基础课程的训练培养了良好的科学素养。从他们的身上可以看出，重视基础理论教学的院校（系）培养的人才在计算科学领域中的优势。

从中国计算科学发展的历程看人才的成长也具有充分的说服力。我国1956年在原中国科学院数学研究所部分研究室的基础上组建了中国科学院计算技术研究所。由于当时高等学校没有计算科学专业，于是，从数学、机电、电工、电子学、机械、自动控制等学科、专业抽调了一批毕业生。若干年之后，人们发现，在各个实验室挑大梁的骨干人员大多数是数学类专业的毕业生。20世纪80年代之后，在对国内五批（含特批）由国务院学位委员会审批公布的博士生导师进行专业背景统计时，人们惊奇地发现，曾经是大学数学类专业的毕业生占了总数的三分之二以上。

当然，这样一种现象是历史的原因形成的，并不等于说今天各专业用人单位应该接受数学类专业的毕业生而不是计算机类专业的毕业生。根据我们对国内外同行人才的观察，由数学类专业毕业转而直接从事计算机行业工作的毕业生已经较20世纪70年代前大大减少，因为学科发展已经积累了大量的知识，形成了庞大的知识体系和结构，学科发展的日渐深化决定了一个大学非计算机类专业的毕业生不经过系统的专业学习和训练，将难于有效地开展工作。当然，有些读者会提出这样的问题：既然以前从数学专业毕业的人才从事计算科学专业有优势，现在很多人获得了较高的学术地位，这是否预示着我们应该继续从数学专业高年级学生中选拔学生，培养计算科学专业人才呢？这个设想本身并不是不可行的，但需要增加学制时间。实际上清华大学计算机科学系（理科实验班）、中国科学技术大学计算机科学系（五年制）都曾经进行过一些试验，有许多可取之处，也积累了一些成功的经验。但是，要在四年制的时间和空间中继续加强数学教学，提出

比 A 类模式更高的数学教学要求是不可取的,原因是一方面过多的数学课程冲击了专业基础和专业课程的教学任务,另一方面也还没有充分的理由一定要上那么多数学类课程。如果读者进一步问为什么老一辈学者可行而我们不行?答案是他们属于伴随着整个学科的发展而成长起来的一代专业技术人员。今日大量重要而内容庞大的专业知识需要学生在大学学习期间完成,留给四年制学生在大学里的学习时间和空间是十分有限的,每一个同学应该倍加珍惜,努力,努力,再努力。

跟踪近年来毕业进入计算机公司的部分本科毕业生,凡是在大学学习阶段比较注重实际而忽视基础课程学习的学生,即使在毕业时操作计算机已达到如火纯青的水平(如有的学生在全国计算机操作比赛中获得大奖),但往往毕业后不出几年,便会感到一种对专业的茫然。这倒不是说他们不能胜任本职工作,而是面对比大学更好的实验技术条件,却不知如何开发能参与竞争的、技术含量较高的软硬件产品。因为,那些大多数专业人员都能开发的软硬件产品现在已做得很多了,即使能够开发出一些看似先进的产品,但因科技含量不高,不可能长期保持领先地位和市场份额。也不能说大学教育远远跟不上学科的发展,因为大学只能教给学生最基本的基础知识和学习方法,注重学生能力的培养,教学质量的优劣只能通过学生的基本知识、科学素养和专业能力反映出来。而计算科学发展又是如此之快,学校现有的条件根本无法迅速地适应学科发展的变化,于是,大量的专业知识也只能靠毕业生自己去更新了。

中国有句成语,叫做厚积薄发。我们对实验和基础理论知识的学习作上述这样一种相对的重要性比较,丝毫没有降低实验重要性的意思。我们有时候常常要在有限的时间和空间中对自己的活动作出规划,以获取最大效益为目标,这是一种最优规划的问题。在学习计算科学时,对实验技术和基础理论知识的重视程度方面也存在这样一种规划。根据多年的教学实践经验和人才成长的规律,对大多数学生来说,如果要考虑将来在工作中不断更新知识,逐步上层次的问题,最优规划中对实验能力的要求应该是较好地掌握基本实验技术,建立正确的思想方法,掌握一定的实验技能。一旦学生在校期间错过了学习基础课程和专业基础课程知识的时机,未来要想补上这些知识是很困难的。

每一个学生在实验教学中所反映出来的能力是不同的。除了自身努力外,这还与中学教育,家庭成长环境有关。对实验动手能力比较差的学生,应主动争取更多一些的机会和时间进行练习,赶上整体平均水平。按照课程进度表,某一学期前面实验完成得不好的同学应争取在本学期最后一周或两周的实验时间内补课。每一个学生应该知道,学校强调的是学生在校期间应在本学科的基本实验技术和实验方法方面打下良好的基础,其良好的重要标志应该是学生实验动手能力的提高,而不是单纯完成实验的速度。

实验教学要达到以下几个目的:

(1) 学生应该通过实验教学理解课堂上讲授的原理、方法和技术怎样通过实验反映出来,即怎样在软件 and 硬件的设计、实现和调试中反映出来。

(2) 学生应该通过实验教学了解哪些是计算科学最基本的实验技术并掌握这些技术,如何进一步掌握其他一些实验技术,即学会掌握实验室技术的一般方法。

(3) 学生应该通过实验教学认识到实验方法的重要性。可以从实验目标与技术要求,构思设计试验,实际操作实现步骤,实验数据的统计分析,研究结果的正确陈述,与其他实验的比较,以及思考如何总结和改进实验、构思新实验中获得体会。

(4) 学生应该通过实验教学养成良好的实验习惯,重视理论联系实际,正确设计实验,完成基本操作,通过实验和实验报告反映正确的思想方法和实验能力。

在实验课程教学中,当一个单元实验结束后,每一个学生应该独立完成实验报告,这将有助于学生在撰写科技报告和论文方面得到训练,在正规的实验教学中得到基本的训练。

上述科学的分析,使我们对计算机科学实验课程在计算科学专业教学中的作用和地位有了比较清晰的认识。我们一直主张,教师和学生 在实验教学中应该严格实验过程的规范操作,同时,教师应该在实验检查过程中弱化对于实验结果的评判,以此获得实验课程教学要求的准确定位。

4.3.4 提高专业技术能力的途径与方法

要回答如何提高专业技术能力这个问题,首先要弄清计算科学专业能力包含哪些内容。参考前面章节对计算科学学科发展特点、规律等内容的介绍,从特征分析的角度,我们不难得出,计算科学专业能力主要通过以下几方面的具体能力体现出来。

- (1) 借助于科学技术文献资料,迅速掌握新知识的能力;
- (2) 分析现有的软硬件产品,进行仿制开发、二次开发和维修的能力;
- (3) 根据新思想,设计软硬件系统进行试验开发的能力;
- (4) 对实际计算问题,应用现有设备,进行计算处理的能力;
- (5) 在前人工作的基础上,进行新概念、新思想、新方法、新技术创新研究的能力。

从技术能力的角度观察,计算科学专业能力主要集中在以下几个方面:

- (1) 阅读、理解科学技术文献上的新知识,特别是用数学形式表述的科学论文、技术报告的内容,能够较快地掌握新知识;
- (2) 计算机实际操作,工具的使用,以及软硬件实验应用操作的能力;

- (3) 硬件设计、数字逻辑系统设计及其实现、维修的能力;
- (4) 软件设计、算法设计、程序设计、程序证明的能力;
- (5) 在前人工作的基础上, 提出新思想、新概念、新方法、新技术、新理论, 并加以数学论证, 或通过设计与实验验证的能力。

尽管列出了上述专业能力的项目, 但并不是要求所有人都无一遗漏地全面掌握。事实上, 作为现代科学技术的一个学科, 一个人只要具有较好的专业基础知识, 较好地掌握其中某一、二个方面的能力就已经相当不容易了。作为本科学生, 大多数人从今后从事的职业工作特点考虑, 比较关心(2), (3), (4)几方面的能力是否能达到。根据多年的教学实践经验, 计算机实际操作, 工具的使用, 以及软硬件实验应用操作的基本能力是所有学生都比较容易达到的。读者只需要根据学校的安排, 认真完成实验课程的学习任务, 并在今后的工作岗位上不断实践, 就能具备较强的能力。但是, (1)和(5)的能力却不是那么容易提高的。一个人要想在这两方面得到提高, 必须要从一年级的数学基础课程开始, 几年如一日地在学习数学、理论计算机科学、计算机科学技术理论等课程中, 从严要求自己, 日积月累, 同时注重学习正确的思想方法, 才能厚积薄发。关于硬件设计、数字逻辑系统设计及其实现、维修的能力和软件设计、算法设计、程序设计的能力, 读者应该注意, 这两方面的能力并不是通过对应的专业(基础)课程学习就能够得到迅速提高的, 而是要通过其他相关课程的学习和实践才能真正较快地提高专业技术能力。

例如, 对于算法设计能力, 由于算法涉及面广, 算法设计与分析课程主要是介绍算法的基本理论、基本设计方法和复杂性分析方法, 而教材中的实例不可能很复杂, 否则将影响算法基本设计方法的介绍。因此, 算法设计与分析的能力只可能通过离散数学、概率、数学分析、代数学、操作系统、编译原理、体系结构、并行处理等一系列课程的学习, 对相关算法问题及其设计方法的了解, 以及今后的反复实践, 才能真正使自己在算法设计的能力上得到提高。要掌握数理逻辑、集合论、图论、组合数学方面的算法设计, 必须学好离散数学; 欲了解图形学与图像处理方面的算法, 必须要有较好的分析数学(含几何)的基础; 要掌握概率算法, 必须要有好的概率论基础; 欲学习代数化简算法, 必须有较强的代数知识和数学修养; 要掌握并行与分布式算法理论, 必须要有计算机体系结构的基础和并发程序设计的理论, 并行与分布式计算模型。又如, 程序设计能力的提高, 仅仅通过高级语言程序设计的学习是不够的, 只有通过汇编程序设计、系统分析、计算机组成原理、系统结构、(分布式与并行)算法设计、数理逻辑、抽象代数、高级语言、程序设计方法学、程序理论、计算模型等一系列课程的学习和反复实践, 才能得到根本的提高。

由于计算科学是一个有相当学术深度的学科，是建立在数学学科、逻辑与电子科学技术基础之上的学科，因此，学习计算机科学与技术专业各门课程的学习方法与学习数学、哲学和技术学科课程的方法具有许多相似之处。对每一个学生来说，首先，静下心来，然后认真听课，反复念书，大量做题，仔细思考，大胆质疑、总结体会、揣摩和联想，完成实验，这些学习的方式方法必将伴随着一个未来有希望、有前途的专业技术人才度过难忘的大学生活。

我们所以指出这一点，是因为目前许多学生对基础理论课程不重视，往往在学习了高级语言程序设计课程之后，就开始把大量时间花费在程序设计和计算机调试上，大量的工作始终停留在计算机实际操作，工具的使用，以及软硬件实验应用操作能力的训练层面，丢掉了许多本来应该花费在学习基础知识上的大量宝贵时间，影响了后续学习任务的高质量完成，也影响了今后自己独立走向学科的深入。

此外，在具备了较好的专业基础知识的基础上，要想更快地提高专业技术能力，除了坚持走理论与实践相结合的发展道路之外，更重要的是要建立科学的思想方法，依靠学科范型等科学哲学的内容，善于在工作中总结前人的工作经验和教训，按照科学的方式方法探索前进，勇于实践。

4.3.5 科学思想方法的训练与形成

在本书的第一章，我们曾经向读者提出了 10 个看上去希奇古怪的问题，实际上，类似的问题不仅在科学发展中可以找到，即使在我们日常的生活实践中也不鲜见。下面结合我们一直倡导的科学的思想方法和工作方式有选择地来试图回答其中一些问题，相信读者可以从中感悟到这里所说的理解科学与科学素养决不是一种空洞的口号。其他几个问题以及更多的读者可能关心的问题的解答，可以参考本书的教学辅导书。

问：考虑一般情况下，字应该怎样书写才算写得比较好？请总结一下判别字写得好的一般标准与写字应该遵循的准则。

答：按照一般的科学思想方法，要回答这个问题，显然应该首先弄清楚字的一些基本的性质。我们说，语言文字作为一种交流的工具，一般情况下写出来的字首先应该能够让别人看得懂，然后才能考虑其书法上的美学价值。如果一个人写的字导致绝大多数人无法看懂，那么，他的字即使是一种艺术品的话，也只能获得极少数经过系统书法训练的专家的认同，而一种艺术形式只能为极少数人鉴赏时，她必然不属于一般的作品。可见，一般情况下，要使字写得好，要获得书法艺术上的认同，字应该首先能够为多数人所认同。其次，从书法的练习活动来看，所有的人写字都是从正楷起步的，这不仅因为字的创造在长期的社会发展过

程中经历了字型统一、标准化的过程,而且,即使从心理学的角度来看,遵守书法规范,沿袭传统技法,字型简单,容易感受到美的心理满足也支持了更多的人倾向于正楷。但是,为什么不用印刷体的字模来代替所有的书法作品呢?这就涉及到人是一种有生命的动物,人的肌体的活力和思想的活动导致了每一个人的身上都跃动着一种追求一定程度刺激和求新、求变的心理需求,正是这样一种生理的、心理的、复杂的同时又是有限复杂的生命形式,决定了人对于语言文字的书法的美学评价必然要求有某种变化。因此,结合古今中外的书法实例,我们可以总结得到判别字写得好的一般标准与写字应该遵循的准则,以及下面的补充说明结论:

以正为主,以草为辅,正中有草,正草结合,以草领型,在正中求变。

所有的普通人写字,首先应该贯彻上述写字的准则和方式,否则他(她)的字将难于获得广泛而持久的认同。至于不按照上述准则写字而所写的字能够获得相当广泛而持久认同的人,一般不在普通人之列,或者是因为其他因素获得承认,或者是开创了书法的一个新天地,成为一代宗师。

问:“凡是美的,都是简单的。”这句话对吗?如果认为不正确,请给出反例。如果认为正确,请就这句话结合你的生活、学习经验给出自己的解释,并用自己的一段话对它进行概括。

答:这个问题涉及到美的概念,因此,按照一般的科学思想方法,首先应该弄清楚什么是美?人们怎样才能感受到美?在弄清了美的概念和人们如何才会对某个事物或对象产生美感之后,我们将不难回答这个问题。

“凡是美的,都是简单的。”这句话实际上指出了人们的审美观和对美的潜意识中包涵了简单性原则。这种简单不是笼统的对于事物苛求其从形式到内容都应该符合越简单越好的要求,而是通过事物在人的某种视觉、听觉、逻辑思维和心理活动等过程中所反映出来的协调性给人带来的精神上的愉悦和快感,较之其他在形式与内容方面的同类事物具有更好的生理与心理感受;同时,它也指出了人类在进行任何具有美学意义的活动中,应该贯彻的一个创作原则。

问:什么是艺术?请你根据自己的理解和体会给艺术下一个比较准确的定义。

答:为了回答这个问题,我们需要先对艺术的范畴进行分析、概括和总结,然后才可能给艺术这个概念下一个比较准确的定义。我们认为,第一,这个问题是要给艺术概念下一个科学的定义。一个事物是否属于艺术的范畴主要取决于人们的感知并能够给人以某种程度的协调美感。第二,用外延的方法给艺术下定义,由于艺术的门类比较多,艺术品的种类比较多,艺术的表现形式很多,显然难于准确概括。用内涵的方法可能比较合适。第三,用内涵的方式给出定义,首先应该从大量艺术事物在静态、动态、形态等多方面给人们带来的协调美感所内涵的

本质特征的提取方面下功夫。由此，概括得到：

艺术是对具有下列特性的全体事物的总称：

- (1) 能够激起人们对于生活的热情，给人以美的享受；
- (2) 它来源于自然和生活，高于自然和生活；
- (3) 有创造性，符合一定的审美准则。

问：如何培养创新人才？教师、学生和社会应该怎么做，才能说我们在培养创新人才方面做得比较好？

答：怎么理解创新？我们说，创新需要有坚实的基础，没有深入就没有真正意义上的创新。创新要靠科学素养和理解科学，靠科学的思想方法。那种仅凭个人的小聪明在工作和生活中完成的所谓创新与发明，顶多不过是一些技巧层面上的“小把戏”，对科学发展和社会进步的作用十分有限。在科学技术和学术研究领域，有了坚实的基础，有了科学的思想方法，我们就能够有针对性地在以后的学习和工作中，对于认识的对象进行多层次、多视角、全方位、立体地审视和观察，运用已经掌握的知识和科学的方式方法展开工作，在深入中依靠科学的思想方法摆脱教条的束缚，有针对性地进行探索前进，而不是盲目发展。科学的发展存在着外延和内涵两种发展方式。当学科处于发展早期时，由于人们对该领域的具体知识了解甚少，往往只能采用外延方式。但是，谁具备好的基础和科学的素养，谁就更容易转入内涵发展方式。

科学技术创新研究包括思想的创新，概念的创新，方法的创新，技术的创新，原理的创新，实现创新不仅需要依靠坚实的专业基础知识，更要靠良好的科学素养和人文精神，需要有科学的思想方法和文化底蕴的支撑。只有把前人的工作真正、彻底地认识清楚，完整、准确地把握了事物发展的客观规律，依靠正确的思想方法，保持良好的心态，才能在科学研究中正确把握发展的方向，找到解决问题的正确的方式方法，形成自己独立的科学见解和主张，建立自己的科学理论。

科学技术的持续发展需要勇于创新的人才去推动。一个没有创新能力的民族，难以屹立于世界民族之林。培养创新人才是新世纪高等教育面临的一个重大难题。

培养创新人才，不能停留在口号的阶段。问题是教师、学生和社会应该怎么做，才能说我们在培养创新人才方面做得比较好？总结历史的经验，我们有理由认为，在今后的人才培养工作中，教师和学生应该努力将下面的 32 字贯穿在教学与科学活动的始终：

见多识广、博闻强记、功底深厚、富于联想

科学理念、人文精神、心态沉稳、持之以恒

前 16 个字主要解决一般层面上创新人才的培养和成长问题，加上后面的 16 个字，主要解决重大创新人才的培养和成长问题。不难看出，要培养重大创新人

才,更主要的功夫不在具体的学问方面,而在于科学精神、人文素养和科学思想方法的塑造方面。因为,任何具体的学问,总是相对比较容易的。学习或了解一个科学理论,不会比发现或建立这个科学理论更困难。任何学问的研究,如果一味地在技术的层面不断地加深对科学技术知识的理解,其深度将是十分有限的。套用一位已故著名科学家的名言:“科学研究不能迷失在具体的细节中。”高水平的科学研究更需要有学术深度的方向、思想、概念、方法、原理、技术、理念方面寻求创新。这 32 个字的方针,体现了在理工科人才培养的过程中,我们应该在重视一级学科基础教学的基础上,更多地依靠科学哲学的思想方法和人文素养,培养创新人才。

此外,社会环境对于创新人才的大量涌现和脱颖而出十分重要。毋庸置疑,人文科学与文化建设,社会环境对人才成长的影响是不可低估的。只有人文精神与科学理念以及优秀的文化才是人才成长不竭的动力和源泉。建设创新文化,营造优良的创新环境,提高公众的科学素质,培育公众的科学精神与科学思维方式,在人类文明发展进程中将显得越来越重要。大学能否培养出一批又一批具有创新能力的人才,社会能否提供适合创新人才脱颖而出的环境、氛围和机制将是关键的、十分重要的因素之一。营造一个对科学技术研究与开发领域提倡宽松、宽容和多元化,支持学术与文化交流,发扬人文与科学批判精神,在探索性创新研究中主张卓尔不群优先于团队精神,在政策上鼓励创新的社会文化氛围是中国国家科技创新体系中政府和人民应该共同致力于一个长期的、基本的学术发展环境目标。

我们应该坚信最高层次的竞争是文化的竞争,只有文化才是持久推动社会进步的第一重要因素。中国具有悠久、灿烂的历史文化,在当今科学技术和学术研究不断深化的进程中,文化底蕴对于各类专门人才在创新研究中保持良好的心理状态具有特别重要的意义,文化的积淀对于创新工作的重要性日益突显,中国人应该具有明显的参与创新竞争的文化优势。

问:有人说,20 世纪计算机科学是领头学科,21 世纪生物学或生命科学是领头学科,此话对吗?

答:关于 20 世纪计算机科学是领头学科,21 世纪生物学或生命科学是领头学科的说法盛行,目前,媒体和舆论对此似乎已经形成共识,但这是不正确的。

一门学科要成为领头学科,应能尽可能多地带动其他学科的发展。下面我们对生物学与计算机科学与技术进行一些简单的分析。

生物学与生命科学:在高精尖实验设备方面,在未来可能的生物计算机与生物计算方面带动一批学科的发展,其本身在促进生物学研究和应用方面可能推动多个产业的巨大发展,如基因工程,生物制药,生物芯片等方面。

计算机科学与技术：计算机科学与技术的主要数学基础是代数和逻辑，由于人工智能研究在计算机科学与技术研究中的引入，进一步推动了逻辑学引起的思维结构方面的研究，而逻辑研究对于科学技术研究具有普遍意义，任何一门科学都离不开逻辑作为其发展的基础；计算机科学与技术与语言之间存在密切联系，各种计算离不开语言的描述。在语言学引起的语法、语义和语用方面，研究工作正在走向深入，极大地丰富了语言学的研究内容和内涵，为人类在不同层面上提供不同的描述语言提供了重要的基础；在高性能计算机系统引起的高精尖设备方面，在渗透、拓展、推动各行各业的发展，带动所有学科的发展方面，计算机科学与技术正在发挥巨大的作用；特别应该注意到的是，以计算机科学与技术为核心，形成了广义的计算机科学与技术学科，即计算科学。计算科学不仅是一门科学，而且还是人类从事科学研究的三种形态之一：即与传统的理论科学、实验科学并列成为第三种学科形态：计算科学。在未来的科学研究和科学发现中，人类未必一定要通过理论研究的推导或实验研究的总结来发现自然规律，建立各种科学理论，而可以通过计算来进行科学研究。例如，美国通过长期核试验积累的数据，已经设计了模拟核试验的计算机程序，通过运行程序可计算得到多种当量的核试验的非常接近真实的实验数据和结果，大大节约了科研经费和科技产品的研制周期。又如，汽车驾驶员和飞机驾驶员的训练可以通过计算机程序非常逼真地在计算机系统上进行，不必每次到现场进行实际操作训练，这已经不是神话。

显然，在领导学科发展方面，生物学与生命科学无法与计算机科学与技术相比较。因为计算机科学与技术的研究与应用，本质上是为了在十分广泛的领域内，在很大程度上代替人的劳动。一门学科，只有当它的研究范畴和内容是为代替人的大量劳动为目的，为众多学科提供不同层面上重要的工具支持，为各学科发展拓展思维空间和科学方法的时候，它才可能成为真正的领头学科。显然，计算机科学与技术具备这一特点。当然，上述两个学科的发展都可能引起人类对社会伦理观念的改变。这样的分析丝毫没有贬低生物学和生命科学的作用和意义的意思。实际上，单从产业和可能产生的市场经济效益来看，两者之间有可能都是巨大的。但是，生物学与生命科学目前还难于看出其在语言和逻辑等方面的领头作用，也难于看出它作为其他大量学科的基础和工具的作用。所以，我们认为，21 世纪的领头科学仍然是计算机科学与技术。

这个事例清楚地告诉我们，重要的是自己怎么来观察和看待事物，而不是人云亦云，盲目从众。

通过上面的问答，读者应该能够从这些内容中初步感悟出如何训练和形成科学的思想方法。但是，读者也应该认识到，科学思想方法的形成是一个比较漫长的过程，需要不断地总结经验和教训，运用哲学的、逻辑的、心理学的科学方法，

在实践中经受锻炼，逐步提高。而且，每个人的方式方法不同，世上并不存在一个严密的、统一的、机械的、程序化的方式方法。科学的思想方法来源于深厚的生活与学习的积累，一个人如果只对自己从事的很狭窄的领域内或方向上的事情感兴趣，而对发生在其他领域内或方向上的事情不闻不问，甚至对发生在自己身边的事情漠不关心，遇事不思考，不总结，那么，他的头脑中必然比较空洞，很难产生广泛的联想，更难以产生创新思维和冲动。从许多著名科学技术人才成长的道路进行观察和总结，不难发现：一个优秀的科学技术专业人才，往往兴趣广泛，对许多事情都有自己独立的看法，有时甚至让普通人感到异想天开。可是，真理的天平往往向他们倾斜。除了他们对科学事业有着孜孜不倦的追求，勤奋努力外，我们还能看到他们尤其重视发生在自己身边的生活中有趣的事情，从分析、思考、归纳、总结中获得灵感，受益无穷。毕竟生活是思想的源泉，否则，人的正确思想从哪里来呢？

根据上面的内容，读者已经感受到了科学思想方法的重要性。实际上，从思维科学的角度看，一般认为存在两种主要的思维方式，一种是发散式思维方式，另一种是收敛式思维方式。人的发散式思维方式是一种自由奔放的思考，几乎不受任何成规的约束，而收敛式思维方式是受一定传统约束的思维方式，是在某个科学时代科学界或科学共同体内共同约定的一套原则和方法的约束下进行思考的思维方式。两种思维方式同等重要，是“一个钱币的两面”，“互补”而不可偏废。一个人只有发散式思维方式，就会胡思乱想，一事无成。仅有收敛式思维则囿于成规，毫无创新。一个训练有素的科学技术人才，应该同时具备和发展这两种思维方式，因为这两种思维方式对于科学技术的进步同样重要。针对具体的科学问题、科学研究工作，这两种思维方式必然会发生冲突，因此，要善于在两者之间保持一种张力。这种张力来源于科学家对具体研究对象的认知和必备的知识基础，来源于对科学领域认知的哲学观，来源于对从事该领域科学研究工作的范型的准确理解和熟练运用，来源于对社会伦理道德和科学良知的深刻反省。

本节所述的内容，并不神秘和晦涩，直接来源于科学发展史的经验总结，来源于一大批杰出的科学家身体力行和世范行为之中。因此，希望有条件的读者，注意收集和阅读一些著名科学人物的传记材料。我相信，前辈大师们献身科学，追求真理的足迹，一定能够带给后人思想的启迪。

4.4 理解科学与科学素养

作为第四章的结束语，我们要强调两点，即理解科学和培养良好的科学素养。理解科学和培养良好的科学素养，包括许多方面的内容，我们下面来分别进行讨

论。

4.4.1 理解科学与科学素养

所谓理解科学是指一个人对多种科学知识的综合结构的了解。其中包括最基本的科学原理,科学思想之间的关系,形成这些关系的原因,如何利用这些科学知识解释和预测自然现象和各种人工实验现象,以及认识和理解发生在我们身边的事情,而不是背诵科学知识。理解科学同时还包括分辨科学和伪科学的能力,在前人工作的基础上探索未知世界和未知领域的能力。

理解科学,必然要求教学从过去单纯以传授知识和记忆知识为主转向以学生通过学习知识,提高理解科学的能力为主。这就要求学生要有较为宽广的、公共的科学基础知识,具有了解自己所从事专业以外领域的基本知识的愿望。要注意,对大学生具有较为宽广的、公共的科学基础知识应主要体现在一级学科和与本学科相关的基础学科知识上,而不是面面俱到。

理解科学,必然要求教学从过去单纯以学生被动接受知识的过程为主转向以教师引导学生主动获取知识的积极学习过程为主。积极的学习过程主要是指学习者身体和头脑的一种状态。在这种状态下,学生已不能满足课堂上学到的知识而常常对发生在自己身边的事物产生兴趣,并积极探索,学生也已不能满足动手活动和简单的实验,而愿意理论联系实际,系统地、科学地进行思考、阐述。

理解科学,必然要求教学从大量要求学生死记硬背知识的教学方式中解脱出来,而突出最重要的知识,贯彻少而精,加强科学技术能力方面的培养,使学生具有科学哲学的思想基础,正确的思想方法。

理解科学,实际上是素质教育的必然要求。要从应试教育转向素质教育,计算科学专业就必须在专业教学中贯彻基础厚实,理论联系实际,激发、探索新知识的兴趣,培养创新能力,塑造科学精神的教育思想,真正从应试教育转向素质教育,还教育功能的本来面貌。

我们在计算科学专业的学生培养目标中将具有良好的科学素养作为一项指标,这看起来难以把握,其实,是很具体的。

所谓科学素养是指一个人参加人类的智力活动所必须具备的科学概念、知识水平和对智力活动过程的理解能力。在日常生活中,科学素养反映在人们对感兴趣的事情充满好奇心,能够理解事情、发现问题、提出问题、参与讨论、解决问题或找到解决问题的途径和方法。在所从事的专业工作中,科学素养反映在人们对自己的工作具有创造性和较高的学术深度,按照科学规律办事,不满足已经取得的成就。

要培养学生良好的科学素养,就必须在教学中特别注重贯彻正确的思想方法。

正确的思想方法不是一个人生来就具有的，也不是从天上掉下来的，它只能来源于学生的实践。对每一个大学计算科学专业的学生来说，正确的思想方法只能在科学的基本原理、原则的基础上，在教与学的过程中，在理论与实践相结合的教学活动中，通过从具体到抽象，从抽象到具体的反复学习、思考、练习、实践和体会，由自己总结得到。

例如，许多人在处理具体问题的时候，往往根据自己的知识和经验，首先匆忙判断自己是否有能力解决这个问题，然后在有可能解决这个问题的时候，立即着手处理这个问题。但是，这样一种匆忙判断和处理问题的方式不是一种好的方式。由于受到各种主客观因素的影响，根据已有的知识和经验，难免夸大或忽视了自己的能力，也不利于充分发挥个人的创造性聪明才智。即使一个人有能力解决某个问题，若无严格的时间限制等特殊原因，我们也并不赞成立即着手处理这个问题。一般地说，对于任何一个问题，正确的做法应该是首先对问题作一个初步的分析，弄清这是一个什么性质的问题，了解同行目前在处理这个问题时的基本情况，然后对处理这样一类问题应该需要什么样的知识和条件有一个清楚的认识。在此基础上，作出自己是否有可能处理这个问题的判断。接着，如果决定处理这个问题，那就应该寻找或建立解决这个问题的方法，创造解决问题的条件，并根据这组方法和条件建立一套科学的操作程序，操作程序中还应包括各个阶段的检验办法。最后，才是具体操作处理这个问题。归纳起来，我们提倡的是：一个对问题的正确的思想认识，一组解决问题的科学方法，一套严密的操作程序。一个人按照这样一种思想方法开展工作，实际上也就是初步具备了科学的态度和正确的思想方法，处理问题的结果也常常比较好，这也是一个人是否具有良好的科学素养的重要标志之一。

科学素养也包含着一个人的科学精神，表现在具有实事求是的科学态度，脚踏实地的工作作风，平常而又良好的心态与科学道德，坚持和维护真理的秉性，献身人类进步事业的精神。

4.4.2 对科学研究活动和一些热点问题的重新认识

本书的读者中，有许多人大学毕业后将走上科技工作岗位，从事科学研究活动。对于年轻一代科技工作者来说，许多人都希望自己能够在科学研究中做出成绩，尽快获得学术界的认可，因而难免急功近利，浪费了最宝贵的时光。为此，在这一章要结束的时候，我们专门选择了两个当前高等学校的热门话题，阐述自己的观点¹⁾。正如读者已经看到的，本书的内容与目前的一些流行的观点往往并

1) 其他关于许多热点问题的讨论，读者可以参考本书作者的其他文章或学术著作。

不一致,我们希望读者能够从中获得某种启迪,对许多问题作更深入的思考。这些内容虽然看似与本书的主题存在一定的距离,但是,它们对于一个大学毕业生在校期间保持良好的心态,端正学习态度,顺利完成学业,在高年级或毕业后顺利进入科学园地,正确地选择自己未来科学生涯的发展道路将是有益的。

1. 关于高等教育改革的技术目标和建设世界一流大学的问题

国外目前的世界一流大学,并非就是科学意义上最理想的大学。为什么呢?要回答这个问题,还得回到办高水平大学的根本目的和需要的认识问题上来:什么是一流的人才?什么是一流的大学?

我们认为,一般地说,一流的人才应该具备下列三个标准:

- ① 具有良好的道德品质和人文素养。
- ② 具有坚实的学科专业基础,专业功底深厚。
- ③ 具有创新意识和科学的思想方法。

如果要求大学培养这样的人才,那么世界上并非现在的一些一流大学就是科学意义上的理想大学。因为,要培养一流的学生,一流的大学不仅要成为教学的中心,科学研究的中心,而且应该成为文化的中心。因为只有成为“三个中心”,在三方面都做得出色的大学才能够成为民族的脊梁,社会的中坚,科学思想库和人民的精神乐园。培养一流的人才,离不开科学研究与文化建设,但一流的科学研究与文化建设水准,未必就能够培养一流的人才,因为培养一流人才还涉及到培养方案是否科学等一系列因素。一所高等学校人才培养的质量更多地要通过学校培养的毕业生对社会的贡献和对未来社会发展与进步的影响体现出来。北京大学之所以在过去的国际大学排名¹⁾中远位于中国其他高等学校的前面,如果单纯从一些重要的科研(成果)指标看似并不合理,主要是因为这所学校在中国过去的近百年里,对中国社会的发展和进步产生了深远的影响。

一流的大学必然要有一流的科学研究。每一个科学工作者都应该明白,科学研究和学术研究工作,一般可以分为四个层次:

- (1) 创立一套理论。
- (2) 开辟一个研究方向。
- (3) 解决某个难题。
- (4) 改进前人的工作,丰富和完善某一领域的知识。

四个层次的工作难易程度不完全具有可比性,但一般地说,对科学技术和学术发展的贡献依次由高到低是学术界比较公认的。作为高水平世界一流大学的客

1) 这样一种排名并不具有权威性。

观要求, 我们目前特别欠缺的是第一、二层次的学术人才, 也就是缺大师级的人才, 缺学术上的挂帅和领军人物。我们的大学, 更缺的是一套培养大师级和学术骨干人才的方式方法。近年来回国的留学生包括“长江学者计划”聘请的人才, 如果按照这个标准去衡量, 未必有多少人的工作真正进入了第一、二层次, 他们大多是在国外学术导师开辟的方向和学术领域内做出了丰富这一领域知识的工作和成果。当然, 应该承认, 这些学者中的大多数在本学科领域还是相当优秀的人才。

一流的大学必然要有一流的教师和一流的教学。第一流的教师一定是第一流的科学家或学者, 但反之则不一定成立。众所周知, 一个学者取得第一流的科研成果, 与其知识基础, 工作条件, 学术环境与氛围, 选择的专业与方向, 个人的努力程度, 机遇与运气等诸多因素之间存在密切联系, 不单纯取决于其学术水平和勤奋程度。这说明, 第一流的教师包括部分已经获得第一流科研成果的科学家和学者, 但同时还包括有实力但因种种原因尚未获得第一流科研成果的一大批教师。没有这样一大批学术底蕴和功力深厚, 忠于职守, 尽心尽力地服务于教学第一线培养人才的科学家, 任何高等学校难于成批地培养高素质的各类专门人才。对于新学人从事科学研究工作, 第三、四层次的研究工作是进行高水平科学研究的前奏, 是一种练习, 在丰富某一专业学科领域知识的同时, 更多地是为未来从事高层次、高难度的学术研究提供必要的经验和积累。事实上, 单纯从培养人才的质量的角度考虑, 对一个教师在科学研究和教学方面投入精力所带来的综合收益方面做深入思考和总结, 研究型大学的科学研究更应该落实在(应用)基础研究方面, 对成果的第一要求是其质量而不是数量, 一般性的成果数量对于高等学校培养高素质的专门人才不具有重要性。培养高质量的人才如同诺贝尔奖的评选一样, 教师队伍的科研成果数量并不是决定人才培养质量最重要的因素, 决定人才培养的主要因素是学校教师队伍的整体素质和学术水平、学校的学风和校风、教师的敬业精神和教学经验、人才培养计划与培养方案的科学性、教学实验设备、图书资料与信息查询支持设施条件等。其中, 教师队伍的整体素质和学术水平是最重要的因素, 直接影响到学风和校风的形成, 具有难以替代的特点。类似地, 众所周知, 从事科学研究工作的人员的数量对于一个国家是否能够获奖并不具有必然的因果关系, 而主要取决于有实力、能够问鼎诺贝尔奖, 目前正在从事重要的科学研究的有效竞争者的数量, 因为他们是未来诺贝尔奖获奖者潜在的候选人群, 而其他许多虽从事科学研究, 经常发表大量一般性科研成果的科研人员, 并不是诺贝尔奖的有效竞争者。我们不难设想, 如果一个学科专业的教师普遍具有第一流教师的素质, 他们都在既从事教学, 又从事重要的、有学术深度的科学研究工作, 那么, 即使这个专业的教师很少出一般性成果, 只要有个别教师

能够拿出震惊学术界的标志性成果，这个学科的人才培养质量和学术影响仍将是巨大的，其学术水准仍是第一流的。众所周知，哥德尔教授一身只发表了少数几篇论文，但他关于形式系统不完备性的论文奠定了他在 20 世纪逻辑学方面不可动摇的学术地位。哥德尔教授一身都在思考许多重大的科学问题，他不愿意写一些没有多大创新和科学意义的文章。可是，由于哥德尔教授的论著数量较少，当年他在普林斯顿大学晋升教授时受到了一些人的阻挠和不公平的对待。其时，同在普林斯顿任教的冯·诺依曼教授深知哥德尔取得的科学成果的份量，他曾为哥德尔鸣不平，说：“这是不公正的。哥德尔早应该成为教授了，因为连我们这些人都是，何况他呢？”这些例子说明，在严肃的学者看来，与重大的标志性成果相比，其他一般性成果即使数量很大，也难与重大标志性成果相匹敌。设想，如果普林斯顿大学的计算机科学系云集了类似于像哥德尔这样的教授，我们能仅仅因为他们科研成果的数量少而武断地认为该系学术水平和教学质量不高吗？

同样，哲学大师维特根斯坦（L.Wittgenstein）一辈子只出了一本书，即他的《逻辑哲学论》，而且薄薄的不超过 80 页，以后一直没有著书和新作发表。直到他去世以后，别人才将他的笔记、讲义整理编辑成书发表。正是大师这本逐浪淘沙，拾贝拣金的小册子，成就了他一辈子的盛名，奠定了他不可动摇的学术地位。大师这样一种集谦逊、浪漫和勇气于一身的对学问的真诚态度，令后人无限敬佩。大师的学术精神，不仅来自于他深厚的积累和深入的思考，来自于他良好的职业操守和道德情怀，也来自于他优雅的气质和恰当的自我认知，更来自于他对科学的虔诚和对真理的信念。大师的风范，是学术界的一杆标尺，既标识涵养和道德的高度，又为后辈学人指明了方向。

一流的大学需要有一流的校园文化。优秀的校园文化应该以校园的理想、科学精神和人文精神作为其重要的支柱，具有较高的文化品位，深深地扎根于本地域的传统文化之中，在继承、发展传统文化和构建新文化的过程中，倡导高尚的品德，培育美丽的心灵，为每一个学人提供自强不息的精神动力和勇往直前的力量源泉，在各方面成为社会的表率，新生活的代表，使大学的主张和人类文明的理想推广开来，渗透到社会的每一个层面，每一个角落。

一流的大学要有一流的管理，还要有宽松的外部社会环境和良好的社会氛围。而到目前为止，我们对于如何办高水平的一流大学还缺乏比较科学的认识基础和比较全面的科学发展观。面对社会从计划经济向市场经济的转轨，面对大学的快速扩张和社会对高等学校多方面的期待等诸多因素，许多大学的管理工作出现顾此失彼的现象，高等学校亟需对未来的发展道路和办学方式方法作出正确的选择。

我们不要一味地迷信国外的大学，因为世界上并没有一所大学真正成为“三个中心”。例如，美国原财政部长萨莫斯出任哈佛大学校长后，对学校一些学科的

名教授很少给学生授课与实质性的指导提出严肃批评，特别批评了一些教授为了掩饰其因学术研究精力投入过多而在教学方面敷衍了事的现象，批评一些教授通过降低优秀标准，给许多学生成绩“A”以便取悦于学生的做法。这些问题不仅使我们看到了国外高水平大学存在的问题，更促使我们思考如何来认识研究型大学中的科学研究与教学等工作，如何平衡大学内各方面的工作。加强国际学术交流十分必要，但拿来主义不能解决中国办高水平世界一流大学的问题。其实，我们应该问：为什么那么多优秀的中国人才进入世界第一流大学，但真正成为大师的学人并不像我们获得中学生奥林匹克比赛金奖的人数那么多呢？有人总是把这一现象的原因归结为国内的应试教育，但这缺乏足够的根据。我们觉得这应该与一些人失去了人生的崇高理想和民族文化的依存有关，与我们的教育工作者和社会氛围中有时充满功利主义的倾向而不是努力去培养更为纯粹意义上的学人和文化人有关。一个人失去了民族文化的根基，失去了坚定的信念和献身祖国的志向，失去了对科学真正的热爱，不可能静下心来从事那些高风险的，有难度的，对科学技术、教育文化发展具有根本性影响的，可能需要自己终身为之奋斗的研究课题，其结果必然使自己在进入到一定的深度之后成了无本之木，悬在空中。可见，只有优秀的文化才能对学者取得更大的成功产生持久而深远的积极影响。由此不难看出，不仅是中国，就是世界上所有的高水平大学，在建设一流的校园文化的工作中，还存在比较明显的不足。

一流的大学不是自封的，更不能将建设世界一流大学的希望寄托在一场社会运动的终极目标上。建设一流大学需要学校师生长期的艰苦奋斗和共同努力，需要全社会的理解和支持，需要一个宽松和谐的外部发展环境，需要一大批脚踏实地，甘于奉献的骨干人才发挥创造性的、独特的作用。我们相信，真正世界一流的大学决不是外国留学生有多少比例，每年科研经费的总量有多少，每年发表的SCI, EI 的学术论文有多少这样一些表面化的东西，而在于学校对科学技术的发展和人类社会的进步究竟做出了什么重大贡献，在传承文明，开拓创新，推动整个社会全面进步的过程中，究竟培养了多少愿意与自己的国家和人民同呼吸、共命运，真正担当起社会发展的中流砥柱的精英人才，在于学校能够真正成为社会发展的思想库，人民群众的思想乐园。

高等教育和高等学校的办学属于科学的范畴。然而，高等教育和高等学校几百年来一直沿用经验方式而非科学方式办学，传授科学知识的方式方法往往并不科学，从未在任何一个学科建立人才培养的科学（理论）体系。虽然经验办学包含科学的成分，但并不等同于科学办学。科学办学是对高等教育界传统办学模式和思维方式的一个挑战，尽管在认识上，人们不难理解：科学办学是经验办学的最高形式，经验办学应该成为科学办学的有益补充。

现在的问题是：什么是科学办学？

我们认为，针对每一个学科，高等学校的科学办学是在对每一个学科的定义、范畴、特点、发展的内在规律和发展趋势，以及学科教育与教学内在规律全面了解的基础上，结合社会实际和学校的定位，围绕高质量人才的属性要求，首先建立学科人才培养的科学理论体系，然后按照科学理论体系来指导办学实践，全面建立人才培养的科学体系。先进的文化能够对人类社会发展和进步产生持久的推动力。为了培养高素质的优秀人才，高水平大学应该建设成为教学、科研、文化三个中心，依托先进的文化体系，在新型产学研合作体系框架内准确融入社会，通过全面建立人才培养的科学体系来实现科学办学，保证科学办学的思想、理念和主张得到全面贯彻和落实。

政府作为社会资源的管理者和社会秩序的维护者，作为高等教育资源的组织者和协调者，在科学办学的理念下，应该按照高等教育发展的一般规律，结合市场经济条件下的国情，在高等教育的办学过程中从法律与行政法规的制定、教育制度与学位体系的创立、政策设计与运行模式的建立，投资与奖惩机制的设立等多个方面，利用行政、法律、经济、政策等多种管理方式和手段进行宏观调控，形成教育资源布局合理，分布均衡，促进各类人才合理、均衡分布，形成分层次分类办学格局，促进充分、合理、有序的竞争，形成市场经济条件下公开、公平、公正的竞争秩序。同时，改善高等学校的外部办学环境，促进学校与科技研发、产业发展、文化建设等部门之间的合理分工与相互合作，帮助高等学校在人才培养、科技研发、文化建设、产业发展、社会服务等方面合理定位，准确融入社会，发挥各高等学校应有的作用，推动社会的全面进步，实现投资与办学效益的最大化。

有关科学办学，内涵发展办高水平大学的更详细的内容，读者可进一步参考相关的文献资料。

2. 关于团队精神与卓尔不群取向的讨论

大学是社会中的一个特殊的机构，其首要任务是培养合格的各类专门人才。大学的一个基本的社会职责是传承文明，开拓创新，培养高素质的各类专门人才。在承担社会职责的过程中，大学与一般的社会机构存在很大的不同，尽管大学的教师也可以在传承文明、开拓创新的过程中发挥自己的聪明才智，在科学研究、技术开发、社会服务、文化建设、推动社会进步等诸多方面做出自己应有的贡献，但所有这一切，都应该是围绕培养高质量的人才这样一个目标而开展的一些必须的或重要的活动。大学落实社会职责的首要方式是通过培养一代又一代的合格毕业生，通过他们对社会的具体贡献来间接落实其社会职责，体现其存在价值。培养高质量的人才，必须要有一支高素质的教师队伍。一所大学科学研究尤其是（应

用)基础研究的水准是反映学校教师队伍业务素质最重要的方面。科学研究重在创新,大学承担社会职责的成效如何,更多地要依靠其培养的学生水准间接地体现出来,否则,这样的大学就不可能成为高水平的大学,甚至不成其为大学。可见,为了培养高素质的各类专门人才,落实大学的社会职责,创新必然成为高水平大学的灵魂。

大学是社会的一个细胞,不可能完全脱离社会而存在。但是,大学是一个特殊的机构,有着自己不同于社会其他机构的任务、性质、理想、追求和处事的原则,社会各个方面不能也不应该将一切看似合理的思想、观念、管理模式和运作方式简单地移植到校园。例如,很多单位招聘人才,要求应聘者具有团队精神。但是,在研究型大学,除非招聘的人才主要从事公共事务管理、教学辅助工作或参与完成某项大的特定的任务,否则研究型大学更看重的应该是一个是否具有独立的科学思想和科学精神,是否具有坚定的信念,崇高的理想和原则性,即是否具有个性一词所包含的正面涵义。而对团队精神,研究型大学应该是较少考虑的因素。因为研究型大学关心创新,尤其是重大创新,研究型大学承担着为人类社会的发展与进步、为本民族的繁荣与富强探索未来发展道路的职责,而在创新研究过程中,真理常常首先掌握在少数人手上。一个学者在重大问题的探索研究中,在向学术深度进军和向科学高峰攀登的过程中,常需要质疑前人的工作和发展道路,深入分析和多视角地对前人的工作进行讨论和质疑,对不同的学术认识可能需要力排众议,独立做出判断和方向性选择,甚至在今后的科研中成为孤独的行路人。研究型大学是探讨学问和追求真理的场所,教师经常需要对各种问题明确回答是还是非,行还是不行,社会也常常期待着大学能够对各种困难的问题给出正确的认知、解释和解决问题的办法,不希望大学沿用政府部门工作人员的一些思想工作方法回避这些问题,更不希望大学套用外交辞令,含糊其词。因为大学是一个国家人民的精神家园,民族意志和道德伦理的最后防线。如果我们不是突出学术标准,而是更多地将各种关系放在重要的地位加以考虑,对大量问题和事务采取机会主义的态度,决定自己如何表态或是沉默,那么,研究型大学就不可能做出重大创新的成果和重大的社会贡献,而只能是将学校的科学研究与人才培养引向平庸,使社会和人民群众对大学和校园内的学人失去信心。如果一所研究型大学将团队精神置于卓尔不群之上,那么,这所学校必然是平庸的,很难真正成为世界一流大学。

一个人如果过分重视人际关系,习惯于在团队精神一边倒的氛围中生活和开展科学研究,很容易在工作中形成思维定式和固定的工作方式,对探索性研究需要的反思、大胆质疑、假设、严密论证、求真、求是等工作方式反而不习惯,不敏感,这可能是许多非常勤奋的科学家没有成为大师级人物,为科学技术或学术

研究做出重大贡献的主要原因之一。

因此,大学需要很好地处理团队精神与卓尔不群两者之间的关系。既不能片面强调团队精神而压制创新,也不能一味地倡导卓尔不群导致校内外纷争不断,或为了标新立异故意与他人唱反调,而是一切要从实际出发,实事求是地根据科学技术发展的规律来开展工作,按照科学的思想方法来开展工作。我们以为,总体上,(重点)研究型大学在办学中应该倡导卓尔不群优先于团队精神,但在完成各项具体的一般性事务中,应该鼓励大家形成良好的团队精神。可见,做好这项工作的关键在于提高学校管理层的素质,创造良好的学术氛围,创造一个积极向上,宽松和谐的工作环境。

最后,我们还要指出,科学研究的投入与科学研究的成果之间存在重要的关系,但并不存在必然的因果关系。历史上,许多重要的科学研究成果往往是在得不到支持或没有支持的情况下做出来的,往往是在一个学者处于相对困难的条件下甚至逆境中做出来的。有时候科研条件好了,高水平的成果反而产生不出来了,说明科学研究及其结果和成效具有极大的不确定性。在中国,首届国家最高科学技术奖的两位获得者吴文俊教授和袁隆平研究员,都是在几乎没有国家经费支持的情况下做出了世界一流的高水平科学研究成果的。这应该引起所有人的深思。

作为本章的结尾,我们将计算科学史上的一段佳话写下来,希望与读者共勉。

历史上,由于冯·诺依曼对存储程序式电子数字计算机的杰出贡献,许多人都推举他为“计算机之父”。然而,正如曾给冯·诺依曼当过助手的弗兰克尔(S. Frankel)所说的,“……我相信他本人从来不会促成这个错误。或许,他可以被恰当地称为助产士,但是他曾向我,并且我肯定他也曾向别人坚决强调:如果不考虑巴贝治等人的工作和他们早先提出的有关计算机和程序设计的一些概念,计算机的基本思想来源于图灵。”

多么诚实的话语啊!

事实上,在冯·诺依曼身边工作过的不少人,如戈德斯坦等都曾不止一次地听说过他本人就图灵对计算科学贡献的高度评价。冯·诺依曼谦虚、诚实、实事求是的科学态度和人文精神,丝毫不损科学大师的光辉形象。相反,虽然他已故去多年,但崇高的精神和风范却永远留在了后人的心中。

在中国,同样不乏学术精深,品德高尚的学者。由厦门大学和贵州大学等学校共同发起的全国计算机科学与技术暑期高级研讨班,是在教育与教学改革最新成果和理念的引导下,为加强全国高等学校计算科学专业师资培训,推进学科专业教育与教学改革而采取的一个重要的措施,得到全国一大批高等学校专家、学者的积极响应和有力支持。在教育部和国内一批高等学校领导和专家的支持下,高级研讨班先后邀请了一批知名的专家学者参与高级研讨班的授课,其中,包括

已年过 60 的陆汝占教授、陆鑫达教授、陈国良教授、康立山教授、罗里波教授、朱洪教授、帅典勋教授等学者。虽然高级研讨班给予专家的报酬很低,但是,专家们毫无怨言,因为他们希望看到在中国的土地上,自己开创的事业和发展的学术方向后继有人,兴旺发达,希望看到国家计算科学事业辉煌的未来。特别值得年轻一代学习的是康立山教授,他虽年近七旬,仍活跃在科研第一线,在自己不断做出创新研究成果的同时,培养和造就了许多优秀人才,带出了一支高水平的科研队伍。近年来康教授大力支持全国计算机科学与技术暑期高级研讨班(贵阳),身体力行,亲自授课,在急功近利,沽名钓誉之风渐盛的今天,犹如寒冬的暖流,温暖着许多后辈莘莘学子的心。

第五章 布尔代数基础

本章主要介绍布尔代数的基本知识,它可以为学生今后学习计算机逻辑代数,数字逻辑,计算机组成原理,二进制运算,以及数理逻辑等课程提供一个基础。

5.1 集合的基本概念与基本运算

由事物或对象组成的集体,无论它们是由其成员通过枚举方式直接表示出来的,还是由其成员所具有的某些本质属性表示出来的,都称为集合。例如,一个班的全体学生构成一个集合;一个工厂里某个车间内的所有机床构成一个集合;一个人的头发、他体内的细胞、基因、他出生以来所到过的国家,所有这些都构成集合。今后,可以用英文字母等符号标记一个集合,如全体运动员的集合可以标记为 A ,也可以标记为 B 。

称两个集合是相等的,如果构成这两个集合的成员完全相同。集合的成员也叫元素。

一个集合 A 中元素的个数叫做集合 A 的基数,记为 $|A|$ 。请注意,□这不是基数的严格的定义。对有穷集合,这样定义基数勉强可行,但对无穷集合不恰当。有关集合基数的概念和知识将来读者会在离散数学或集合论等课程中系统地学习,目前,我们暂且使用这种不严格的直观上的定义。

5.1.1 从属与包含关系

通常用大写英文字母作为集合的名称。若某事物 a 是集合 A 的一个元素,则记为

$$a \in A$$

并说“ a 属于 A ”,或者说“ a 在 A 中”。

若 a 不是集合 A 的元素,则记为

$$a \notin A$$

当以枚举形式表示一个集合时,我们用一个大大括号把这些枚举的元素括起来以表示这个集合。

例 1 {麻雀, 黄鹂, 杜鹃, 白鹭, 红嘴鸥} 是一个由五种鸟组成的集合;
 $\{a, b, c, d, \dots, x, y, z\}$ 是由英语的 26 个小写字母组成的集合。

这种枚举表示方法可以表示所有有穷集合和相当小的集合。特别, 可以表示空集合 $\{\}$ 。空集合是没有元素的集合。空集合在本书中记为 ϕ 。但是, 这种枚举的方式在表示大的集合, 即含有大量元素的集合, 特别是含有无穷多个元素的集合时常常是不适用的。此时, 我们可以使用内涵的表示方式和带省略的枚举形式来表示。带省略的枚举形式也是一种简化了的内涵的表示方式。

例 2 $A = \{x \mid ax^2 + bx + c = 0 \text{ 且 } a, b, c \in R\}$, 其中, R 是实数集。

这表示, A 是由所有一元二次方程 $ax^2 + bx + c = 0$ 的根所组成的集合。

例 3 $B = \{a, b, aa, ab, bb, aaa, aab, abb, bbb, aaaa, \dots\}$ 。

这种带省略的表示形式实际上可表示一些集合元素有某种出现规律的无穷集合。这种出现规律实际上是集合中所有元素所共同拥有的性质, 只是有时候用解析表达式或数学公式表示性质可能比较困难。

集合的表示应当注意以下两点:

- (1) 同一个集合中的元素是互不相同的。
- (2) 集合中的元素之间不规定次序。

与空集合相对应的一个大的集合是所谓的全集合, 即一切事物构成的集合。这是一个虚构的集合, 但它在布尔代数的运算中是有用的。我们用 1 来表示这个虚设的全集合。

考虑两个集合 A 和 B 。若集合 A 的每一个元素也是集合 B 的一个元素, 则称集合 B 包含集合 A , 也称集合 A 包含在集合 B 中, 记为

$$A \subseteq B$$

若 $A \subseteq B$, 则称 A 是 B 的子集合, B 是 A 的母集合。显然, 对任何集合 A , 有 $A \subseteq A$ 。若集合 A 和 B 之间存在 $A \subseteq B$ 且 $A \neq B$, 则称 A 是 B 的真子集合, 记为 $A \subset B$ 。若既有 $A \subseteq B$, 又有 $B \subseteq A$, 则可以得出结论 $A = B$ 。这是因为 A 的元素都是 B 的元素, 而 B 的元素也是 A 的元素, 说明 A, B 中拥有相同的元素, 所以 A 和 B 相同, 故相等。显然, 对任何集合 A , $A \subseteq 1$ 。特别地, $\phi \subseteq 1$ 。

5.1.2 集合的基本运算和基本关系

1. 集合的交、并、补

设 A, B 是两个集合, 定义 A 和 B 的公共元素构成的集合 C 为集合 A 和集合 B 的交集, 简称 A 和 B 的交, 记为 $C = A \cap B$ 。将集合 A 的元素和集合 B 的元素合并在一起, 组成一个新的集合 C , 那么, 这样得到的集合 C 就定义为集合 A 和集合 B 的并集, 简称 A 和 B 的并, 记为 $C = A \cup B$ 。

如果从全集合 1 中取出部分元素构成集合 A , 剩下的元素构成集合 B , 那么,

集合 A 与集合 B 就形成互补关系。我们称集合 B 为集合 A 的补集合, 记为 $A' = B$ 。显然, 也有 $B' = A$ 。而且, 此时下列定理 1 的一系列等式成立。

定理 1 设 A 为一个集合, B 为 A 的补集合, 则

- (1) $A \cap B = \phi$;
- (2) $A \cup B = 1$;
- (3) $\phi' = 1$;
- (4) $1' = \phi$;
- (5) $(A')' = A$;
- (6) $(1')' = 1$;
- (7) $(\phi')' = \phi$ 。

我们选择证明其中的(1), 其余的证明留给读者作为练习。今后, 我们将采用这种方式, 把一部分证明工作留给读者。

证明(1) 用反证法。设 $A \cap B \neq \phi$, 则必存在非空元素 $x \in A \cap B$, 故 $x \in A$ 且 $x \in B$, 此与 B 为 A 的补集合矛盾。所以, $A \cap B = \phi$ 。 $\square$

例 4 设 $A = \{a, b, c, d\}$, $B = \{c, d, e, f\}$, $C = \{e, f, g, h\}$, 则

$$A \cap B = \{c, d\}$$

$$A \cup B = \{a, b, c, d, e, f\}$$

$$A \cap C = \phi$$

$$A \cup C = \{a, b, c, d, e, f, g, h\}。$$

根据定义, 不难证明定理 2。

定理 2 对任意集合 A 和集合 B , 有

- (1) $A \cap B = B \cap A$; (交换律)
- (2) $A \cup B = B \cup A$; (交换律)
- (3) $A \cap A = A$;
- (4) $A \cup A = A$; (幂等律)
- (5) $A \cap A' = \phi$;
- (6) $A \cup A' = 1$;
- (7) $A \cap \phi = \phi$;
- (8) $A \cup \phi = A$ 。

我们选择证明其中的(2), 其余的读者可以自己试着去证明。

证明(2) 我们分两部分来证明。首先证明左边的集合是右边集合的子集合, 然后再证明右边的集合是左边集合的子集合。这样, 若两个集合互为子集合时, 必然相等。

① 设任意元素 $x \in A \cup B$, 则 $x \in A$ 或者 $x \in B$, 也可表述为 $x \in B$ 或者 $x \in A$,

故 $(A \cup B) \subseteq (B \cup A)$;

② 同理可证, $(B \cup A) \subseteq (A \cup B)$ 。

所以, $A \cup B = B \cup A$ 。 □

定理 2(2) 的上述证明方法是证明两个集合相等时最常用的方法, 也是最基本的方法。

2. 集合上的一些基本关系

下面我们在集合互补、包含、交与并的基础上, 首先来建立一些基本关系。

定理 3 设 A, B 为任意集合, 则

(1) $(A \cap B) \subseteq A$;

(2) $(A \cap B) \subseteq B$;

(3) $A \subseteq (A \cup B)$;

(4) $B \subseteq (A \cup B)$ 。

我们选择证明其中的(1), 其余的读者可以自己试着去证明。

证明 (1) 设任意元素 $x \in A \cap B$, 由交的定义知, $x \in A$, 故 $(A \cap B) \subseteq A$ 。 □

定理 4 设 A, B 为任意集合, 则

(1) $A \subseteq B$, 当且仅当 $A \cap B = A$;

(2) $A \cap B = A$, 当且仅当 $A \cup B = B$;

(3) $A \cup B = B$, 当且仅当 $A \subseteq B$ 。

这个定理说明了一个循环等价关系。像这样的等价关系, 今后可用来作等价的概念定义。循环等价关系是数学和计算科学中常见的定理形式。

证明 (1) 我们分两部分来证明。

先证明定理的必要性。假定 $A \subseteq B$, 我们来证明 $A \cap B = A$ 。

① 设任意元素 $x \in A \cap B$, 可知 $x \in A$, 由此可得 $(A \cap B) \subseteq A$;

② 又设任意元素 $x \in A$, 由 $A \subseteq B$ 知 $x \in B$, 故 $x \in A \cap B$, 可得 $A \subseteq (A \cap B)$ 。

综合①, ②, 知 $A \cap B = A$ 。

再证明结论的充分性。假定 $A \cap B = A$, 我们来证明 $A \subseteq B$ 。这是显然的。

因为对于任意元素 $x \in A$, 由 $A \cap B = A$ 知 $x \in A \cap B$, 即由任意 $x \in A$ 可直接导出 $x \in B$, 由此可知 $A \subseteq B$ 。 □

其他的证明留给读者。

上面的证明中, 仍然采用了论证两个集合相等最基本的方法, 即设法先证明等式两边的集合互为子集合。若两个集合互为子集合, 那么, 显然这两个集合是相等的。

证明一个定理, 有时候需要从定义出发推导、论证, 有时候需要引用已知的

结论来简化证明步骤。事实上, 定理 4(1)的证明中第一部分可以直接引用定理 3 的(1)。今后我们应该逐步习惯于简化的证明方式。

定理 5 (De Morgan 律) 设 A, B 为任意集合, 则

$$(1) (A \cap B)' = A' \cup B';$$

$$(2) (A \cup B)' = A' \cap B'.$$

证明 我们只要证明其中的一个就可以了, 因为这两个式子中的任何一个是在另一个在互补关系下的直接结果。下面, 我们证明(1)。

设任意元素 $x \in (A \cap B)'$, 则 $x \notin (A \cap B)$, 因此 $x \notin A$ 或者 $x \notin B$ 。换言之 $x \in A'$ 或者 $x \in B'$, 故 $x \in (A' \cup B')$ 。所以, $(A \cap B)' \subseteq (A' \cup B')$;

反之, 设任意元素 $x \in (A' \cup B')$, 则 $x \in A'$ 或者 $x \in B'$ 。换言之 $x \notin A$ 或者 $x \notin B$, 因此 $x \notin (A \cap B)$, 故 $x \in (A \cap B)'$ 。所以, $(A' \cup B') \subseteq (A \cap B)'$ 。

从而, 可知 $(A \cap B)' = A' \cup B'$ 。 □

定理 6 对任意的集合 $A, \phi \subseteq A$ 。

证明 使用反证法。假设定理不成立, 即存在集合 A , 使得 $\phi \subseteq A$ 不成立。这就是说, 存在元素 $x \in \phi$, 但 $x \notin A$, 这与 ϕ 是空集合相矛盾。故定理成立, 空集合是一切集合的子集合。 □

定理 7 空集合是惟一的。

证明 设 ϕ_1 和 ϕ_2 都是空集合, 由定理 6 知

$$\phi_1 \subseteq \phi_2 \text{ 且 } \phi_2 \subseteq \phi_1,$$

所以 $\phi_1 = \phi_2$ 。 □

3. 集合上的一些运算定律

与中学数学中加法和乘法的结合律、分配律一样, 集合的运算除了满足我们已经看到的幂等律、交换律之外, 也满足结合律和分配律。采用类似于定理 5 的证明方法, 容易证明定理 8。

定理 8 设 A, B, C 是任意的集合, 则

$$(1) A \cup (B \cap C) = (A \cup B) \cap (A \cup C); \text{ (结合律)}$$

$$(2) A \cap (B \cup C) = (A \cap B) \cup (A \cap C); \text{ (结合律)}$$

$$(3) A \cup (B \cap C) = (A \cup B) \cap (A \cup C); \text{ (分配律)}$$

$$(4) A \cap (B \cup C) = (A \cap B) \cup (A \cap C). \text{ (分配律)}$$

证明(3) 设 $x \in A \cup (B \cap C)$, 则 $x \in A$ 或 $x \in (B \cap C)$ 。若是前一种情况, 那么 $x \in (A \cup B)$ 且 $x \in (A \cup C)$, 因而 $x \in ((A \cup B) \cap (A \cup C))$; 若是后一种情况, 那么 $x \in B$ 且 $x \in C$, 因此 $x \in (A \cup B)$ 且 $x \in (A \cup C)$, 也有 $x \in ((A \cup B) \cap (A \cup C))$ 。这就证明了

$$(A \cup (B \cap C)) \subseteq ((A \cup B) \cap (A \cup C))$$

反之, 设 $x \in ((A \cup B) \cap (A \cup C))$, 则 $x \in (A \cup B)$ 且 $x \in (A \cup C)$ 。若 $x \in A$, 那么 $x \in (A \cup (B \cap C))$; 若 $x \notin A$, 那么必有 $x \in B$ 且 $x \in C$, 因此 $x \in (B \cap C)$, 也有 $x \in (A \cup (B \cap C))$ 。这就证明了

$$((A \cup B) \cap (A \cup C)) \subseteq (A \cup (B \cap C))$$

所以, $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$ 。 $\square$

其他的证明留给读者。

从上面的一些公式中我们看到, 为了帮助理解, 使用了不少圆括弧。如果一个式子比较复杂, 圆括弧的数量就可能很多, 这显然是一个累赘。能不能减少圆括弧的数量呢? 答案是肯定的。减少圆括弧的思路可以通过定义“ $\in$ ”, “ $\cap$ ”, “ $\cup$ ”, “ $\subseteq$ ”等运算符之间的优先顺序, 减少不必要的圆括弧, 类似于在四则混合运算中规定“加”、“减”、“乘”、“除”的优先顺序。读者不妨自己试一试。

4. 集合上的其他一些关系

利用集合上的基本关系和基本运算, 可以导出集合上的其他一些关系。

定理 9 设 A, B, C 是任意的集合, 那么下列关系成立:

- (1) 若 $A \subseteq B$ 且 $A \subseteq C$, 则 $A \subseteq (B \cap C)$;
- (2) 若 $A \subseteq C$ 且 $B \subseteq C$, 则 $(A \cup B) \subseteq C$ 。

证明留给读者。

定理 10 设 A, B, C 是任意的集合, 若 $A \subseteq B$, 则

- (1) $(A \cap C) \subseteq (B \cap C)$;
- (2) $(A \cup C) \subseteq (B \cup C)$ 。

证明 (1) 由定理 3 和定理 9 易得。 $\square$

(2) 的证明留给读者。

由定理 3 和定理 10 易证定理 11。

定理 11 设 A, B 是任意的集合, 则有 $A \cap (A \cup B) = A$ 。

证明 依定理 3 有 $(A \cap (A \cup B)) \subseteq A$ 。又 $A = A \cap A$, $A \subseteq (A \cup B)$, 因此, 由定理 10 知 $(A \cap A) \subseteq (A \cap (A \cup B))$, 故 $A \subseteq (A \cap (A \cup B))$ 。所以, $A \cap (A \cup B) = A$ 。 $\square$

定理 12 设 A, B 是任意的集合, 则 $A \subseteq B$, 当且仅当 $A \cap B' = \phi$ 。

证明留给读者。

定理 13 设 A 是一个集合, 那么下列等式成立:

- (1) 若对任意的集合 B , 都有 $A \subseteq B$, 则 $A = \phi$;
- (2) 若对任意的集合 B , 都有 $B \subseteq A$, 则 $A = 1$ 。

证明(1) 由题设, 特别当 $B=\phi$ 时, 有 $A\subseteq\phi$, 但 $\phi\subseteq A$, 所以 $A=\phi$. $\square$

(2) 的证明留给读者。

定理 14 设 A, B 是两个集合, 那么下列等式成立:

(1) 若 $A\cup B=\phi$, 则 $A=\phi$ 且 $B=\phi$;

(2) 若 $A\cap B=1$, 则 $A=1$ 且 $B=1$ 。

证明(1) 因 $A\subseteq A\cup B=\phi$, 故 $A=\phi$; 同理可证, $B=\phi$. $\square$

(2) 的证明留给读者。

5. 集合的差与对称差

除了上面介绍的一些运算之外, 还有两种重要的集合运算也是需要掌握的, 它们是集合的差与对称差。

定义 1 集合 A, B 的差 $A-B$ 定义为一切属于 A 但不属于 B 的元素构成的集合, 即 $A-B=A\cap B'$ 。

对任意集合 A, B , 不难证明定理 15。

定理 15 设 A, B, C 都是集合, 那么下列等式成立:

(1) $1-A=A'$;

(2) $A-B=\phi$, 当且仅当 $A\subseteq B$;

(3) $(A-B)\cup B=A\cup B$;

(4) $C\cap(A-B)=(C\cap A)-(C\cap B)$. (分配律)

证明留给读者。

交关于差是可分配的。但是, 并关于差却不是可分配的。读者可以通过思考和举一反例来认识这一点。

定义 2 集合 A, B 的对称差 $A\oplus B$ 定义为 A 与 B 之差和 B 与 A 之差的并, 即 $A\oplus B=(A-B)\cup(B-A)$ 。

显然, A 与 B 的对称差也可以写成如下的定义形式:

$$A\oplus B=(A\cap B')\cup(B\cap A').$$

由定义 2, 我们还可以得到对称差的其他几种表示形式或等价定义。读者可以根据自己已经掌握的知识和对定义 2 的理解试给出其他的等价定义。

定理 16 若 A, B 是两个集合, 则

(1) $A\oplus B=(A\cup B)\cap(A'\cup B')$;

(2) $A\oplus B=(A\cup B)-(A\cap B)$;

(3) $A\oplus B=A'\oplus B'$ 。

证明留给读者。

作为一种运算, 对称差满足一些定律。

定理 17 设 A, B, C 都是集合, 那么下列等式成立:

- (1) $A \oplus B = B \oplus A$; (交换律)
- (2) $(A \oplus B) \oplus C = A \oplus (B \oplus C)$; (结合律)
- (3) $A \cap (B \oplus C) = (A \cap B) \oplus (A \cap C)$; (分配律)
- (4) $A \cup (B \oplus C) \neq (A \cup B) \oplus (A \cup C)$ 。(并关于对称差不满足分配律)

$$\begin{aligned}
 \text{证明 (3)} \quad A \cap (B \oplus C) &= A \cap ((B - C) \cup (C - B)) \\
 &= (A \cap (B - C)) \cup (A \cap (C - B)) \\
 &= ((A \cap B) - (A \cap C)) \cup ((A \cap C) - (A \cap B)) \\
 &= (A \cap B) \oplus (A \cap C)
 \end{aligned}$$

其他的证明留给读者。 □

定理 18 设 A, B 是两个集合, 那么下列等式成立:

- (1) $A \oplus A = \phi$;
- (2) 若 $A \oplus B = \phi$, 则 $A = B$ 。

证明留给读者。

定义 3 设 A, B 为集合, $A \oplus B$ 的补定义为 A 与 B 的叉, 记为 $A \times B$, 即

$$A \times B = (A \cap B) \cup (A' \cap B').$$

类似于对称差的性质, 易证定理 19。

定理 19 设 A, B, C 都是集合, 那么下列等式成立:

- (1) $A \times B = A' \times B'$;
- (2) $A \times B = B \times A$; (交换律)
- (3) $(A \times B) \times C = A \times (B \times C)$; (结合律)
- (4) $A \times A = 1$;
- (5) $A \times B = A' \oplus B' = A \oplus B'$;
- (6) $A \times B' = A' \times B = A \oplus B$ 。

证明留给读者。

6. 幂集合及其性质

前面已经讨论了集合的一些最基本的内容。众所周知, 到现在为止所讨论的集合的元素都是以现实世界中的个体作为参照对象的。如果考虑构成集合的元素不必是现实世界中某一类事物中的个体, 而允许它是一个集合, 那么, 就可以研究由集合作为元素构成的集合, 这样就产生了幂集合的概念。

定义 4 设 A 为一个集合, 称由 A 的全体子集合构成的集合为 A 的幂集合, 记为 $P(A)$, 即

$$P(A) = \{x \mid x \subseteq A\}$$

由定义可知, $P(\phi) = \{\phi\}$, $P(\{\phi\}) = \{\phi, \{\phi\}\}$ 。

如果将幂集合的定义看作是由一个集合到另一个集合的幂集运算, 那么集合的幂集运算就具有下列性质:

定理 20 设 A, B 是任意两个集合, 则

- (1) $A \subseteq B$, 当且仅当 $P(A) \subseteq P(B)$;
- (2) $P(A \cap B) = P(A) \cap P(B)$;
- (3) $P(A) \cup P(B) \subseteq P(A \cup B)$;
- (4) $P(A - B) \subseteq ((P(A) - P(B)) \cup \{\phi\})$ 。

证明留给读者。

对任意一个有穷集合 A , $P(A)$ 的元素个数即它的基数是多少呢? 利用中学学过的数学知识, 不难证明, $|P(A)| = 2^{|A|}$ 。

在本章开始的时候, 我们曾经不严格地说明了一个集合的基数就是集合中元素的个数。这样的定义性说明对于有穷集合勉强是可以的, 但对于无穷集合就不行了。例如, 对于自然数的集合和偶数集合来说, 这两个集合的基数是相同, 但我们显然不能说这两个集合具有同样多的元素, 因为后者还是前者的真子集合。关于集合基数的严格的定义及其作用, 我们将来留到集合论课程中去讨论, 现在还不需要读者对这个问题进行深究。

5.2 自对偶的公理系统

5.2.1 布尔代数公理系统

在上一节中, 我们学习了集合代数的一些知识。细心的读者可能已经发现, 按照前面章节中关于布尔代数的简单介绍, 好像集合代数与布尔代数之间不是一回事。因为布尔代数是一种逻辑系统, 而上面的内容似乎与逻辑之间缺乏明显的联系。于是, 产生疑问: 为什么要引入集合代数?

下面, 我们作一些通俗的背景介绍。

1. 布尔代数系统的公理

人们在社会生活中, 不仅需要计算, 也需要推理, 需要在许多情况下根据自己已有的知识对许多事情作出判断和推断, 这就产生了逻辑。

古典逻辑是采用自然语言建立概念、断言和推理系统。随着人们对自然现象和社会事物的认识不断深化, 用自然语言来描述事物有时就显得既不严格, 也不方便, 甚至难以理解。哲学的发展急需要一种类似于像数学那样的强有力的工具。

布尔为了使逻辑中的推理能像数学计算一样地进行, 将代数符号引入逻辑系统, 用符号来表示由自然语言描述的各种抽象的断言及其结构, 通过对符号的代数演算来反映用自然语言描述推理的过程及其所作的断言。根据这一思想, 布尔建立了一个二值运算的代数系统, 可以将人类的一类思维推理方式和过程先从自然语言的表述转化为抽象的符号和数学计算, 然后把计算的结果通过解释的方法返回到非形式化的定义和描述中去。这样一来, 逻辑推理就有可能化为一种数学计算, 优点是显而易见的。

为了叙述布尔代数系统, 我们先引入类的概念。

我们讨论的所有对象和事物的全体构成论域, 也叫类。不太严格地讲, 可以将类看作 1, 即全集合。布尔代数是将从 1 看成由所有命题构成的一个特殊的类。

所谓命题, 是指其含义确定, 取值为“真”或“假”值的断言。

布尔将其建立的这个代数系统描述成一个公理系统, 它一共有十条公理, 具体如下:

设 κ 代表类, $\subseteq$ 表示集合的包含关系符号, 那么, 系统的公理为

- (1) 如果 $A \subseteq \kappa$ 且 $B \subseteq \kappa$, 则 $A \cap B \subseteq \kappa$;
- (2) 如果 $A \subseteq \kappa$ 且 $B \subseteq \kappa$, 则 $A \cup B \subseteq \kappa$;
- (3) 存在 ω , $\omega \subseteq \kappa$, 使得对任意的 A , 如果 $A \subseteq \kappa$, 则 $A \cap \omega = A$;
- (4) 存在 ϕ , $\phi \subseteq \kappa$, 使得对任意的 A , 如果 $A \subseteq \kappa$, 则 $A \cup \phi = A$;
- (5) $A \cap B = B \cap A$, $A \subseteq \kappa$, $B \subseteq \kappa$;
- (6) $A \cup B = B \cup A$, $A \subseteq \kappa$, $B \subseteq \kappa$;
- (7) $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$, $A \subseteq \kappa$, $B \subseteq \kappa$;
- (8) $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$, $A \subseteq \kappa$, $B \subseteq \kappa$;
- (9) 对任意的 A , 存在一个 A' , 使得 $A \cup A' = \kappa$, $A \cap A' = \phi$;
- (10) 在类 κ 中至少存在两个不同的元素。

有了公理, 我们只要再给出一些从已知的命题出发得到另一个命题的推理规则, 就能从布尔代数的公理出发, 设法推导我们需要的命题。注意, 这些推理规则有语法推理规则和语义推理规则之分。采用语法推理规则, 推导只是形式上的, 只关心从怎样的一些已知的形式上的命题 (形式前提) 推出一些以前未知的形式上的命题 (形式结论), 而不考虑这些已知的形式上的命题具体是一些什么样的命题, 也不考虑它们和推导出的形式上的命题之间在逻辑真值方面是否一致。采用语义的推理规则, 推导将是语义上的或逻辑上的, 必须考虑这些已知的命题为真时与推导出的命题的真值之间在逻辑上是否一致, 因为我们不能设想前提为假的条件下推出的结论是可靠的, 能够令人信服。由于人类在进行推导时, 总是在前提为真的条件下试图去推出一个命题成立或不成立, 因此, 从推导或计算方便的

角度考虑,我们自然希望语法推理规则与语义推理规则之间保持一致,这样就可以根据实际情况和需要,任选一种方式及其推理规则来进行推导并保证推导的正确性。这就涉及到布尔代数公理系统的系统特征,更具体地说涉及到一个公理系统的推理的可靠性和完备性。这里所谓的可靠性是指凡形式上的推理所反映的前提与结论之间的关系,在语义上的逻辑推理中都是成立的,形式推理可靠地反映了语义上的逻辑推理;所谓完备性是说凡在语义上的逻辑推理中成立的前提与结论之间的关系,形式上的推理也都能反映,即形式推理在反映语义上的逻辑推理时无一遗漏。

鉴于读者的基础知识和本书是面向一年级大学生的原因,我们暂时只能简要地证明布尔代数公理系统的推理的完备性。关于这方面更深入的内容,一年级的大学学习时可能有困难,不过没有关系,待读者系统地学习了数理逻辑课程后,将会接触到这方面的内容,获得深入而又完整的理解。目前,没有必要作深入的探讨。

当我们注意到在集合代数中任意一个集合 A 与 A' 互为补集合, ϕ 的补集合为 1, 1 的补集合为 ϕ 时,如果我们将这里的每一个集合看成是一个命题,该集合的补集合看成是这个命题的否命题,特别, 1 表示一种无论什么情况下都为“真”的恒真命题(也叫永真命题), ϕ 表示一种无论什么情况下都为“假”的恒假命题(也叫永假命题)的话,那么,集合代数就可以看成是一种建立在命题表示基础之上的代数演算系统,所有命题只取“真”、“假”两个值的逻辑代数。一旦真值的“真”用 1 表示,“假”用 0 代表,那么,永真命题和永假命题既可以以其命题符号 1 参加运算,也可以以其值 1 参加运算,对永假命题也一样,两者在形式上完全一致。此外,对其他任意命题,它既可以取值为“真”,也可以取值为“假”。一个命题 A ,如果它取值为“真”,那么它的否命题 A' 就取值为“假”。

下面,我们采用公理系统演算的方法,从公理出发,运用一组推理规则,将与上一节对应的逻辑代数的一些性质和关系推导出来。剩下的一些性质和关系,读者不妨自己动手练习一下,试着证明。

2. 代数系统的形式演算

事实上,对于大多数的逻辑代数的性质和关系都可以从上面给出的公理 3-9 出发推导出来。为了今后使用方便,我们按照上面关于命题取值的说明,先将这几条公理改写成如下形式:

$$[1] \quad A \cap 1 = A;$$

$$[2] \quad A \cup 0 = A;$$

$$[3] \quad A \cap B = B \cap A;$$

$$[4] \quad A \cup B = B \cup A;$$

$$[5] \quad A \cap (B \cup C) = (A \cap B) \cup (A \cap C);$$

$$[6] \quad A \cup (B \cap C) = (A \cup B) \cap (A \cup C);$$

$$[7] \quad A \cap A' = 0;$$

$$[8] \quad A \cup A' = 1.$$

其中, 1 表示永真命题, 0 表示永假命题。

另外, 我们需要对“=”作假定, 它满足自反性、对称性和传递性, 即满足:

(1) 对任意 A , $A=A$; (自反性)

(2) 若 $A=B$, 则 $B=A$; (对称性)

(3) 若 $A=B$, $B=C$, 则 $A=C$ 。(传递性)

由于只考虑“真”、“假”两个值, 所以, 如果 $A=B$, 则 $A'=B'$ 。这就是说, 如果两个命题的值相等, 它们否命题的值也相等。

定理 21 0 和 1 都是惟一的。任意 A , A 的补 (否命题) A' 也是惟一的。

证明 用反证法。设若不然, 0_1 和 0_2 是两个不同的 0, 则

$$0_1 = 0_1 \cup 0_2 \quad (\text{根据}[2])$$

$$= 0_2 \quad (\text{根据}[2], [4])$$

矛盾。这就证明了 $0_1=0_2$, 0 是惟一的。

同理, 可证 1 是惟一的, 对任意 A , A' 也是惟一的。□

注意, 这里的证明中借用了公理, 在每一推导步骤的右边标明了使用的公理的编号, 说明了这样一种推导过程是采用公理化的方法进行的, 并提供了每一步推导的依据。至于反证法, 需要时也可以形式化, 作为逻辑推理 (派生) 规则的一部分。

定理 22 设 A 为一个命题, A' 为 A 的补, 则 $A''=A$ 。

证明 因为 $A \cap A' = 0$, $A \cup A' = 1$ (根据[7], [8]), 这说明 A 是 A' 的补, 但补是惟一的, 所以 $A''=A$ 。□

定理 23 对任意的 A 和 B , 有

$$(1) \quad A \cap A = A; \quad (\text{幂等律})$$

$$(2) \quad A \cup A = A; \quad (\text{幂等律})$$

$$(3) \quad A \cap 0 = 0;$$

$$(4) \quad A \cup 1 = 1.$$

证明 (4) 因为

$$A \cup 1 = (A \cup 1) \cap 1 \quad (\text{根据}[1])$$

$$= 1 \cap (A \cup 1) \quad (\text{根据}[3])$$

$$= (A \cup A') \cap (A \cup 1) \quad (\text{根据}[8])$$

$$=A \cup (A' \cap 1) \quad (\text{根据}[6])$$

$$=(A \cup A') \quad (\text{根据}[1])$$

$$=1 \quad (\text{根据}[8])$$

所以, $A \cup 1 = 1$. □

从上面三个定理可以看出, 我们在推导中隐含地使用了几条推导规则, 它们是

(1) 等式的传递推导。如果已知 $A=B$, $B=C$, 那么, 就可以推出 $A=C$;

(2) 反证法。如果假定一个命题不成立, 由此可以得出矛盾的结论, 那么, 原命题得证;

(3) 代换推导。如果一个命题 A 中的某一命题 B (也可以就是这个命题本身), 用与它相等的命题 C 代换之后 (即用 C 代换 A 中的一个或多个的 B) 得到命题 D , 那么, $A=D$ 。

这些推导规则也可以用形式化的方式来表示, 不过暂时无此必要。至于其他的推导规则, 读者可以从今后的定理证明过程中自己试着去总结。需要指出的是, 读者如果感到从定理证明过程中总结推导规则有困难时, 完全不必担忧, 因为在数理逻辑课程中将会系统地学习这方面的知识。所以这样写教材, 是希望读者在今后的学习中, 特别是在数学基础课程的学习中, 努力去掌握读书、练习、分析、归纳、思考、提炼、总结、体会、探索的读书方式和方法。

定理 24 对任意的 A 和 B , 有

$$(1) A \cap (A \cup B) = A; \quad (\text{吸收律})$$

$$(2) A \cup (A \cap B) = A. \quad (\text{吸收律})$$

证明 (1) 从等式左边出发, 推导出右边。

$$A \cap (A \cup B) = (A \cup 0) \cap (A \cup B) \quad (\text{根据}[2])$$

$$= A \cup (0 \cap B) \quad (\text{根据}[6])$$

$$= A \cup 0 \quad (\text{定理 23(3)})$$

$$= A \quad (\text{根据}[2]) \quad \square$$

定理 25 若对任意的 A , B 和 C , 有 $A \cap B = A \cap C$, $A \cup B = A \cup C$, 则 $B = C$ 。

证明 因为

$$B = B \cap (B \cup A) \quad (\text{由定理 24(1)})$$

$$= B \cap (C \cup A) \quad (\text{由题设})$$

$$= (B \cap C) \cup (B \cap A) \quad (\text{根据}[5])$$

$$= (B \cap C) \cup (C \cap A) \quad (\text{由题设})$$

$$= C \cap (A \cup B) \quad (\text{根据}[5])$$

$$= C \cap (A \cup C) \quad (\text{由题设})$$

$$=C \quad (\text{定理 24(1)})$$

所以, $B=C$. □

定理 26 若对任意的 A, B 和 C , 有 $A \cup B = A \cup C$, $A' \cup B = A' \cup C$, 则 $B=C$.

证明 因为

$$\begin{aligned} (A \cup B) \cap (A' \cup B) &= B \cup (A \cap A') \quad (\text{根据[6]}) \\ &= B \cup 0 \quad (\text{根据[7]}) \\ &= B \quad (\text{根据[2]}) \end{aligned}$$

又有

$$\begin{aligned} (A \cup B) \cap (A' \cup B) &= (A \cup C) \cap (A' \cup C) \quad (\text{由题设}) \\ &= C \cup (A \cap A') \quad (\text{根据[6]}) \\ &= C \cup 0 \quad (\text{根据[7]}) \\ &= C \quad (\text{根据[2]}) \end{aligned}$$

所以, $B=C$. □

定理 27 若对任意的 A, B 和 C , 有 $A \cap B = A \cap C$, $A' \cap B = A' \cap C$, 则 $B=C$.

证明 留给读者。

下面我们证明并的结合律, 同时, 请读者自己补上每一步推理的依据。

定理 28 对任意的 A, B 和 C , 有

- (1) $A \cap (B \cap C) = (A \cap B) \cap C$;
- (2) $A \cup (B \cup C) = (A \cup B) \cup C$ 。

证明 (2) 设 $L = A \cup (B \cup C)$, $M = (A \cup B) \cup C$, 那么

$$A \cap L = A \cap (A \cup (B \cup C))$$

$$=A$$

且

$$\begin{aligned} A \cap M &= A \cap ((A \cup B) \cup C) \\ &= (A \cap (A \cup B)) \cup (A \cap C) \\ &= A \cup (A \cap C) \\ &= A \end{aligned}$$

故

$$A \cap L = A \cap M$$

又

$$\begin{aligned} A' \cap L &= A' \cap (A \cup (B \cup C)) \\ &= (A' \cap A) \cup (A' \cap (B \cup C)) \\ &= 0 \cup (A' \cap (B \cup C)) \\ &= (A' \cap (B \cup C)) \end{aligned}$$

$$=(A' \cap B) \cup (A' \cap C)$$

且

$$\begin{aligned} A' \cap M &= A' \cap ((A \cup B) \cup C) \\ &= (A' \cap (A \cup B)) \cup (A' \cap C) \\ &= ((A' \cap A) \cup (A' \cap B)) \cup (A' \cap C) \\ &= (0 \cup (A' \cap B)) \cup (A' \cap C) \\ &= (A' \cap B) \cup (A' \cap C) \end{aligned}$$

故

$$A' \cap L = A' \cap M = A$$

于是, 由定理 27, 可得 $L=M$ 。这就证明了并的结合律。 $\square$

定理 29 若对任意的 A 和 B , 有 $A \cap B = A \cup B$, 则 $A=B$ 。

证明 因为

$$\begin{aligned} A &= A \cap (A \cup B) && \text{(定理 24(1))} \\ &= A \cap (A \cap B) && \text{(由题设)} \\ &= (A \cap A) \cap B && \text{(结合律)} \\ &= A \cap B && \text{(定理 23(1))} \\ &= A \cap (B \cap B) && \text{(定理 23(1))} \\ &= (A \cap B) \cap B && \text{(结合律)} \\ &= (A \cup B) \cap B && \text{(由题设)} \\ &= B && \text{([3], 定理 24(1))} \end{aligned}$$

所以, $A=B$ 。 $\square$

下面证明 De Morgan 律, 也请读者自己补上每一步推理的依据。

定理 30 (De Morgan 律) 设有任意的 A, B , 则

$$(1) (A \cap B)' = A' \cup B';$$

$$(2) (A \cup B)' = A' \cap B'.$$

证明 (1) 因为

$$\begin{aligned} (A \cap B) \cap (A' \cup B') &= (A \cap B \cap A') \cup (A \cap B \cap B') \\ &= ((A \cap A') \cap B) \cup (A \cap (B \cap B')) \\ &= (0 \cap B) \cup (A \cap 0) \\ &= 0 \cup 0 \\ &= 0 \end{aligned}$$

而

$$\begin{aligned} (A \cap B) \cup (A' \cup B') &= (A \cup A' \cup B') \cap (B \cup A' \cup B') \\ &= ((A \cup A') \cup B') \cap ((B \cup B') \cup B') \end{aligned}$$

$$\begin{aligned}
 &= (1 \cup B') \cap (A' \cup 1) \\
 &= 1 \cap 1 \\
 &= 1
 \end{aligned}$$

由定理 21 和定理 22 知, $A' \cup B'$ 是 $A \cap B$ 的补, 故(1)成立。 $\square$

在上一节定理 4 的证明中, 曾经证明了如下三个循环等价关系, 并提到这些关系可用来进一步定义概念。现在, 我们就来叙述这些关系的应用。

- (1) $A \subseteq B$, 当且仅当 $A \cap B = A$;
- (2) $A \cap B = A$, 当且仅当 $A \cup B = B$;
- (3) $A \cup B = B$, 当且仅当 $A \subseteq B$ 。

细心的读者可能已经注意到, [1]—[8]并没有与“ $\subseteq$ ”有关的公理, 那么, 上一节中与“ $\subseteq$ ”有关的定理怎么在公理系统中推导出来呢?

注意到循环等价关系(1)—(3)中, $A \subseteq B$ 分别等价于 $A \cap B = A$ 和 $A \cup B = B$, 因此, 只要任取其一就可以引入 $A \subseteq B$ 的定义, 另一个可以推导出来。这样, 实际上也就引入了符号“ $\subseteq$ ”, 从而, 上一节中与“ $\subseteq$ ”有关的定理也可望在公理系统中获得证明。

下面, 我们引入“ $\subseteq$ ”关系, 并导出有关的定理。推导的依据, 全部留给读者。

定义 5 设有任意的 A, B , 定义 $A \subseteq B$ (读作 A 包含于 B 中, 或 B 包含 A) 为 $A \cap B = A$ 。

定理 31 设有任意的 A, B , 若 $A \subseteq B$, 则 $A \cap B = A$, 当且仅当 $A \cup B = B$ 。

证明 由题设 $A = A \cap B$ 即 $A \subseteq B$, 知

$$\begin{aligned}
 A \cup B &= (A \cap B) \cup B \\
 &= (A \cup B) \cap (B \cup B) \\
 &= (A \cup B) \cap B \\
 &= B
 \end{aligned}$$

反过来, 设 $A \cup B = B$, 也有

$$\begin{aligned}
 A \cap B &= A \cap (A \cup B) \\
 &= (A \cap A) \cup (A \cap B) \\
 &= A \cup (A \cap B) \\
 &= A
 \end{aligned}$$

$\square$

推论 1 对任意的 A , $0 \subseteq A$, $A \subseteq 1$ 。

定理 32 设有任意的 A, B 和 C , 若 $A \subseteq B$ 且 $B \subseteq C$, 则 $A \subseteq C$ 。

证明 依定义 5, 我们只要证明: 若 $A = A \cap B$ 且 $B = B \cap C$, 则 $A = A \cap C$ 即可。由题设, 因为

$$\begin{aligned}
 A \cap C &= (A \cap B) \cap C \\
 &= A \cap (B \cap C) \\
 &= A \cap B \\
 &= A
 \end{aligned}$$

所以, $A = A \cap C$. □

定理 33 设有任意的 A, B , 则

(1) $A \subseteq B$, 当且仅当 $A \cap B' = 0$;

(2) $A \subseteq B$, 当且仅当 $A' \cup B = 1$ 。

证明 (1) 由题设 $A \subseteq B$ 知, $A = A \cap B$. 因为

$$\begin{aligned}
 A \cap B' &= (A \cap B) \cap B' \\
 &= A \cap (B \cap B') \\
 &= A \cap 0 \\
 &= 0
 \end{aligned}$$

反过来, 如果 $A \cap B' = 0$, 则

$$\begin{aligned}
 A &= A \cap 1 \\
 &= A \cap (B \cup B') \\
 &= (A \cap B) \cup (A \cap B') \\
 &= (A \cap B) \cup 0 \\
 &= A \cap B
 \end{aligned}$$

此即 $A \subseteq B$. □

定理 34 对任意的 A, B , 有

(1) $(A \cap B) \subseteq A$;

(2) $A \subseteq (A \cup B)$ 。

证明 (1) 因为

$$\begin{aligned}
 (A \cap B) \cap A &= (A \cap A) \cap B \\
 &= A \cap B
 \end{aligned}$$

所以, 由定义 5, $(A \cap B) \subseteq A$. □

定理 35 对任意的 A, B , 若 $A \subseteq B$ 且 $B \subseteq A$, 则 $A = B$.

证明 由题设 $A \subseteq B$ 且 $B \subseteq A$, 据定义 5 和交换律立即可得

$$\begin{aligned}
 A &= A \cap B \\
 &= B \cap A \\
 &= B
 \end{aligned}$$
□

定理 36 对任意的 A, B , 若 $A \subseteq B$ 且 $A \subseteq C$, 则 $(A \subseteq B) \cap C$.

证明 因为

$$\begin{aligned}
 A \cap (B \cap C) &= (A \cap B) \cap C \\
 &= A \cap C \\
 &= A
 \end{aligned}$$

所以, $A \subseteq (B \cap C)$. □

下面, 我们仿照上一节集合的差和对称差的概念引入差和对称差的概念, 然后继续给出若干定理. 至于定理的证明, 则留给读者作为练习.

定义 6 设有任意的 A, B , 定义差 $A - B$ (读作 A 与 B 的差) 为 $A \cap B'$.

定理 37 对任意的 A, B , 若 $A \subseteq C$ 且 $B \subseteq C$, 则 $(A \cup B) \subseteq C$.

定理 38 对任意的 A, B 和 C , 若 $A \subseteq B$, 则 $(A \cap C) \subseteq (B \cap C)$ 且 $(A \cup C) \subseteq (B \cup C)$.

定理 39 设有任意的 A , 则

(1) 若对任意的 B , 都有 $B \subseteq A$, 则 $A = 1$;

(2) 若对任意的 B , 都有 $A \subseteq B$, 则 $A = 0$.

定理 40 对任意的 A, B, C , 下列关系成立:

(1) $A = 1 - A'$;

(2) $A - B = 0$, 当且仅当 $A \subseteq B$;

(3) $(A - B) \cup B = A \cup B$;

(4) 若 $A \cap B = 0$, 则 $A - B = A$;

(5) $C \cap (A - B) = (C \cap A) - (C \cap B)$.

定义 7 设有任意的 A, B , 定义对称差 $A \oplus B$ (读作 A 与 B 的对称差) 为 $(A - B) \cup (B - A)$.

定理 41 对任意的 A, B 和 C , 下列关系成立:

(1) $A \oplus B = (A \cup B) \cap (A' \cup B')$;

(2) $A \oplus B = B \oplus A$;

(3) $(A \oplus B) \oplus C = A \oplus (B \oplus C)$;

(4) $A \oplus B = (A \cup B) - (A \cap B)$;

(5) $A \oplus B = A' \oplus B'$;

(6) $A \cap (B \oplus C) = (A \cap B) \oplus (A \cap C)$; (分配律)

(7) $A \cup (B \oplus C) \neq (A \cup B) \oplus (A \cup C)$; (并关于对称差不满足分配律)

(8) $A \oplus A = 0$;

(9) 若 $A \oplus B = 0$, 则 $A = B$.

定义 8 设有任意的 A, B , 定义 $A \oplus B$ 的补 $A \times B$ (读作 A 与 B 的叉) 为 $(A \cap B) \cup (A' \cap B')$.

定理 42 对任意的 A, B 和 C , 下列关系成立:

- (1) $A \times B = A' \times B'$;
- (2) $A \times B = B \times A$; (交换律)
- (3) $(A \times B) \times C = A \times (B \times C)$; (结合律)
- (4) $A \times A = 1$;
- (5) $A \times B = A' \oplus B = A \oplus B'$;
- (6) $A \times B' = A' \times B = A \oplus B$ 。

5.2.2* 标准形式和公理系统的完备性

由命题 $A, B, C, \dots$ 有限次使用 $\cap, \cup, ', -, \oplus, \times$ 等所构成的任何式子, 都可以简化为下列标准形式:

$$S_1 \cap S_2 \cap S_3 \cap \dots \cap S_{n-1} \cap S_n$$

其中, 每个 $S_i (1 \leq i \leq n)$ 是一些不再含有 $\cap, \cup, ', -, \oplus, \times$ 的命题或否命题有限次使用 $\cup$ 而成, 或者简化为第二标准形式:

$$T_1 \cup T_2 \cup T_3 \cup \dots \cup T_{n-1} \cup T_n$$

其中, 每个 $T_i (1 \leq i \leq n)$ 是一些不再含有 $\cap, \cup, ', -, \oplus, \times$ 的命题或否命题有限次使用 $\cap$ 而成。

利用下列等式对任意给定的一个命题公式进行替换, 确实能够实现化为标准形式的目标。

- (1) $A = (A')'$;
- (2) $(A \cap B)' = (A' \cup B')$;
- (3) $(A \cup B)' = (A' \cap B')$;
- (4) $A \cap B = B \cap A$;
- (5) $A \cup B = B \cup A$;
- (6) $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$;
- (7) $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$;
- (8) $A \cap (B \cap C) = (A \cap B) \cap C$;
- (9) $A \cup (B \cup C) = (A \cup B) \cup C$ 。

事实上, 前面所有定理中的关系或等式, 均可用于化简。将一个式子化简在实际工作中有着重要的应用。例如, 许多数字逻辑系统的设计, 从对系统功能最直观的认识和思想出发, 常常能够很快设计出正确的逻辑电路。然而, 正确的逻辑电路不一定是简单的逻辑电路。为了简化电路, 降低生产成本, 提高器件运算速度, 就需要运用布尔代数的一些关系进行化简。这在表面上看起来好像是一个逻辑电路的问题, 而实质上是一个布尔代数中的数学问题。

上面给出的各项定理, 都是采用形式推导的方式给出的, 我们并没有去关心

定理中的那些命题都代表一些什么样的命题, 更没有考察它们是“真”是“假”, 或有时候为“真”, 有时候为“假”, 以及前提“真”时, 结论是否为“真”。所有这些推导都只是形式上的, 即根据公理和推导规则, 从这种形式可以推出那种形式。

但是, 这样一种推导显然不能令人满意, 因为我们所关心的是这样一些推导: 当前提为“真”时, 结论也为“真”。于是, 需要对每一个命题作出解释, 这种解释, 并不是去关心每一个命题究竟表示一个什么样的具体命题, 而只是考察该命题的真值。因为, 一旦一个命题 A 被具体对应到一个实实在在的命题, 如“上海动物园内现生活着 20 只大熊猫”, 那么, A 的真值也就被确定了。

设 $f(A_1, A_2, \dots, A_n)$ 表示任意一个由命题 $A_1, A_2, \dots, A_n$ 通过有限次使用 $'$, $\cap$, $\cup$ 所构成的式子, 则用 $f(A_1, A_2, \dots, A_{i-1}, 0, A_{i+1}, \dots, A_n)$ 表示以 0 代替 $f(A_1, A_2, \dots, A_n)$ 的 A_i 而得到的式子。

定理 43 任意式子 $f(A_1, A_2, \dots, A_n)$ 都可以由 $A_1, A_2, \dots, A_n$ 及其补, 连同着仅仅包含 0, 1 的式子而得到。

证明 我们首先证明

$$(Th) \quad f(A) = (A \cup f(0)) \cap (A' \cup f(1))$$

设 $f(A) = A$, 因为

$$\begin{aligned} A &= A \cap 1 \\ &= (A \cup 0) \cap (A' \cup 1) \\ &= (A \cup f(0)) \cap (A' \cup f(1)) \end{aligned}$$

所以, 当 $f(A)$ 为只包含一个命题的式子时, 结论成立。而且, 当 (Th) 对 $f(A)$ 成立时, 对 $f'(A)$ 即 $f(A)$ 的补也成立, 理由是

$$\begin{aligned} f'(A) &= (A' \cap f'(0)) \cup (A \cap f'(1)) \\ &= (A' \cap 1) \cup (A \cap 0) \\ &= A' \cup 0 \\ &= 0 \cup A' \\ &= (A \cup 1) \cap (A' \cup 0) \\ &= (A \cup f'(0)) \cap (A' \cup f'(1)) \end{aligned}$$

其次, 设 $f(A)$ 满足 $f(A) = (A \cup f(0)) \cap (A' \cup f(1))$, $g(A)$ 满足 $g(A) = (A \cup g(0)) \cap (A' \cup g(1))$, 即 (Th) 对 $f(A)$, $g(A)$ 成立, 则

$$\begin{aligned} &(A \cup f(0)) \cap (A' \cup f(1)) \cap (A \cup g(0)) \cap (A' \cup g(1)) \\ &= (A \cup f(0)) \cap (A \cup g(0)) \cap (A' \cup f(1)) \cap (A' \cup g(1)) \\ &= (A \cup (f(0) \cap g(0))) \cap (A' \cup (f(1) \cap g(1))) \end{aligned}$$

类似地, 有

$$\begin{aligned}
& ((A \cup f(0)) \cap (A' \cup f(1))) \cup ((A \cup g(0)) \cap (A' \cup g(1))) \\
&= ((A \cup 0) \cap (A' \cup 1)) \cup ((A \cup 0) \cap (A' \cup 1)) \\
&= ((A \cup 0) \cup ((A \cup 0) \cap (A' \cup 1))) \cap ((A' \cup 1) \cup ((A \cup 0) \cap (A' \cup 1))) \\
&= ((A \cup 0) \cup (A \cup 0)) \cap ((A \cup 0) \cup (A' \cup 1)) \cap ((A' \cup 1) \cup (A \cup 0)) \cap ((A' \cup 1) \cup (A' \cup 1)) \\
&= (A \cup 0 \cup 0) \cap (A \cup 0 \cup A' \cup 1) \cap (A' \cup 1 \cup A \cup 0) \cap (A' \cup 1) \\
&= (A \cup 0 \cup 0) \cap (A \cup A' \cup 1) \cap (A \cup A' \cup 1) \cap (A \cup A' \cup 1) \\
&= (A \cup 0 \cup 0) \cap (A \cup A' \cup 1) \\
&= (A \cup 0 \cup 0) \cap (A' \cup 1 \cup 1) \\
&= (A \cup f(0) \cup g(0)) \cap (A' \cup f(1) \cup g(1))
\end{aligned}$$

这就是说, (Th)对 $f(A)$ 和 $g(A)$ 的 $\cap$ 和 $\cup$ 都成立。因为每个式子都是由 $'$, $\cap$, $\cup$ 构成的, 所以(Th)对所有式子 $f(A)$ 都成立。特别, 当 $f(A)=I$, $f(A)$ 是一个不含 A 的式子时, (Th)对 $f(A)$ 也成立。因为

$$\begin{aligned}
(A \cup I) \cap (A' \cup I) &= (A \cap A') \cup I \\
&= I
\end{aligned}$$

现在, 设 $f(A, A_1, A_2, \dots, A_n)$ 是由 $A, A_1, A_2, \dots, A_n$ 构成的式子, 我们来证明

$$(Th^*) \quad f(A, A_1, A_2, \dots, A_n) = (A \cup f(0, A_1, A_2, \dots, A_n)) \cap (A' \cup f(1, A_1, A_2, \dots, A_n)).$$

分三种情形证明:

(1) $f(A, A_1, A_2, \dots, A_n)$ 中不包含 A , 令 $f(A, A_1, A_2, \dots, A_n)=I$ 。于是由

$$(A \cup f(0, A_1, A_2, \dots, A_n)) \cap (A' \cup f(1, A_1, A_2, \dots, A_n)) = I$$

知(Th*)成立;

(2) 设 $f(A, A_1, A_2, \dots, A_n) = A \cap g(A_1, A_2, \dots, A_n)$, 因为 $g(A_1, A_2, \dots, A_n)$ 中不含 A , 所以由

$$\begin{aligned}
f(A, A_1, A_2, \dots, A_n) &= (A \cup f(0, A_1, A_2, \dots, A_n)) \cap (A' \cup f(1, A_1, A_2, \dots, A_n)) \\
&= (A \cup (0 \cap g(A_1, A_2, \dots, A_n))) \cap (A' \cup (1 \cap g(A_1, A_2, \dots, A_n))) \\
&= A \cap (A' \cup g(A_1, A_2, \dots, A_n)) \\
&= (A \cap A') \cup (A \cap g(A_1, A_2, \dots, A_n)) \\
&= (A \cap g(A_1, A_2, \dots, A_n))
\end{aligned}$$

知(Th*)对 $f(A, A_1, A_2, \dots, A_n) = A \cap g(A_1, A_2, \dots, A_n)$ 成立;

(3) 设 $f(A, A_1, A_2, \dots, A_n) = A \cup g(A_1, A_2, \dots, A_n)$, 因为 $g(A_1, A_2, \dots, A_n)$ 中不含 A , 所以由

$$\begin{aligned}
f(A, A_1, A_2, \dots, A_n) &= (A \cup f(0, A_1, A_2, \dots, A_n)) \cap (A' \cup f(1, A_1, A_2, \dots, A_n)) \\
&= (A \cup (0 \cup g(A_1, A_2, \dots, A_n))) \cap (A' \cup (1 \cup g(A_1, A_2, \dots, A_n))) \\
&= (A \cup g(A_1, A_2, \dots, A_n)) \cap 1
\end{aligned}$$

$$=(A \cup g(A_1, A_2, \dots, A_n))$$

知 (Th^*) 对 $f(A, A_1, A_2, \dots, A_n)=A \cup g(A_1, A_2, \dots, A_n)$ 成立。

综上所述, 如果 (Th^*) 对某些式子成立, 则对于它们有限次的补、并、交构成的式子也成立。所以, (Th^*) 对一切由命题有限次使用', $\cap$, $\cup$ 构成的式子都成立。□

根据定理 41, 我们已经知道, 依照 (Th^*) , 可以逐步将所有式子表示成需要的形式。首先, 对于 $f(A, B)$, 应用 (Th) 和 (Th^*) , 有

$$f(A, B)=(A \cup f(0, B)) \cap (A' \cup f(1, B))$$

但 $f(0, B)$ 和 $f(1, B)$ 是至多只含有 B 的式子, 类似地有

$$f(0, B)=(B \cup f(0, 0)) \cap (B' \cup f(0, 1))$$

$$f(1, B)=(B \cup f(1, 0)) \cap (B' \cup f(1, 1))$$

所以

$$f(A, B)=(A \cup B \cup f(0, 0)) \cap (A \cup B' \cup f(0, 1)) \cap (A' \cup B \cup f(1, 0)) \cap (A' \cup B' \cup f(1, 1))$$

同理, 对 $f(A, B, C)$, 有

$$\begin{aligned} f(A, B, C) &= (A \cup B \cup C \cup f(0, 0, 0)) \cap (A \cup B \cup C' \cup f(0, 0, 1)) \\ &\quad \cap (A \cup B' \cup C \cup f(0, 1, 0)) \cap (A \cup B' \cup C' \cup f(0, 1, 1)) \\ &\quad \cap (A' \cup B \cup C \cup f(1, 0, 0)) \cap (A' \cup B \cup C' \cup f(1, 0, 1)) \\ &\quad \cap (A' \cup B' \cup C \cup f(1, 1, 0)) \cap (A' \cup B' \cup C' \cup f(1, 1, 1)) \\ &\quad \vdots \end{aligned}$$

对上面的每个式子 $f(I_1, I_2, \dots, I_n)$, 当 $I_i=0$ 或 1 时, $(1 \leq i \leq n)$, $f(I_1, I_2, \dots, I_n)$ 是等于 0 或 1 的。因此, 对于

$$(Th) \quad f(A)=(A \cup f(0)) \cap (A' \cup f(1))$$

如果取遍 $f(0)$, $f(1)$ 所有可能的值的组合, 就可得到所有只含有 A 的不同的化简式子:

$$(A \cup 0) \cap (A' \cup 0)=A \cap A'=0$$

$$(A \cup 0) \cap (A' \cup 1)=A \cap 1=A$$

$$(A \cup 1) \cap (A' \cup 0)=1 \cap A'=A'$$

$$(A \cup 1) \cap (A' \cup 1)=1 \cap 1=1$$

这就是说, 每一个只含有 A 的式子, 无论多么复杂, 都可以简化为 0 , 1 , A , A' 中的一个, 它们相等。

同理, 每个只含有 A, B 的式子与下列 16 个式子中的一个相等:

$$(A \cup B \cup 0) \cap (A \cup B' \cup 0) \cap (A' \cup B \cup 0) \cap (A' \cup B' \cup 0)=0$$

$$(A \cup B \cup 0) \cap (A \cup B' \cup 0) \cap (A' \cup B \cup 0) \cap (A' \cup B' \cup 1)=A \cap B$$

$$\begin{aligned}
&(A \cup B \cup 0) \cap (A \cup B' \cup 0) \cap (A' \cup B \cup 1) \cap (A' \cup B' \cup 0) = A - B \\
&(A \cup B \cup 0) \cap (A \cup B' \cup 0) \cap (A' \cup B \cup 1) \cap (A' \cup B' \cup 1) = A \\
&(A \cup B \cup 0) \cap (A \cup B' \cup 1) \cap (A' \cup B \cup 0) \cap (A' \cup B' \cup 0) = B - A \\
&(A \cup B \cup 0) \cap (A \cup B' \cup 1) \cap (A' \cup B \cup 0) \cap (A' \cup B' \cup 1) = B \\
&(A \cup B \cup 0) \cap (A \cup B' \cup 1) \cap (A' \cup B \cup 1) \cap (A' \cup B' \cup 0) = A + B \\
&(A \cup B \cup 0) \cap (A \cup B' \cup 1) \cap (A' \cup B \cup 1) \cap (A' \cup B' \cup 1) = A \cup B \\
&(A \cup B \cup 1) \cap (A \cup B' \cup 0) \cap (A' \cup B \cup 0) \cap (A' \cup B' \cup 0) = A' \cup B' \\
&(A \cup B \cup 1) \cap (A \cup B' \cup 0) \cap (A' \cup B \cup 0) \cap (A' \cup B' \cup 1) = (A + B)' \\
&(A \cup B \cup 1) \cap (A \cup B' \cup 0) \cap (A' \cup B \cup 1) \cap (A' \cup B' \cup 0) = B' \\
&(A \cup B \cup 1) \cap (A \cup B' \cup 0) \cap (A' \cup B \cup 1) \cap (A' \cup B' \cup 1) = A \cup B' \\
&(A \cup B \cup 1) \cap (A \cup B' \cup 1) \cap (A' \cup B \cup 0) \cap (A' \cup B' \cup 0) = A' \\
&(A \cup B \cup 1) \cap (A \cup B' \cup 1) \cap (A' \cup B \cup 0) \cap (A' \cup B' \cup 1) = A' \cup B \\
&(A \cup B \cup 1) \cap (A \cup B' \cup 1) \cap (A' \cup B \cup 1) \cap (A' \cup B' \cup 0) = A' \cup B' \\
&(A \cup B \cup 1) \cap (A \cup B' \cup 1) \cap (A' \cup B \cup 1) \cap (A' \cup B' \cup 1) = 1
\end{aligned}$$

由此和前面建立的布尔代数公理系统及其形式演算, 我们有如下的公理系统完备性定理。

定理 44 如果对于 A_i 的值 0 或 1, f, g 两个式子相等, 即

$$f(A_1, A_2, \dots, A_n) = g(A_1, A_2, \dots, A_n)$$

则 $f(A_1, A_2, \dots, A_n) = g(A_1, A_2, \dots, A_n)$ 可以由公理[1]—[8]来证明。

证明 我们只需证明 f 和 g 含两个命题的情形即可, 其他情况可以类似证明。

设 $f(0,0)=g(0,0)$, $f(0,1)=g(0,1)$, $f(1,0)=g(1,0)$, $f(1,1)=g(1,1)$, 因此

$$\begin{aligned}
f(A,B) &= (A \cup B \cup f(0,0)) \cap (A \cup B' \cup f(0,1)) \cap (A' \cup B \cup f(1,0)) \cap (A' \cup B' \cup f(1,1)) \\
&= (A \cup B \cup g(0,0)) \cap (A \cup B' \cup g(0,1)) \cap (A' \cup B \cup g(1,0)) \cap (A' \cup B' \cup g(1,1)) \\
&= g(A,B)
\end{aligned}$$

由于 5.1 中引入的集合, 集合的补, 空集合和全集合与 5.2 节中的命题, 命题的补, 0 和 1 对应, 因此, 凡能由 4.1 节的方法建立起来的集合代数的每个关系都可以由公理[1]—[8]来证明。公理[1]—[8]作为一个集合代数的解释是完备的。□

因为 5.2 节通过公理系统及其演算建立的布尔代数中的各项定理是(能够)反映 5.1 节中建立的集合代数的各个关系的, 所以, 凡是能由形式推导得出的 5.2 节中的定理, 也可以使用上面解释的方法给出证明。在许多简单的情况下, 这样的证明只需要画出一张真值表即可。

例 5 用解释的方法证明关系:

$$(A' \cap B) \cup B' = A' \cup B'.$$

证明 我们作出表 5.1。

表 5.1

A	B	A'	B'	$A' \cap B$	$(A' \cap B) \cup B'$	$A' \cup B'$
0	0	1	1	0	1	1
0	1	1	0	1	1	1
1	0	0	1	0	1	1
1	1	0	0	0	0	0

从表中可以看出, 无论 A, B 取值组合怎样, $(A' \cap B) \cup B'$ 与 $A' \cup B'$ 的值总相等, 说明两个式子相等, 即 $(A' \cap B) \cup B' = A' \cup B'$ 。□

习 题

1. 证明 $A \subseteq B$ 与 $A' \cup B = 1$ 等价。

2. 证明

(1) $(A \cap B \cap C)' = A' \cup B' \cup C'$;

(2) $(A \cup B \cup C)' = A' \cap B' \cap C'$;

(3) $(A_1 \cap A_2 \cap \cdots \cap A_n)' = A_1' \cup A_2' \cup \cdots \cup A_n'$;

(4) $(A_1 \cup A_2 \cup \cdots \cup A_n)' = A_1' \cap A_2' \cap \cdots \cap A_n'$ 。

3. 证明

(1) 若 $X \subseteq A$ 且 $X \subseteq A'$, 则 $X = 0$;

(2) 若 $A \subseteq X$ 且 $A' \subseteq X$, 则 $X = 1$;

(3) 若 $A \subseteq C$ 且 $B \subseteq D$, 则 $A \cup B \subseteq C \cup D$;

(4) 若 $A \subseteq C$ 且 $B \subseteq D$, 则 $A \cap B \subseteq C \cap D$ 。

4. 证明

(1) $(A - B) \oplus B = A \cup B$;

(2) $(A - B) \cap B = 0$;

(3) $A \cap (A - B) = A - B$;

(4) $A - (B - C) = (A - B) \cup (A \cap C)$;

(5) $(A - B) - C = (A - C) - (B - C)$;

(6) $(A \cup B) \cup (B - A) = A \cup B$ 。

5. 证明下列关系:

(1) $(A \cap B) \cup (A \cap B') \cup (A' \cap B) = A \cup B$;

- (2) $A \oplus B = (A \cap B') \cup (A' \cap B)$;
- (3) $A \oplus (A \cup B) = B - A$;
- (4) $(A \cup B) \cap (A' \cup C) = (A' \cap B) \cup (A \cap C)$ 。

6. 将下列各式变换成两种标准形式:

- (1) $(A \cup B)' \cap A \cup B$;
- (2) $(A \cup B \cup C)' \cap (A \cup B)$;
- (3) $A \oplus (A \cup B) - A$;
- (4) $((A \cup B) \cap (A' \cup C)) \cup ((A' \cup B) \cap (A \cup C))$ 。

7. 尽可能补上书中定理的全部证明。

附录 计算机科学系一年级计算科学实验参考目录

配合计算机科学系大学本科一年级“计算机科学与技术导论”课程的开设,可同时开设“计算科学实验”课程。参考学时数为 54—72 学时。排课时,“计算科学实验”课程应安排在一个下午(3—4 学时,名义上为 54 学时),每周一次。宁可少安排一个学时,也不要分两次上实验课。要注意提高学生板块时间的利用效率,防止实验课程冲击一年级更为重要的基础课程。

计算机科学系大学本科一年级(上)的“计算科学实验”课程实行因材施教。根据学生入学时计算机实际操作能力和水平,依次从下列实验单元中选取合适的实验单元组织实施,由实验室向学生提供《上机手册和实验内容提纲》,明确内容、要点和要求。

根据教学改革的研究成果,“计算科学实验”课程的教学重在基本实验方法、基本实验技术和规范的实验工作方式方法、工作流程的基本训练,理论联系实际的基本素养的培养,而不是突出实验的速度和实验结果的质量。因此,实验教学中必须强化实验过程的规范操作训练,弱化对实验结果的评判。即实验的结果是次要的,重要的是每一个学生应该按照实验的要求,理论联系实际,严格按照实验过程的规范要求和实验规程完成实验,提高自身的实践能力。

建议“计算机科学与技术导论”和“计算科学实验”课程考核方式为考查,包括平时实际操作和读书报告两部分。

一年级(上)计算科学实验单元参考目录(54—72 学时)

配合计算机科学系大学本科一年级(上)“计算机科学与技术导论”课程的教学,“计算科学实验”课程主要围绕熟悉计算机操作使用,提高感性认识这一目标进行教学。参考学时数 54-72 学时。

第一单元: 看一段有关计算科学学科简介的录像,参观计算机机房,听取实验室教师对实验工作的一般要求(2 学时,也可配合入学教育进行);

第二单元: 在计算机上练习打字(8—16 学时);

第三单元: 在计算机上学习、练习由《上机手册和实验内容提纲》给出的 DOS 操作系统的常用命令(4—6 学时);

第四单元: 学习英文、汉字(拼音编码)输入、文件编辑和打印输出(4 学时);

第五单元：在计算机上学习、练习由《上机手册和实验内容提纲》给出的 Windows 操作系统的基本操作（6—8 学时），运行由《上机手册和实验内容提纲》中给的一至两个高级语言程序，提高实际程序在计算机上运行的感性认识；

第六单元：在 Windows 或 DOS 操作系统的支持下，学习、练习由《上机手册和实验内容提纲》给出的一些应用软件和工具软件的基本操作。如软硬件系统或设备的安装、参数设置和卸载，PCtools、汉字库造字程序、计算机作画程序、计算机音乐创作程序、计算机辅助设计程序，计算机动画制作程序、办公自动化程序、Word、Excel、PowerPoint、科学论文编辑排版程序等软件的基本操作（20—30 学时，不必全部掌握）；

第七单元：学习和练习由“上机手册和实验内容提纲”给出的一些计算机网络软件的常用基本操作。如 E-mail 的发送和接收，访问网络上的其他结点，访问和查询 WWW 上的资源，查询科学技术文献库（剩余时间），网络连接的参数设置。

一年级(下)计算科学实验单元参考目录（54—72 学时）

在计算机科学系大学本科一年级（下）的课程教学中，由于尚未开设计算机专业（基础）课程，因此，学习各种软件如何使用的任务可在这学期完成。此时，“计算科学实验”课程可继续上一学期的教学。

第八单元：软件使用练习（3） 24—32 学时（第 1—8 周，每周一次）

学习和练习由《上机手册和实验内容提纲》给出的一些应用软件和工具软件的基本操作。如网页制作工具、科学论文编辑排版程序等软件的基本操作。注意掌握国际流行的(英文)科学论文排版软件；

第九单元：软件使用练习（4） 24—32 学时（第 9—16 周，每周一次）

学习和练习由《上机手册和实验内容提纲》给出的一种或两种流行数据库管理系统的使用。掌握库结构操作、库记录操作的一般方法，了解数据库语言程序设计的简单方法，了解应用数据库管理系统设计信息管理系统的基本流程；

第十单元：软件使用练习（5） 6—8 学时（第 17—18 周，每周一次）

学习和练习由《上机手册和实验内容提纲》给出的计算机绘图软件和图形工具软件的基本操作。本单元的教学内容在二年级（上）将另行安排一个单元继续四周。因此，有条件的学校可考虑让学生初步了解或掌握三维图形软件的使用。

后 记

1992年秋天，我从中国科学院数学研究所毕业回到武汉，在中国地质大学计算机科学与技术系讲授编译原理与人工智能课程。大约是11月上旬的一天，同系的孟永良教授突然向我提出一连串的问题：“你能否告诉我究竟什么是计算机科学？像现在这样，教学中注重计算机实际操作知识的传授和技能训练行吗？计算机科学导论应该讲些什么内容？怎样正确看待计算机科学中理论与实践的统一？……”进一步，他坦言道：“这些问题，我曾请教过几个计算机专业毕业的教师，他们似乎也没有说清楚。”

永良教授与我早在1982年就相识。他是一个地球物理学家，曾在大西北野外从事过多年的地质勘探工作，也曾在《地球物理学报》发表论文。10年前，为了照顾家庭，他改行调到正拟筹建的学校计算机科学与技术系任教。永良教授担任计算机科学与技术系计算机科学导论与高级语言程序设计等课程的主讲教师，尽管他常常谦虚地说“自己从事计算机科学是‘半途出家’，至今仍是‘门外汉’”，但他集多年的教学实践经验，以科学家敏锐的眼光和惟真求实的科学态度提出的这些问题，确实值得我们深思。老实说，他提的问题，以前我从未认真考虑过，甚至没有把它们看成是问题。在我的认识中，如果老师连这些问题都回答不了，何以教学？现在看来，永良教授的提问是正确的，在大学计算机科学系的教师中深入讨论这些问题很有必要。从那以后，我经常与这位坦诚的教授交流学术思想，探讨学问，促膝谈艺，获益良多。在我们的交流中，他曾不止一次地希望我能有机会系统地给系内的老师讲一讲对计算机科学一些基本问题的认识，并主讲几门难度较大的课程，借以起到培养青年教师的作用。虽然，永良教授的设想没能实现，但他提出的许多问题和观点使我认识了自己不熟悉的领域中一个科学家的思想方法与科学态度。这再次使我相信，科学的不同学科之间在更高的层面上是相通的。

上过大学的人都有切身的体会，数度寒窗，岁月如梭，一个人在大学学习的几年中获取的知识是十分有限的，更多的知识需要在今后的工作中去学习和掌握。正因为如此，一个高水平的大学，教学的主要目标与重点不在于具体教会了学生多少知识，而在于让学生为未来发展打下一个坚实的基础，帮助他们建立一种正确的研究问题的思想方法，掌握读书和获取新知识的本领。授人以鱼，还是授人以渔？这个浅显的道理大家都明白，但真正做起来就难了。高水平的大学之所以教学水平高，就在于有一批通过学术研究掌握了如何授人以渔要领的教授。

市场经济，人们每天都会面对新的挑战、机遇和诱惑，也许这些名词本来就是同义词，就看是谁来选择对号入座了。历史早已一再证明，社会思潮不是所有的人都能像对待疾病那样抵御得了的。面对现实，当计算机科学系的毕业生可以成为大公司中年收入6万—15万元的工程师，也可以成为收入不菲，福利待遇堪优的银行、海关、邮电等部门的职员时，科学的神圣与崇高或许正在远离一个又一个莘莘学子童年时代精心编织的梦和铭记胸怀的理想。当然，重视计算机实用技术的学习也是不错的，没有什么不好，这在不少职业技术大学早已进行了富有成效的实践，社会也确实需要各种类型的人才。谁都知道，学一门手艺，扬一技之长，曾是中国社会千百年来无数家庭教育子女的一种训诫，一种传统，毕竟社会的进步离不开能工巧匠，更离不开工程师。然而，面对非职业技术大学的学生，每一位教师所在的岗位有自己的任务、目标和不可推卸的责任。计算机科学终究不是一门单纯的技术，也不是所有能工巧匠的技艺能够覆盖的技术科学。如果没有高深的理论支持，计算机科学也决不可能成为一门万众瞩目的高科技。

反复推敲，仔细思考，永良教授的一连串的问题其实提出了一个我们明天的社会究竟需要什么样的科学家和工程师的问题，也就是说，我们到底只需要少量的计算机科学家，还是需要一大批计算机科学家？我们到底是需要能够从事“短、平、快”项目开发的工程师，还是需要用更多的计算机科学理论知识“武装”起来的工程师？这个认识问题不解决，计算机科学类专业教育必然还会各行其是。

“我们的大学什么时候能培养出有实力问鼎图灵奖的计算机科学人才？我们的国家什么时候才能结束靠大量出口农副产品、工艺品和劳动密集型产品换取高科技计算机产品的历史？”我无法回答也没有能力回答许多学生提出的这类问题，只能以本书献给那些有志于计算机科学事业的同仁和后来者，但愿它能起到一颗铺路石的作用。

平凡的小事有时也能给我们许多联想，发人深省。一日，我在农贸市场买水果，与一位从浙江来厦门种草莓的农夫闲聊。当我得知每年他需千里迢迢从北方购进草莓苗栽培时，大吃一惊，连忙向他请教是何缘故。他告诉我，厦门气温太高，在这里培养幼苗，虽然长得快，但产量低，易生病。农夫的文化水平不高，具体道理也讲不太清楚。于是，带着问题，我向书本请教，得到答案：原来，许多植物的生长有一定的规律，幼苗在合理的低温条件下栽培，能够起到“延迟生长”和为日后生长加速“积累能量”的作用，俗称“冻一冻”。联想到我们人类，一个小孩如果儿童时期长得比较快，以后青春期“发育”时生长速度会相对放慢，反之，则会加快。

十年树木，百年树人。我常想，人才的培养与树木的生长规律应该是相似的。幼苗时期不打下良好的基础，不经风霜雨雪的磨炼，是树，不可能成材，是人，

也不可能成才。我主张厚积薄发,更欣赏大器晚成。就像初春时节绽开的腊梅一样,“不经严寒风雨雪,哪得梅花百日香。”

近年来,由于承担国家教育部面向21世纪教学内容与课程体系改革项目的原由,我参加了有关的几个会议,与不少大学的师生进行过一些讨论。我有一种直觉,在目前的社会背景下,有关计算机科学类专业教育的一些问题很可能在一个相当长的时期内得不到解决,原因是复杂的。其实,若干有条件的学校,只要经过几届学生的试点就能检验究竟是哪一种观点对,从高等教育学理论上进行科学的讨论和论证也不是一件困难的事。尽管许多人都主张实践是检验真理的标准,但遗憾的是,这项很有意义的检验真理的工作似乎既感兴趣又有能力为之的人不多。

世界上恐怕没有一个国家像中国一样在计算机科学领域集中了如此多的优秀人才。从20世纪50年代起一直到今天,高等学校计算机科学类专业学生入学成绩一直在各校名列前茅,历经几十年不衰。近年来,随着国家经济的发展,实力的增强,高等学校计算机科学类专业的实验条件有了很大改善,少数重点高校的实验条件已同国外先进水平相差无几。乍一看,似乎我们已在人力和物力上为赶超西方发达国家计算机科学发展水平做好了准备。然而,实际情况并非如此,许多迹象都表明仍有大量基础准备工作尚待完成,尤其是对计算机科学学科的认识。21世纪正在向我们走来,她令人欣喜、激动、振奋,也掺杂着几许让人不安的遐想。

到厦门大学工作后,因为离得远,平时大家工作又都比较忙,虽然与永良教授始终保持联系,但交流少了许多。今年春节前,我曾寄给他一本小册子,里面对计算机科学一词的叫法也像这本教材一样改称为计算科学。这不是我的创造,而是美国人开的先例,现在也已为越来越多的人所接受:本来就应该这样。我猜测,远在武汉的良师益友——永良教授必定已经注意到了这一点。

作者

1997年7月20日

Images have been losslessly embedded. Information about the original file can be found in PDF attachments. Some stats (more in the PDF attachments):

```
{
  "filename": "MTEyNjc5NDUuemlw",
  "filename_decoded": "11267945.zip",
  "filesize": 30544768,
  "md5": "a4f35417c87a842c042cdba448f80464",
  "header_md5": "8ca891343525baeb3ba9f50e20ff3445",
  "sha1": "8399ce0c96be48df28d25896c3dabe0c10930561",
  "sha256": "1c0bf5ccf9e3aa662d105103d723a9946626a63a3a5a2ec687c476bc9eeee21a",
  "crc32": 3167475206,
  "zip_password": "",
  "uncompressed_size": 31332010,
  "pdg_dir_name": "\u8ba1\u7b97\u673a\u79d1\u5b66\u5bfc\u8bba\u7b2c\u4e09\u7248_11267945",
  "pdg_main_pages_found": 272,
  "pdg_main_pages_max": 272,
  "total_pages": 291,
  "total_pixels": 1537966080,
  "pdf_generation_missing_pages": false
}
```